

NEA Chess AI

Harry Foster

Candidate Number:

Centre Number:

ANALYSIS	5
BACKGROUND	5
EVIDENCE OF RESEARCH	9
<i>How a chess engine works</i>	10
Generation	10
Evaluation	11
Searching	11
Opening Books:	12
EXISTING SOLUTIONS	14
PROGRAMMING LANGUAGES TO USE	15
OBJECTIVES	16
DESIGN	19
ENGINE	19
OVERVIEW	19
ENGINE CLASS PURPOSE TABLE	20
ENGINE UML DIAGRAM	22
ALGORITHMS:	23
<i>NegaMax</i>	23
<i>Make Move</i>	23
<i>Opening book probe</i>	25
DATA STRUCTURES:	27
<i>Bitboard</i>	27
<i>Stack</i>	28
<i>Move</i>	28
<i>Transposition Table</i>	29
UI	31
TESTING	40
EVALUATION	49
OBJECTIVE ANALYSIS	49
USER FEEDBACK	52
HOW THE SYSTEM COULD BE IMPROVED	54
REFLECTION OF THE PROCESS	55
TECHNICAL SOLUTION	56
ENGINE	56
<i>Program</i>	56
<i>Command Handler</i>	56
<i>Representation</i>	65
Bitboard	65
BitboardTools	74
BoardState	77
Tools	78
<i>Generation</i>	84
Generator	84
Magic	96
Move	107
<i>PolyGlut</i>	108
PolyEntry	108
Polyglut	109
Keys	115
Zobrsit	118

<i>Search</i> -----	122
MoveOrderer-----	122
Perft-----	124
Searcher-----	129
<i>Evaluation</i> -----	134
Evaluator-----	134
PieceTables-----	137
Transposition table-----	140
UI-----	143
<i>MainWindow</i> -----	143
XAML-----	143
C#-----	144
<i>Views</i> -----	144
HomeView-----	144
Screen-Shot-----	144
XAML-----	144
C#-----	145
MainMenu-----	146
Screen-Shot-----	146
XAML-----	147
C#-----	147
Settings-----	149
Screen-Shot-----	149
XAML-----	149
C#-----	151
Manage Engines popup-----	155
Screen-Shot-----	155
XAML-----	155
C#-----	156
BoardPreviewView-----	159
Screen-Shot-----	159
XAML-----	159
C#-----	160
NewGameView-----	160
Screen-Shot-----	160
XAML-----	160
C#-----	163
NewGameBotSelect-----	168
Screen-Shot-----	168
XAML-----	168
C#-----	168
NewPlayerSelectView-----	169
Screen-Shot-----	169
XAML-----	169
C#-----	170
TimePerGame-----	170
Screen-Shot-----	170
XAML-----	170
C#-----	173
UnlimitedTime-----	175
Screen-Shot-----	175
XAML-----	175
C#-----	175
PlayingScreen-----	176
Screen-Shot-----	176
XAML-----	176
C#-----	177
GameInformationView-----	179

Screen-Shot	179
XAML	179
C#	180
PlayerInformationView	181
Screen-Shot	181
XAML	181
C#	182
BoardView	187
Screen-Shot	187
XAML	187
C#	187
Promotion	188
Screen-Shot	188
XAML	188
C#	188
GameOver	190
Screen-Shot	190
XAML	190
C#	191
AnalysisScreen	191
Screen-Shot	191
XAML	191
C#	192
EngineAnalysisView	193
Screen-Shot	193
XAML	193
C#	195
AnalysisBoardView	197
Screen-Shot	197
XAML	197
C#	197
<i>ViewModels</i>	<i>198</i>
BoardViewModelBase	198
C#	198
BoardViewModel	205
C#	205
AnalysisBoardViewModel	207
C#	207
BoardPreviewModel	209
C#	209
<i>Models</i>	<i>210</i>
User	210
C#	210
UISettings	211
C#	211
EngineInfo	214
C#	214
EngineOption	214
C#	214
Square	215
C#	215
Game	215
C#	215
CurrentGame	217
C#	217
AnalysisGame	221
C#	221
UIMove	224

Candidate Number:

Harry Foster

Centre Number:

C#	224
Player	226
C#	226
Bot	227
C#	227
AnalysisBot	229
C#	229
OnlinePlayer	231
C#	231
<i>Client</i>	231
Program	231
C#	231
<i>Server</i>	234
Program	234
C#	234

Analysis

Background

Due to the pandemic, chess has skyrocketed in popularity worldwide. The Netflix Series, “The Queens Gambit” which released in late October of 2020 helped chess become a household activity. Since then, chess has found its way into the mainstream media a few times with the scandal involving Hans Niemann being one of the most prevalent. Within schools the chess club has become one of the most favourable and popular for young players which has caused pupils to want to improve their skills and become better chess players. There are popular sites such as chess.com or lichess, but the school has put measures in place to block these sites from students as they may access them during inappropriate times. I plan to produce a chess program which will allow students to play against an engine of different strengths so that they are able to progress and improve upon their skills as well as allow them to play against each other. The system should also allow the students to setup a position and analyse it as well as allow them to look through a game that they have played.

To get a better understanding of what my program should do so that it will be helpful to the chess club I interviewed the current head of the chess club Mr Dennis. Some of the questions and responses were:

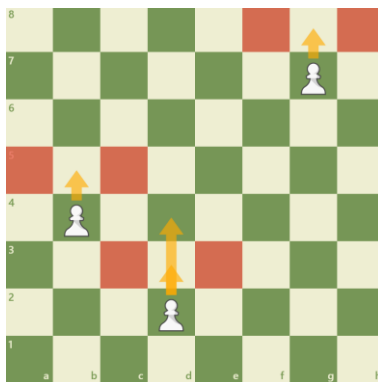
Question	Response
Would a chess program be useful?	I think that a chess program would be very useful for students and for the club since we have around 50 students each week and we don't have enough physical boards so some need to wait to play. Students can't use their phones during school hours and devices as we block chess sites.
What would you look for in a chess program?	I would generally look for a program that would allow students to play on their own against a computer and would also let them play against each other
What is the most important feature that you would want?	The most important feature would be the ability for students to analyse their games that they played.
What other features might make a chess program more appealing?	Customisation is a very important for students, I would like to see a program where students can change colours of the UI and make it personalised to them.

Chess is a very complex game with a lot of different rules which need to be validated during move generation. Here are the rules of the game so that we can understand what the problem is:



This is the starting position for a classical game of chess, there are two sides white and black. White always starts. The board is made up of an 8x8 grid with a chequered pattern, the square a1 (the bottom left square) is always a dark square. The board is made up of files and ranks, each column is called a file and corresponds to the letters of the alphabet whereas the rows are called ranks and are numbered e.g., the first file is the A file, the rightmost the H file etc, and the bottom row is the first rank, and the top is the eighth rank.

There are 6 different types of pieces each with their own rules and legal moves:



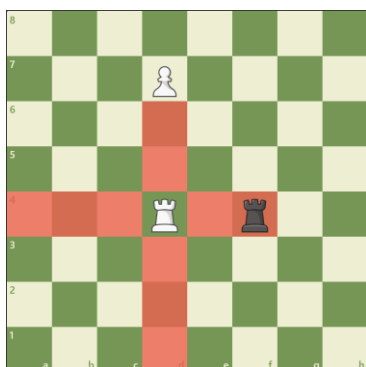
Pawns

The pawn is most abundant piece on the board and has a value of 1. Each player starts with eight pawns one on each file. If a pawn is on its starting square can move two spaces forwards in its respective direction. After, the pawn can move one space forward. The pawn can capture on its forward diagonals and only if it is an enemy piece, these squares are highlighted in red. If a pawn reaches the end of the board, it can promote and become a higher value piece; it is able to become a bishop, rook, knight or queen.



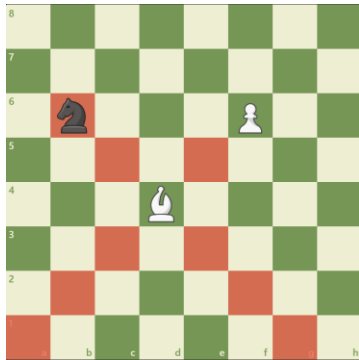
En Passant

This is a very special case which is called En-passant. This is where on the previous move a pawn has moved two places and is now on the same rank as an enemy pawn. In this position, it is black to play and white played c4 on the previous move, this means that the black pawn is able to capture on c3 (but in this position it is not able to as it would leave the king in check).



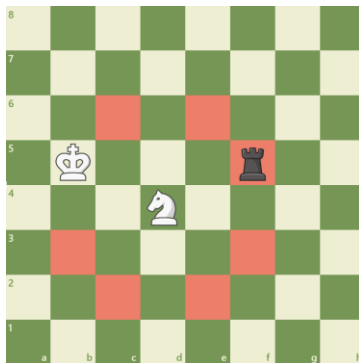
Rooks

The rook starts on the corner squares and are orthogonal sliding pieces and is worth about 5 points of material. The rook can move anywhere on the same file or rank. If a piece is in its way, then it obstructs its movement. In the diagram shown the red squares show all the places the rook can move; it cannot take the white pawn as it is one of its own pieces whereas it is able to take the black rook. Both the pawn and the rook block its movement as it is not able to move to the edge of the board.



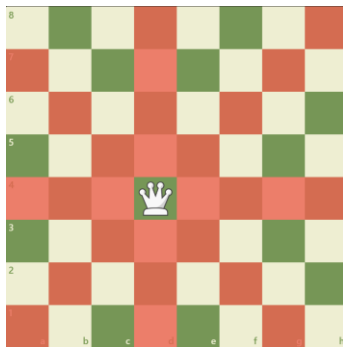
Bishops

The bishop is very similar to the rook where it can move anywhere on the same diagonal hence it called a diagonal sliding piece. The bishop is worth less than a rook with about 3 points of material. It also obeys the same rules when presented with blockers. This can be seen in the diagram where the white pawn blocks the movement of the bishop.



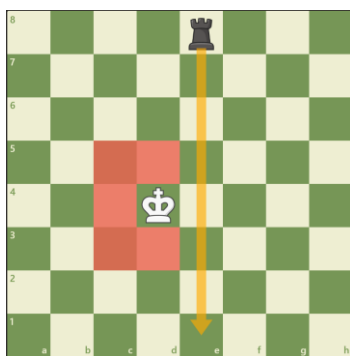
Knights

The knight has a more unusual move rules as it can jump over pieces. The knight can move in L-Shapes as shown by the red squares on the diagram. When a knight puts a king in check (when it attacks a king) there are only two responses, either for the king to move away from the knight or to somehow capture the knight.



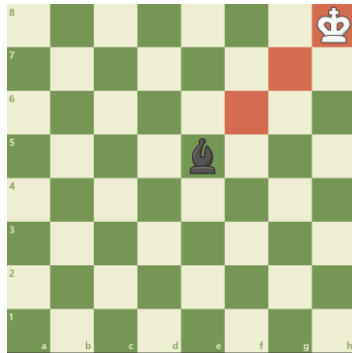
Queen

The queen is the most valuable attacker in the starting of the game, this is because it has the most mobility as it is a combination of a rook and bishop. The queen also obeys the same mobility rules to that of a rook and a bishop, if a piece is in its path it blocks its mobility.



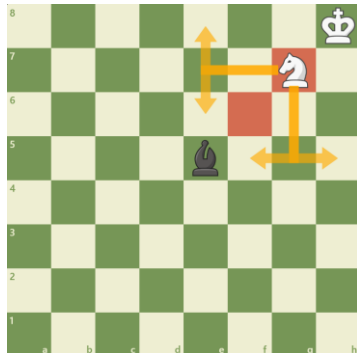
King

The king is the most important piece on the board as the objective of chess is to checkmate the opponent's king. Checkmate occurs when the king has no available squares, it is in check and no other piece can capture the checking piece. In terms of mobility, the king is only able to move 1 square in any direction. The king is not able to move into squares attacked by an opponent's piece. This can be seen in the diagram where the king is unable to move into the e file as the black rook prevents it from doing so.



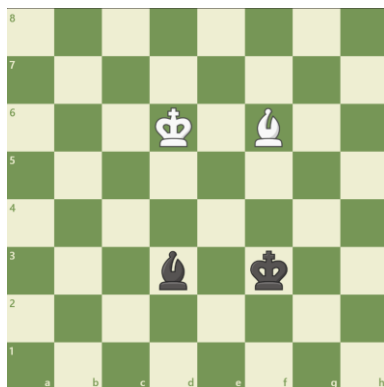
Check

In this position the opponent's bishop is attacking the white king, this is called check as if it were black's turn it would be able to capture the king. When in check the player must either move out of the way of the attacking piece, block the attacking piece, or capture the attacking piece. If two pieces were attacking the king at the same time it is known as double check and the only response the player can do is to move the king.



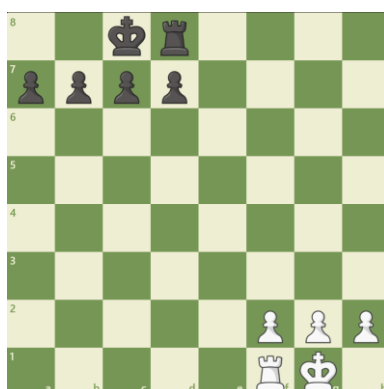
Pins

This is the same position as before but there is a knight on the g7 square. This knight is unable to move as if it did it would allow the bishop to take the king. This is known as a pin. If the knight were replaced by a bishop, then it would be able to move about the pin and capture the bishop.



Stalemate

Stalemate occurs in a few different ways, the most common being when the player has no available moves, yet they are not in check. In this case the game is a draw. Another being the 50-move rule which is where it has been 50 moves since the last pawn move or capture. Three-fold repetition is where the same position has occurred three times during the same game, in this case it is a draw. The final way to draw is by insufficient material, this is where there aren't enough pieces on the board to mate the king. Pawns do not count as insufficient material as they have the potential to become a different type.



Castling

Castling is another special move in chess. Castling can happen when both the king and the rook have not moved and the squares between them are completely free. The king can castle on either side if it is legal, to castle the king moves two places and the rook is moved to the side of the king. On the diagram we can see white has castled kingside (moved from e1-g1) and black has castled queenside (e8-c8). Castling can be prevented as the king cannot castle through check, into check nor as a method to get out of check.

The problem is to produce a program which will be able to play a full game of chess without making any illegal moves. The algorithm will also be able to evaluate a given position and output the strongest move.

Evidence of Research

History of chess engines

Chess has a rich history that can be traced back over 1500 years. Chess has evolved into what we know today in the 16th century yet there weren't any clocks or standardised pieces. It wasn't until the 19th century until the first official world championship came about. Since the 19th century many different books highlighting different principles used in the game such as openings, middle game strategies or endgames. The emergence of chess engines was started in the early 20th century by Leonardo Torres Quevedo named "El Ajedrecista" (the chess player) which was able to play a limited form of chess. The earliest known computer chess engine was in 1948 by Alan Turing and David Champenowne and was called "Turochamp" and was able to play a full game of chess verses a computer but it was never completed as the developed algorithm was too complex to be ran by the earliest of computers which caused Turing to test the algorithm by hand. A close colleague of Turing, Claude Shannon, would later publish his paper "Programming a Computer for Playing Chess" that would outline different ways to create a chess engine and is one of the most influential papers for the creation of chess engines. Later in 1957, IBM programmers would create "The Bernstein Chess Program" which was the first complete chess program and was able to play an entire game of chess.

Into the 1960's and 1970's chess engines grew largely in popularity as many different computer scientists worked on making the best chess engine. During this time chess grandmasters would devote their time to improve chess programming by many different methods, the most notable being the improvement searching by introducing move selection techniques as well as opening and endgame databases. During the 1970's hardware advanced at a rapid rate which had a significant effect on chess programming as hardware was able to perform brute-force evaluation, this is where the algorithm checks every possible move which was previously unfeasible. The first computer to win against a human master would be in 1978 where David Levy would be beaten once during a six-game match by "Chess 4.7" with the resulting score being 4.5-1.5. After this point chess engine programming would become extremely competitive with many monetary prizes for the best algorithms to be developed. In 1980 Edward Fredkin would offer \$10,000 for the first engine to reach grandmaster level and \$100,000 to beat the world champion. This famously lead to the development of Deep Blue which was the first chess computer to beat the world champion, Gary Kasparov. Deep Blue originally started as "Chip Test" in 1985 and would win the North American Computer Chess Championships in the following year. After this the team would develop a better engine Deep Thought in 1988 which was significantly better than its predecessor and become the first engine to defeat a chess grandmaster and would win the competition for 5 more times. After, in 1994 IBM would take an interest in the project and would sponsor the team which lead to the creation of Deep Blue. In 1997, Gary Kasparov would be defeated.

Going into the new millennium, the chess engine would continue to improve with hardware improvements with the strongest algorithms such as stockfish emerging. In 2017 AlphaZero would become a new revolution in chess programming as it would take a different approach than all other chess engines. Where traditional chess engines would construct game trees and search through every possibility of moves and evaluating positions, AlphaZero would use neural networks for its analysis. In a 100-game match, AlphaZero against Stockfish, AlphaZero would win 28 games and would draw a further 72. Since then, the strongest of chess engines would adopt some sort of neural networks to further improve its strength.

How a chess engine works

Generation

There are many different approaches to making a chess engine. The three main parts to a chess engine are: Generation, Evaluation and Searching. Generation deals with generating all the legal moves in a given position and is the most important part of a chess engine as if illegal moves are generated it will cause incorrect lines to be evaluated. To make sure the algorithm is generating all of the correct legal moves we can use perft testing which is where we count all of the strictly legal moves generated at a certain depth and compare it to a pre-computed table of values. Perft results can be found on https://www.chessprogramming.org/Perft_Results which I will be using to test my system. To represent a position, you can use integers to represent the type of piece as well as the colour in an arrangement such as:

	Pawn	Knight	Bishop	Rook	Queen	King
White	0001	0010	0011	0100	0101	0110
Black	1001	1010	1011	1100	1101	1110

This is useful as the colour of the piece can be known by a bitwise and with 8. One problem is that 64 integers will be needed to represent a position which is not ideal. Another method of representation is using bitboards where a 64-bit integer is used to represent a certain piece type. For example, 12 bitboard can be used to represent a position, one for each piece type and colour by using 1 or more arrays of 64-bit integers. This could look like:

Array – index	0	1	2	3	4	5
White Pieces	Pawn Bitboard	Knight Bitboard	Bishop Bitboard	Rook Bitboard	Queen Bitboard	King Bitboard
Black Pieces	Pawn Bitboard	Knight Bitboard	Bishop Bitboard	Rook Bitboard	Queen Bitboard	King Bitboard

You can reduce this to 8 bitboard if needed, one for each type of piece and one for each colour. So, this would look like:

Array – index	0	1	2	3	4	5	6	7
Pieces	White Pieces	Black Pieces	All Pawns Bitboard	All Knights Bitboard	All Bishops Bitboard	All Rooks Bitboard	All Queens Bitboard	King Bitboard

The advantages to using 8 bitboards is to save memory but the trade-off is that you will need to manually compute the bitboard that corresponds to just the piece and colour required. For example, to get the bitboard that corresponds to only the white pawns you would bitwise and the white pieces and the pawns bitboards. Although this is not difficult and is also inexpensive it is another step that the processor will need to take which will increase the overall processing times. To generate moves there are two main different methods, to calculate position on the fly or to use a pre-computed table of moves. Calculating positions on the fly are very computationally intensive and can slow the system down quite drastically, whereas precomputed tables is more memory intensive. This can speed up the system, for this reason, the trade-off between memory usage and computation time is not such a big problem because of the amount of ram in a modern computer. One problem that arises is when you need to know the type of piece on a given square, it is quite

computationally intensive as you need to calculate all of the piece and colour types and then loop over, one common solution is to keep a separate representation of the board so that it is easily identifiable.

Evaluation

The second main part of a chess engine is evaluation. As mentioned previously, new neural-network based chess evaluation algorithms have become the new industry standard. Before this, the most common way was to create a Hand-Crafted-Evaluation (HCE) which would be a heuristic function which its focus is speed and accuracy. The most basic evaluation function would simply return the sum of all the piece values. This is not very accurate as in general, the position of the pieces is much more important. Building off this we can add more factors to make the evaluation more accurate such as the mobility of the pieces (how many legal moves can be played) as in general the more legal moves a player can do the more control the player has. Chess can be described as 3 main stages, the opening, middlegame and endgame. The middlegame generally starts where it is a unique game of chess with the position never being reached before, it is in this stage of the game where chess engine as most utilised. By knowing which stage, the game is in we can use pre-set factors to evaluate the position as in general in the middle game it is better to have your pieces controlling the centre of the board or the king to be castled and in safety which is what we want to encourage the engine to do. However, as the game progresses into the endgame these factors can become a hinderance to the engine as commonly it is better to progress the king to the centre of the board in the endgame as it has the most mobility and is a useful attacker. Calculating which game stage, the position is in can create problems such as “evaluation discontinuity” which is where the same position is marked as being in different game stages which can cause issues with how it is evaluated and cause a worse decision to be made. To avoid this error “tapered evaluations” are used which gradually changes the game state using a decimal which is calculated by the number of pieces and their types still left on the board. (maybe mention genetic algorithms)

Searching

The final main part of a chess engine is searching. As previously stated, the basic premise of a chess engine is to look at every possible move and pick the ones that lead to the best outcome for the chosen side. The main algorithm used to do this is the minimax algorithm, to understand how it works take the following graph:

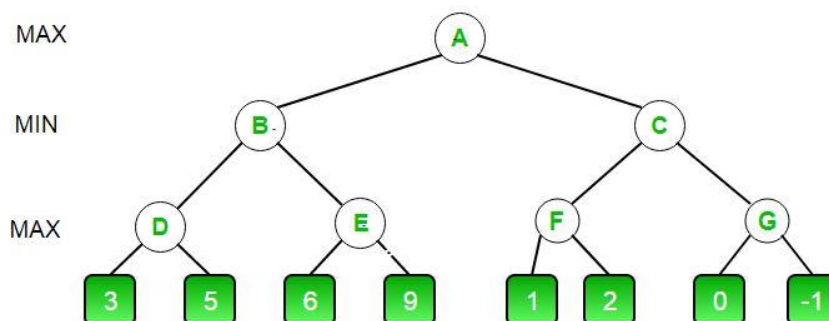


Figure 1: <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-4-alpha-beta-pruning/amp/>

In this situation we want to maximize A, starting at the bottom of the graph at D, we want to maximize this level, between 3 and 5, 5 is bigger so D is 5. An extension of the minimax algorithm is called alpha-beta pruning which is where we “prune” branches off the tree so that it becomes more efficient to search through as we don’t need to search every individual node. Using the graph above as an example, we know that B needs to be minimised so B must be less than or equal to 5 so when we evaluate E we can see that the first one is 6, which means that E will be greater than or equal to 6 which is greater than 5 so we don’t need to look the 9. If we continue this, we get the following result:

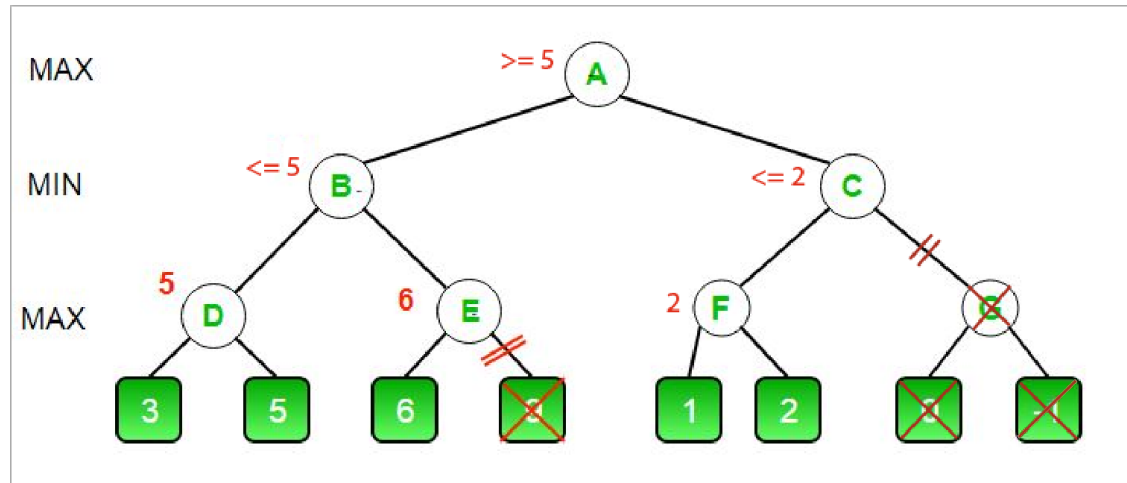


Figure 2: Figure 1: <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-4-alpha-beta-pruning/amp/>

Which tells us that decision B is the best one to take as it leads to the best outcomes. There are a few things we can do to speed up this process, one of which is sorting the moves so that the most likely best moves are evaluated first which means that the algorithm will prune more branches and as a result will not need to evaluate through as many positions making the computation time faster. For example, if we would want to look at pawn capture moves first as it is most likely that the pawn will capture a higher value piece or maybe look at checks as it would force the opponent to respond and will give the player more time to improve its position. To put this into context, the root nodes shown in the graph above would be the evaluation of the position after a series of moves. A very popular way to reduce search computation is to use “Transposition table” which would be a hash table of previously computed moves so that if we find the same position, we can just look up the evaluation instead of computing a new evaluation. As the game progresses the stored position will become less relevant as it will most likely not be able to reach some of the positions stored. To counteract this, we set a certain limit to how much data can be stored within the table (ideally as much as possible) and when it is near to full capacity, we clear the table. The main problem that arises with this problem is how to convert a position to a hash so we can use it as an index but for it to be a perfect hashing function with no collisions. There are many ways to solve this problem, but the most well-known and best current algorithms is Zobrist hashing. The reason Zobrist hashing is one of the best solutions is because it once a move has been made you do not need to recalculate the Zobrist hash but by instead modifying the hash by performing a bitwise xor on the changed pieces and squares.

Opening Books:

Opening books is an integral part for chess engines. The most common types of opening books are, polyglot, ABK (Arena's book format), and CTG (ChessBase Opening Tree of Games). CTG is a closed

source proprietary opening book format owned by German company Chessbase. ABK and polyglot on the other hand is opensource and free to use. ABK consists of an array of entries that contain the following structure:

```
{
  char from;
  char to;
  char promotion;
  char priority;
  int ngames;
  int nwon;
  int nlost;
  int plycount;
  int nextMove;
  int nextSibling;
}
```

The size of each entry is 28 bytes.

Polyglot is the other commonly used opening book format. Polyglot also contains an array of entries and uses 16 bytes per entry and stores the following information:

```
U64 key;
U16 move;
U16 weight;
U32 learn;
```

The key is the Zobrist hash of the board position and is calculated using the random numbers which can be found on: http://hgm.nubati.net/book_format.html. The hash is calculated by the exclusive or of all the pieces on the board, the castling rights, the en-passant value and the players turn. The file is sorted from the lowest hash value to the highest which allows searching algorithms such as the binary search to be used. The move is stored as a 16-bit unsigned integer which is in the following format:

N/A	Promotion type	From row	From file	To row	To file
0	000	000	000	000	000

Where the a-file is represented as 0 and the h-file is represented as 7,

Where the first rank is represented as 0 and the 8th rank is represented as 7,

Where the promotion type is as follows:

Type	Value
None	0
Knight	1
Bishop	2
Rook	3
Queen	4

For this project I have decided to implement the polyglot book opening standard as each entry is a lot smaller which is beneficial as I will load the data into main memory instead of opening and reading the file each time. As well as this it will also be beneficial as I am calculating the Zobrist hashing anyway since I am using it for the transposition table.

Existing Solutions

Stockfish is currently the most popular chess engine in the world and makes use of the Universal Chess Interface (UCI). This allows Stockfish to communicate with any user interface which supports the UCI protocol. This is a very useful feature which I would like to implement within my project since it will make my engine portable which allows me to link with existing platforms such as Lichess. It will also make it easier to compare the strength of engines as using chess GUIs' such as "Arena" which is mainly used for making two engines play many games against each other to see how they compare to each other.

As previously mentioned, stockfish uses neural networks to evaluate positions. For simplicity I would like to stay away from creating neural networks and would like to create a hand-crafted evaluation function. Perhaps in the future I could improve on my engine by substituting the HCE for a neural network as a fun side project.

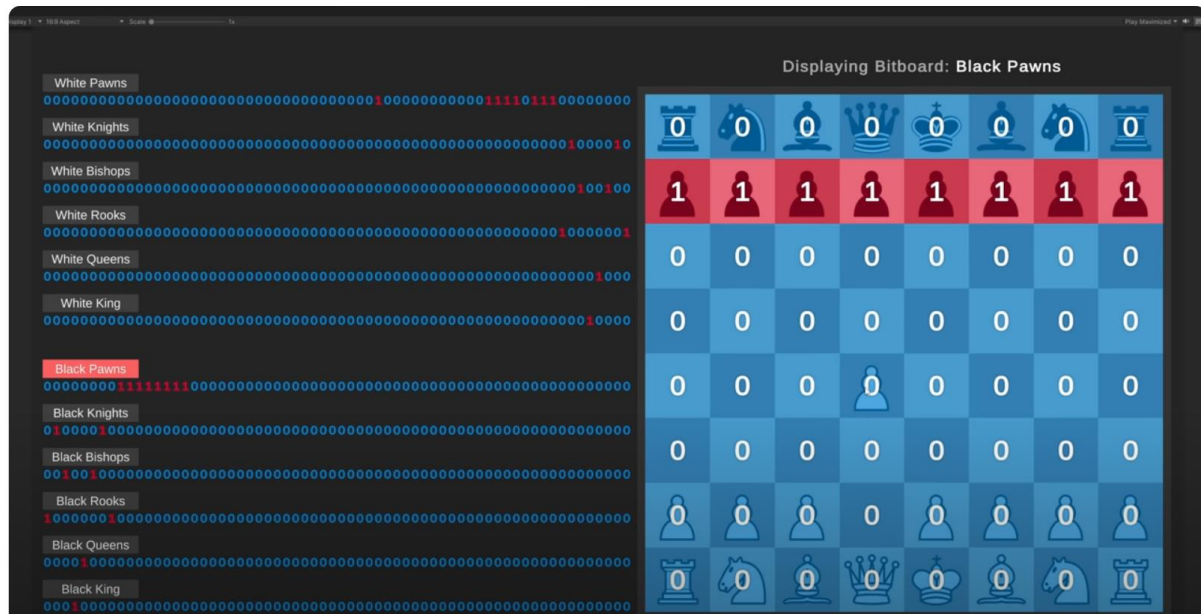


"Lucas Chess" is a GUI created by Lucas Monge using python and Qt. The software is completely free and is open source. It is quite minimalistic with only the board and the drop down menus at the top. The GUI has many features such as being able to add any engine as long as it uses the UCI protocol. General gameplay has a nice feel to it, but I did not like that when clicking on the piece the available moves were not shown nor was there any indication of which piece was currently selected. Once moving the piece an arrow would be displayed indicating the last move played, while this was a very nice idea, I found the arrows to be visually cluttering, I would prefer this to be done by changing the

colours of the start and destination squares. The software has lots of options which allowed you to change every aspect of the board such as the pieces, the square colour, the arrow colour and much more while also having many pre initialised popular options.

Another software I found was SCID which has a very nice yet more complex GUI than Lucas Chess. I liked the notation and comment editor windows which shows the history of the current game played in standard algebraic chess notation. Another feature you can do is traverse the game and jump to any previous position. I liked the comments that store any annotations the player creates within a specific position so that they can be used later. While playing, the GUI changes the border of the squares to indicate the last moves played, while this is a nice way that the arrows used by Lucas Chess, SCID is too minimalistic to where the last move played is not noticeable on the board. The general gameplay was quite nice to use, and the program did highlight the currently selected piece but did not show the available moves.

Sebastian Lague



While researching about chess engines, I came across Sebastian Lague who has created his own chess engine using unity. His idea of visually displaying the bitboard like so, is useful for debugging purposes.

Programming Languages to use

I intend to use C# and WPF since C# is a fully featured and highly regarded programming language used often in industry designed specifically for windows and offers a wide range of tools that I can use to build my program. C# offers built in memory management using garbage collection which will be helpful to make sure the program isn't bloated with unnecessary data which decrease the total amount of main memory the program uses. C# is well documented which is good for fixing bugs, as well as this it also has a large community which offers a lot of help and coding examples online using resources such as GitHub and stack-overflow and chess-programming-wiki. Moreover, C# is an object orientated language which will help me to modularise my code so that it is easier to debug

and work on. Being a compiled language, it offers more speed than that of an interpreted language such as python, this is especially better because of the amount of computation the program will have to perform.

I am confident that by using the canvas tool in XAML borders to create the squares of the board I will be able to add images to display the pieces.

Objectives

- a. The program must be able to play a full game of chess without any illegal moves. The program must:
 - a. Calculate all of the legal moves given any position
 - b. Be able to recognise the state of the game
 - i. if the position is checkmate, stalemate or a draw
 - c. Be able to handle special moves:
 - i. En-Passant
 - ii. Castling
 - iii. Promotion
- b. There must be a user interface which must:
 - a. Support 2 players
 - b. Store the move history in standard chess notation
 - c. Allow the user to interact with the chess board
 - i. Allow the user to click on a piece and show all of the legal moves for a selected piece
 - ii. The user should be allowed to drag and drop the piece using the cursor
 - iii. The legal moves will be indicated by a red border shown around the squares
 - iv. If the user tries to make an illegal move the program will not allow it and will reset the piece back to its original position
 - d. Allow the user to resign
 - e. Allow the user to offer a draw if playing against a player
 - f. Allow the user to request a takeback
 - g. Be able to communicate with the engine using the uci standard
 - h. Be able to switch to analysis mode which should:
 - i. Allow the user to input a custom board (FEN String)
 - ii. Show the output of the engine:
 1. The moves looked during searching (PV) in uci algebraic notation
 2. Display the number of moves generated at a given depth
 - i. When creating a new game there must be a window which:
 - i. Allows the user to choose an engine or player
 - ii. Allows the user to select a time control:
 1. Unlimited
 2. Time per game:
 - a. Input an amount of time
 - b. Input an increment time
 - iii. Allow the user to input the:

1. Site
 2. Round number
 3. Date
 4. Event type – Casual or competitive
- j. A setting screen which must:
- i. Change the UI colours:
 1. The background
 2. The buttons
 3. The panels
 4. The text
 5. The Light square
 6. The dark square
 7. The available moves
 8. The arrows
 - ii. Allow the user to restore dark or light colour scheme pre-set
 - iii. Change the pieces that are displayed
 - iv. Allow the user to manage engines:
 1. Import new engines
 2. Delete engines
 - v. Show a preview of the board
- c. Search
- a. Minimax algorithm with alpha beta pruning
 - b. The search tree should be able to search to a depth of at least 5 plies in a reasonable amount of time
 - c. Use of Transposition Tables to prevent unnecessary evaluation which must:
 - i. Use Zobrist hashing
 - d. Sort the moves so that the most likely good moves are searched first
- d. The evaluation algorithm must consider:
- a. The game stage (Opening, middlegame, endgame):
 - i. It must calculate which stage the game is currently in
 - b. The position of the pieces
 - i. Should use piece square tables
- e. Generation should:
- a. Use Bitboards to store the position
 - b. Make use of Magic bitboards to store rook and bishop attack tables
 - c. Store Pre-calculated attack tables for:
 - i. Knights
 - ii. Kings
 - iii. Pawns
 - d. Generate all of the square that the opponent is attacking
 - e. Calculate check and double check
 - f. Generate check rays and pin rays
 - g. Pass all performance tests and should be bug free

- f. Opening Book should:
 - a. Be able to open and read any valid polyglot opening book
 - b. Use a binary search to query the data

Design

When deciding how to break up the program for development, I decided that would use object-oriented programming because it would allow me to develop each part of the program separately and allow me to test each one as I go along. After I also decided to split the UI and the Engine into two separate projects so that I could work on them separately and for portability, as I wanted the engine to be able to work on any platform. To allow the two projects to interact with each other I imported the Engine project into the UI project as an assembly reference so that the resources and the code used within the engine project could be utilised. The UI communicates with the engine using the UCI standard which opens the possibilities for the UI to connect and communicate with any compatible engine.

Engine

I will first explain about the engine project. The project is separated into many different classes, below is a table outlining them all as well as their function. Following this is a UML diagram to illustrate on a basic level how they interact with each other.

Overview

The general way that the engine works is based on the UCI protocol. Once a command is entered by the user, the engine parses and executes the command. The most important commands are the position command which tells the engine what the current position is. The user has the option to either load this position as a FEN string which is a standard way of representing any chess position or tell the engine that it originated from the start position which is given by the parameter "startpos". The general structure of a position command is:

```
position [startpos/fen] {fen string} {moves} {list of moves eg e2e4 e7e5}
```

When this command is entered the engine will parse the first argument, if it is "startpos" then it creates a new bitboard. If it is "fen" then the engine will get the fen string and try and parse it into a bitboard. If these are successful, then the engine will check to see if the "moves" argument is given. If it is then it will split the list of moves up and execute each move, if an invalid move is given then the engine will throw an error.

The other most important command is the go command. There are many parameters that can be passed into this command such as the desired depth that the engine will search to, the amount of time the engine should search for, the amount of time left for both players. When the go command is entered by the user the command is parsed and the searcher begins. The searcher runs an iterative deepening search using the minimax algorithm. This means that if a maximum depth of 5 was given, for example, then the algorithm would first search at a maximum depth of 1 then 2 then 3 and so on up to 5. During this search each possible move will be generated and played out until the maximum depth was reached which is the number of turns played since the initial position. Once this has been reached then the position is evaluated and a value for how good the position is for the player is returned and this value is used to find the optimal move which the engine will then display to the user.

Engine class purpose table

Class/Structure	Purpose
Command Handler	A class which takes the command inputted from the console and parses it
Searcher	Performs a search on the given position and returns the best move
Perft	Runs a performance test on the given position up until a specified depth and outputs debugging/test information.
PerftTest	A structure that contains the information that will be passed to the perft class. The structure is only used when performing bulk tests which are converted from a json file in the engine folder.
MoveOrderer	Given a list of moves/a position it will sort the moves in the following order: killer moves, captures, quiet moves. The moves are sorted this way so that the best moves are searched first which will likely create more beta cutoffs and as a result it will not have to search more unnecessary lines.
Evaluator	This will be given a position and will return a numerical value that represents how good the position is for the side to play. This is calculated by how many piece on the board as well as their locations.
PieceTables	A static class which initialises and is used by the evaluator to retrieve values associated for a piece on a specified square.
BoardInfo	Stores information about the board such as: the number of white and black pawns, knights, bishops ect. Contains lambda functions which calculates the material score based on the constant material values stored in the struct.
Transposition Table	This is a specialised hash table which stores the evaluations for positions which have been calculated by the search so that they do not need to be calculated again.
TTEnter	The information stored within the transposition table and consists of: The hash of the position, the depth the position was searched at, the value of the position, whether it caused a cut off.
GameState	An enum which represents the current game state. The three options are: In-progress, Checkmate, Stalemate
Tools	A static class which contains useful functions such as converting a FEN string to a bitboard, printing a bitboard and converting a move into the long algebraic notation.

BitBoardTools	A static class which contains useful functions and data which helps dealing with bitboards such as getting the least significant bit. It also stores pre-computed attack tables for pawns, knights and kings.
Move	A struct that represents a move as a 16 bit unsigned integer. The struct contains useful masks which helps to decipher what move is being played
Bitboard	A class that stores all the information about a position such as the castling rights, the en-passant file, the piece bitboards and more.
BoardState	A struct which stores information that cannot be obtained from a move struct about a position. It is used within the undo move to recover the previous positions information.
Generation	This will generate and return a list of all the legal moves that can be played at any given position
Magic	This is a static class which stores all of the possible combinations of pseudo legal moves a rook or bishop can produce for any combination of blocking pieces
PolyGlott	A class which opens a polyglott book, reads it and stores the values into a array of entries.
PolyEntry	The information stored within one entry of a opening book and consists of: the Zobrist hash of the position, the move that will be played, the weight of the move, the learn value of the move.
Zobrist	A static class that loads the hashing values that are used to generate the final Zobrist hash and contains functions to manipulate and change the hash.
ZobrsitKey	A structure that contains the separate keys that form to make the Zobrist hash.

Page 22 of 236

Algorithms:

NegaMax

The search class uses a key algorithm which is known as the minimax algorithm. This was briefly discussed in the analysis section. An alternative to this but yields the same output is the Negamax algorithm which uses the same principal as the minimax algorithm but uses the mathematical relation:

$\max(a, b) = -\min(-a, -b)$ and vice versa. Using this relation, only one function is needed whereas in the traditional minimax two separate functions are utilised, one for minimising and the other for maximising. The parameters passed into the function are the depth remaining, the values of alpha and beta and if it is maximising or minimising. The function checks whether the depth is 0, if it is then it is at the maximum depth needed to search and will return the static evaluation of the position. Then we check to see if the position is the end of the game, if it is we return the appropriate values (0 for stalemate). After this we probe the transposition table to check if the current position has already been evaluated elsewhere during the search. If it has, we can simply use this value and return up the tree. If we have not, then we must evaluate it and so we generate all of the legal moves for the position and alter the board to reflect the move being made and then recursively call the function so that a value be assigned. The value returned will be compared to the other moves within the local search and based on if we are maximizing or minimizing it will choose the newly searched move or discard it. Based on the values of alpha and beta we may be able to prune branches off the search tree which reduces the number of positions that are needed to evaluate. The pseudo-code for such an algorithm is shown below:

```

1. FUNCTION NegaMax(depth, alpha, beta)
2.   IF (depth == 0 OR GameOver) THEN
3.     return (EvaluationOfBoard())
4.   ENDIF
5.   TTValue  $\Leftarrow$  (Probe Transposition Table)
6.   IF (TTValue is not null) THEN
7.     return TTValue
8.   ENDIF
9.   LegalMoves  $\Leftarrow$  generateLegalMoves()
10.  FOREACH Move in LegalMoves
11.    MakeMove(Move)
12.    Score  $\Leftarrow$  -NegaMax(depth-1, -alpha, -beta)
13.    UndoMove(Move)
14.    IF Score >= beta THEN
15.      Return beta
16.    ENDIF
17.    IF Score > alpha THEN
18.      alpha = Score
19.    ENDIF
20.  END FOREACH
21.  return alpha
22. END FUNCTION

```

Make Move

The Make move function is one of the most essential functions within the search engine as if the resulting position is incorrect then the wrong moves will be generated, and the wrong positions will be evaluated. The make move function is a part of the bitboard class and uses all the internal attributes. There is one parameter which is the move that is to be made which for my engine is a

structure. The first thing that is done is the state of the current board is saved into a stack so that when the undo move function is called, the values of the variables can be recovered all of the internal variables are explained in the structures section in the bitboard. After the ply count is incremented and the captured piece is changed to -1 along with the En-passant variable. The bitboards are then altered to reflect the move given and if a capture is made the type of piece captures is saved. After the move flags such as castling, promotion or En-passant captures/setting are delt with. The function then changes the castling rights based on if the king or rooks move. Finally, we change the Zobrist hashing and switch the player turn. The pseudo code for the function is shown below:

```

1. FUNCTION MakeMove(move)
2.   boardStateHistory.Push(zobristKey, castlingRights, plyCount, moveCount,
   capturedPiece, enPassantFile, state, WhiteToPlay)
3.   start ⇐ move.startIndex
4.   destination ⇐ move.destinationIndex
5.   MoveColour ⇐ WhiteToPlay ? 0 : 1
6.   EnemyColour ⇐ 1 - MoveColour
7.
8.   zobristKey.moveKey ⇐ 0
9.   zobristKey.enPassantKey ⇐ 0
10.  zobristKey.castlingKey ⇐ 0
11.
12.  enPassantFile ⇐ -1
13.  capturedPiece ⇐ -1
14.
15.  plyCount ⇐ plyCount+1
16.
17.  IF Capture THEN
18.    capturedPiece ⇐ getPieceType(Board, destinationBitboard)
19.    Board[EnemyColour] ^= destinationBitboard //removes the piece from the
    enemy bitboard
20.    Board[capturedPiece] ^= destinationBitboard //removes the piece from the
    piece bitboard
21.
22.    plyCount ⇐ 0; // piece has been captured so the 50 move rule resets
23.  ENDIF
24.
25.  pieceType ⇐ getPieceType(Board, startBitboard);
26.
27.  IF (pieceType is Pawn) THEN
28.    plyCount ⇐ 0; // pawn has advanced so the 50 move rule resets
29.  ENDIF
30.
31.  Board[MoveColour] ^= destinationBitboard | startBitboard
32.  Board[pieceType] ^= destinationBitboard | startBitboard
33.  zobristKey.pieceKey ⇐ AlterPieceKey (start, pieceType)
34.  zobristKey.pieceKey ⇐ AlterPieceKey(destination, pieceType)
35.
36.  // deal with flags
37.  IF (move.flag ≠ 0) THEN
38.    IF (move.flag is Promotion) THEN
39.      promotionType = move.pieceIndex + 3;
40.      Board[promotionType] ^= destinationBitboard;
41.      Board[2] ^= destinationBitboard;
42.    ENDIF
43.
44.    ELSE IF (move.flag is castling) THEN
45.      MoveKing()
46.      MoveRook()
47.      RemoveKingCastlingRights()

```

```

48.         ENDIF
49.
50.
51.         ELSE IF (move.flag is enPassantSet) THEN
52.             enPassantFile = CalculateFile(destination)
53.         ENDIF
54.
55.         ELSE IF (move.flag is enPassantCapture) THEN
56.             enPassantSquare ⇐ pushPawn(destinationBitboard, EnemyColour)
57.             Board[PAWNS] ^= enPassantSquare
58.             Board[EnemyColour] ^= enPassantSquare
59.             zobristKey.pieceKey ⇐ AlterPieceKey(enPassantSquare, pieceType)
60.         ENDIF
61.     ENDIF
62.
63.     IF (King moves) THEN
64.         removeKingCastlingRights()
65.     ENDIF
66.     ELSE IF (rook moves) THEN
67.         removeRookCastlingRights()
68.     ENDIF
69.
70.     IF (NOT WhiteToPlay) THEN
71.         moveCount ⇐ moveCount+1
72.         zobristKey.moveKey ⇐ Zobrist.MoveHash
73.     ENDIF
74.
75.     Zobrsist.castlingRights ⇐ CalculateCastlingHash(CastlingRights)
76.
77.     IF (enPassantFile ≠ -1) THEN
78.         index ⇐ enPassantFile - 1;
79.         IF (Squares next to pawn are enemy pawns) THEN
80.             zobristKey.enPassantKey ⇐ CalculateEnPassantHash(enPassantFile)
81.         ENIF
82.     ENDIF
83.
84.     WhiteToPlay ⇐ !WhiteToPlay;
85.
86. END FUNCTION

```

Opening book probe

The binary search is one of the most popular and well-known searching algorithms to date. The engine utilises a binary search when probing the polyglot move book as the book is already in order by the Zobrist hash/key values with the lowest key values at the start of the book and the highest at the end. The book does although contain multiple occurrences of the same key as many moves can be played in the same position which means that the start of the occurrences must be found. The binary search algorithm is a divide and conquer algorithm that starts in the middle of the array and compares the value to the one it is looking for, if it is larger than the end of the search scope becomes in the middle of the array and the rest is discarded. The middle of this new search scope is looked at and the process is repeated until the value is found or the desired value is not in the array. After this index is returned. The probe then uses this value and finds the start of the occurrences of the key. If a random move is to be played, then the range of possible book moves is calculated and a random index between the start and end is returned otherwise the move with the highest weight is returned which is often the start move. The pseudo code for the binary search algorithm is as follows:

```

1. FUNCTION binarySearch(left, right, desiredKey)
2.   IF (right >= left) THEN
3.
4.     mid  $\Leftarrow$  left + (right - left) / 2;
5.
6.     IF (entries[mid].key == desiredKey) THEN
7.       return mid;
8.     ENDIF
9.
10.    IF (entries[mid].key > desiredKey) THEN
11.      return binarySearch(left, mid - 1, desiredKey);
12.    ENDIF
13.
14.    ELSE THEN
15.      return binarySearch(mid + 1, right, desiredKey);
16.    END ELSE
17.
18.  END IF
19.  return -1
20. END FUNCTION

```

And the pseudo code for the probing function is:

```

1. FUNCTION probeBook(kingloc, desiredKey)
2.
3.   index  $\Leftarrow$  binarySearch(0, entries.Length, desiredKey);
4.
5.   IF (index is -1) THEN
6.     return EMPTYMOVE // the position did not exist in the opening book
7.   ENDIF
8.
9.   startIndex  $\Leftarrow$  findStartIndex(index, desiredKey);
10.
11.  bestWeight  $\Leftarrow$  -10000
12.
13.  TRY
14.    IF (entries[startIndex].weight is entries[startIndex + 1].weight) // if
    the next weight is the same then there is no "best move" so we find the range and
    pick a random move
15.      range  $\Leftarrow$  findRange(startIndex, desiredKey);
16.      index  $\Leftarrow$  RandomInt(startIndex, startIndex + range) // random index
    within the range
17.    END IF
18.  ELSE
19.    index  $\Leftarrow$  startIndex
20.  END ELSE
21.  ENDTRY
22.  CATCH
23.    index  $\Leftarrow$  startIndex
24.  END CATCH
25.
26.  entry  $\Leftarrow$  entries[index];
27.  fromLoc  $\Leftarrow$  getStartLoc(entry)
28.  toLoc  $\Leftarrow$  getEndLoc(entry)
29.
30.  // castling moves are in a untraditional format so we need to correct this
  (e1h1 replace with e1g1)
31.  IF (fromLoc is kingloc) THEN
32.
33.    IF (king on black startRank) THEN
34.      toLoc  $\Leftarrow$  8 * entry.destinationRow + (8 - entry.destinationFile)
35.    ENDIF
36.
37.    ELSE IF (king on white startRank) THEN

```

```

38.         toLoc ⇐ 8 * entry.destinationRow + (5 - entry.destinationFile)
39.         ENDIF
40.     ENDIF
41.
42.     promotion ⇐ getPromotionPieceType(entry)
43.
44.     return new Move(fromLoc, toLoc, promotion) // move in the format that the
         engine can use
45. END FUNCTION

```

Check attacks – determine pins and find all checks

Data Structures:

Bitboard

I have made a bitboard data structure that represents a position. The class contains a array of eight 64 bit unsigned integers which each represent information about the board; the structure of this array was discussed in the analysis. The least significant bit represents the h1 square and the most significant bit represents the a8 square of a chess board. To visualise this, the starting bitboard for the kings looks like this:

```
00001000 00000000 00000000 00000000 00000000 00000000 00000000 00001000
```

Which is easier to understand why I have chosen this representation when it is viewed in this format:

```

8 00001000
7 00000000
6 00000000
5 00000000
4 00000000
3 00000000
2 00000000
1 00001000
  abcdefgh

```

The class also contains the castling rights which are stored using 4 bits:

0001 - White King can castle kingside

0010 - White King can castle queenside

0100 - Black King can castle kingside

1000 - Black King can castle queenside

The plycount is the amount of half moves played and resets every time a pawn is moved, or a piece is captured. The plycount enforces the fifty-move rule which ends the game and results in a draw.

The variables enPassantFile and capturedPiece both use -1 to indicate that they do not have a value. The enPassantFile stores the file when a pawn has moved to spaces forward which is used within the generator to create En-passant moves. The capturedPiece variable stores which piece was captured on the last move so that when the undoMove function is called the correct piece can be replaced. The Zobrist hash is stored and is a custom data type. The Zobrist hash represents the current position as a single unique 64-bit integer and is incrementally updated during the makeMove function. The board state history is a stack which stores the previous board states and all their internal variables so that when the undoMove function is called we don't need to do any processing to obtain the original values we can just retrieve them.

Stack

The bitboard makes use a stack to hold the previous positions. A stack is a first-in last-out data structure which means that the last item to be added to the structure is the first item to be removed. A stack is used as during searching the only operations that are performed on a bitboard are the makeMove and undoMove function which are always operated with the intention of modifying the position by only one ply and the undoMove function is always called after a make move function. A stack has two main methods the add and pop method; the add method appends an item to the top of the stack and the pop method removes the top item returns it and sets the pointer to the next item in the stack. The stack stores a struct named "BoardState" which stores the information about a certain position which would otherwise be lost or would take a lot of computation to recover. These variables are:

Name	Type	Purpose
zobristKey	ZobrsitKey	Stores the hash of the previous position
CastlingRights	U16	Stores the castling rights of the position. This information could be otherwise unrecoverable depending on the situation.
ply	U16	The ply count
Moves	U16	The current move number
Enpassant_flag	I16	Stores the file that a possible enPassant capture can be made on, this can be recovered but would require computation
whiteToPlay	bool	If the current position is whites turn
capturedPiece	I32	Stores the type of piece which was captured on the move that is being taken back. This information is otherwise unrecoverable.

Move

I have made a structure to represent a move. The structure is made of a 16-bit unsigned integer in the following way:

Flag	End square	Start square
0000	000000	000000

Where the flags are:

0000	No flag
0001	En passant capture

0010 Castling
 0011 En Passant set
 0100 Knight Promotion
 0101 Bishop Promotion
 0110 Rook Promotion
 0111 Queen Promotion

For example, to move a pawn from e2 to e4 the move would be represented as:

0011_011011_001011

My original idea for representing moves was to use the flag to represent a capture and the type of piece captured as if it was stored within the move then it wouldn't have to be stored within the bitboard. Ultimately, I could do this because I forgot to account for the fact that pawns that are able to promote when capturing. Using the original design, I would have to store both the captured piece and the promotion piece which is not possible using the available number of bits for the flag. In the end, I decided to only store a promotion. The struct contains many useful members such as the bitmask for retrieving the move and lambda functions that return information about the move.

Transposition Table

I have made a transposition table class which acts as a hash table which stores the information about previously searched positions. A hash table is a data structure which allows for fast lookup speeds in constant time. It achieves this by using a hashing function which maps the zobrsit-key of the position to a unique output which is the index used to lookup or store the information and is always within the bounds of the array. This significantly reduces the amount of time needed to search as moves that have been previously searched do not need to be searched again and instead can be simply looked up. The table is constructed using an array of TTEEntry classes which is a custom container of 32 bytes and stores the hash of the position, the depth that it was searched at the evaluation of the position and the best following move. TTEEntry is a class and not a struct as a class is nullable whereas a struct is not, we can check if no entry is stored by just checking if it is not null. We store the depth because a higher depth means a more accurate search result. If we are looking at the same position at a higher depth than it is stored in the transposition table, then we need to evaluate the position manually and replace the entry in the hash table with the new value as the old value is less accurate and is not suitable to use. The index hashing function is calculated by $index = zobrsit \bmod numEntries$. A big problem with using a hash table is that there is a possibility of a hash collision which is where two different inputs, which in our case is two different positions, produce the same output/index. There are many different solutions to avoid this problem, one of which is for each index in the hash table to store a list of entries and then use a linear search to find the correct entry. I have instead implemented an "always replace approach" where the new value will always replace the old value since most of the positions stored within the transposition table are useless and not need for searching as they stem from a lower search. As well as this, a lot of the positions are from older searches which are not useful for the current/future searches. The class contains a few methods such as add which stores the values given, a get method which performs a lookup and clear table which sets all the entries to null. For ease of debugging, I have added these variables as well as a function to reset and another to display them.

Variable

Purpose

Candidate Number:

Harry Foster

Centre Number:

successfulAccess	Counts the number of times that the table is accessed, and a value has been found and returned
unsuccessfulAccess	Counts the number of times that the table is accessed where a value has not been found and null has been returned
timesAccessed	Counts the number of times that the table has tried to be accessed
Writes	Counts the number of times that a new value has been added
Rewrites	Counts the number of times that a position has been rewritten at a higher depth
rewriteCollisions	Counts the number of times that a hash collision has occurred and has been rewritten.

UI

UI class purpose table

The UI is a project that uses the WPF library to create a visual display for the user. The project is split into many different classes which serve a unique purpose. The UI mostly contains controllers and code for allowing the user to interact with the screen. One of the most important classes is the bot class which created a new thread which communicates with the engine and passes the output into the game class to be dealt with and displayed to the user whether it be to move a piece or to show analysis information. The UI does however have some backend code which are shown below:

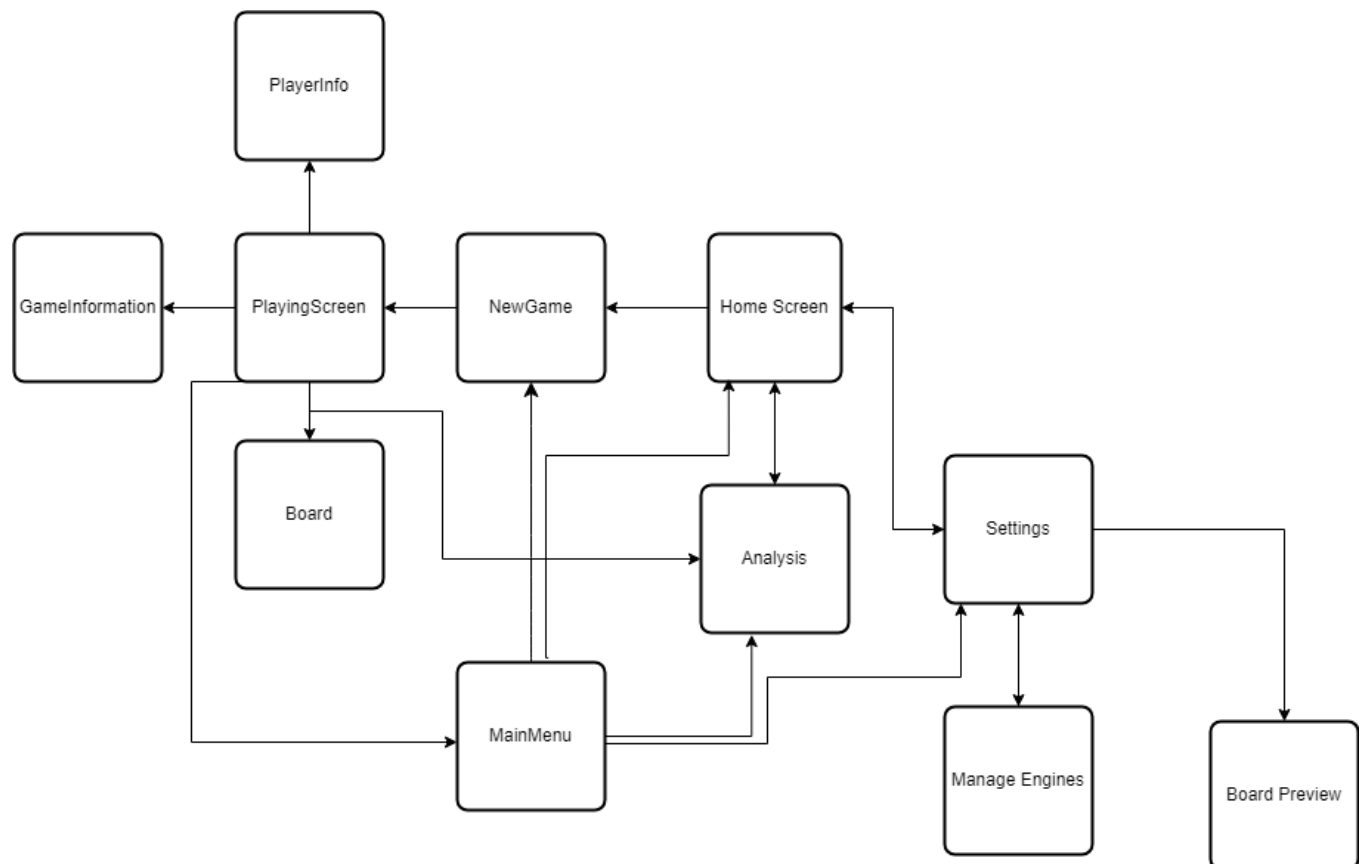
Class	Purpose
Player	This class contains the code that stores the player information for the game and contains methods.
Bot	This class contains the information about the bot such as the name of the bot and the path of the exe file. The class contains methods to communicate with the engine such as sending position and search commands as well as code to parse and receive information to be shown to the user.
AnalysisBot	This class inherits from the bot class and overrides the HandleOutput method as it is only concerned with scraping the UCI info lines and passing them to the analysis
Online player	This class is similar to the player class and the bot class as it has methods which communicates and sends the moves to the other online user as well as to parse the moves sent to it. The online user class also contains information about the user such as the IP address and the port of the recipient as well as the inherited properties of the player class.
User	This is a static class which stores information about the user of the program. It has a UISettings property. The class also contains a list of the available engines that the user can play against, this can be edited within the settings page.
Game	The parent class of CurrentGame and AnalysisGame, acts as a template which stores all the information about the game and contains many functions which control the board and alter the bitboard.
CurrentGame	The child class of Game and contains the functions which deal specifically with the game screen and switching the players and clocks.
AnalysisGame	The other child class of Game which overrides the make move function to communicate with

	the analysis bot. It also overrides the display move function to accommodate the exploration of alternate lines.
UISettings	This is a class which contains all the colours and settings which are to be used throughout the program. This includes the background colour, the text colour, the piece folder, the colours of the squares ect.
UIMove	The UIMove class represents a move that has been made in the current game. The class contains the move in the format which can be used by the engine that can be passed into the bitboard and in a string format which can be used to construct a position command to be sent to the player or the engine.

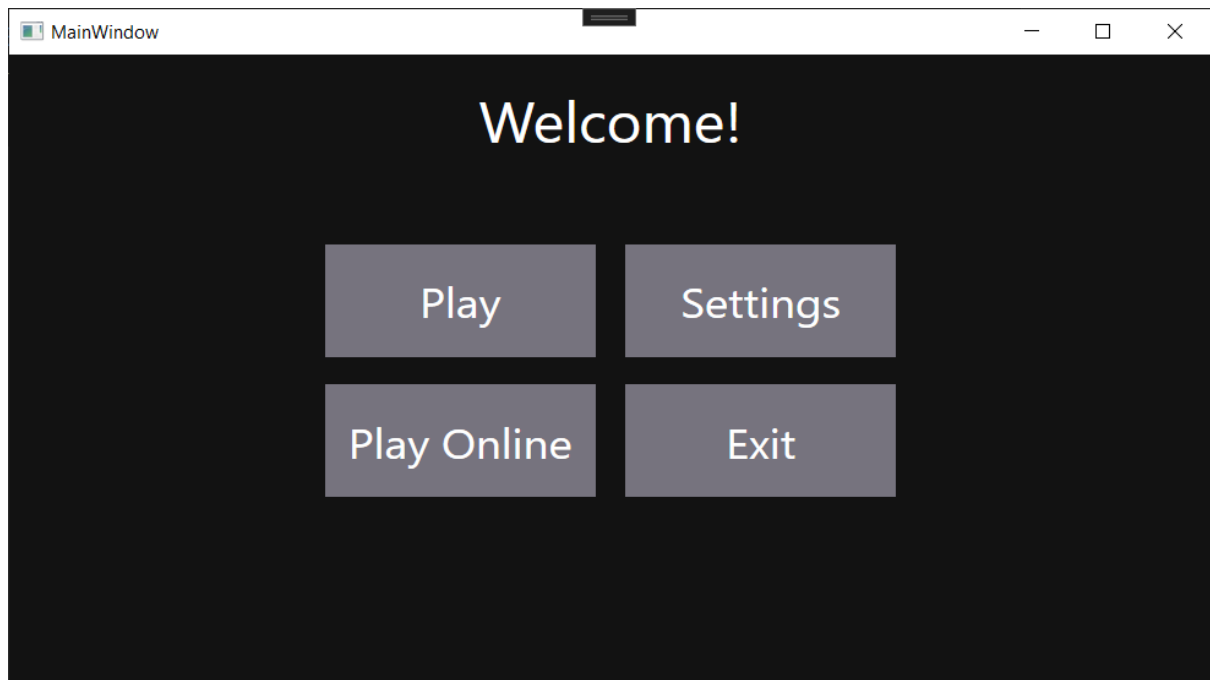
User Interface design

In this section, I will discuss the user interface of my program and explain the reasons behind some elements of it. The UI is a separate project from the engine and interacts with it using the UCI protocol. The UI is made up of one main window which is populated with a “viewbox” that contains a user control. I have decided to use a “viewbox” so that the program will be able to be scaled to fit any size of screen rather than one static size. I am using user controls mainly for simplicity as the way that the page is changed is done by setting the viewbox’s child attribute to a new user control. To design the user interface, I have chosen to use adobe XD as it is a very easy to use software and has useful plugins which allows me to make the design stage as easy and as quick as possible. Each page of the application will have the same standard background colour, button colour, panel input colours and text colours which can all be changed in the settings window so that the UI is as intuitive to the user as possible.

The user flow diagram for the user interface is shown below:

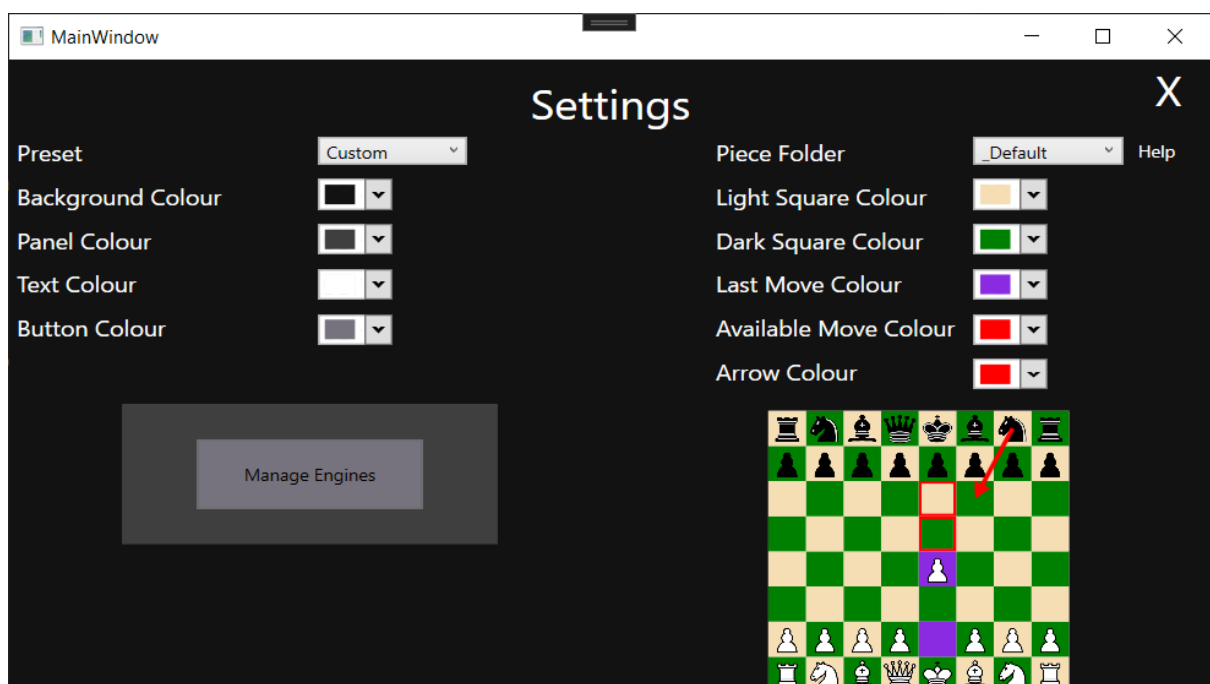


Home Page

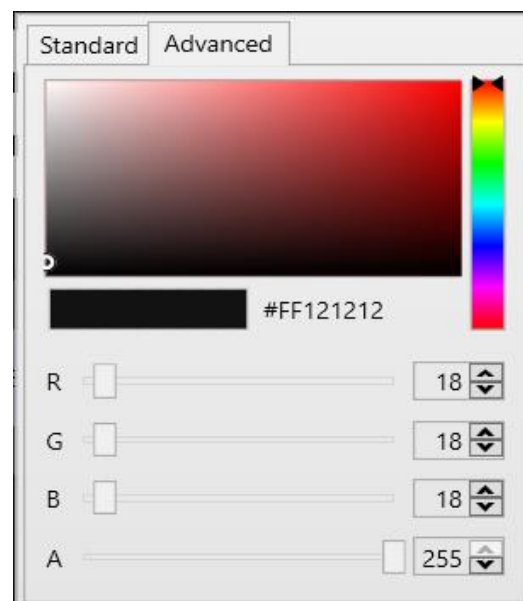


This is the window displayed to the user on start-up. I have chosen to keep the home page minimalistic so that it is intuitive for all users. The main colour theme that I have chosen is a dark theme as it is currently the most popular colour scheme to use and I wanted the UI to be modern looking.

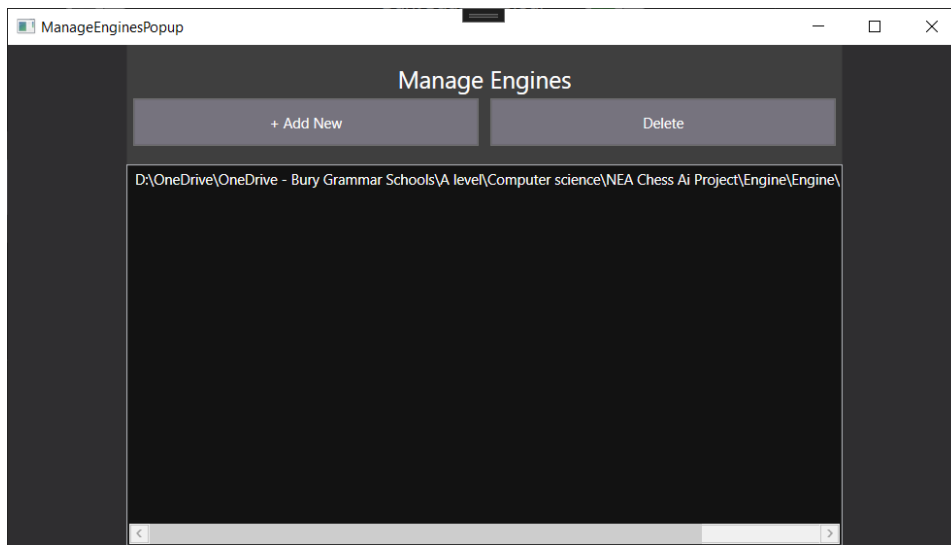
Settings Screen



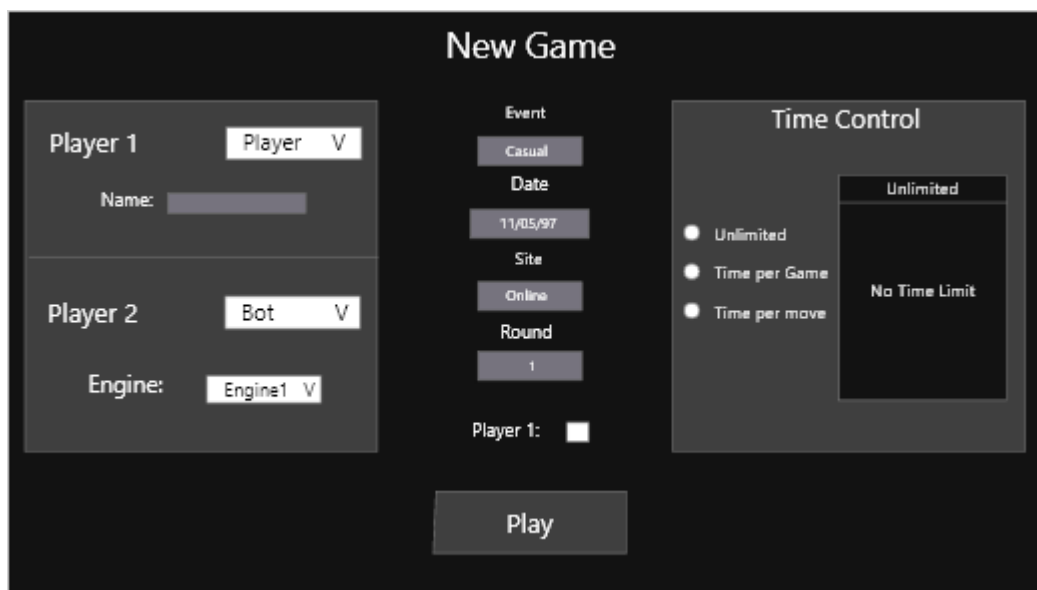
The settings window allows the user to customise the application to their liking. The preset selection window is always on the custom setting when the window starts because the selection boxes have a selection change event which triggers when the window opens, if I set the selection to dark or light mode as the default selection then the code that resets the window to the pre-set will run and will lose all of the user's settings. I have used a combo-box to select the piece folder so that the user can only select from a limited number of options. The options on the left are all associated with the overall colour scheme of the application while the right contains all the controls for the visual appearance of the board which I have done to logically separate the two options to be more intuitive for the user. The piece folder selection window is populated by the different folders in the projects assets folder which allows users to add their own piece designs to be used which adds another layer of personalisation to the application. The "Manage Engines" button is contained within a grey panel as it is the colour that is used for all the panels within the app. The colour selection popup looks like this:



I am using the extended wpf toolkit by "Xceed" to do this. This immediately gives the user a lot more choice to select their own desired colour and allows them to use a hex colour value as well as to select one from the standard colour list.

Manage Engines pop-up

The popup is a very simplistic and offers very little complexity to the user. The add new button opens a file explorer window which is filtered for exe files. The user can click the delete button to remove an engine from the list but cannot delete the standard engine, if they attempted to delete it a message box would appear and would tell the user that they could not remove it.

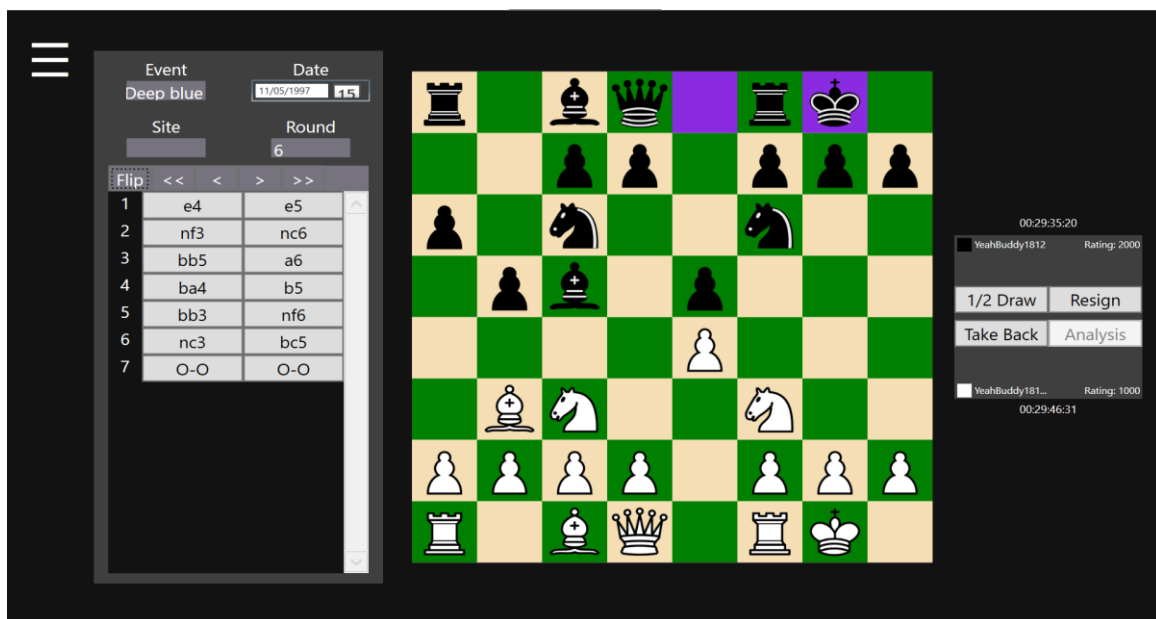
New game select

The new game select gives the user many options to setup the match. For simplicity I have chosen for player 1 to always play white. On the diagram the player option is on the left panel. The user can select if each player is a bot or a human. When selecting a bot, the player must select which engine they want to play against using the drop-down menu. If a player is selected the user can enter their display name. Each of the elements in the middle of the screen are input boxes which stores the information about the game such as the event type which there are two options casual or competitive. The date the game is played, the site which is by default set to online but can be changed to reflect where the game may be played and the round number. The time control panel on the right side allows the user to select the type of time control they want. Unlimited time means

that each player has unlimited time to play each move. When the time per game is selected each player has a set amount of time for the entire game and an increment time can be entered which is the amount of time each player gains per move played. When time per move is selected the player can set the amount of time each move has to be made and resets with each move, if the player fails to make a move in the allotted time, then the game will end. The when the "Time per game" option is selected the following screen is displayed:

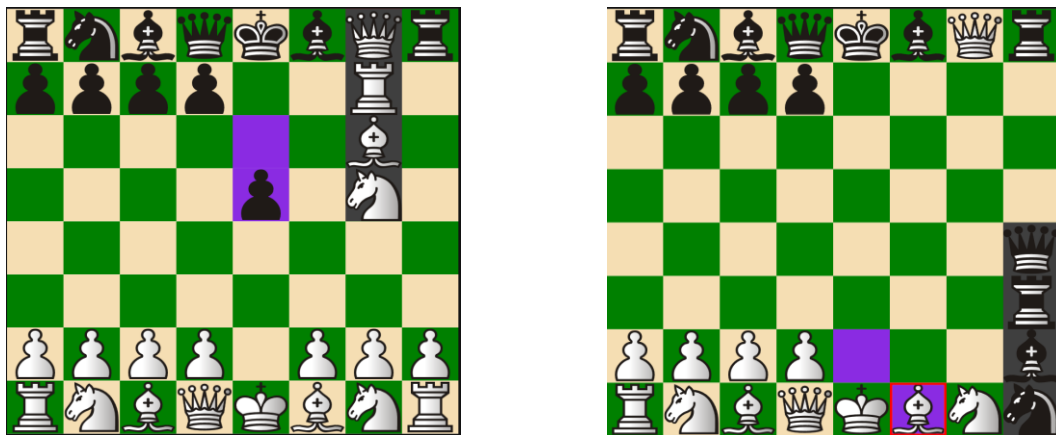
The panel is split into the time and increment areas where the user can select the amount of time for each. These times will be the same for each player. In the time section the player selects the number of hours then minutes then seconds each player has for the entire game. For each of the entries, if the down and up arrows are clicked then the box will be incremented or decremented with the limits of 0 and 99. The user can also type their own desired value and will reject if the user enters a non-numerical value or between 0 and 99.

Playing screen



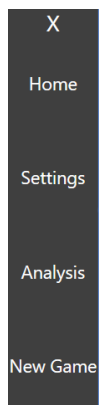
The playing screen is the most complex screen in the program. The screen is composed of many different user controls such as the game information on the left panel which shows the moves played, the event name, the date of the game, the site, and the round number. The user cannot interact with the game information e.g. the date or the event name as this is entered in the new game selection. The user can interact with the rest of the buttons such as the flip button and the rest of the navigation options. If the user clicks on one of the moves played in the scrollable window

the board will change to reflect the position when the move was played, while in this view the user can add comments such as arrow or red squares and view the available moves that a piece can make but cannot make a move. In the middle of the screen is the current board which the user can interact with, they can left click on a piece to show the available moves that it can make, they can click and drag a piece. The user can also right click to turn the square red, or they can hold the right mouse button to create an arrow from which ever square they started on. The piece images are stored in the SVG file format which the standard way of loading this into an image control which wpf can handle is to use the "SvgImageSource" within the media.imaging library. Unfortunately, the target builds versions to use this package is higher than what my program uses, because of this I am instead using the "SvgImage" package by "DotNetProjects" to handle loading and converting the svg file into a format which can be used by wpf. Promotion is one of the key aspects of the game. The user control which is shown when a promotion is to take place looks like the following:

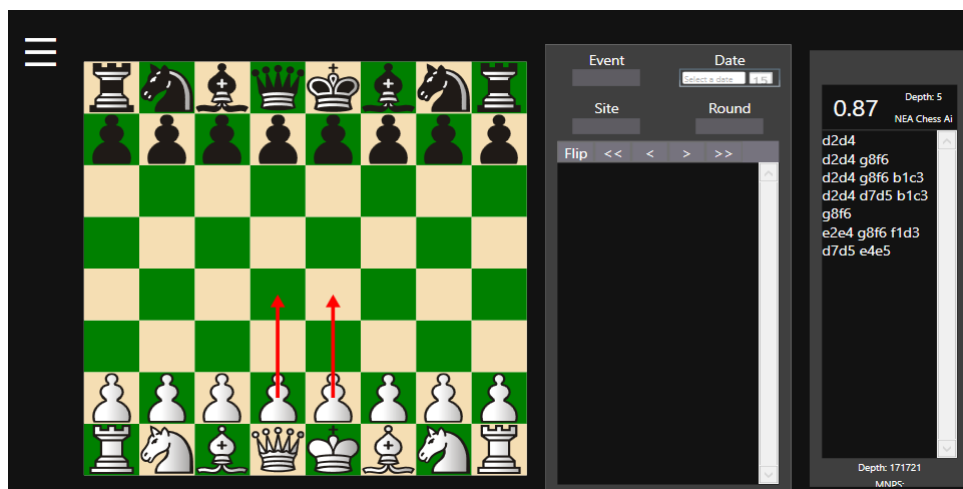


The colour of promotion panel is displayed in the location which the user will promote to. The background colour is the same as the panel colour which is used throughout the program and the pieces are the same as the current piece set and the colour which is to be promoted to. If the promotion is at the bottom of the screen, then the promotion panel is moved upwards as shown in the right picture above. The panel on the right-hand side is the player information view which shows the user their information as well as their opponents. The panel also shows the amount of time each player has left and when move is plays the players clock will stop and the opponent's clock will start counting down. The "Analysis" button is disabled while the game is being played and is enabled when the game is over. If the match is casual, then the "take back" button will be enabled, and a take back request will be sent to the opponent.

When the menu button in the top left-hand corner is pressed the following screen is displayed:



Analysis screen



The analysis screen is very similar in design to the playing screen, it uses the same board and game information panel but instead of the player information panel the engine information is shown. The board and game information panel work the same as the playing screen, but the user can navigate to older positions and make moves and explore alternate lines. The engine is always active whilst on this screen and will continually update the engine information panel with the new information that the engine outputs. The text within the panel is the principal variation lines which are the best continuations from the current position. The moves that the engine outputs also add arrows onto the screen which can be removed which shows the user what the best move is. If in the playing screen after the game has ended the user clicks on the analysis button, this screen will show but with the game information panel being populated with the game that was just played so that the user can navigate through their game and analyse each position to see where they went wrong.

Testing

Test No.	Test Description	Input Data/User Action	Type	Expected Result	Pass / Fail
1 Testing the UI					
1.1 Testing the Home Page					
1.1.1	The user can click on the Play button	The Play button is clicked	Expected	The new game screen should be shown	Passed
1.1.2	The user can click on the Settings button	The Play button is clicked	Expected	The screen should change to display	Passed
1.1.3	The user can click on the Analysis button	The Analysis button is clicked	Expected	The analysis screen should be shown with a blank game	Passed
1.1.4	The user can click on the exit button	The exit button is clicked	Expected	The program will quit.	Passed
1.2 Testing the Settings Page					
	The user can change the pre-set to dark	The user clicks on the pre-set combo-box and selects the dark option	Expected	The colour scheme of the app will revert to the dark pre-set	Passed
1.2.1.2	The user can change the pre-set to light	The user clicks on the pre-set combo-box and selects the light option	Expected	The colour scheme of the app will change to a light colour scheme	Passed
1.2.2.1	The user can change the background colour	The user clicks on the background colour picker and selects a colour	Expected	The background colour of all the screens will change to the selected value	Passed
1.2.2.2	The user can change the panel colour	The user clicks on the panel colour picker and selects a colour	Expected	The background colour of all the panels on all of the screens will change to the selected value	Passed
1.2.2.3	The user can change the text colour	The user clicks on the text colour picker and selects a colour	Expected	The colour of all the text on all the screens will change to the selected value	Passed
1.2.2.4	The user can change the button colour	The user clicks on the button colour picker and selects a colour	Expected	The button colour on all screens will change to the selected value	Passed
1.2.3.1	The user can change the piece folder	The user clicks on the piece folder combo-box and selects an option.	Expected	The piece images on the preview board will change to the selected value	Passed
1.2.3.2	The user can change the light square colour	The user clicks on the light square colour picker and selects a colour	Expected	The background colour of all the light squares will change to the selected value	Passed
1.2.3.3	The user can change the dark square colour	The user clicks on the dark square colour picker and selects a colour	Expected	The background colour of all the dark squares will change to the selected value	Passed

1.2.3.4	The user can change the last move square colour	The user clicks on the last move colour picker and selects a colour	Expected	The background colour of the last move square will change to the selected value	Passed
1.2.3.5	The user can change the available move colour	The user clicks on the available move colour picker and selects a colour	Expected	The border colour of all the available moves will change to the selected value	Passed
1.2.3.6	The user can change the arrow/comment colour	The user clicks on the arrow colour picker and selects a colour	Expected	The colour of all the arrows / coloured squares will change to the selected value	Passed
1.2.3.7	The user should not be able to interact with the preview	The user tries to click or move a piece or interacts with the board in any way.	Erroneous	Nothing should happen if the user clicks a piece on the preview board	Passed
1.2.4.1	The user should be able to click the manage engines button	The user clicks on the "Manage Engines" button	Expected	The manage engines screen should be visible to the user in a popup window.	Passed
1.2.5	The user can click the X button	The user clicks on the "X" button	Expected	The user will be taken back to the home screen	Passed
1.3 Testing the manage engine screen					
1.3.1.1	The user can click the add button	The user clicks on the add button	Expected	A file explorer popup shows, and the files are filtered for only EXEs	Passed
1.3.1.2	The user adds a non-compliant engine	The user selects an EXE which is not an engine or one that does not support UCI	Erroneous	The selected file will not show in the engines list and a popup will display to indicate to the user the exe was not valid	Passed
1.3.1.3	The user adds a valid engine	The user selects an EXE which is a valid engine	Expected	The selected file will show in the list of engines.	Passed
1.3.1.4	The user tries to add an engine that is already added	The user selects an EXE which is a valid engine and is already in the list	Erroneous	The selected file will not show in the list of engines.	Passed
1.3.2.1	The user can click the remove button	The user clicks on the remove button when a item is selected	Expected	The current engine selected will be removed from the list	Passed
1.3.2.2	The user tries to remove the standard engine	The user clicks on the remove button when the standard engine is selected	Erroneous	A popup will display to indicate to the user that the action is not valid, and the engine will not be removed from the list	Passed
1.3.3.1	The user can set another engine to the default	The clicks the "Make default" button when an engine is selected	Expected	The previous default will remove its background colour and the currently selected one will change to green to indicate it is the default engine.	Passed
1.3.3.2	The default engine will switch to the standard engine if the default engine is removed	The user selects the default engine and clicks the remove button	Expected	The selected engine will be removed, and the default engine will now be the standard engine	Passed

1.3.3.3	The user tries to set a default engine when no engine is selected	The user clicks the "make default" button when nothing is selected	Erroneous	A popup will display and indicate to the user that no engine is currently selected	Passed
1.4 Testing the New Game Screen					
1.4.1.1	The user can change the player type to bot	The user clicks on the player combo-box and selects the bot option	Expected	The screen will change to prompt the user choose the desired engine to use	Passed
1.4.1.2	The user can change the player type to player	The user clicks on the player combo-box and selects the player option	Expected	The screen will change to prompt the user to enter the players details	Passed
1.4.2.1	The user can select the game type	The user clicks on the type combo-box and selects the "Casual" or "Competitive" option	Expected	The combo-box will change according to the users selection	Passed
1.4.2.2	The user can change the date	The user clicks on the date picker and selects a date	Expected	The date shown is now the selected date	Passed
1.4.2.3	The user changes the date into the future	The user clicks on the date picker and selects a date greater than today's date	Erroneous	The date picker will reject the input and will revert to the original selection	Passed
1.4.2.4	The user can change the site name	The user enters "Chess.com"	Expected	The input box will change to the valid input	Passed
1.4.2.5	The user can change the round number	The user enters "2"	Expected	The input box will change to the valid input	Passed
1.4.2.6	The user changes the round number to a character	The user enters "a" into the round	Erroneous	The input box will reject the input and revert to the last valid input	Passed
1.4.2.7	The user changes the round number to less than 1	The user enters "-1" or "0"	Erroneous	The input box will reject the input and revert to the last valid input	Passed
1.4.2.8	The user de-focuses the round number when nothing is in the box	The user removes the current value in the round number and then de focuses the box	Erroneous	The input box will change to 1	Passed
1.4.3.1	The user can change the time control to time per game	The user clicks on the radio control for time per game	Expected	The user-control to the right of the radio buttons will change to the time per game control and will remove the old radio button	Passed
1.4.3.2	The user clicks the increment button	The user clicks the arrow button above the input box	Expected	The value will be incremented by one	Passed
1.4.3.3	The user clicks the decrement button	The user clicks the arrow button below the input box	Expected	The value will be decremented by one	Passed
1.4.3.4	The user clicks the decrement button when the time field is on 00	The user clicks the arrow button below the input box	Erroneous	The value will not change and will stay at 00	Passed

1.4.3.4	The user tries to click the increment button when the time field is on 99	The user clicks the arrow button above the input box	Erroneous	The value will not change and will stay at 99	Passed
1.4.3.5	The user enters a valid time in the field	The user clicks on the input field and enters 3	Expected	The value will change to the value that the user enters	Passed
1.4.3.6	The user enters an alpha character in the field	"a"	Erroneous	The input will reject the input and will revert to the last valid input	Passed
1.4.3.7	The user enters a negative number in the field	-1	Erroneous	The input will reject the input and will revert to the last valid input	Passed
1.4.3.8	The user enters the maximum in the field	99	Boundary	The value will change to 99	Passed
1.4.4.1	The user plays the game with unlimited time selected	The user clicks the play game button with the unlimited time radio button selected	Expected	The timer on the playing screen will not show and the user will have an unlimited amount of time to make a move	Passed
1.4.4.2	The user plays with a time control	The user clicks the play game button with a valid amount of time selected	Expected	The timer on the playing screen will show and will be the value that was inputted by the user	Passed
1.4.4.3	The user tries to play a game with no time	The user clicks the play game button when the playing time is 0 for each field	Erroneous	The playing screen will not be shown and an error message will be displayed to the user.	Passed
1.5 Testing the playing screen					
1.5.1.1	The user can click a valid piece	Clicks on the e2 pawn in the starting position	Expected	The valid moves are shown (e2e3 and e2e4)	Passed
1.5.1.2	The user can drag a piece	The user clicks and drags the e2 pawn	Expected	The pawn is removed from the board and follows the mouse	Passed
1.5.1.4	The user clicks on an invalid move	The user clicks on a square which is not highlighted	Erroneous	The piece selected does not move	Passed
1.5.1.5	The user drags the piece to an invalid square	Drags the e2 pawn to an invalid place	Erroneous	The piece is reverted to its original position	Passed
1.5.1.6	The user clicks on a valid move	E2 pawn selected and the e4 square is clicked	Expected	The pawn moves to the e4 square	Passed
1.5.1.7	The user drags a valid move	Drags a piece to a valid move e2 pawn dragged to the e4 square	Expected	The pawn moves to the e4 square	Passed
1.5.1.8	The user tries to make a move when it is not their turn	The user clicks on a black piece when it is whites turn	Erroneous	The moves will not be displayed	Passed
1.5.2.1	The user can flip the board	The user clicks the flip button	Expected	The board will flip around	Passed
1.5.2.2	The user can make a move while the board is flipped	The user clicks on a piece and makes a move while the board is flipped	Expected	The move will be made, and the board will stay in the flipped stage	Passed

1.5.2.3	The user tries to make an invalid move while the board is flipped	The user clicks on the opposite colour pieces in the flipped state	Erroneous	The program should not react, and no moves should be displayed	Passed
1.5.2.4	The user can create an arrow	The user clicks and hold the right mouse button	Expected	The program should create an arrow which starts from the square they clicked on and ends on the one they released on	Passed
1.5.2.5	The user can create an attack comment	The user clicks on the square	Expected	The background colour of the square they clicked	Passed
1.5.2.6	The user can remove an arrow	The user drags the right mouse over an existing arrow	Expected	The arrow should be removed	Passed
1.5.2.7	The user can remove an attack comment	The user right clicks on the square which is already coloured in	Expected	The background colour reverts to what it was before the square was highlighted	Passed
1.5.3.1	The user can play against another player	The user makes a move	Expected	Once a player has made their turn it is then the other players turn	Passed
1.5.3.2	The user plays against an engine	The user makes a move	Expected	Once a move has been made it is the engines turn and it will think and make a move	Passed
1.5.3.3	The engine is playing white and makes the first move	The user selects the engine to play white on the new game screen	Expected	The engine will make a move	Passed
1.5.4.1	When the game type is competitive the "Take Back" button is disabled	The user selects the game type to be "competitive"	Expected	The Take back button is disabled	Passed
1.5.4.2	When the game type is casual the "Take Back" button is enabled	The user selects the game type to be "casual"	Expected	The Take back button is enabled	Passed
1.5.4.3	The user requests a take back when it is not their turn and playing a bot	The user clicks the take back button	Erroneous	The program should not react and will not do anything	Passed
1.5.4.4	The user requests a take back when it is their turn and playing a bot	The user clicks the take back button	Expected	The board will change to when the user last made a move and will remove the moves from the move history	Passed
1.5.4.5	The user requests a take back when it is the first move and playing a bot	The user clicks the take back button	Erroneous	The program should do nothing	Passed
1.5.4.6	The user requests a take back when playing another player	The user clicks the take back button	Expected	The board will go back one move and will remove the last move from the move history	Passed
1.4.4.7	The user requests a take back when it is the first move, and they are playing another player	The user clicks the take back button	Erroneous	The program should do nothing	Passed
1.4.5.1	The move history is updated when the user plays a move	A move is played	Expected	The move in the long algebraic format will be added to the move history	Passed

1.5.5.2	The user clicks on a previous move	The user clicks on a move in the move history	Expected	The board will change to what it was at that position	Passed
1.5.5.3	The user tries to play a move while in a previous position	The user clicks on a move	Erroneous	The move should not be played, and the piece should return to its original position	Passed
1.5.5.4	The user clicks the previous move button	The user clicks the "<" button	Expected	The board will change to what the previous position was	Passed
1.5.5.5	The user clicks the previous move button when they are on the first move	The user clicks the "<" button	Erroneous	Nothing will happen	Passed
1.5.5.6	The user clicks on the next move button	The user clicks the ">" button	Expected	The next move that was played will be shown on the board	Passed
1.5.5.7	The user clicks the next move button when they are on the current position	The user clicks the ">" button	Erroneous	Nothing will happen	Passed
1.5.5.8	The user clicks on the first move button	The user clicks the "<<" button	Boundary ?	The board will change to the position after the first move	Passed
1.5.5.9	The user clicks on the last move button	The user clicks the ">>" button	Boundary ?	The board will change to the position at the end of the game/the current position	Passed
1.5.6.1	The user promotes to a piece	A pawn is moved to the promoting rank	Expected	The promotion screen will be displayed to the user at the square where they are promoting to	Passed
1.5.6.2	The user cancels the promotion	The user clicks another piece or another move	Expected	The promotion screen is removed from the board	Passed
1.5.6.3	A checkmate position is reached	The player reaches a position where it is checkmate	Expected	The game should end, and the checkmate message should show with the player who wins	Passed
1.5.6.4	A stalemate position is reached	The player reaches a position where it is stalemate	Expected	The game should end, and the stalemate message should show	Passed
1.5.6.5	A player runs out of time	The clock runs out of time while on the players turn	Expected	The game should end and the player with time left on the clock should win	Passed
1.5.6.6	resign	The player clicks the resign button	Expected	The game should end, and the resignation message should show with the player who resigns loses	Passed
1.5.6.7	A player clicks on the draw button	The player clicks the draw button	Expected	The game should end, and the draw message should show	Passed
1.5.7.1	The players clocks start counting down when a move is played	The player makes a move	Expected	The players clock will stop counting and the opponents will start	Passed
1.5.7.2	The increment is added when a move is played	The player makes a move	Expected	The opponent's clock will increase by the given increment	Passed
1.5.7.3	The user clicks the analysis button when the game is over	The user clicks on the analysis screen	Expected	The screen will change to the analysis screen and the moves that the user played will be shown in the game information panel	Passed

Candidate Number:

Harry Foster

Centre Number:

1.5.7.4	The user resigns and goes to another screen	The user clicks ok to accept the resignation	Expected	The game will end, and the screen will switch to the desired page	Passed
1.6 Testing the analysis screen					
1.6.1	The position will start to be analysed by the position	N/A	Expected	When the analysis screen is shown the current position selected should start being analysed and the information should start being displayed	Passed
1.6.2	An arrow will be displayed which shows the engines output	N/A	Expected	The best moves and pv lines which the engine outputs should be displayed by adding an arrow to indicate which move should be played	Passed
1.6.3	The engines depth and evaluation will be shown	N/A	Expected	The evaluation and depth that the engine outputs should be displayed to the user on the rightmost panel	Passed
1.6.4	When a move is played the screen will reset	The user makes a move	Expected	After any move has been played the move should show in the game information panel and the engine should start analysing this new position	Passed
1.6.5	The user can revisit a move	The user clicks on a previous move	Expected	Once the user clicks on a previous move the engine should switch to analysing this position and will display the information	Passed
2 Testing the engine					
2.1 Testing position command					
2.1.1.1	Testing the startpos argument	Position startpos	Expected	The current position will revert back to the starting position	Passed
2.1.2.1	Testing the fen argument	Position fen r1k4r/p2nb1p1/2b4p/1p1n1p2/2PP4/3Q1NB1/1P3PPP/R5K1 b - - 0 19	Expected	The current position will change to what the fen string encodes	Passed
2.1.2.2	Testing an incorrect fen	Position fen	Erroneous	An error message will be displayed and the position will not be setup	Passed
2.1.2.3	Testing a partial fen	Position fen	Erroneous	An error message will be displayed and the position will not be setup	Passed
2.1.3.1	Testing move arguments	Position startpos moves e2e4 e7e5	Expected	The position will change to the new position	Passed
2.1.3.2	Entering an incorrect move	Position startpos moves e2e4 b8e5	Erroneous	An error message will be displayed and the position will not change	Passed
2.2 Testing the generation					
2.2.1	Perft test depth 2 at the startpos	Position startpos go perft 2	Expected	Nodes: 400	Passed
2.2.2	Perft test depth 3	Position startpos go perft 3	Expected	Nodes: 8,902	Passed
2.2.3	Perft test at depth 5 using the kiwipete position	position fen r3k2r/p1ppqpb1/bn2pn p1/3PN3/1p2P3/2N2Q1	Expected	Nodes: 193690690	Passed

		p/PPPBPPP/R3K2R w KQkq - - - Go perft depth 5			
2.3 Testing the Zobrist hashing					
2.3.1	Testing Zobrist hash on starting position	startpos	Expected	463b96181691fc9c	Passed
2.3.2	Testing the zobrsit hash with castling rights changed	rnbqkbnr/ppp1pppp/8/3pP3/8/8/PPPP1PPP/RNBQKBNR b KQkq - 0 2	Expected	662fafb965db29d4	Passed
2.3.3	Testing Zobrist hash with an en passant	rnbqkbnr/p1pppppp/8/8/PpP4P/8/1P1PPPP1/RNBQKBNR b KQkq c3 0 3	Expected	3c8123ea7b067637	Passed
2.3.4	Testing zobrsit hashing with an enpasant which isn't added	rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1	Expected	823c9b50fd114196	Passed
2.3.5	Testing Zobrist hashing with white to play	rnbqkbnr/ppp1pppp/8/3p4/4P3/8/PPPP1PPP/RNBQKBNR w KQkq d6 0 2	Expected	0756b94461c50fb0	Passed
2.4 Testing the opening book					
2.4.1	Testing the book searcher	Position startpos go	Expected	The search will return a move from the book	Passed
2.4.2	Testing the show book	Poly	Expected	The weight and the move name will be shown	Passed
2.4.3	Testing an erroneous input command	Not "Poly"	Erroneous	A error message should be shown that tells the user that the command was not recognised	Passed
2.4.4	Testing if a book move was not found	Probe the book in the position	Expected	The book should start a search instead as nothing was found	Passed
2.4.5	Testing if UseBook is false	Setoption name OwnBook value false	Expected	The program should not search the book and will instead perform a search	Passed
2.4.6	Test if the book handles castling moves correctly	Probe book in the position	Expected	The move shown should be e1g1 not e1h1	Passed
2.4.7	The user changes the book location to a valid book	Setoption name BookName value gm2600	Expected	The new book should be read into memory	Passed
2.4.8	The user changes the book location to an invalid book	Setoption name BookName value stockfish	Erroneous	An error should be displayed, and the invalid book should not be read into memory	Passed
2.5 Testing the searcher					
2.5.1	Testing mate in 1 ply	rnbqkbnr/pppp1ppp/3p4/4P2Q/2B5/8/PPPP1PP/P/RNB1K1NR w KQkq - 0 1	Expected	h5f7	Passed

Candidate Number:

Harry Foster

Centre Number:

2.5.2	Testing mate in 2 plys	KR1R4/2P5/4B1PP/q1b1 BP2/N3P3/pQ2p1p1/1p r1b2p/1k5r w - - 1 2	Expected	a8b7	Passed
2.5.3	Testing mate in 3 plys	K7/P3q2P/Q1P5/P4n2/5 P2/2p5/2k2Bpp/8 b - - 3 3	Expected	e7d8	Passed
2.5.4	Testing mate in 4 plys	8/8/8/8/1BB5/NN6/krQ 5/7K w - - 0 1 8/8/8/N3B3/8/6NP/2kP 1P2/R3K2R w KQ - 0 1	Expected	c2h7 d2d3	Passed

Evaluation

Objective Analysis

- g. The program must be able to play a full game of chess without any illegal moves. The program must:
 - a. Calculate all of the legal moves given any position (**achieved**) This was achieved and can be seen in the perft section of testing, for any position it will return a list of all the legal moves.
 - b. Be able to recognise the state of the game (**achieved**) The game state is changed during the generation
 - i. if the position is checkmate, stalemate or a draw (**achieved**) This is done when the moves are generated
 - c. Be able to handle special moves:
All special moves are generated and handled correctly
 - i. En-Passant (**achieved**)
 - ii. Castling (**achieved**)
 - iii. Promotion (**achieved**)
- h. There must be a user interface which must:
 - a. Support 2 players (**achieved**) The user can play in 2 player mode if both of the player types are players when selected in the new game popup
 - b. Store the move history in standard chess notation (**semi-achieved**) This is achieved as the move is converted from uci notation to long algebraic notation but there are some key elements that are missing such as a + denoting check and # denoting checkmate
 - c. Allow the user to interact with the chess board (**achieved**) This is achieved and is shown the playing screen tests
 - i. Allow the user to click on a piece and show all of the legal moves for a selected piece (**achieved**)
 - ii. The user should be allowed to drag and drop the piece using the cursor (**achieved**)
 - iii. The legal moves will be indicated by a red border shown around the squares (**achieved**)
 - iv. If the user tries to make an illegal move the program will not allow it and will reset the piece back to its original position (**achieved**)
 - d. Allow the user to resign (**achieved**) This is when the user clicks on the resign button in the playing screen and ends the game and also shows that the player lost
 - e. Allow the user to draw if playing against a player (**achieved**) This is when the user clicks on the draw button in the playing screen and ends the game and shows that the game is a draw
 - f. Allow the user to request a takeback (**achieved**) The user can click the takeback button when the game is casual and the last move played will be removed from the history and the board will go to the previous position.
 - g. Be able to communicate with the engine using the uci standard (**achieved**) This is not something the user can see but the UI opens a new process and sends uci commands to the engine and then parses the output for the user to see eg when a bestmove is returned
 - h. Be able to switch to analysis mode which should:

- i. Allow the user to input a custom board (FEN String) (**not achieved**) This was something that I planned to do but unfortunately ran out of time. I wanted to make a similar screen to that of the lichess board editor where a user can drag and drop pieces onto the board and setup a unique position which they can then analyse
- ii. Show the output of the engine:
 - 1. The moves looked during searching (PV) in uci algebraic notation (**achieved**) This is done when the user goes onto the analysis screen and the rightmost panel populates with the PV and the engines other outputs
 - 2. Display the number of moves generated at a given depth (**achieved**) Again this is shown to the user in the rightmost panel at the bottom where there is a counter that shows the number of nodes searched
- i. When creating a new game there must be a window which:
 - i. Allows the user to choose an engine or player (**achieved**) In the playing screen there is a combo box for each player where the user can select either a player or a bot.
 - ii. Allows the user to select a time control: On the right panel there is a radio control where you can select unlimited or time per game
 - 1. Unlimited (**achieved**)
 - 2. Time per game:
 - a. Input an amount of time (**achieved**) When time per game is selected a new panel is shown to the right of the radio control which has two sections, the top of this section is the playing time where the user can input the amount of time they want
 - b. Input an increment time (**achieved**) The second section is the amount of increment the user can have and this can be a non zero field
- iii. Allow the user to input the: In the middle of the new game select screen the user can set the details of the game
 - 1. Site (**achieved**) This is the third field and allows the user to input the site/place the game is being held, this field is set to "Online" but can be changed to anything and can be left blank
 - 2. Round number (**achieved**) This is the fourth field and it only accepts numeric characters and it must be above 0
 - 3. Date (**achieved**) This is the second field and allows the user to input a date using the date picker widget. The user can select a date less than or equal to the current date
 - 4. Event type – Casual or competitive (**achieved**) This is done using a combo box at the top of the screen and allows users to select either casual or competitive
- j. A setting screen which must:
 - i. Change the UI colours: the colour UI settings are in the left hand panel of the settings screen and all of them can be changed using the colour picker from the wpf extended toolbox
 - 1. The background (**achieved**)
 - 2. The buttons (**achieved**)

3. The panels (achieved)
 4. The text (achieved)
 5. The Light square (achieved)
 6. The dark square (achieved)
 7. The available moves (achieved)
 8. The arrows (achieved)
 - ii. Allow the user to restore dark or light colour scheme pre-set (achieved) This is the combo box at the top of the settings page and is set to custom whenever a setting is changed. If the selected item is the preset then all of the settings are changed to that preset
 - iii. Change the pieces that are displayed (achieved) This is the combo box at the top of the right hand side of the settings screen and is populated with the contents of the pieces folder which have a valid piece in. When the user selects a different piece folder the preview board is changed
 - iv. Allow the user to manage engines: This is done by a manage engines screen which is accessed by clicking the manage engines button in the settings menu
 1. Import new engines (achieved) On the manage engines screen the user can add an engine by clicking the add button. A file explorer window will open with a filter for exes. Once the user clicks on one the program will verify that it is an engine
 2. Delete engines (achieved) On the manage engines screen the user can delete an engine by clicking the delete button when an engine is selected from the list. The engine will then be removed from the list
 - v. Show a preview of the board (achieved) This is the last element in the settings screen at the bottom right hand side. It is updated in real time when the piece folder or any colour associated with the board is changed eg the light square colour, the previous move colour.
- i. Search
- a. Minimax algorithm with alpha beta pruning (achieved) This is done using the negamax algorithm instead of the minimax which completes the same function but uses less lines of code to do so.
 - b. The search tree should be able to search to a depth of at least 5 plies in a reasonable amount of time (achieved) This is a very ambiguous objective because a reasonable amount of time is subjective. The algorithm normally takes around 30 seconds to finish 5 plies but of course in more complex positions it may take longer since there are move to search. Overall this objective has been achieved.
 - c. Use of Transposition Tables to prevent unnecessary evaluation which must: The searching algorithm uses the transposition table to lookup the current position each time the negamax is called
 - i. Use Zobrist hashing (achieved) Zobrist hashing is used not only in the opening book but is also used to calculate the transposition table hash to store the entry
 - d. Sort the moves so that the most likely good moves are searched first (semi-achieved) This is semi achieved as the move order does use most vulnerable victim and least vulnerable attacker to sort the capture moves from best to worst. After researching

this in more depth I had planned to add killer move tables and move history tables to make the sorting more efficient but I did not have enough time

- j. The evaluation algorithm must consider:
 - a. The game stage (Opening, middlegame, endgame):
 - i. It must calculate which stage the game is currently in (**achieved**) This is done in the evaluation function and uses it to either interpolate or use exact opening or endgame piece square values
 - b. The position of the pieces
 - i. Should use piece square tables (**achieved**) This is done and uses the piece square class to lookup values from the tables
- k. Generation should:
 - a. Use Bitboards to store the position (**achieved**) A bitboard class serves as the main medium for the entire project.
 - b. Make use of Magic bitboards to store rook and bishop attack tables (**achieved**) This is seen in the static magic class where on startup both rooks and bishop tables are created and populated
 - c. Store Pre-calculated attack tables for:

This is seen in the bitboard tools where on startup all of the aforementioned tables are created

 - i. Knights (**achieved**)
 - ii. Kings (**achieved**)
 - iii. Pawns (**achieved**)
 - d. Generate all of the square that the opponent is attacking (**achieved**) In the generation class all of the squares that the opponent attacks is calculated at the start of the generation process
 - e. Calculate check and double check (**achieved**) The program does calculate check and double check in the aptly named "CalculateCheckAttacks" function
 - f. Generate check rays and pin rays (**achieved**) The algorithm generates all of the rays and pinrays and stores them in the generators fields
 - g. Pass all performance tests and should be bug free (**achieved**) The engine does pass all performance test and appears to be bug-free but there is no absolute guarantee that it will work for any position
- l. Opening Book should:
 - a. Be able to open and read any valid polyglot opening book (**achieved**) This was achieved but the program does load the entire book into main memory which for large books is unfeasible
 - b. Use a binary search to query the data (**achieved**) A binary search is used to locate one instance of the opening book then a linear search is used to find the first instance.

User Feedback

I collected user feedback by use of an online form which asked people questions about the program. Below are the questions included in the form as well as the average score (where applicable) and

sample answers. The main feedback I collected was from pupils who used the system as well as from members in my local chess club.

1. How intuitive was the program? *Average score: 4.1*
2. Rate the engines difficulty and overall skill. *Average score: 4.33*
3. Are there any comments about the engine's playstyle.
Responses:
"The opening is generally quite good, and it can easily find winning captures"
"It plays quite well yet sometimes the moves can be quite sporadic"
"It can either be extremely good at finding checkmate in endgames or it just shuffles about"
"It took very little time and played very well but it played the same openings"
4. Did you find the analysis tool useful?
Responses:
"I found it very useful to see which moves I should have played"
"When using the default engine, it sometimes suggests very strange moves"
5. Are there any comments about the user experience?
Responses:
"I liked the customisation features and the ability to revert to the original colour scheme, but I thought that the light colour scheme looked off"
"Its very easy to use and looks similar to Chess.com and Lichess so it feels very familiar"
6. Can you suggest any improvements that could be made?
Responses:
"I would have liked for local multiplayer so I could play with a friend"
"The only improvement I would like to see would be to save your settings"
"I think that it could be interesting if there was some kind of game statistic for each user so you could see your win rate"
"Other game modes could add more excitement"

The feedback collected was quite limited which means that there may be skew in the data, yet I tried to ask a diverse sample group. The main people that responded and used the system were pupils in the school chess club, these were the youngest and most computer literate but was not as skilled in the game. The other smaller subset of people was those who were from my local chess club and have been playing the game for decades. After reading all the responses most of them agreed that the program was relatively simplistic to use and had a familiar feel to common chess sites such as chess.com and lichess. Some of the older users commented that the add engines was complicated and not easy to use as they did not know how to download alternative engines. This could be resolved by having common chess engines pre-downloaded for the user to use without having them to add the engine themselves. As for the playing screen all the users found it quite intuitive to use. Most of the younger users who used other chess sites knew to use the right mouse button to add arrows and highlight squares yet for most of the older users did know this and did not use this feature. To aid user understanding a small message could be displayed to the user to inform them of this feature. Most of the responses showed that the time control settings were a bit confusing since the time had hours minutes and seconds whereas the increment had hours minutes seconds and milliseconds which users did not understand what each was for. To rectify this, the initial contents of the box could be changed from 00 to the time span they represent eg, Hh, Mm, or Ss to aid understanding and useability.

In regard to the playstyle and difficulty of the engine there were some mixed responses. Of the more beginner users, they found the engine to be quite challenging, yet some of the more amateurish they found some of the bots playing to be difficult yet manageable. The more professional users found the engine to be quite easy to beat even on the highest setting, yet they did commend the engines opening which was the result of the opening book. Most of the users who played the engine numerous times did comment on the lack of variety of openings. This because the default book that it uses is one which is weighted which means that each book move has a value associated with it thus it picks the highest one; if the book were to be changed to an unweighted one the opening would be randomised. The most mentioned this about the playstyle of the engine was the lack of three-fold repetition and that the engine kept playing the same move many times. One method to easily fix this is when searching the position, a hash table of the current line looked at would store if the current move has already been played two previous times, then the search iteration would return 0 as a draw. I am not entirely sure if this idea is viable since there may be hash collision problems.

How the system could be improved

There are many ways that the system could be improved, one of which would be to replace the evaluation function with a machine learning algorithm. Hand crafted evaluation algorithms have been out of favor for a long time. Implementing a machine learning approach to evaluating a board position allows for a more sensitive result which would consider more factors given the training data. Speed is one of the most important factors for chess engine as the more nodes searched per second means a better and more accurate result which means a better game played; by rewriting the code in a faster language, such as C, the execution performance can be improved.

Another feature that could be implemented is to add compatibility with another standard communication protocol for the engine such as xboard. This would allow the engine to be used on many more platforms and would give users more options as to how they can use the engine. This could be achieved in many ways; my thought was that I could create a superclass of the "CommandHandler" which then subclasses which are designed to support one of the communication protocols would inherit from the base class. In the generation class the method which generates the pawn moves is very inefficient as it loops over every pawn. If I were to revisit this project, I would improve this by using bitmasks to filter out the pawns that would not be able to move in that specific position this would cut down the amount of processing per position as less pawns would need to be generated for.

A very important part of the searching algorithm is the move order. In my program the only thing that the move order dealt with was captures which it sorted them using MVVLVA (Most vulnerable victim, least vulnerable attacker). The move ordering could greatly be improved by using killer move tables, history tables as well as PV tables which helps the searcher to increase the amount of beta cutoffs. With this addition I could also implement late move reduction which is where moves later in the list which are likely to be worse are searched to a lower depth which reduces the amount of nodes searched thus making the searcher a lot more efficient.

I did explore the possibility of adding an online feature where players could play on the lan network. This is evident by the online player class and the server project. This idea was scrapped when I found out that my school's local network blocks UDP broadcasting from students' computers. It would be quite challenging to implement since there is quite a lot for me to learn about sockets and the best practices and how I could link it with the current system. If I were to continue this endeavor, I would

first get the communication to work then I would create another screen which would allow the user to find, connect and host a game. After they have picked one, I would then try and use TCP/IP and communicate with the UI similar to how it is already done with the engine.

One user suggested that I add a system to store the user's settings. This was again another idea which I thought I might have added yet decided not to due to time issues. There are two approaches to how this could have been implemented, one taking the database approach and one being the local approach. If I had enough time, I think I would have opted for the database approach which would have contained data such as: id, username, password email address, ELO rating, piece settings, colours etc. When initially thinking about this feature I did foresee a few issues such as setting portability since piece folders and engines are added manually by the user, storing them in a database may lead to issues since the user may log into a device where these custom settings are not available. The ELO rating system is synonymous with chess and with information being stored about a user it wouldn't be difficult to add an ELO system that would change based on how you perform against other players. If I were to add this, I would take inspiration from lichess and would only allow the ELO rating to be changed on matches that are labeled as competitive.

A suggestion that caught my eye was one to add other game modes such as antichess or blind chess. Depending on which other game mode I would choose to implement it could be a herculean or trivial task. For example, blind chess would be a relatively simple game mode since for each player they cannot see any of the pieces so to implement this, the piece folder can be changed to one that is blank so that no piece is displayed, and it requires little extra code. Whereas a game mode which changes the rules or adds a new piece or functionality is much harder since the existing code is not compatible so a new class will have to be created to accommodate the new game mode. As well as this an engine would not be available to play against.

Reflection of the process

Overall, I have thoroughly enjoyed most of this project, it has helped me immensely in improving my programming and research skills. I started with the development of the engine and as soon as I got a very crude working version, I started the UI. This was about the time where I created the perf testing framework where for the next few weeks I became hyper-focused on making small improvements to get the nodes per second counter as high as possible. In doing so I lost valuable time that could be used elsewhere yet I did learn about the tools available to measuring performance. The two main changes that made were using a static generator in the perf class instead of creating a new one every time in the function which added one hundred thousand more nodes per second. The second was changing the bitboard to index function so that it used a lookup table since there are only 64 possible indexes, this was by far the most impactful change I made and resulted in adding five hundred thousand more nodes per second. The UI was one of the most difficult parts of the project; I started by using Adobe XD to plan out what I wanted the UI to look like and created it using XAML. The code behind the UI is by far the most complex programs I have written, and I regret not planning it out in much more detail since the classes and how they interact with each other was a mess. If I were to rewrite the project, I would lay out the code in a much better way so that it is easier to maintain. As with any project, debugging has been one of if not the most gruelling task. There are many people who have aided me with this yet the most instrumental are Arin Patel, William Fairclough and Nicole Makin who both never fail to find the smallest and

most unpredictable of bugs that I then proceed to spend hours pulling my hair out to fix. The two most troublesome aspects of the entire project was debugging the BoardViewModel and the Generator. Since I had the support of the perf testing framework the Generator did not take as long as the BoardViewModel to debug but I still spent many days over the Christmas break dedicated to eradicating it of bugs. A big regret I have was not commenting the code in detail as I went along since when I went back to older parts of the code which I had not seen in a few months I forgot how most of the functions worked which the code comments did give me a brief idea but not a good understanding; I have truly realised the importance of clean readable code with useful code comments.

Technical solution

Engine

Program

```
1. using Engine.Search;
2. using System;
3. using System.Collections.Generic;
4.
5. namespace Engine
6. {
7.     internal class Program
8.     {
9.         static void Main(string[] args)
10.        {
11.            string command = "";
12.            CommandHandler handler = new CommandHandler();
13.
14.            do
15.            {
16.                command = Console.ReadLine();
17.                handler.ProcessCommand(command);
18.
19.            } while (command != "exit");
20.        }
21.    }
22. }
```

Command Handler

```
1. using Engine.Evaluation;
2. using Engine.PolyGlut;
3. using Engine.Search;
4. using Newtonsoft.Json;
5. using System;
6. using System.Collections.Generic;
7. using System.Diagnostics;
8. using System.IO;
9. using System.Net.Http.Headers;
10. using System.Threading;
11. using System.Threading.Tasks;
12.
13. namespace Engine
14. {
15.     internal class CommandHandler
```

```

16.     {
17.
18.         Bitboard b = new Bitboard(); // the current positon
19.         Searcher searcher = new Searcher(); //
20.         Generation generator = new Generation();
21.         Perf ft = new Perf ft();
22.         string loc = Directory.GetParent(Directory.GetCurrentDirectory()).FullName.Split(new
string[] { "UI", "Engine" }, StringSplitOptions.None)[0];
23.         bool UseBook = true; // bool to indicate whether or not the engine can use the opening
book
24.         Thread actionThread; // the current proccess which is being executed
25.         public static bool Debug = false; // if debug information should be shown, used
throughout the program
26.
27.         /// <summary>
28.         /// Handles the UCI commands
29.         /// </summary>
30.         public CommandHandler() { }
31.
32.         /// <summary>
33.         /// Gets the input from the console and passes it to the corresponding parser
34.         /// </summary>
35.         /// <exception cref="Exception">Throws an error if the command is invalid</exception>
36.         public void ProcessCommand(string command)
37.         {
38.             command = command.Trim();
39.             try
40.             {
41.                 if (command == "uci")
42.                 {
43.                     Console.WriteLine("id name NEA Chess Engine");
44.                     Console.WriteLine("id author Harry Foster");
45.                     Console.WriteLine("option name OwnBook type check default true");
46.                     Console.WriteLine("option name BookName type string");
47.                     Console.WriteLine("uciok");
48.                 }
49.
50.
51.                 else if (command == "ucinewgame")
52.                 {
53.                     b = new Bitboard();
54.                     searcher = new Searcher();
55.                     generator = new Generation();
56.                     perft = new Perf ft();
57.                 }
58.
59.                 else if (command == "isready")
60.                 {
61.                     Console.WriteLine("readyok");
62.                 }
63.
64.                 else if (command == "stop")
65.                 {
66.                     stopThreadProcess();
67.                 }
68.
69.                 else if (command.Contains("position"))
70.                 {
71.                     HandlePositionCommand(command);
72.                 }
73.
74.                 else if (command.Contains("setoption"))
75.                 {
76.                     HandleSetOptionCommand(command);
77.                 }
78.

```

```

79.         else if (command.Contains("go"))
80.         {
81.             HandleGoCommand(command);
82.         }
83.
84.         else if (command == "d")
85.         {
86.             Tools.printPieceBoard(b);
87.         }
88.
89.         else if (command == "help")
90.         {
91.             ListCommands();
92.         }
93.
94.         else if (command == "poly")
95.         {
96.             HandlePolyCommand(command);
97.         }
98.
99.         else if (command == "quit") {
100.             Environment.Exit(0);
101.         }
102.
103.         else if (command == "static")
104.         {
105.             Evaluator evaluator= new Evaluator();
106.             Console.WriteLine(evaluator.Evaluate(b));
107.         }
108.
109.         else if (command == "")
110.         {
111.         }
112.
113.         else { throw new Exception("type help for a list of available commands");
114.     }
115.
116.     catch (Exception e)
117.     {
118.         Console.WriteLine("Could not recognise command: " + command);
119.         Console.WriteLine(e.Message);
120.
121.
122.         if (Debug)
123.         {
124.             Console.WriteLine(e.InnerException);
125.             Console.WriteLine(e.StackTrace);
126.             Console.WriteLine(e.Source);
127.         }
128.     }
129. }
130.
131.     /// <summary>
132.     /// Parse the position command and will execute the specified instruction
133.     /// </summary>
134.     /// <exception cref="Exception">Throws an error if the command is
invalid</exception>
135.     public void HandlePositionCommand(string command)
136.     {
137.         Bitboard oldpos = this.b;
138.
139.         // check if the command has enough arguments
140.         if (command.Split(' ').Length >= 2)
141.         {
142.             // if the second argument is starpos then create a new bitboard

```

```

143.         if (command.Contains("startpos"))
144.         {
145.             b = new Bitboard();
146.         }
147.
148.         else if (command.Contains("fen"))
149.         {
150.             // extracts everything after "fen"
151.             string fenstring = command.Split(new string[] { "fen " },
StringSplitOptions.None)[1];
152.
153.             string[] fenArray = fenstring.Split(' ');
154.             fenstring = "";
155.             // create the fen string
156.             for (int i = 0; i < 6; i++) // 6 as there are 6 parts in a fen
string: {board} {w/b} {Castling} {enpassant} {plys} {moves}
157.             {
158.                 fenstring += fenArray[i] + ' ';
159.             }
160.
161.             b = Tools.FENtoBitboard(fenstring);
162.         }
163.
164.         // check if the command makes any moves
165.         if (command.Contains("moves"))
166.         {
167.             // extract everything after "moves "
168.             string MovesString = command.Split(new string[] { "moves " },
StringSplitOptions.None)[1];
169.             // converts it into an array of moves
170.             string[] moves = MovesString.Split(' ');
171.             foreach (string move in moves)
172.             {
173.                 // try and convert the move from a string
174.                 if (stringToMove(move, out Move target))
175.                 {
176.                     b.MakeMove(target);
177.                 }
178.
179.                 // the conversion failed and a move was not found therefore it
must be invalid
180.                 else {
181.                     this.b = oldpos;
182.                     throw new Exception($"Invalid Move: {move}");
183.                 }
184.             }
185.         }
186.
187.     }
188.
189.     else
190.     {
191.         throw new Exception("Incorrect position command expected starpos or
fen");
192.     }
193. }
194.
195.     /// <summary>
196.     /// Parse the setOption command and will execute the specified instruction
197.     /// </summary>
198.     /// <exception cref="Exception">Throws an error if the command is
invalid</exception>
199.     public void HandleSetOptionCommand(string command)
200.     {
201.         string[] param = command.Split(' ');
202.

```

```

203.         if (param[2] == "BookName")
204.         {
205.             // creates a new book using the new book name
206.             Polyglot.BookPath = param[4];
207.         }
208.
209.         else if (param[2] == "OwnBook")
210.         {
211.             // converts the input to a bool
212.             UseBook = bool.Parse(param[4]);
213.         }
214.     }
215.
216.     /// <summary>
217.     /// Displays the books moves for the current position
218.     /// </summary>
219.     public void HandlePolyCommand(string command)
220.     {
221.         // show the book moves for the current position
222.         PolyGlott.Polyglot.showBookMoves(b.zobristKey.Key);
223.     }
224.
225.     /// <summary>
226.     /// Parse the go command and will execute the specified instruction
227.     /// </summary>
228.     /// <exception cref="Exception">Throws an error if the go command is
invalid</exception>
229.     public void HandleGoCommand(string command)
230.     {
231.         int depth;
232.         // check if it is a performance test command
233.         if (command.Contains("perft"))
234.         {
235.             // the command should contain at least 3 arguments
236.             if (command.Split(' ').Length < 3)
237.             {
238.                 throw new Exception("Invalid perft command expected format: go
<perft> <int>");
239.             }
240.
241.             // do a capture performance test
242.             if (command.Contains("captures"))
243.             {
244.                 // the depth of search should be the third argument
245.                 depth = int.Parse(command.Split(' ')[2]);
246.                 actionThread = new Thread (() => perft.PerftTestCaptures(b, depth));
247.                 actionThread.Start();
248.                 return;
249.             }
250.
251.             // do a bulk performance test
252.             else if (command.Contains("bulk"))
253.             {
254.                 // load the json data from the file
255.                 string bulkFileLoc = Path.Combine(loc, "PerftTestPositions.txt");
256.                 string json = File.ReadAllText(bulkFileLoc);
257.                 // serialise the json into an array of PerftTest classes
258.                 PerftTest[] perftTests =
                JsonConvert.DeserializeObject<PerftTest[]>(json);
259.                 actionThread = new Thread (() => perft.PerftTestBulkTest(b,
perftTests));
260.                 actionThread.Start();
261.                 return;
262.             }
263.
264.             // does a performance test and compares the output with the text file

```

```

265.         else if (command.Contains("results"))
266.         {
267.             depth = int.Parse(command.Split(' ')[2]);
268.             // loads the expected outputs
269.             string resultsFileLoc = Path.Combine(loc, "PerftResults.txt");
270.
271.             string[] results = File.ReadAllLines(resultsFileLoc);
272.             // performs the test
273.             actionThread = new Thread(() => perft.PerftTestDebug(b, depth,
                results));
274.             actionThread.Start();
275.             return;
276.         }
277.
278.         // regular perft test
279.         depth = int.Parse(command.Split(' ')[2]);
280.         actionThread = new Thread(() => perft.PerftTest(b, depth));
281.         actionThread.Start();
282.     }
283.     else
284.     {
285.
286.         if (command.Split(' ').Length < 2 || command.Contains("infinite")) // it
            is only "go" so an infinite search will start
287.         {
288.             // probe the opening book
289.             if (tryFindBook(out Move foundBookMove))
290.             {
291.                 Console.WriteLine($"bestmove
                {Tools.MoveToString(foundBookMove)}");
292.                 return;
293.             }
294.
295.             // start a infinite search
296.             actionThread = new Thread(() => searcher.Search(b));
297.             actionThread.Start();
298.         }
299.
300.         if (command.Contains("time"))
301.         { // the command contains wtime <int> and btime <int>
302.             try
303.             {
304.                 if (tryFindBook(out Move foundBookMove))
305.                 {
306.                     Console.WriteLine($"bestmove
                {Tools.MoveToString(foundBookMove)}");
307.                     return;
308.                 }
309.
310.                 int whiteInc = 0;
311.                 int blackInc = 0;
312.
313.                 int whiteTime = int.Parse(command.Split(new string[] { "wtime "
                }, StringSplitOptions.None)[1].Split(' ')[0]);
314.                 int blackTime = int.Parse(command.Split(new string[] { "btime "
                }, StringSplitOptions.None)[1].Split(' ')[0]);
315.
316.                 if (command.Contains("inc"))
317.                 {
318.                     whiteInc = int.Parse(command.Split(new string[] { "winc " },
                StringSplitOptions.None)[1].Split(' ')[0]);
319.                     blackInc = int.Parse(command.Split(new string[] { "binc " },
                StringSplitOptions.None)[1].Split(' ')[0]);
320.                 }
321.
322.                 runTimedSearch(whiteTime, blackTime, whiteInc, blackInc);

```

```

323.                }
324.                catch
325.                {
326.                    throw new Exception("Invalid go command expected format: go
<wtime> <int> <winc> <int> <btime> <int> <binc> <inc>");
327.                }
328.            }
329.
330.            else if (command.Contains("depth"))
331.            {
332.                if (tryFindBook(out Move foundBookMove))
333.                {
334.                    Console.WriteLine($"bestmove
{Tools.MoveToString(foundBookMove)}");
335.                    return;
336.                }
337.                try
338.                {
339.                    depth = int.Parse(command.Split(' ')[2]);
340.                    actionThread = new Thread(() => searcher.Search(b, depth));
341.                    actionThread.Start();
342.                }
343.                catch
344.                {
345.                    throw new Exception("Invalid go command expected format: go
<depth> <int>");
346.                }
347.            }
348.
349.        }
350.    }
351.
352.    /// <summary>
353.    /// Lists all of the vaild commands a user can enter
354.    /// </summary>
355.    internal void ListCommands()
356.    {
357.        Console.WriteLine("uci - switch to uci mode and show the possible engine
settings");
358.        Console.WriteLine("isready - check if the engine is ready to recive
commands");
359.        Console.WriteLine("ucinewgame - create a new game");
360.        Console.WriteLine("setoption name <OptionName> value <value> - change a
option that was listed in uci");
361.        Console.WriteLine("help - show this screen");
362.        Console.WriteLine("d - display the current board");
363.        Console.WriteLine("position [fen <fenstring> | startpos ] moves <move1> ....
<movei> - sets up the specified position");
364.        Console.WriteLine("go - starts an infinite search");
365.        Console.WriteLine("go depth <int> - starts a search for a limited depth");
366.        Console.WriteLine("go perft <int> - runs a performance test for a limited
depth");
367.        Console.WriteLine("go perft <int> <bulk/capture/results> - runs a performance
test and displays info for debugging for a limited depth");
368.        Console.WriteLine("poly - shows all of the moves found in the opening book
for the current position");
369.        Console.WriteLine("quit - exits the engine");
370.        Console.WriteLine("stop - stops the current process as soon as possible");
371.        Console.WriteLine("For more information please refer to the UCI documentaion
online");
372.    }
373.
374.
375.    /// <summary>
376.    /// Method <c>tryFindBook</c>
377.    /// will run a polyglot book search if the option is enabled

```

```

378.          /// </summary>
379.          /// <param name="bookmove">The move that was returned by the opening book
    prober</param>
380.          /// <returns>A bool that indicates if the search was successful</returns>
381.          internal bool tryFindBook(out Move bookmove)
382.          {
383.              try
384.              {
385.                  bookmove = new Move(); // create empty move
386.                  if (UseBook)
387.                  {
388.                      // get the current king index
389.                      int kingloc =
        BitBoardTools.BitboardToIndex(Bitboard.getKings(b.Board, (b.WhiteToPlay ? 0 : 1)));
390.                      // probe the opening book
391.                      Move result = Polyglot.queryBook(kingloc, b.zobristKey.Key);
392.                      bookmove = result;
393.
394.                      if (result.isEmpty)
395.                      {
396.                          return false;
397.                      }
398.
399.                      return true;
400.                  }
401.
402.                  // useBook was false
403.                  else return false;
404.              }
405.              catch {
406.                  bookmove = new Move();
407.                  return false;
408.              }
409.          }
410.
411.          /// <summary>
412.          /// Method <c>stopThreadProcess</c>
413.          /// Will call the stop search function in the searcher and the perft which will
    terminate the thread process as fast as it can
414.          /// </summary>
415.          private void stopThreadProcess() {
416.              if (actionThread != null && actionThread.IsAlive) {
417.                  searcher.stopSearch();
418.                  perft.stopsearch();
419.              }
420.          }
421.
422.          /// <summary>
423.          /// Method <c>stringToMove</c>
424.          /// Will convert the given move from a string into a Move struct
425.          /// </summary>
426.          /// <param name="move">The move that will be converted</param>
427.          /// <param name="move1">The converted move in the Move struct format</param>
428.          /// <returns>A bool that indicates if the conversion was successful</returns>
429.          public bool stringToMove(string move, out Move move1) {
430.              string start = move.Substring(0, 2);
431.              string end = move.Substring(2, 2);
432.              int promotion = -1;
433.
434.              // check if it is a promotion
435.              if (move.Length > 4)
436.              {
437.                  switch (move[4])
438.                  {
439.                      case 'n':
440.                          promotion = 0;

```



```
441.         break;
442.
443.         case 'b':
444.             promotion = 1;
445.             break;
446.
447.         case 'r':
448.             promotion = 2;
449.             break;
450.
451.         case 'q':
452.             promotion = 3;
453.             break;
454.     }
455. }
456.
457. byte startIndex = Tools.SquareToIndex(start);
458. byte endIndex = Tools.SquareToIndex(end);
459.
460. List<Move> movelist = generator.GenerateMoves(b);
461.
462. bool found = false;
463. Move target = new Move();
464. move1 = new Move();
465.
466. // loop over the legal moves and find the target move
467. foreach (Move i in movelist)
468. {
469.     // check if the start index and destination index are the same
470.     if (i.startIndex == startIndex && i.destinationIndex == endIndex)
471.     {
472.         if (promotion != -1)
473.         {
474.             // get the correct promotion
475.             if (i.isPromotion && i.pieceIndex == promotion)
476.             {
477.                 target = i;
478.                 found = true;
479.                 break;
480.             }
481.         }
482.
483.         // it is not a promotion
484.         else
485.         {
486.             target = i;
487.             found = true;
488.             break;
489.         }
490.     }
491. }
492. // the target move was found
493. if (found)
494. {
495.     move1= target;
496.     return true;
497. }
498.
499. // the target move was not found
500. else
501. {
502.     return false;
503. }
504. }
505.
506. /// <summary>
```

```

507.          /// Method <c>runTimedSearch</c>
508.          /// Will calculate the amount of time the program will search for.
509.          /// If the program does not finish in the specified time then it will cancel the
    search
510.          /// and return the best evaluation found
511.          /// </summary>
512.          /// <param name="whiteTime">the amount of time in ms that white has left on the
    clock</param>
513.          /// <param name="blackTime">the amount of time in ms that black has left on the
    clock</param>
514.          /// <param name="whiteInc"> the amount of time in ms that white will add every
    move played</param>
515.          /// <param name="blackInc"> the amount of time in ms that black will add every
    move played</param>
516.          internal void runTimedSearch(int whiteTime, int blackTime, int whiteInc, int
    blackInc)
517.          {
518.              int Searchtime = Searcher.CalculateSearchTime(whiteTime, blackTime, whiteInc,
    blackInc, b.WhiteToPlay);
519.              Task.Factory.StartNew(() => {
520.                  Thread.Sleep(Searchtime);
521.                  stopThreadProcess();
522.              });
523.          }
524.      }
525.  }

```

Representation

Bitboard

```

1. using Engine.Representaion;
2. using System;
3. using System.Collections.Generic;
4. using System.IO;
5. using System.Runtime.Serialization.Formatters.Binary;
6. using static Engine.BitBoardTools;
7. using static Engine.Tools;
8.
9. namespace Engine
10. {
11.
12.     public enum GameState
13.     {
14.         InProgress,
15.         Checkmate,
16.         StaleMate,
17.     };
18.
19.     [Serializable]
20.     public class Bitboard
21.     {
22.         public ulong[] Board = new ulong[8]; // Eight bitboards
23.
24.         // Board[0] = White Pieces eg. 0x 00000000 00000000 00000000 00000000 00000000 00000000
    11111111 11111111
25.         // Board[1] = Black Pieces eg. 0x 11111111 11111111 00000000 00000000 00000000 00000000
    00000000 00000000
26.         // Board[2] = Pawns          eg. 0x 00000000 11111111 00000000 00000000 00000000 00000000
    11111111 00000000
27.         // Board[3] = Knights         eg. 0x 01000010 00000000 00000000 00000000 00000000 00000000
    00000000 01000010

```

```

28.      // Board[4] = Bishops      eg. 0x 00100100 00000000 00000000 00000000 00000000 00000000
      00000000 00100100
29.      // Board[5] = Rooks      eg. 0x 10000001 00000000 00000000 00000000 00000000 00000000
      00000000 10000001
30.      // Board[6] = Queens      eg. 0x 00010000 00000000 00000000 00000000 00000000 00000000
      00000000 00010000
31.      // Board[7] = Kings      eg. 0x 00001000 00000000 00000000 00000000 00000000 00000000
      00000000 00001000
32.
33.      public ushort castlingRights;
34.
35.      /*
36.          0001 - White King can castle kingside
37.          0010 - White King can castle queenside
38.          0100 - Black King can castle kingside
39.          1000 - Black King can castle queenside
40.      */
41.
42.      public ushort plyCount = 0;
43.
44.      public ushort moveCount = 1;
45.
46.      public short enPassantFile = -1;
47.
48.      public int capturedPiece = -1;
49.
50.      public bool WhiteToPlay = true;
51.
52.      public ZobristKey zobristKey; // the hash of the current position
53.
54.      [NonSerialized]
55.      public Generation generator = new Generation();
56.
57.      public GameState state = GameState.InProgress;
58.
59.      Stack<BoardState> boardStateHistory = new Stack<BoardState>();
60.
61.      public Bitboard()
62.      {
63.
64.          Board[0] =
        0b00000000_00000000_00000000_00000000_00000000_00000000_11111111_11111111;
65.          Board[1] =
        0b11111111_11111111_00000000_00000000_00000000_00000000_00000000_00000000;
66.          Board[2] =
        0b00000000_11111111_00000000_00000000_00000000_00000000_11111111_00000000;
67.          Board[3] =
        0b01000010_00000000_00000000_00000000_00000000_00000000_00000000_01000010;
68.          Board[4] =
        0b00100100_00000000_00000000_00000000_00000000_00000000_00000000_00100100;
69.          Board[5] =
        0b10000001_00000000_00000000_00000000_00000000_00000000_00000000_10000001;
70.          Board[6] =
        0b00010000_00000000_00000000_00000000_00000000_00000000_00000000_00010000;
71.          Board[7] =
        0b00001000_00000000_00000000_00000000_00000000_00000000_00000000_00001000;
72.
73.          castlingRights = 0b1111;
74.          zobristKey = Zobrist.CalculateZobrist(this);
75.      }
76.
77.      #region Make/Take move
78.      public void MakeMove(Move move)
79.      {
80.          boardStateHistory.Push(new BoardState(zobristKey, castlingRights, plyCount,
        moveCount, capturedPiece, enPassantFile, state, WhiteToPlay)); // add zobrist hash

```

```

81.         byte start = move.startIndex;
82.         byte destination = move.destinationIndex;
83.         ulong startBitboard = IndexToBitboard(start);
84.         ulong destinationBitboard = IndexToBitboard(destination);
85.         int MoveColour = WhiteToPlay ? 0 : 1;
86.         int EnemyColour = 1 - MoveColour;
87.
88.         zobristKey.moveKey = 0;
89.         zobristKey.enPassantKey = 0;
90.         zobristKey.castlingKey = 0;
91.
92.         enPassantFile = -1;
93.         capturedPiece = -1;
94.
95.         plyCount++;
96.
97.
98.         // if capture
99.         if ((Board[EnemyColour] & destinationBitboard) > 0)
100.            {
101.
102.                capturedPiece = getPieceType(Board, destinationBitboard);
103.                Board[EnemyColour] ^= destinationBitboard;
104.                Board[capturedPiece] ^= destinationBitboard;
105.                Zobrist.changePieceSquare(destination, (capturedPiece - 2) * 2 +
EnemyColour, ref zobristKey.pieceKey);
106.
107.                plyCount = 0; // piece has been captured so the 50 move rule resets
108.            }
109.            int pieceType = getPieceType(Board, startBitboard);
110.
111.            if (pieceType == 2)
112.            {
113.                plyCount = 0; // pawn has advanced so the 50 move rule resets
114.            }
115.
116.            Board[MoveColour] ^= destinationBitboard | startBitboard;
117.            Board[pieceType] ^= destinationBitboard | startBitboard;
118.
119.
120.
121.
122.            Zobrist.changePieceSquare(start, (pieceType - 2) * 2 + MoveColour, ref
zobristKey.pieceKey);
123.            Zobrist.changePieceSquare(destination, (pieceType - 2) * 2 + MoveColour,
ref zobristKey.pieceKey);
124.
125.            // deal with flags
126.            if (move.flag != 0)
127.            {
128.
129.                if (move.isPromotion)
130.                {
131.                    int promotionType = move.pieceIndex + 3;
132.                    Board[promotionType] ^= destinationBitboard;
133.                    Board[2] ^= destinationBitboard;
134.                }
135.
136.                else if (move.flag == Move.castling)
137.                {
138.                    switch (destination)
139.                    {
140.                        case 1: // white kingside
141.                            castlingRights = clearCastleBit(castlingRights,
(ushort)Tools.Castling.KC);

```

```

142.                                castlingRights = clearCastleBit(castlingRights,
    (ushort)Tools.Castling.QC);
143.
144.                                Board[MoveColour] ^= 1;
145.                                Board[(int)BB.Rooks] ^= 1;
146.                                Zobrist.changePieceSquare(0, 6 + MoveColour, ref
    zobristKey.pieceKey);
147.
148.
149.
150.                                Board[MoveColour] ^= 4;
151.                                Board[(int)BB.Rooks] ^= 4;
152.                                Zobrist.changePieceSquare(2, 6 + MoveColour, ref
    zobristKey.pieceKey);
153.
154.                                break;
155.
156.                                case 5:
157.                                castlingRights = clearCastleBit(castlingRights,
    (ushort)Tools.Castling.KC);
158.                                castlingRights = clearCastleBit(castlingRights,
    (ushort)Tools.Castling.QC);
159.
160.
161.                                Board[MoveColour] ^= 128;
162.                                Board[(int)BB.Rooks] ^= 128;
163.                                Zobrist.changePieceSquare(7, 6 + MoveColour, ref
    zobristKey.pieceKey);
164.
165.                                Board[MoveColour] ^= 16;
166.                                Board[(int)BB.Rooks] ^= 16;
167.                                Zobrist.changePieceSquare(4, 6 + MoveColour, ref
    zobristKey.pieceKey);
168.                                break;
169.
170.                                case 57:
171.                                castlingRights = clearCastleBit(castlingRights,
    (ushort)Tools.Castling.kc);
172.                                castlingRights = clearCastleBit(castlingRights,
    (ushort)Tools.Castling.qc);
173.
174.
175.                                Board[MoveColour] ^= 72057594037927936; // 2^56
176.                                Board[(int)BB.Rooks] ^= 72057594037927936;
177.                                Zobrist.changePieceSquare(56, 6 + MoveColour, ref
    zobristKey.pieceKey);
178.
179.                                Board[MoveColour] ^= 288230376151711744; // 2^58
180.                                Board[(int)BB.Rooks] ^= 288230376151711744;
181.                                Zobrist.changePieceSquare(58, 6 + MoveColour, ref
    zobristKey.pieceKey);
182.                                break;
183.
184.                                case 61:
185.                                castlingRights = clearCastleBit(castlingRights,
    (ushort)Tools.Castling.kc);
186.                                castlingRights = clearCastleBit(castlingRights,
    (ushort)Tools.Castling.qc);
187.
188.
189.                                Board[MoveColour] ^= 9223372036854775808; // 2^63
190.                                Board[(int)BB.Rooks] ^= 9223372036854775808;
191.                                Zobrist.changePieceSquare(63, 6 + MoveColour, ref
    zobristKey.pieceKey);
192.
193.                                Board[MoveColour] ^= 1152921504606846976; // 2^60

```

```

194. Board[(int)BB.Rooks] ^= 1152921504606846976;
195. Zobrist.changePieceSquare(60, 6 + MoveColour, ref
    zobristKey.pieceKey);
196. break;
197. }
198. }
199.
200. else if (move.flag == Move.enPassantSet)
201. {
202.     enPassantFile = CalculateFile(destination);
203. }
204.
205. else if (move.flag == Move.enPassantCapture)
206. {
207.     ulong enPassantSquare = pushPawn(destinationBitboard, EnemyColour);
208.     Board[2] ^= enPassantSquare;
209.     Board[EnemyColour] ^= enPassantSquare;
210.     Zobrist.changePieceSquare((ushort)(destination + 8 * (MoveColour ==
1 ? 1 : -1)), 0 + EnemyColour, ref zobristKey.pieceKey);
211. }
212. }
213.
214. // if the king moves or the rook moves change the castling rights
215. changeCastlingRights(start);
216. changeCastlingRights(destination);
217.
218.
219. if (!WhiteToPlay)
220. {
221.     moveCount++;
222.     zobristKey.moveKey ^= Zobrist.zobristTables[14][0];
223. }
224.
225. ulong tempCastling = castlingRights;
226. while (tempCastling != 0)
227. {
228.     ulong piece = BitBoardTools.popLSB(ref tempCastling);
229.     int index = BitBoardTools.BitboardToIndex(piece);
230.     zobristKey.castlingKey ^= Zobrist.zobristTables[12][index];
231. }
232.
233. if (enPassantFile != -1)
234. {
235.     int index = enPassantFile - 1;
236.     // if the squares directly next to the pushed pawn are enemy pawns then
    apply the hash
237.     ulong pawnTempLoc = pushPawn(destinationBitboard, EnemyColour);
238.     ulong ajecentMask = BitBoardTools.getPawnAttacks(pawnTempLoc,
    MoveColour);
239.     ulong enemyPawns = Bitboard.getPawns(Board, EnemyColour);
240.     if ((ajecentMask & enemyPawns) > 0)
241.     {
242.         zobristKey.enPassantKey ^= Zobrist.zobristTables[13][index];
243.     }
244. }
245.
246. WhiteToPlay = !WhiteToPlay;
247.
248. }
249.
250. private void changeCastlingRights(byte location) {
251.
252.     switch (location)
253.     {
254.         case 0:

```

```

255.                castlingRights = clearCastleBit(castlingRights,
256.                (ushort)Castling.KC);
257.                break;
258.            case 3:
259.                castlingRights = clearCastleBit(castlingRights,
260.                (ushort)Castling.KC);
261.                castlingRights = clearCastleBit(castlingRights,
262.                (ushort)Castling.QC);
263.                break;
264.            case 7:
265.                castlingRights = clearCastleBit(castlingRights,
266.                (ushort)Castling.QC);
267.                break;
268.
269.            case 56:
270.                castlingRights = clearCastleBit(castlingRights,
271.                (ushort)Castling.kc);
272.                break;
273.
274.            case 59:
275.                castlingRights = clearCastleBit(castlingRights,
276.                (ushort)Castling.kc);
277.                castlingRights = clearCastleBit(castlingRights,
278.                (ushort)Castling.qc);
279.                break;
280.            case 63:
281.                castlingRights = clearCastleBit(castlingRights,
282.                (ushort)Castling.qc);
283.                break;
284.
285.            default: break;
286.        }
287.    }
288.
289.    public void UndoMove(Move move)
290.    {
291.        // unload previous board state
292.        BoardState previousState = boardStateHistory.Pop();
293.        WhiteToPlay = !WhiteToPlay;
294.
295.        byte start = move.startIndex;
296.        byte destination = move.destinationIndex;
297.        ulong startBitboard = IndexToBitboard(start);
298.        ulong destinationBitboard = IndexToBitboard(destination);
299.
300.        int MoveColour = WhiteToPlay ? 0 : 1;
301.        int EnemyColour = 1 - MoveColour;
302.
303.        int pieceType = getPieceType(Board, destinationBitboard);
304.
305.        // if capture - replace the piece
306.        if (capturedPiece != -1)
307.        {
308.            Board[EnemyColour] ^= destinationBitboard;
309.            Board[capturedPiece] ^= destinationBitboard;
310.        }
311.
312.        Board[MoveColour] ^= destinationBitboard | startBitboard;

```

```

313.         Board[pieceType] ^= destinationBitboard | startBitboard;
314.
315.
316.         // deal with flags
317.         if (move.flag != 0)
318.         {
319.             // if promotion - replace the pawn
320.             if (move.isPromotion)
321.             {
322.                 Board[pieceType] ^= startBitboard;
323.                 Board[2] ^= startBitboard;
324.             }
325.
326.             else if (move.flag == Move.castling)
327.             {
328.                 // if castle - reset rook position
329.                 switch (destination)
330.                 {
331.                     case 1: // white kingside
332.                         Board[MoveColour] ^= 1;
333.                         Board[(int)BB.Rooks] ^= 1;
334.                         Board[MoveColour] ^= 4;
335.                         Board[(int)BB.Rooks] ^= 4;
336.                         break;
337.
338.                     case 5: // white queenside
339.                         Board[MoveColour] ^= 128;
340.                         Board[(int)BB.Rooks] ^= 128;
341.                         Board[MoveColour] ^= 16;
342.                         Board[(int)BB.Rooks] ^= 16;
343.                         break;
344.
345.                     case 57: // black kingside
346.                         Board[MoveColour] ^= 72057594037927936; // 2^56
347.                         Board[(int)BB.Rooks] ^= 72057594037927936;
348.                         Board[MoveColour] ^= 288230376151711744; // 2^58
349.                         Board[(int)BB.Rooks] ^= 288230376151711744;
350.                         break;
351.
352.                     case 61: // black queenside
353.                         Board[MoveColour] ^= 9223372036854775808; // 2^63
354.                         Board[(int)BB.Rooks] ^= 9223372036854775808;
355.                         Board[MoveColour] ^= 1152921504606846976; // 2^60
356.                         Board[(int)BB.Rooks] ^= 1152921504606846976;
357.                         break;
358.                 }
359.             }
360.
361.             // enpassant capture - add pawn to board
362.             else if (move.flag == Move.enPassantCapture)
363.             {
364.                 ulong enPassantSquare = pushPawn(destinationBitboard, EnemyColour);
365.                 Board[2] ^= enPassantSquare;
366.                 Board[EnemyColour] ^= enPassantSquare;
367.             }
368.
369.
370.         }
371.         // reset the board variables
372.         capturedPiece = previousState.capturedPiece;
373.         enPassantFile = previousState.enpassant_flag;
374.         castlingRights = previousState.castlingRights;
375.         plyCount = previousState.ply;
376.         moveCount = previousState.moves;
377.         state = previousState.gameState;
378.         zobristKey = previousState.zobristKey;

```



```

379.         }
380.
381.         #endregion
382.
383.         public bool isMate()
384.         { // this does not need to be called in the searching algorithm as when the
moves are being generated it already calculates if it is mate, this is only used for checking
in the ui
385.             generator.board = this;
386.             generator.InitiliseVars(false);
387.             List<Move> moves = generator.GenerateMoves(this);
388.             if (moves.Count == 0 && generator.isCheck) return true;
389.             return false;
390.         }
391.
392.         public bool isStalemate()
393.         { // this does not need to be called in the searching algorithm as when the
moves are being generated it already calculates if it is mate, this is only used for checking
in the ui
394.             generator.board = this;
395.             generator.InitiliseVars(false);
396.             List<Move> moves = generator.GenerateMoves(this);
397.             if (moves.Count == 0 && !generator.isCheck) return true;
398.             return false;
399.         }
400.
401.         public bool isCheck()
402.         {
403.             generator.board = this;
404.             generator.InitiliseVars(false);
405.             generator.CalculateCheckAttacks();
406.             if (generator.isCheck) return true;
407.             return false;
408.         }
409.
410.         public static int getPieceType(ulong[] Board, ulong square)
411.         { // will only work if there is a piece on given square
412.             if ((Board[2] & square) > 0) return 2;
413.             else if ((Board[3] & square) > 0) return 3;
414.             else if ((Board[4] & square) > 0) return 4;
415.             else if ((Board[5] & square) > 0) return 5;
416.             else if ((Board[6] & square) > 0) return 6;
417.             else if ((Board[7] & square) > 0) return 7;
418.
419.             return -1; // there was not a piece on the given square
420.         }
421.
422.         public static bool canCastleKingside(ushort castling, int turn)
423.         {
424.             if (turn == 0)
425.             {
426.                 return ((castling & 1) == 1);
427.             }
428.
429.             return ((castling & 4) == 4);
430.         }
431.
432.         public static bool canCastleQueenside(ushort castling, int turn)
433.         {
434.             if (turn == 0)
435.             {
436.                 return ((castling & 2) == 2);
437.             }
438.
439.             return ((castling & 8) == 8);
440.         }

```

```
441.
442.         #region get piece board functions
443.         public static ulong getPawns(ulong[] Board, int turn)
444.         { // return a bitboard of the pawns of the given colour
445.             return (Board[(int)BB.Pawns] & Board[turn]);
446.         }
447.
448.         public static ulong getKnights(ulong[] Board, int turn)
449.         {
450.             return (Board[(int)BB.Knights] & Board[turn]);
451.         }
452.
453.         public static ulong getBishops(ulong[] Board, int turn)
454.         {
455.             return (Board[(int)BB.Bishops] & Board[turn]);
456.         }
457.
458.         public static ulong getRooks(ulong[] Board, int turn)
459.         {
460.             return (Board[(int)BB.Rooks] & Board[turn]);
461.         }
462.
463.         public static ulong getQueens(ulong[] Board, int turn)
464.         {
465.             return (Board[(int)BB.Queen] & Board[turn]);
466.         }
467.
468.         public static ulong getKings(ulong[] Board, int turn)
469.         {
470.             return (Board[(int)BB.King] & Board[turn]);
471.         }
472.         #endregion
473.
474.         public bool IsColour(ulong loc, int colour)
475.         {
476.             if ((loc & Board[colour]) > 0)
477.             {
478.                 return true;
479.             }
480.
481.             return false;
482.         }
483.
484.         public bool IsDiagonal(ulong loc)
485.         {
486.             if ((loc & Board[(int)BB.Bishops]) > 0)
487.             {
488.                 return true;
489.             }
490.             else if ((loc & Board[(int)BB.Queen]) > 0) return true;
491.
492.             return false;
493.         }
494.
495.         public bool IsOrthogonal(ulong loc)
496.         {
497.             if ((loc & Board[(int)BB.Rooks]) > 0)
498.             {
499.                 return true;
500.             }
501.             else if ((loc & Board[(int)BB.Queen]) > 0) return true;
502.
503.             return false;
504.         }
505.     }
506.
```

```

507.     }
508.
509.     // allow a deep copy to be made of the bitboard
510.     public static class Extensions
511.     {
512.         public static T DeepClone<T>(this T obj)
513.         {
514.             using (MemoryStream stream = new MemoryStream())
515.             {
516.                 BinaryFormatter formatter = new BinaryFormatter();
517.                 formatter.Serialize(stream, obj);
518.                 stream.Position = 0;
519.
520.                 return (T)formatter.Deserialize(stream);
521.             }
522.         }
523.     }

```

BitboardTools

```

1. using System;
2. using System.Collections.Generic;
3.
4. namespace Engine
5. {
6.     public static class BitBoardTools
7.     {
8.         public const ulong AFile =
9.             0b10000000_10000000_10000000_10000000_10000000_10000000_10000000_10000000;
10.         public const ulong HFile = AFile >> 7;
11.         public const ulong Rank1 = 0b11111111;
12.         public const ulong Rank8 = Rank1 << (8 * 7);
13.         public const ulong Edges = AFile | HFile | Rank1 | Rank8;
14.         public const ulong notAFile = ~AFile; //
15.             0b01111111_01111111_01111111_01111111_01111111_01111111_01111111_01111111
16.         public const ulong notHFile = ~(AFile >> 7);
17.         public const ulong notABFile = ~AFile & ~(AFile >> 1);
18.         public const ulong notGHFile = ~(AFile >> 6) & ~(AFile >> 7);
19.         public static readonly ulong[,] CastlingMasks = { { 6, 432345564227567616 }, { 112,
20.             8070450532247928832 } };
21.         public static readonly byte[] DirectionMagnitudes = { 8, 1, 8, 1, 7, 9, 7, 9 };//
22.             {N,E,S,W,NE,SE,SW,NW}
23.         public static readonly ulong[] EdgeDirectionMask = { Rank8, HFile, Rank1, AFile, Edges,
24.             Edges, Edges, Edges };// {N,E,S,W,NE,SE,SW,NW}
25.         public static readonly ulong[] usefulPawnRanks = { Rank8, Rank1, Rank1 << 8, Rank8 >>
26.             8, Rank8 >> 8 * 3, Rank1 << 8 * 3 }; // white promotion, black promotion, white start, black
27.             start, white EnPassant, BlackEnPassant
28.         public static readonly ulong[] wPawnAttacks;
29.         public static readonly ulong[] bPawnAttacks;
30.         public static readonly ulong[] KnightAttacks;
31.         public static readonly ulong[] KingAttacks;
32.         public static readonly Dictionary<ulong, byte> BitboardIndexDictionary = new
33.             Dictionary<ulong, byte>();
34.         public static byte CalculateFile(ulong position)
35.         {
36.             byte index = BitboardToIndex(position);

```

```
36.
37.         return (byte)Math.Abs((index % 8) - 8);
38.     }
39.
40.     public static byte CalculateRank(ulong position)
41.     {
42.         byte index = BitboardToIndex(position);
43.         return (byte)((index / 8) + 1);
44.     }
45.
46.     public static byte CalculateFile(byte position)
47.     {
48.         return (byte)Math.Abs((position % 8) - 8);
49.     }
50.
51.     public static byte CalculateRank(byte position)
52.     {
53.         return (byte)((position / 8) + 1);
54.     }
55.
56.     internal static ulong wPawnWestAttacks(ulong whitePawns)
57.     {
58.         return (whitePawns << 9) & notHFile;
59.     }
60.
61.     internal static ulong wPawnEastAttacks(ulong whitePawns)
62.     {
63.         return (whitePawns << 7) & notAFile;
64.     }
65.
66.
67.     internal static ulong bPawnWestAttacks(ulong blackPawns)
68.     {
69.         return (blackPawns >> 7) & notHFile;
70.     }
71.
72.
73.     internal static ulong bPawnEastAttacks(ulong blackPawns)
74.     {
75.         return (blackPawns >> 9) & notAFile;
76.     }
77.
78.
79.     public static ulong pawnAttacks(ulong pawns, int WhiteToPlay)
80.     {
81.
82.         if (WhiteToPlay == 0) return (wPawnWestAttacks(pawns) | wPawnEastAttacks(pawns));
83.
84.         else return (bPawnWestAttacks(pawns) | bPawnEastAttacks(pawns));
85.     }
86.     public static ulong pawnWestAttacks(ulong pawns, int WhiteToPlay)
87.     {
88.
89.         if (WhiteToPlay == 0) return wPawnWestAttacks(pawns);
90.
91.         else return bPawnWestAttacks(pawns);
92.     }
93.
94.     public static ulong pawnEastAttacks(ulong pawns, int WhiteToPlay)
95.     {
96.
97.         if (WhiteToPlay == 0) return wPawnEastAttacks(pawns);
98.
99.         else return bPawnEastAttacks(pawns);
100.    }
101.    public static ulong getPawnAttacks(ulong pawns, int WhiteToPlay)
```

```

102.      {
103.          int index = BitboardToIndex(pawns);
104.          if (WhiteToPlay == 0) return wPawnAttacks[index];
105.          return bPawnAttacks[index];
106.      }
107.
108.      public static ulong pushPawn(ulong start, int WhiteToPlay)
109.      {
110.          if (WhiteToPlay == 1)
111.          {
112.              return start >> 8;
113.          }
114.          return start << 8;
115.      }
116.
117.      // returns the least significant bit
118.      public static ulong LSB(ulong x)
119.      {
120.          return x &= ~x + 1;
121.      }
122.
123.      public static ulong popLSB(ref ulong x)
124.      {
125.          ulong lsb = x & ~x + 1;
126.          x ^= lsb;
127.          return lsb;
128.      }
129.
130.      // converts the given index 0-63 into the relevent bitboard
131.      // eg index 2 would give
132.      // 00000000
133.      // 00000000
134.      // 00000000
135.      // 00000000
136.      // 00000000
137.      // 00000000
138.      // 00000000
139.      // 00000100 this would represent the square f1
140.      public static ulong IndexToBitboard(Byte index)
141.      {
142.          return 1UL << (index);
143.      }
144.
145.      public static byte BitboardToIndex(ulong a)
146.      {
147.          return BitboardIndexDictionary[a];
148.      }
149.
150.      public static byte CalculateBitboardToIndex(ulong a)
151.      {
152.          byte b = 0;
153.          while (a > 1)
154.          {
155.              a >>= 1;
156.              b++;
157.          }
158.          return b;
159.      }
160.
161.      // Generate the knight attack mask for any given knight bitboard
162.      public static ulong knightAttacks(ulong knight)
163.      {
164.          ulong attacks = 0UL;
165.          attacks |= (knight << 17) & notHFile; // 2 up 1 right
166.          attacks |= (knight >> 15) & notHFile; // 2 up 1 left
167.          attacks |= (knight >> 17) & notAFile; // 2 down 1 right

```

```

168.         attacks |= (knight << 15) & notAFile; // 2 down 1 left
169.
170.
171.         attacks |= (knight >> 10) & notABFile; // 2 right 1 down
172.         attacks |= (knight << 6) & notABFile; // 2 right 1 up
173.         attacks |= (knight >> 6) & notGHFile; // 2 left 1 down
174.         attacks |= (knight << 10) & notGHFile; // 2 left 1 up
175.
176.         return attacks;
177.     }
178.
179.     public static ulong genkingAttacks(ulong king)
180.     {
181.         ulong attacks = 0UL;
182.         attacks |= (king >> 8);
183.         attacks |= (king << 8);
184.
185.         attacks |= (king >> 9) & notAFile;
186.         attacks |= (king >> 7) & notHFile;
187.         attacks |= (king << 9) & notHFile;
188.         attacks |= (king << 7) & notAFile;
189.
190.
191.         attacks |= (king >> 1) & notAFile;
192.         attacks |= (king << 1) & notHFile;
193.
194.
195.         return attacks;
196.     }
197.
198.     static BitBoardTools()
199.     {
200.         wPawnAttacks = new ulong[64];
201.         bPawnAttacks = new ulong[64];
202.         KnightAttacks = new ulong[64];
203.         KingAttacks = new ulong[64];
204.
205.         for (byte i = 0; i < 64; i++)
206.         { // sets up the attack bitboards
207.             ulong b = IndexToBitboard(i);
208.             wPawnAttacks[i] = pawnAttacks(b, 0);
209.             bPawnAttacks[i] = pawnAttacks(b, 1);
210.             KnightAttacks[i] = knightAttacks(b);
211.             KingAttacks[i] = genkingAttacks(b);
212.             BitboardIndexDictionary.Add(b, i);
213.         }
214.
215.
216.
217.     }
218.
219. }
220. }

```

BoardState

```

1. using System;
2.
3. namespace Engine.Representaion
4. {
5.     [Serializable]
6.     internal struct BoardState
7.     {

```

```

8.      public ZobristKey zobristKey;
9.
10.     public ushort castlingRights;
11.
12.     public ushort ply;
13.
14.     public ushort moves;
15.
16.     public short enpassant_flag;
17.
18.     public bool WhiteToPlay;
19.
20.     public int capturedPiece;
21.
22.     public GameState gameState;
23.
24.     public BoardState(ZobristKey zobristkey, ushort castling, ushort plyCount, ushort
    Moves, int CapturedPiece, short EnpassantFlag, GameState state, bool Turn)
25.     {
26.         zobristKey = zobristkey;
27.         castlingRights = castling;
28.         ply = plyCount;
29.         moves = Moves;
30.         enpassant_flag = EnpassantFlag;
31.         capturedPiece = CapturedPiece;
32.         gameState = state;
33.         WhiteToPlay = Turn;
34.     }
35.
36. }
37. }

```

Tools

```

1. using Engine.Representaion;
2. using System;
3. using System.Collections.Generic;
4.
5. namespace Engine
6. {
7.     public static class Tools
8.     {
9.         public enum BB : int
10.        {
11.            WhitePieces,
12.            BlackPieces,
13.            Pawns,
14.            Knights,
15.            Bishops,
16.            Rooks,
17.            Queen,
18.            King,
19.        };
20.
21.        public enum Castling
22.        {
23.            KC = 1,
24.            QC = 2,
25.            kc = 4,
26.            qc = 8,
27.        };
28.
29.        /// <summary>

```

```

30.    /// Outputs a visual representation of the bitboard. 0's are outputted as '.'
31.    /// </summary>
32.    /// <param name="b">The bitboard</param>
33.    public static void printBitboard(ulong b)
34.    {
35.        // converts the bitboard into binary
36.        string binary = Convert.ToString((long)b, 2);
37.        // pads zeros so that it is formatted correctly
38.        binary = binary.PadLeft(64, '0');
39.        // replaces 0 to . for comfort
40.        binary = binary.Replace('0', '.');
41.
42.        Console.WriteLine(
43.            "8 " + binary.Substring(0, 8) + "\n" +
44.            "7 " + binary.Substring(8, 8) + "\n" +
45.            "6 " + binary.Substring(16, 8) + "\n" +
46.            "5 " + binary.Substring(24, 8) + "\n" +
47.            "4 " + binary.Substring(32, 8) + "\n" +
48.            "3 " + binary.Substring(40, 8) + "\n" +
49.            "2 " + binary.Substring(48, 8) + "\n" +
50.            "1 " + binary.Substring(56, 8) + "\n" +
51.            " abcdefgh\n");
52.    }
53.
54.
55.    /// <summary>
56.    /// Outputs the ascii representation of the given board
57.    /// </summary>
58.    /// <param name="b">The board to be displayed</param>
59.    public static void printPieceBoard(Bitboard b)
60.    {
61.        Console.WriteLine(PieceBoardToString(b));
62.    }
63.
64.    /// <summary>
65.    /// Creates string which is the ascii representation of the given board
66.    /// </summary>
67.    /// <param name="b"></param>
68.    /// <returns></returns>
69.    public static string PieceBoardToString(Bitboard b)
70.    {
71.        string output = "";
72.        string[] board = new string[64];
73.        for (byte i = 0; i < 64; i++) // loop over each square
74.        {
75.            int pieceType = Bitboard.getPieceType(b.Board,
76.                BitBoardTools.IndexToBitboard(i));
77.            string ascii = string.Empty;
78.            // get the type of piece
79.            switch (pieceType)
80.            {
81.                case 2: ascii = "p"; break;
82.                case 3: ascii = "n"; break;
83.                case 4: ascii = "b"; break;
84.                case 5: ascii = "r"; break;
85.                case 6: ascii = "q"; break;
86.                case 7: ascii = "k"; break;
87.                default: ascii = " "; break;
88.            }
89.
90.            // if it is black then make it upper case

```



```

95.         if (b.IsColour(BitBoardTools.IndexToBitboard(i), 0))
96.         {
97.             ascii = ascii.ToUpper();
98.         }
99.         // store it in the ascii representation of the board
100.        board[63 - i] = ascii;
101.    }
102.
103.
104.        int counter = 7; // start at 7 since we count down
105.
106.        for (int i = 0; i < 17; i++) // 17 is the total number of rows used to
create the table
107.        {
108.            if (i % 2 == 0)
109.            {
110.                output += "\n  +---+---+---+---+---+---+---+";
111.            }
112.            else
113.            {
114.                int rowNumber = 8 - ((i - 1) / 2);
115.                output += (" \n {rowNumber} | {board[counter - 7]} |
{board[counter - 6]} | {board[counter - 5]} | {board[counter - 4]} | {board[counter - 3]} |
{board[counter - 2]} | {board[counter - 1]} | {board[counter]} |");
116.                // go to the next row
117.                counter += 8;
118.            }
119.        }
120.        output += ("\n      a   b   c   d   e   f   g   h\n");
121.
122.        output += (" \n Turn: {(b.WhiteToPlay ? "White" : "Black")}");
123.        output += (" \n Castling Rights: {castlingToString(b.castlingRights)}");
124.        output += (" \n En Passant: {(b.enPassantFile == -1 ? -1 :
IndexToFile((byte)b.enPassantFile))}");
125.        output += (" \n Zobrist Key: {b.zobristKey.Key.ToString("X")}");
126.        return output;
127.    }
128.
129.    /// <summary>
130.    /// Counts the amount of 1's in a binary 64-bit int
131.    /// </summary>
132.    /// <param name="u">A 64-bit integer</param>
133.    /// <returns>The number of 1's</returns>
134.    public static byte countOnes(ulong u)
135.    {
136.        byte a = 0;
137.
138.        while (u != 0)
139.        {
140.            a++;
141.            u = u & (u - 1);
142.        }
143.
144.        return a;
145.    }
146.
147.    /// <summary>
148.    /// Converts the bit representation of castling rights to a string
149.    /// </summary>
150.    /// <example>1111 would return "KQkq"</example>
151.    /// <param name="castling"></param>
152.    /// <returns>The string of the castling rights</returns>
153.    public static string castlingToString(ushort castling)
154.    {
155.        string result = "";
156.        ulong ulongcastling = (ulong)castling;

```

```

157.         while (ulongcastling != 0)
158.         {
159.             ulong castlingright = BitBoardTools.popLSB(ref ulongcastling);
160.             switch (castlingright)
161.             {
162.                 case 1:
163.                     result += "K";
164.                     break;
165.                 case 2:
166.                     result += "Q";
167.                     break;
168.                 case 4:
169.                     result += "k";
170.                     break;
171.                 case 8:
172.                     result += "q";
173.                     break;
174.             }
175.         }
176.         if (result == "")
177.         {
178.             return "-";
179.         }
180.         return result;
181.     }
182.
183.     public static Bitboard FENtoBitboard(string FEN)
184.     {
185.
186.         FEN = FEN.Trim();
187.         if (FEN.Length == 0)
188.         {
189.             throw new Exception("Invalid FEN String");
190.         }
191.         string[] FENSplit;
192.         string[] FENBoard;
193.
194.         FENSplit = FEN.Split(' ');
195.
196.         if (FENSplit.Length < 6)
197.         {
198.             throw new Exception("Invalid FEN String");
199.         }
200.
201.         FENBoard = FENSplit[0].Split('/');
202.         Bitboard b = new Bitboard();
203.
204.         b.Board[(int)BB.WhitePieces] = FENToBinary(FENBoard, new List<char>() {
205.             'P', 'R', 'B', 'N', 'Q', 'K' });
206.         b.Board[(int)BB.BlackPieces] = FENToBinary(FENBoard, new List<char>() {
207.             'r', 'n', 'b', 'q', 'k', 'p' });
208.         b.Board[(int)BB.Pawns] = FENToBinary(FENBoard, new List<char>() { 'p', 'P'
209.             });
210.         b.Board[(int)BB.Knights] = FENToBinary(FENBoard, new List<char>() { 'n',
211.             'N' });
212.         b.Board[(int)BB.Bishops] = FENToBinary(FENBoard, new List<char>() { 'b',
213.             'B' });
214.         b.Board[(int)BB.Rooks] = FENToBinary(FENBoard, new List<char>() { 'r', 'R'
215.             });
216.         b.Board[(int)BB.Queen] = FENToBinary(FENBoard, new List<char>() { 'q', 'Q'
217.             });
218.         b.Board[(int)BB.King] = FENToBinary(FENBoard, new List<char>() { 'k', 'K'
219.             });
220.
221.         if (FENSplit[1] == "w") b.WhiteToPlay = true;
222.         else b.WhiteToPlay = false;

```

```

215.
216.         b.castlingRights = FENToCastling(FENSplit[2]);
217.
218.         if (FENSplit[3] != "-")
219.         {
220.             b.enPassantFile = SquareToFile(FENSplit[3]);
221.         }
222.
223.         if (FENSplit[4] != "-")
224.         {
225.             b.plyCount = ushort.Parse(FENSplit[4]);
226.         }
227.
228.         if (FENSplit[5] != "-")
229.         {
230.             b.moveCount = ushort.Parse(FENSplit[5]);
231.         }
232.
233.         b.zobristKey = Zobrist.CalculateZobrist(b);
234.
235.         return b;
236.     }
237.
238.     /// <summary>
239.     /// Converts the given index of the square to the chess notation format (e1,
    g5, b3)
240.     /// </summary>
241.     /// <param name="index">the index of the square</param>
242.     /// <returns>The string format of a square</returns>
243.     public static string IndexToSquare(byte index)
244.     {
245.         byte file = BitBoardTools.CalculateFile(index);
246.         byte rank = BitBoardTools.CalculateRank(index);
247.         file += 96;
248.
249.         return Convert.ToChar(file) + rank.ToString();
250.     }
251.
252.
253.     public static char IndexToFile(byte file)
254.     {
255.         file += 96;
256.         return Convert.ToChar(file);
257.     }
258.
259.
260.     public static ulong clearBit(byte index, ulong b)
261.     {
262.         return b & ~(1UL << index);
263.     }
264.
265.     public static ushort clearCastleBit(ushort castling, ushort right)
266.     {
267.         return (ushort)(castling & ~right);
268.     }
269.
270.     public static byte SquareToIndex(string square)
271.     {
272.         square = square.ToLower();
273.
274.         char file = square[0];
275.         int intFile = file;
276.         intFile -= 96;
277.
278.         char rank = square[1];
279.         int intRank = rank - 48;

```

```
280.
281.         return (byte)((intRank - 1) * 8 + Math.Abs(intFile - 8));
282.     }
283.
284.     public static byte SquareToFile(string square)
285.     {
286.         square = square.ToLower();
287.         char file = square[0];
288.         int intFile = file;
289.         return (byte)(intFile - 96);
290.     }
291.
292.     internal static ushort FENToCastling(string rights)
293.     {
294.         ushort result = 0;
295.         if (rights == "-") return result;
296.
297.         foreach (char i in rights)
298.         {
299.             switch (i)
300.             {
301.
302.                 case 'K':
303.                     result += (ushort)Castling.KC;
304.                     break;
305.
306.                 case 'Q':
307.                     result += (ushort)Castling.QC;
308.                     break;
309.
310.                 case 'k':
311.                     result += (ushort)Castling.kc;
312.                     break;
313.
314.                 case 'q':
315.                     result += (ushort)Castling.qc;
316.                     break;
317.
318.             }
319.
320.         }
321.         return result;
322.     }
323.
324.     internal static ulong FENToBinary(string[] FEN, List<char> target)
325.     {
326.         string result = "";
327.         foreach (string row in FEN)
328.         {
329.
330.             foreach (char c in row)
331.             {
332.                 if (target.Contains(c))
333.                 {
334.                     result += "1";
335.                 }
336.                 else if (int.TryParse(c.ToString(), out int a))
337.                 {
338.                     for (int i = 0; i < a; i++)
339.                     {
340.                         result += "0";
341.                     }
342.                 }
343.                 else result += "0";
344.             }
345.         }
```

```

346.         return (ulong)Convert.ToInt64(result, 2);
347.     }
348.
349.     internal static string MoveToString(Move move)
350.     {
351.         string result = string.Empty;
352.         byte destination = move.destinationIndex;
353.         byte start = move.startIndex;
354.
355.         result = $"{IndexToSquare(start) + IndexToSquare(destination)}";
356.
357.         if (move.isPromotion)
358.         {
359.             switch (move.pieceIndex)
360.             {
361.                 case 0:
362.                     result += "n";
363.                     break;
364.
365.                 case 1:
366.                     result += "b";
367.                     break;
368.
369.                 case 2:
370.                     result += "r";
371.                     break;
372.
373.                 case 3:
374.                     result += "q";
375.                     break;
376.
377.             }
378.         }
379.
380.         return result;
381.     }
382.
383.     internal static string BoardToFEN(Bitboard b)
384.     {
385.         throw new NotImplementedException();
386.     }
387.
388. }
389.

```

Generation

Generator

```

1. using System.Collections.Generic;
2.
3. namespace Engine
4. {
5.     public class Generation
6.     {
7.         public enum Promotions { All, QueenOnly, QueenAndKnight }
8.
9.         public Promotions promotionsToGenerate = Promotions.All;
10.        public static readonly ushort[][] PromotionFlags = { new ushort[] { 4, 5, 6, 7 }, new
        ushort[] { 7 }, new ushort[] { 7, 4 } }; // stores the promotion flags which are passed to the
        move constructor
11.

```

```

12.     static private readonly bool[] directionConsts = { true, false, false, true, true,
false, false, true }; // the directions where the LSB is moving away or towards the king square
(true for moving away)
13.
14.     int MoveColour; // the index whose turn (0 white, 1 black)
15.     int EnemyColour; // the other index
16.
17.     byte KingSquare; // the index of the current players king
18.     ulong KingSquareBitboard; // the bitboard of the kings location
19.
20.     public bool isCheck;
21.     public bool isDoubleCheck;
22.
23.     bool genQuietMoves; // only generate quiet moves (moves that are not captures)
24.
25.
26.     // If in check, this bitboard contains squares in line from checking piece up to king
27.     // If not in check, all bits are set to 1
28.     ulong checkRayMask;
29.
30.     ulong pinRayMask; // the bitboard with the pins that are on the king
31.     ulong pinH; // the horizontal pins (E and W)
32.     ulong pinV; // the vertical pins (N and S)
33.     ulong pinHV; // the horizontal and verical pins (E, W, N, S)
34.     ulong pinD; // the diagonal pins (NE, SE, SW, NW)
35.     ulong[] pinDirectionMasks = new ulong[9]; // {N, E, S, W, NE, SE, SW, NW, Full
bitboard}
36.     internal ulong opponentAttacks; // all of the square that the opponents can move to /
are attacking
37.     internal ulong opponentAttacksWithoutPawns; // opponents attacks without the opponent
pawnmoves
38.     ulong AllPieces; // all the pieces on the board
39.     ulong notFriendlyPieces; // all of the opponents pieces
40.
41.     internal Bitboard board;
42.
43.     // If only captures should be generated, this will have 1s only in positions of enemy
pieces.
44.     // Otherwise it will have 1s everywhere.
45.     ulong moveTypeMask;
46.
47.     /// <summary>
48.     /// Generates all of the legal moves for any bitboard
49.     /// </summary>
50.     /// <param name="board">the amount of time in ms that white has left on the
clock</param>
51.     /// <param name="capturesOnly">Limits the moves generated to be captures only. Default
value is false</param>
52.     /// <returns>A list of valid moves</returns>
53.     public List<Move> GenerateMoves(Bitboard board, bool capturesOnly = false)
54.     {
55.         List<Move> moves = new List<Move>();
56.
57.
58.         this.board = board;
59.
60.         InitiliseVars(capturesOnly);
61.
62.         CalculateCheckAttacks();
63.
64.         genKingMoves(ref moves);
65.
66.         if (!isDoubleCheck) // in double check only the king can move
67.         {
68.             genPawnMoves(ref moves);
69.             genKnightMoves(ref moves);

```

```

70.         genSlidingMoves(ref moves);
71.     }
72.
73.     if (moves.Count == 0) // if there are no moves then the it is either checkmate or
    stale mate
74.     {
75.         if (isChecked)
76.         {
77.             board.state = GameState.Checkmate;
78.         }
79.         else board.state = GameState.StaleMate;
80.     }
81.
82.     return moves;
83. }
84.
85.     /// <summary>
86.     /// Loops through the bitboard of legal moves and creates a Move struct for each and
    adds them to the list of valid moves
87.     /// </summary>
88.     /// <param name="moves">The list of moves</param>
89.     /// <param name="legalMoves">The bitboard of legal moves (places the piece can move
    to)</param>
90.     /// <param name="start">The bitboard of the start location</param>
91.     /// <param name="insertAtFront">If true inserts the move at the start of the
    list</param>
92.     void addLegalMoves(ref List<Move> moves, ulong legalMoves, ulong start, bool
    insertAtFront = false)
93.     {
94.         while (legalMoves != 0)
95.         {
96.             ulong destination = BitBoardTools.popLSB(ref legalMoves);
97.
98.             if (insertAtFront)
99.             {
100.                 moves.Insert(0, new Move(start, destination));
101.             }
102.             else
103.             {
104.                 moves.Add(new Move(start, destination));
105.             }
106.         }
107.     }
108.
109.     /// <summary>
110.     /// Generates all of the promotions and adds them to the list of moves
111.     /// </summary>
112.     /// <param name="moves">The list of moves</param>
113.     /// <param name="start">The bitboard of the start location</param>
114.     /// <param name="destination">The bitboard of the destination of the
    move</param>
115.     void genPromotions(ref List<Move> moves, ulong start, ulong destination)
116.     {
117.         foreach (ushort i in PromotionFlags[(int)promotionsToGenerate])
118.         {
119.             moves.Insert(0, new Move(start, destination, i));
120.         }
121.     }
122.
123.     /// <summary>
124.     /// Generates all of legal king moves and adds them to the list
125.     /// </summary>
126.     /// <param name="moves">The list of moves</param>
127.     private void genKingMoves(ref List<Move> moves)
128.     {
129.         ulong PseudoLegal = BitBoardTools.KingAttacks[KingSquare];

```

```

130.         ulong kingMoves = PseudoLegal & (~ (opponentAttacks |
board.Board[MoveColour])) & moveTypeMask;
131.         addLegalMoves(ref moves, kingMoves, KingSquareBitboard, false);
132.
133.         // Castling moves
134.
135.         if (isChecked == false && moveTypeMask == ulong.MaxValue)
136.         {
137.             ulong clearedQueenSideAttack = (Tools.clearBit(6, opponentAttacks) &
Tools.clearBit(62, opponentAttacks));
138.             ulong CheckBlockers = clearedQueenSideAttack | AllPieces; // the king
can not move through or into check or if there is a piece blocking the path
139.
140.             if (Bitboard.canCastleKingside(board.castlingRights, MoveColour) &&
(BitBoardTools.CastlingMasks[0, MoveColour] & CheckBlockers) == 0)
141.             {
142.                 byte destination = MoveColour == 0 ? (byte)1 : (byte)57;
143.                 moves.Add(new Move(KingSquare, destination, Move.castling));
144.             }
145.
146.             if (Bitboard.canCastleQueenside(board.castlingRights, MoveColour) &&
(BitBoardTools.CastlingMasks[1, MoveColour] & CheckBlockers) == 0)
147.             {
148.                 byte destination = MoveColour == 0 ? (byte)5 : (byte)61;
149.                 moves.Add(new Move(KingSquare, destination, Move.castling));
150.             }
151.         }
152.     }
153.
154.     /// <summary>
155.     /// Generates all of legal knight moves and adds them to the list
156.     /// </summary>
157.     /// <param name="moves">The list of moves</param>
158.     void genKnightMoves(ref List<Move> moves)
159.     {
160.         ulong knights = Bitboard.getKnights(board.Board, MoveColour);
161.
162.         knights &= ~pinRayMask; // pinned knights can't move
163.
164.         while (knights != 0)
165.         { // for each knight
166.             ulong knightLoc = BitBoardTools.popLSB(ref knights);
167.             ulong knightAttacks =
BitBoardTools.KnightAttacks[BitBoardTools.BitboardToIndex(knightLoc)];
168.             knightAttacks &= checkRayMask & moveTypeMask & notFriendlyPieces; //
legalise the moves
169.
170.             addLegalMoves(ref moves, knightAttacks, knightLoc, false);
171.         }
172.     }
173.
174.     /// <summary>
175.     /// Generates all of legal moves for the sliding pieces (rooks, bishops and
queens) and adds them to the list
176.     /// </summary>
177.     /// <param name="moves">The list of moves</param>
178.     void genSlidingMoves(ref List<Move> moves)
179.     {
180.         // Orthogonal pieces
181.
182.         ulong OrthogonalSliders = Bitboard.getRooks(board.Board, MoveColour) |
Bitboard.getQueens(board.Board, MoveColour);
183.         OrthogonalSliders &= ~pinD; // diagonally pinned rooks can't move
184.         ulong DiagonalSliders = Bitboard.getBishops(board.Board, MoveColour) |
Bitboard.getQueens(board.Board, MoveColour);
185.         DiagonalSliders &= ~pinHV; // orthogonally pinned bishops can't move

```



```

186.
187.         if (isChecked)
188.         { // pinned pieces can not move in check
189.             OrthogonalSliders &= ~pinRayMask;
190.             DiagonalSliders &= ~pinRayMask;
191.         }
192.
193.         while (OrthogonalSliders != 0) // generate moves for each rooks
194.         {
195.             ulong OrthognalPiece = BitBoardTools.popLSB(ref OrthogonalSliders); //
196.             gets the bitboard the pieces location
197.             byte PieceSquareIndex = BitBoardTools.BitboardToIndex(OrthognalPiece);
198.             // converts it into an index
199.             ulong legalMoves = Magic.getRookBlockerAttacks(OrthognalPiece,
200.             AllPieces); // gets all the pseudo-legal moves via the magic bitboard tables
201.             legalMoves &= checkRayMask & moveTypeMask & notFriendlyPieces; //
202.             legalises the pseudo-legal moves
203.
204.             if (IsPinned(PieceSquareIndex))
205.             { // pinned rook can only move horizontally and vertically
206.                 legalMoves &= pinHV;
207.             }
208.
209.             addLegalMoves(ref moves, legalMoves, OrthognalPiece, false);
210.         }
211.
212.         while (DiagonalSliders != 0) // generate moves for each bishops
213.         {
214.             ulong DiagonalPiece = BitBoardTools.popLSB(ref DiagonalSliders);
215.             byte PieceSquareIndex = BitBoardTools.BitboardToIndex(DiagonalPiece);
216.             ulong legalMoves = Magic.getBishopBlockerAttacks(DiagonalPiece,
217.             AllPieces);
218.             legalMoves &= checkRayMask & moveTypeMask & notFriendlyPieces;
219.
220.             if (IsPinned(PieceSquareIndex))
221.             { // pinned bishops can only move along the pinned diagonals
222.                 legalMoves &= pinD;
223.             }
224.
225.             addLegalMoves(ref moves, legalMoves, DiagonalPiece, false);
226.         }
227.     }
228.
229.     /// <summary>
230.     /// Generates all of legal pawn moves and adds them to the list
231.     /// </summary>
232.     /// <param name="moves">The list of moves</param>
233.     internal void genPawnMoves(ref List<Move> moves)
234.     {
235.         ulong Pawns = Bitboard.getPawns(board.Board, MoveColour); // gets the
236.         bitboard of all the pawns to loop through
237.
238.         while (Pawns != 0) // loops through each pawn
239.         {
240.             int pinDirectionMaskIndex = 8;
241.
242.             ulong pawnLoc = BitBoardTools.popLSB(ref Pawns);
243.             if (IsPinned(BitBoardTools.BitboardToIndex(pawnLoc)))
244.             {
245.                 pinDirectionMaskIndex = pinType(pawnLoc); // gets the type of pin
246.             }
247.
248.             if (genQuietMoves)
249.             { // generate pushes
250.
251.                 // Push the pawn once

```

```

246.         ulong pawnPush;
247.
248.         pawnPush = BitBoardTools.pushPawn(pawnLoc, MoveColour);
249.
250.         pawnPush &= ~AllPieces;
251.
252.         if ((BitBoardTools.usefulPawnRanks[MoveColour] & pawnPush) == 0) //
if not a promotion
253.         {
254.
255.             if ((BitBoardTools.usefulPawnRanks[MoveColour + 2] & pawnLoc) >
0 && pawnPush > 0)
256.             { // if it is on the starting rank then generate double pushes
257.                 ulong pawnDoublePush;
258.                 pawnDoublePush =
BitBoardTools.pushPawn(BitBoardTools.pushPawn(pawnLoc, MoveColour), MoveColour); // double push
the pawn
259.
260.                 pawnDoublePush &= ~AllPieces; // if it does not end up on a
piece (the double push is empty)
261.                 pawnDoublePush &= checkRayMask & moveTypeMask &
pinDirectionMasks[pinDirectionMaskIndex]; // make sure the pawn is not pinned or causes a check
262.                 if (pawnDoublePush != 0) // if it can double push add the
move
263.                 {
264.                     moves.Add(new Move(pawnLoc, pawnDoublePush,
Move.enPassantSet));
265.                 }
266.             }
267.             // add the single pawn push
268.             pawnPush &= checkRayMask & moveTypeMask &
pinDirectionMasks[pinDirectionMaskIndex];
269.             addLegalMoves(ref moves, pawnPush, pawnLoc, false);
270.         }
271.
272.         else
273.         { // it is a promotion
274.             pawnPush &= checkRayMask & moveTypeMask &
pinDirectionMasks[pinDirectionMaskIndex];
275.             if (pawnPush != 0)
276.             {
277.                 genPromotions(ref moves, pawnLoc, pawnPush);
278.             }
279.         }
280.     }
281.
282.
283.     // gen captures / enpassant
284.     ulong pawnCaptures = BitBoardTools.getPawnAttacks(pawnLoc, MoveColour);
285.     pawnCaptures &= notFriendlyPieces &
pinDirectionMasks[pinDirectionMaskIndex];
286.
287.     if (pawnCaptures != 0)
288.     {
289.         if ((pawnLoc & BitBoardTools.usefulPawnRanks[MoveColour + 4]) > 0
&& board.enPassantFile != -1)
290.         { // if on the correct enPassant rank && the enPassant flag is
enabled
291.             ulong potentialEnPassant = pawnCaptures & (BitBoardTools.AFile
>> (board.enPassantFile - 1)); // gets the enpassant square that can be taken on
292.             ulong enPassantSquare =
BitBoardTools.pushPawn(potentialEnPassant, EnemyColour); // gets the square that the enemy pawn
is on
293.
294.             if (potentialEnPassant > 0 && (enPassantSquare & checkRayMask) >
0 && notCheckAfterEnPassant(potentialEnPassant, pawnLoc, enPassantSquare))

```

```

295.                {
296.                    moves.Insert(0, new Move(pawnLoc, potencialEnPassant,
Move.enPassantCapture));
297.                }
298.            }
299.
300.            pawnCaptures &= board.Board[EnemyColour] & checkRayMask;
301.
302.            if (pawnCaptures != 0)
303.            {
304.                if ((BitBoardTools.usefulPawnRanks[MoveColour] & pawnCaptures)
== 0) // it is not a capture promotion
305.                {
306.                    addLegalMoves(ref moves, pawnCaptures, pawnLoc, true);
307.                }
308.
309.                else
310.                {
311.                    // it is a capture promotion
312.                    while (pawnCaptures != 0)
313.                    { // loop over each capture
314.                        ulong pawnCapturePromotion = BitBoardTools.popLSB(ref
pawnCaptures);
315.                        genPromotions(ref moves, pawnLoc,
pawnCapturePromotion);
316.                    }
317.                }
318.            }
319.        }
320.
321.    }
322. }
323.
324.    /// <summary>
325.    /// Calculates if the king will be in check after the enpassant
326.    /// </summary>
327.    /// <param name="destination">Where the capturing pawn will end up</param>
328.    /// <param name="start">Where the capturing pawn started</param>
329.    /// <param name="enPassantSquare">Where the EnPassant pawn is</param>
330.    /// <returns>True if it is a valid EnPassant capture, False if not</returns>
331.    bool notCheckAfterEnPassant(ulong destination, ulong start, ulong
enPassantSquare)
332.    {
333.        ulong afterEnPassantAllPieces = AllPieces; // copy the pieces
334.        afterEnPassantAllPieces ^= destination | start | enPassantSquare; // add
the destination pieces and remove the start and captured pawn
335.        ulong attacks = Magic.getRookBlockerAttacks(KingSquareBitboard,
afterEnPassantAllPieces); // gets the rook blocks since they are the only piece which causes
this error
336.
337.        if ((attacks & Bitboard.getRooks(board.Board, EnemyColour)) > 0) // checks
is the rook is attacking the king after the enpassant
338.        {
339.            return false; // not valid
340.        }
341.
342.        return true; // valid move
343.    }
344.
345.    /// <summary>
346.    /// Resets all of the local variables which are used to generate the moves
347.    /// </summary>
348.    /// <param name="capturesonly">Sets if the moves generated are only ones that
capture the opponents pieces useful for quicence searches</param>
349.    public void InitiliseVars(bool capturesonly)
350.    {

```

```

351.         isChecked = false;
352.         isDoubleCheck = false;
353.
354.         checkRayMask = 0;
355.         pinRayMask = 0;
356.
357.         MoveColour = board.WhiteToPlay ? 0 : 1;
358.         EnemyColour = 1 - MoveColour;
359.
360.         KingSquareBitboard = board.Board[(int)Tools.BB.King] &
board.Board[MoveColour];
361.         KingSquare = BitBoardTools.BitboardToIndex(KingSquareBitboard);
362.         AllPieces = board.Board[0] | board.Board[1];
363.         notFriendlyPieces = ~(board.Board[MoveColour]);
364.
365.         opponentAttacks = 0;
366.         opponentAttacksWithoutPawns = 0;
367.
368.         genQuietMoves = !capturesonly;
369.
370.         pinDirectionMasks[0] = 0;
371.         pinDirectionMasks[1] = 0;
372.         pinDirectionMasks[2] = 0;
373.         pinDirectionMasks[3] = 0;
374.         pinDirectionMasks[4] = 0;
375.         pinDirectionMasks[5] = 0;
376.         pinDirectionMasks[6] = 0;
377.         pinDirectionMasks[7] = 0;
378.         pinDirectionMasks[8] = ulong.MaxValue;
379.
380.         pinH = 0;
381.         pinD = 0;
382.         pinV = 0;
383.         pinHV = 0;
384.
385.         moveTypeMask = capturesonly ? board.Board[EnemyColour] : ulong.MaxValue;
386.     }
387.
388.     /// <summary>
389.     /// Calculates the pin and check masks which are used to generate moves and
also builds the bitboard of all of squares the opponents are attacking
390.     /// </summary>
391.     public void CalculateCheckAttacks()
392.     {
393.         CalculatePinMasksAndCheck();
394.
395.         // Knight checks
396.         ulong enemyKnights = Bitboard.getKnights(board.Board, EnemyColour);
397.         ulong knightCheckers = BitBoardTools.KnightAttacks[KingSquare] &
enemyKnights;
398.         if (knightCheckers > 0)
399.         {
400.             isDoubleCheck = isChecked;
401.             isChecked = true;
402.             checkRayMask |= knightCheckers;
403.         }
404.         while (enemyKnights != 0)
405.         {
406.             ulong knight = BitBoardTools.popLSB(ref enemyKnights);
407.             opponentAttacksWithoutPawns |=
BitBoardTools.KnightAttacks[BitBoardTools.BitboardToIndex(knight)];
408.         }
409.
410.         // pawn checks
411.         ulong enemyPawns = Bitboard.getPawns(board.Board, EnemyColour);

```

```

412.         ulong enemyPawnAttacks = BitBoardTools.pawnAttacks(enemyPawns,
    EnemyColour);
413.         ulong pawnCheckers = KingSquareBitboard & enemyPawnAttacks;
414.         if (pawnCheckers > 0)
415.         {
416.             isDoubleCheck = isCheck;
417.             isCheck = true;
418.             ulong pawnAttackers = board.WhiteToPlay ?
    BitBoardTools.wPawnAttacks[KingSquare] : BitBoardTools.bPawnAttacks[KingSquare];
419.             ulong pawnCheckMap = Bitboard.getPawns(board.Board, EnemyColour) &
    pawnAttackers;
420.             checkRayMask |= pawnCheckMap;
421.         }
422.
423.         CalculateOpponentSlidingAttacks(); // all of the squares the opponents
    bishops and rook are attacking
424.
425.         int enemyKingSquare =
    BitBoardTools.BitboardToIndex(Bitboard.getKings(board.Board, EnemyColour));
426.         opponentAttacksWithoutPawns |= BitBoardTools.KingAttacks[enemyKingSquare];
427.
428.         opponentAttacks = opponentAttacksWithoutPawns | enemyPawnAttacks;
429.
430.         pinHV = pinH | pinV;
431.
432.         if (!isCheck)
433.         {
434.             checkRayMask = ulong.MaxValue; // the pieces can move anywhere as they
    dont need to block check
435.         }
436.     }
437.
438.     /// <summary>
439.     /// Calculates the pin and check masks and also if it is check or double check
440.     /// </summary>
441.     public void CalculatePinMasksAndCheck()
442.     {
443.         for (int i = 0; i < 8; i++) // loop through each direction
444.         {
445.             ulong directionMagic = Magic.directions[KingSquare, i]; // gets the
    bitboard of the direction ray from the king square
446.             ulong piecesToCheck = directionMagic & (AllPieces ^
    KingSquareBitboard); // gets the bitboard of the pieces on this ray
447.             ulong maybePin = 0; // the location of a piece that might cause a pin
448.             int friendlyPieces = 0; // counter of friendly pieces found along the
449.
450.             if (directionConsts[i]) // we know the LSB is moving away from the king
    square so the first piece will be a blocking or checking piece.
451.             {
452.                 while (piecesToCheck != 0) // through each piece
453.                 {
454.                     ulong tempPiece = BitBoardTools.popLSB(ref piecesToCheck);
455.                     if (board.IsColour(tempPiece, EnemyColour)) // if it is an
    enemy piece
456.                     {
457.                         if ((i <= 3 && board.IsOrthogonal(tempPiece)) || (i >= 4 &&
    board.IsDiagonal(tempPiece))) // 0-3 are orthoganal rays, 4-7 are diagonal rays
458.                         { // make sure it is the right piece for the right
    direction
459.                             if (friendlyPieces == 0 && maybePin == 0)
460.                             {
461.                                 isDoubleCheck = isCheck;
462.                                 isCheck = true;
463.                             }
464.                             if (maybePin == 0)
465.                             {

```

```

466.         maybePin = tempPiece;
467.     }
468.     }
469.     else
470.     { // if the first piece is not a piece that can not deliver
      check then we dont need to search the rest of the ray
471.         break;
472.     }
473. }
474.
475.     else
476.     { // it is a friendly piece
477.         if (maybePin == 0)
478.         { // there is not a checking piece before it so it could be
      pinned
479.             friendlyPieces++;
480.         }
481.     }
482. }
483.
484.     if (isCheck && maybePin > 0 && friendlyPieces == 0) // it is a
      check
485.     {
486.         checkRayMask |= Magic.ForwardFill(KingSquareBitboard, maybePin,
      BitBoardTools.EdgeDirectionMask[i], BitBoardTools.DirectionMagnitudes[i]);
487.     }
488.
489.     if (friendlyPieces == 1 && maybePin > 0) // forward fill to create
      the pin mask
490.     {
491.         ulong mask = Magic.ForwardFill(KingSquareBitboard, maybePin,
      BitBoardTools.EdgeDirectionMask[i], BitBoardTools.DirectionMagnitudes[i]);
492.         pinRayMask |= mask;
493.         switch (i)
494.         {
495.             case 0:
496.             case 2:
497.                 pinV |= mask;
498.                 break;
499.
500.             case 1:
501.             case 3:
502.                 pinH |= mask;
503.
504.                 break;
505.             default:
506.                 pinD |= mask;
507.                 break;
508.         }
509.         pinDirectionMasks[i] = mask;
510.     }
511. }
512.
513.     else
514.     { // the LSB is moving towards the king
515.         while (piecesToCheck != 0)
516.         {
517.             ulong tempPiece = BitBoardTools.popLSB(ref piecesToCheck);
518.             if (board.IsColour(tempPiece, EnemyColour))
519.             {
520.                 if ((i < 4 && board.IsOrthogonal(tempPiece)) || (i > 3 &&
      board.IsDiagonal(tempPiece)))
521.                 {
522.                     maybePin = tempPiece; // it could be check or a pin
523.                     friendlyPieces = 0;
524.                 }

```

```

525.
526.         else
527.         {
528.             maybePin = 0;
529.         }
530.     }
531.     else
532.     {
533.         if (maybePin != 0) // if it is before a piece that can
check the king then we can ignore it
534.         {
535.             friendlyPieces++;
536.         }
537.     }
538. }
539.
540. // no friendly piece was found but there was pinning piece
541. if (friendlyPieces == 0 && maybePin > 0)
542. {
543.     isDoubleCheck = isCheck;
544.     isCheck = true;
545.     checkRayMask |= Magic.BackwardFill(KingSquareBitboard,
maybePin, BitBoardTools.EdgeDirectionMask[i], BitBoardTools.DirectionMagnitudes[i]);
546. }
547.
548. if (friendlyPieces == 1 && maybePin > 0) // forward fill to create
the pin mask
549. {
550.
551.     ulong mask = Magic.BackwardFill(KingSquareBitboard, maybePin,
BitBoardTools.EdgeDirectionMask[i], BitBoardTools.DirectionMagnitudes[i]); ;
552.     pinRayMask |= mask;
553.     switch (i)
554.     {
555.         case 0:
556.         case 2:
557.             pinV |= mask;
558.             break;
559.         case 1:
560.         case 3:
561.             pinH |= mask;
562.             break;
563.         default:
564.             pinD |= mask;
565.             break;
566.     }
567.     pinDirectionMasks[i] = mask;
568. }
569. }
570.
571. if (isDoubleCheck) // only the king can move in double check so we can
stop searching for pins or checks.
572. {
573.     break;
574. }
575. }
576. }
577.
578. /// <summary>
579. /// Calls the get slider attacks and fills the slider attack bitboards
580. /// </summary>
581. public void CalculateOpponentSlidingAttacks()
582. {
583.     ulong OrthogonalSliders = Bitboard.getRooks(board.Board, EnemyColour) |
Bitboard.getQueens(board.Board, EnemyColour);

```

```

584.         ulong DiagonalSliders = Bitboard.getBishops(board.Board, EnemyColour) |
        Bitboard.getQueens(board.Board, EnemyColour);
585.
586.
587.         GetSliderAttack(OrthogonalSliders, AllPieces ^ KingSquareBitboard, true);
588.         GetSliderAttack(DiagonalSliders, AllPieces ^ KingSquareBitboard, false);
589.
590.     }
591.
592.     internal void GetSliderAttack(ulong pieces, ulong blockers, bool isRook)
593.     {
594.         while (pieces != 0) // loop over each sliding piece
595.         {
596.             ulong loc = BitBoardTools.popLSB(ref pieces); // get the location
597.             if (isRook)
598.             {
599.                 opponentAttacksWithoutPawns |= Magic.getRookBlockerAttacks(loc,
        blockers); // get the pseudo-legal rooks moves
600.             }
601.
602.             else
603.             {
604.                 opponentAttacksWithoutPawns |= Magic.getBishopBlockerAttacks(loc,
        blockers); // get the pseudo-legal bishop moves
605.             }
606.         }
607.     }
608.
609.     /// <summary>
610.     /// Calculates if the piece on the given square is pinned
611.     /// </summary>
612.     /// <param name="square">The index location of the piece</param>
613.     /// <returns>True if the piece is pinned, False if the piece is not
        pinned</returns>
614.     bool IsPinned(byte square)
615.     {
616.         return ((pinRayMask >> square) & 1) == 1;
617.     }
618.
619.     /// <summary>
620.     /// Calculates the type of pin (North pin, North-east pin, East pin, ect.).
621.     /// Should only be called if the given piece is pinned.
622.     /// </summary>
623.     /// <param name="square">The bitboard of the piece</param>
624.     /// <returns>The index of the pin mask in the pinDirectionMasks array. If the
        piece was not pinned the index pointing to a full bitboard is returned</returns>
625.     int pinType(ulong square)
626.     {
627.         for (int i = 0; i < pinDirectionMasks.Length - 1; i++)
628.         {
629.             if ((square & pinDirectionMasks[i]) > 0)
630.             {
631.                 return i;
632.             }
633.         }
634.         return 8; // if it not pinned return the full bitboard
635.     }
636. }
637. }
638. /*
639.     Notes:
640.     Everything should be working so DONT TOUCH ANYTHING. I DONT CARE IF IT MIGHT
        IMPROVE PERFORMANCE JUST DONT TOUCH IT.
641. */

```


Magic

```

1. using System;
2.
3. namespace Engine
4. {
5.     public static class Magic // its magic, don't question it.
6.     {
7.
8.         public static ulong[][] RookMagic = new ulong[64][]; // the list of rook magic numbers
           for each square and blocker combinations
9.         public static ulong[][] BishopMagic = new ulong[64][]; // the list of bishop magic
           numbers for each square and blocker combinations
10.
11.         /*
12.         Piece masks are the psuedo legal moves that they can move to in an empty board without
           the edges
13.
14.         e.g
15.         The pseudo legal moves that a rook on e4 are:
16.         ....1...
17.         ....1...
18.         ....1...
19.         ....1...
20.         1111.111
21.         ....1...
22.         ....1...
23.         ....1...
24.
25.         minus the edges:
26.         .....
27.         ....1...
28.         ....1...
29.         ....1...
30.         .111.11.
31.         ....1...
32.         ....1...
33.         .....
34.
35.         This however is not always the case as the mask for a rook on e1 is:
36.         .....
37.         ....1...
38.         ....1...
39.         ....1...
40.         ....1...
41.         ....1...
42.         ....1...
43.         .111.11.
44.
45.         */
46.
47.         public static ulong[] rookMasks = new ulong[64]; // the mask for each rooks square
48.         public static ulong[] bishopMasks = new ulong[64]; // the mask for each bishop square
49.
50.         internal static Random r = new Random(); // used for the production of magic numbers
51.
52.         public static ulong[,] directions = new ulong[64, 8];
53.
54.         #region RelevancyBits
55.
56.         /*
57.         Relevancy bits are the number of bits that can be changed when looping through each of
           the blockers

```

```

58.
59.     for a rook on e2 the rook mask would be:
60.     .....
61.     .....1..
62.     .....1..
63.     .....1..
64.     .....1..
65.     .....1..
66.     .1111.1.
67.     .....
68.
69.     if you count the number of 1's it is 10, thus there are only 10 relevent bits
70.     The 1's represent a location where a blocker could possibly be
71.
72.     */
73.
74.     public static byte[] rookRelevencyBits = {
75.         12, 11, 11, 11, 11, 11, 11, 12,
76.         11, 10, 10, 10, 10, 10, 10, 11,
77.         11, 10, 10, 10, 10, 10, 10, 11,
78.         11, 10, 10, 10, 10, 10, 10, 11,
79.         11, 10, 10, 10, 10, 10, 10, 11,
80.         11, 10, 10, 10, 10, 10, 10, 11,
81.         11, 10, 10, 10, 10, 10, 10, 11,
82.         12, 11, 11, 11, 11, 11, 11, 12
83.     };
84.
85.     public static byte[] bishopRelevencyBits = {
86.         6, 5, 5, 5, 5, 5, 5, 6,
87.         5, 5, 5, 5, 5, 5, 5, 5,
88.         5, 5, 7, 7, 7, 7, 5, 5,
89.         5, 5, 7, 9, 9, 7, 5, 5,
90.         5, 5, 7, 9, 9, 7, 5, 5,
91.         5, 5, 7, 7, 7, 7, 5, 5,
92.         5, 5, 5, 5, 5, 5, 5, 5,
93.         6, 5, 5, 5, 5, 5, 5, 6
94.     };
95.
96.     #endregion
97.
98.     #region Pre-Generated Magic numbers
99.     static readonly ulong[] RookMagicNums = {
100.         36029072039489824,
101.         5782624189288546370,
102.         144150443348476040 ,
103.         4683778865573660676,
104.         144119895662026896 ,
105.         720580411708801032 ,
106.         216211265088365056 ,
107.         4755801546350985732,
108.         9799973544901223808,
109.         2324068515031552000,
110.         140806277038208 ,
111.         162411130282115104 ,
112.         3754594787358408710,
113.         140823387832448 ,
114.         1189513286240960772,
115.         281477196611602 ,
116.         44530225123969 ,
117.         1157427578673315840,
118.         621497849179283536 ,
119.         11601292431886172168,
120.         37155246891337728 ,
121.         397583954963595776 ,
122.         576465150637377552 ,
123.         4613940017272865028 ,

```

```

124. 2900318510066468896 ,
125. 2305878194659577856 ,
126. 18150536118304 ,
127. 1153361313555028225 ,
128. 19703274141651112 ,
129. 653094515883901056 ,
130. 6751691217168 ,
131. 369311670709030980 ,
132. 1154117780077806240 ,
133. 1373741647501729792 ,
134. 900794829720523008 ,
135. 1225559651530057728 ,
136. 9224075758664680448 ,
137. 99219972248568832 ,
138. 18316253173125128 ,
139. 4908924366961508417 ,
140. 1873567815880966145 ,
141. 616219780227153920 ,
142. 2331102106812432 ,
143. 2533343512035336 ,
144. 2823546145406981 ,
145. 1750211442500567044 ,
146. 5637231283011718 ,
147. 4906127683962929156 ,
148. 5116165043808895232 ,
149. 9153435374985472 ,
150. 1450194285995507968 ,
151. 288802132936622976 ,
152. 9574547388924032 ,
153. 1243556516444242432 ,
154. 4612248973347127808 ,
155. 9232471631597732352 ,
156. 10204586778300706 ,
157. 9809403019975098658 ,
158. 1198133973571207186 ,
159. 2359921408581042433 ,
160. 9548194504113408002 ,
161. 3891391638924298257 ,
162. 9144254366212 ,
163. 288321645490470946
164.
165. };
166.
167. public static readonly ulong[] BishopMagicNums = {
168.     2670656032459719233,
169.     2891315681645586954,
170.     4505188834541572,
171.     582094694981304320,
172.     5188446662528073728,
173.     5476659758951833600,
174.     6918093096971688194,
175.     563096120984576,
176.     18119990850917380,
177.     18023229100673088,
178.     576759823794708736,
179.     36068396625829888,
180.     9965397868544,
181.     9223374240514839812,
182.     36734143124473090,
183.     23645564508325392,
184.     291610412402607616,
185.     4505835431592136,
186.     2328501779207328257,
187.     4902590578690113540,
188.     598460753510536,
189.     9301129542138634248,

```

```

190. 9224499128112910336,
191. 281486805173504,
192. 2332972359193216000,
193. 658125913573296136,
194. 114349477740616,
195. 1134704656978960,
196. 180707005948776448,
197. 4508684885460995,
198. 9223533118596712448,
199. 865818729185837440,
200. 1157471421976316928,
201. 1155340469960443400,
202. 106824429732480,
203. 10160125161842671872,
204. 18015502316376128,
205. 3467780511315837128,
206. 76562456385749632,
207. 9514421561491260048,
208. 2559697529946114,
209. 81636612491969792,
210. 9278260208161542144,
211. 9296566535915045376,
212. 15904994243379712,
213. 8215137535106482336,
214. 73342115827941440,
215. 1732199222897410080,
216. 5633108415096832,
217. 4621001639552225296,
218. 45054277870157904,
219. 2260905690139656,
220. 72198957014586500,
221. 2882339633118220288,
222. 604643580409151488,
223. 9016549402771488,
224. 563505213735936,
225. 9304439037794321152,
226. 9223460000478007296,
227. 10746070016,
228. 2453617385946534144,
229. 4954050875412185600,
230. 40547859336069376,
231. 74311661598670976,
232.
233. };
234. #endregion
235.
236. /*
237.  Rook mask
238.
239.  .....
240.  ....1..
241.  ....1..
242.  ....1..
243.  ....1..
244.  ....1..
245.  ....1..
246.  .1111.1.
247.  */
248.
249. public static ulong genrookMask(ulong rook)
250. { // creates the rook attack mask
251.     byte file = BitBoardTools.CalculateFile(rook);
252.
253.     byte rank = BitBoardTools.CalculateRank(rook);
254.
255.     ulong attack = 0;

```

```

256.
257.         attack = ((BitBoardTools.AFile >> file - 1) & ((~BitBoardTools.Rank1) &
~(BitBoardTools.Rank8))) | ((BitBoardTools.Rank1 << (8 * (rank - 1))) & (~BitBoardTools.AFile &
~(BitBoardTools.HFile)));
258.         attack &= ~rook;
259.         return attack;
260.
261.     }
262.
263.     /*
264.     .....
265.     .....
266.     .....
267.     .1.....
268.     ..1.....
269.     ...1....
270.     ....1.1.
271.     .....
272.     */
273.
274.     internal static ulong bishopNW(ulong bishop)
275.     {
276.         // Calculate Bishop north west
277.
278.         ulong attack = bishop;
279.
280.         const ulong pr1 = BitBoardTools.notHFile & (BitBoardTools.notHFile << 9);
281.         const ulong pr2 = pr1 & (pr1 << 18);
282.
283.         attack |= BitBoardTools.notHFile & (bishop << 9);
284.
285.         attack |= pr1 & (attack << 18);
286.
287.         attack |= pr2 & (attack << 36);
288.
289.         return attack ^ bishop;
290.     }
291.
292.
293.     internal static ulong bishopNE(ulong bishop)
294.     {
295.         // Calculate Bishop north west
296.
297.         ulong attack = bishop;
298.
299.         const ulong pr1 = BitBoardTools.notAFile & (BitBoardTools.notAFile << 7);
300.         const ulong pr2 = pr1 & (pr1 << 14);
301.
302.         attack |= BitBoardTools.notAFile & (bishop << 7);
303.
304.         attack |= pr1 & (attack << 14);
305.
306.         attack |= pr2 & (attack << 21);
307.
308.         return attack ^ bishop;
309.     }
310.
311.     internal static ulong bishopSW(ulong bishop)
312.     {
313.         // Calculate Bishop north west
314.
315.         ulong attack = bishop;
316.
317.         const ulong pr1 = BitBoardTools.notHFile & (BitBoardTools.notHFile >> 7);
318.         const ulong pr2 = pr1 & (pr1 >> 14);
319.

```

```

320.         attack |= BitBoardTools.notHFile & (bishop >> 7);
321.
322.         attack |= pr1 & (attack >> 14);
323.
324.         attack |= pr2 & (attack >> 21);
325.
326.         return attack ^ bishop;
327.     }
328.
329.     internal static ulong bishopSE(ulong bishop)
330.     {
331.         // Calculate Bishop north west
332.
333.         ulong attack = bishop;
334.
335.         const ulong pr1 = BitBoardTools.notAFile & (BitBoardTools.notAFile >> 9);
336.         const ulong pr2 = pr1 & (pr1 >> 18);
337.
338.         attack |= BitBoardTools.notAFile & (bishop >> 9);
339.
340.         attack |= pr1 & (attack >> 18);
341.
342.         attack |= pr2 & (attack >> 36);
343.
344.         return attack ^ bishop;
345.     }
346.
347.     public static ulong genbishopMask(ulong bishop)
348.     {
349.         return (bishopNW(bishop) | bishopNE(bishop) | bishopSW(bishop) |
350.         bishopSE(bishop)) & (~BitBoardTools.Edges);
351.     } // creates the bishop attack mask
352.
353.     // Given the mask and the occupancy we need to calculate the attacks
354.
355.     /*
356.         .....
357.         ....1..
358.         ....1..
359.         ....1..
360.         ....1..
361.         ....1..
362.         .1111.1.
363.     */
364.
365.     /// <summary>
366.     /// Perform a forward fill on the given start position and stops when the edge
367.     /// or a blocker is hit
368.     /// </summary>
369.     /// <param name="Pos">The start bitboard</param>
370.     /// <param name="blocker">The bitboard of the blocker which might obstruct the
371.     /// fill</param>
372.     /// <param name="edgemask">The edge of the board</param>
373.     /// <param name="magnitude">The direction (1 west, 8 north, 7NE, 9NW)</param>
374.     /// <returns>The resultant fill bitboard</returns>
375.     public static ulong ForwardFill(ulong Pos, ulong blocker, ulong edgemask, byte
376.     magnitude)
377.     {
378.         ulong attack = 0;
379.         byte inc = 1;
380.         bool flag = false;
381.         do
382.         {
383.             ulong temp = Pos << (magnitude * inc);
384.             if ((temp & edgemask) > 0 || (temp & blocker) > 0) flag = true;

```

```

382.         attack |= temp;
383.         inc++;
384.
385.     } while (!flag);
386.
387.     return attack;
388. }
389.
390.     /// <summary>
391.     /// Perform a backwards fill on the given start position and stops when the edge
    or a blocker is hit
392.     /// </summary>
393.     /// <param name="Pos">The start bitboard</param>
394.     /// <param name="blocker">The bitboard of the blocker which might obstruct the
    fill</param>
395.     /// <param name="edgemask">The edge of the board</param>
396.     /// <param name="magnitude">The direction (1 east, 8 south, 7 SW, 9 SE)</param>
397.     /// <returns>The resultant fill bitboard</returns>
398.     public static ulong BackwardFill(ulong Pos, ulong blocker, ulong edgemask, byte
    magnitude)
399.     {
400.         ulong attack = 0; // the bitboard generated from the fill
401.         byte inc = 1;
402.         bool flag = false;
403.         do
404.         {
405.
406.             ulong temp = Pos >> (magnitude * inc);
407.
408.             // check if we have hit the edge or a blocker
409.             if ((temp & edgemask) > 0 || (temp & blocker) > 0 || temp <= 0) flag =
    true; // stop the fill
410.
411.             attack |= temp;
412.             inc++;
413.
414.         } while (!flag);
415.
416.         return attack;
417.     }
418.
419.     public static ulong genRookAttack(ulong rookPos, ulong blockers)
420.     {
421.         // bitscan in a certain direction until we hit a blocker
422.         ulong attack = 0;
423.         byte file = BitBoardTools.CalculateFile(rookPos);
424.         byte rank = BitBoardTools.CalculateRank(rookPos);
425.
426.         // NorthFill +8 (8th rank)
427.         if (rank != 8)
428.         {
429.             attack |= ForwardFill(rookPos, blockers, BitBoardTools.Rank8, 8);
430.         }
431.
432.         if (rank != 1)
433.         {
434.             // SouthFill -8 (1st rank)
435.             attack |= BackwardFill(rookPos, blockers, BitBoardTools.Rank1, 8);
436.         }
437.
438.         if (file != 1)
439.         {
440.             // WestFill forward +1 (A file)
441.             attack |= ForwardFill(rookPos, blockers, BitBoardTools.AFile, 1);
442.         }
443.

```

```

444.         if (file != 8)
445.         {
446.             // EastFill -1 H File
447.             attack |= BackwardFill(rookPos, blockers, BitBoardTools.HFile, 1);
448.         }
449.
450.         return attack;
451.     }
452.
453.     public static ulong genBishopAttack(ulong bishopPos, ulong blockers)
454.     {
455.         // bitscan in a certain direction until we hit a blocker
456.         ulong attack = 0;
457.         byte file = BitBoardTools.CalculateFile(bishopPos);
458.         byte rank = BitBoardTools.CalculateRank(bishopPos);
459.
460.         if (rank != 8 && file != 8)
461.         {
462.             // NorthEastFill forward 7
463.             attack |= ForwardFill(bishopPos, blockers, BitBoardTools.Edges, 7);
464.         }
465.
466.         if (file != 8 && rank != 1) // if the bishop is on the HFile there is no
            need to generate the East fills
467.         {
468.             // SouthEastFill backwards 9
469.             attack |= BackwardFill(bishopPos, blockers, BitBoardTools.Edges, 9);
470.         }
471.
472.         if (file != 1 && rank != 1)
473.         {
474.             // SouthWestFill backwards 7
475.             attack |= BackwardFill(bishopPos, blockers, BitBoardTools.Edges, 7);
476.         }
477.
478.         if (file != 1 && rank != 8)
479.         { // if the bishop is on the AFile there is no need to generate the West
            fills
480.             // NorthWestFill forward 9
481.             attack |= ForwardFill(bishopPos, blockers, BitBoardTools.Edges, 9);
482.         }
483.
484.
485.         return attack;
486.     }
487.
488.     // each magic corresponds to one square and acts as a hashing function to
489.     // create a unique index for each blocking pattern
490.     // Generate a magic number, hash every blocker possibility and then assign a
491.     // index to each
492.     // if there is a collision then the magic number is not magic and we need to
493.     // regenerate a new one and clear the table
494.     // To reduce the size of the magic table we only use 2^(relevant bits)
495.     // for the size of the look up array; that is why jagged arrays are used
496.     // This will take more computation but once we have all the magic numbers we
497.     // dont need to regenerate them just use them
498.
499.     public static bool tryGenTable(ulong magic, byte relevancy, ulong mask, byte
500.     index, ref ulong[][] table, bool isRook)
501.     {
502.         // try each variation for a given mask

```



```

503.         {
504.             // calculate magic index
505.             int magicIndex = (int)((occ * magic) >> (64 - relevancy));
506.
507.             // calculate attacks
508.             ulong attacks;
509.
510.             if (isRook)
511.             {
512.                 attacks = genRookAttack(Pos, occ);
513.             }
514.
515.             else
516.             {
517.                 attacks = genBishopAttack(Pos, occ);
518.             }
519.
520.             // check if the index is negative
521.             if (magicIndex < 0)
522.             {
523.                 Array.Clear(table[index], 0, table[index].Length);
524.                 return false;
525.             }
526.
527.             else if (table[index][magicIndex] == 0)
528.             {
529.                 table[index][magicIndex] = attacks;
530.             }
531.
532.             // Check for positive collision (different magic numbers points to the
same attacking bitboard)
533.             else if (table[index][magicIndex] != attacks)
534.             {
535.                 Array.Clear(table[index], 0, table[index].Length);
536.                 return false;
537.             }
538.
539.             occ = (occ - mask) & mask;
540.
541.             } while (occ > 0);
542.
543.             return true;
544.         }
545.
546.         public static ulong getRookBlockerAttacks(ulong pos, ulong blockers)
547.         {
548.             int index = BitBoardTools.BitboardToIndex(pos);
549.             ulong magic = RookMagicNums[index];
550.             byte relevancy = rookRelevancyBits[index];
551.             blockers = blockers & rookMasks[index];
552.             int magicIndex = (int)((blockers * magic) >> (64 - relevancy));
553.
554.             return RookMagic[index][magicIndex];
555.
556.         }
557.
558.         public static ulong getBishopBlockerAttacks(ulong pos, ulong blockers)
559.         {
560.             int index = BitBoardTools.BitboardToIndex(pos);
561.             ulong magic = BishopMagicNums[index];
562.             byte relevancy = bishopRelevancyBits[index];
563.             blockers = blockers & bishopMasks[index];
564.             int magicIndex = (int)((blockers * magic) >> (64 - relevancy));
565.             return BishopMagic[index][magicIndex];
566.         }
567.

```

```

568.     public static ulong getRookBlockerAttacks(byte index, ulong blockers)
569.     //overload for index
570.     {
571.         ulong magic = RookMagicNums[index];
572.         byte relevancy = rookRelevancyBits[index];
573.         blockers = blockers & rookMasks[index];
574.         int magicIndex = (int)((blockers * magic) >> (64 - relevancy));
575.         return RookMagic[index][magicIndex];
576.     }
577.     public static ulong getBishopBlockerAttacks(byte index, ulong blockers)
578.     //overload for index
579.     {
580.         ulong magic = BishopMagicNums[index];
581.         byte relevancy = bishopRelevancyBits[index];
582.         blockers = blockers & bishopMasks[index];
583.         int magicIndex = (int)((blockers * magic) >> (64 - relevancy));
584.         return BishopMagic[index][magicIndex];
585.     }
586.     public static ulong findPossibleMagic()
587.     {
588.         return (genRandom64() & genRandom64() & genRandom64());
589.     }
590.
591.     public static ulong genRandom64()
592.     {
593.         ulong u1, u2, u3, u4;
594.         u1 = (ulong)(r.Next()) & 0xFFFF;
595.         u2 = (ulong)(r.Next()) & 0xFFFF;
596.         u3 = (ulong)(r.Next()) & 0xFFFF;
597.         u4 = (ulong)(r.Next()) & 0xFFFF;
598.
599.         return u1 | (u2 << 16) | (u3 << 32) | (u4 << 48);
600.     }
601.
602.     static Magic()
603.     {
604.
605.         bool flag = false;
606.
607.         // set up the direction maks
608.         for (byte i = 0; i < 64; i++) // loop over each square
609.         { // {N,E,S,W,NE,SE,SW,NW}
610.
611.             ulong loc = BitBoardTools.IndexToBitboard(i);
612.             byte file = BitBoardTools.CalculateFile(loc);
613.             byte rank = BitBoardTools.CalculateRank(loc);
614.
615.             if (rank != 8)
616.             {
617.                 directions[i, 0] = ForwardFill(loc, 0, BitBoardTools.Rank8, 8);
618.             }
619.             if (rank != 1)
620.             {
621.                 // SouthFill -8 (1st rank)
622.                 directions[i, 2] = BackwardFill(loc, 0, BitBoardTools.Rank1, 8);
623.             }
624.             if (file != 1)
625.             {
626.                 // WestFill forward +1 (A file)
627.                 directions[i, 3] = ForwardFill(loc, 0, BitBoardTools.AFile, 1);
628.             }
629.             if (file != 8)
630.             {
631.                 // EastFill -1 H File

```

```

632.         directions[i, 1] = BackwardFill(loc, 0, BitBoardTools.HFile, 1);
633.     }
634.
635.
636.         // NorthEastFill forwards 7 HFile || Rank8
637.
638.         if (rank != 8 && file != 8)
639.         {
640.             // NorthEastFill forward 7
641.             directions[i, 4] = ForwardFill(loc, 0, BitBoardTools.Edges, 7);
642.         }
643.
644.         if (file != 8 && rank != 1) // if the bishop is on the HFile there is
no need to generate the East fills
645.         {
646.             // SouthEastFill backwards 9
647.             directions[i, 5] = BackwardFill(loc, 0, BitBoardTools.Edges, 9);
648.         }
649.
650.         if (file != 1 && rank != 1)
651.         {
652.             // SouthWestFill backwards 7
653.             directions[i, 6] = BackwardFill(loc, 0, BitBoardTools.Edges, 7);
654.         }
655.
656.         if (file != 1 && rank != 8)
657.         { // if the bishop is on the AFile there is no need to generate the
West fills
658.             // NorthWestFill forward 9
659.             directions[i, 7] = ForwardFill(loc, 0, BitBoardTools.Edges, 9);
660.         }
661.     }
662.
663.         // ##### GEN MAGIC TABLES #####
664.
665.         // Bishops
666.         for (int i = 0; i < 64; i++) // repeat for each square
667.         {
668.             // ulong magic; // if you want to generate new magic numbers
669.             ulong magic = BishopMagicNums[i];
670.             ulong mask = genbishopMask(BitBoardTools.IndexToBitboard((byte)i));
671.             bishopMasks[i] = mask;
672.             byte relevancy = bishopRelevancyBits[i];
673.             BishopMagic[i] = new ulong[BitBoardTools.IndexToBitboard(relevancy)];
674.
675.             do
676.             {
677.                 // magic = findPossibleMagic(); // if you want to generate new
magic numbers
678.                 flag = tryGenTable(magic, relevancy, mask, (byte)i, ref
BishopMagic, false);
679.             } while (!flag);
680.
681.             BishopMagicNums[i] = magic;
682.         }
683.
684.         // Rooks
685.         for (int i = 0; i < 64; i++) // repeat for each square for rooks
686.         {
687.             // ulong magic; // if you want to generate new magic numbers
688.             ulong magic = RookMagicNums[i];
689.             ulong mask = genrookMask(BitBoardTools.IndexToBitboard((byte)i));
690.             rookMasks[i] = mask;
691.             byte relevancy = rookRelevancyBits[i];
692.             RookMagic[i] = new ulong[BitBoardTools.IndexToBitboard(relevancy)];
693.

```

```

694.         do
695.         {
696.             // magic = findPossibleMagic(); // if you want to generate new
        magic numbers
697.             flag = tryGenTable(magic, relevancy, mask, (byte)i, ref RookMagic,
        true);
698.         } while (!flag);
699.
700.         RookMagicNums[i] = magic;
701.     }
702. }
703. }
704. }

```

Move

```

1. namespace Engine
2. {
3.
4.     // Each move is represented as a 16bit ushort
5.     // Bits 1-6 will represent the starting square
6.     // Bits 6-12 will represent the destination square
7.     // Bits 12-16 will represent the flag
8.
9.     // For example to move a pawn from e2 to e4 the move would be represented as the following:
10.
11.     // 0010 11011 01011
12.     // enpassant set flag -- 27 is destination -- 11 is the starting square
13.     public struct Move
14.     {
15.         readonly ushort move;
16.
17.
18.         /*
19.         4 bits to play around with max value is 1111 = 15
20.
21.         0000 - 0 - no flag
22.         0001 - 1 - Enpassant capture
23.         0010 - 2 - Castling
24.         0011 - 3 - EnPassant Set
25.
26.         Promotion
27.         0100 - 4 - Knight
28.         0101 - 5 - Bishop
29.         0110 - 6 - Rook
30.         0111 - 7 - Queen
31.
32.
33.         */
34.
35.         const ushort startIndexMask = 0b111111;
36.         const ushort destinationIndexMask = 0b111111_000000;
37.         const ushort flagMask = 0b1111_000000_000000;
38.         public const ushort enPassantCapture = 1;
39.         public const ushort castling = 2;
40.         public const ushort enPassantSet = 3;
41.
42.         public Move(bool empty) // empty move the parameter is because c# does not support
        parameterless structs
43.         {
44.             this.move = 0;
45.         }
46.         public Move(ushort moveValue)
47.         {

```

```

48.         this.move = moveValue;
49.     }
50.
51.     public Move(byte start, byte destination) // we know it is a quiet move so we know the
flag will be 0
52.     {
53.         move = (ushort)(start | (destination << 6));
54.     }
55.
56.     public Move(ulong start, ulong destination) // if we need to pass through the bitboards
instead of the index values
57.     {
58.         start = BitBoardTools.BitboardToIndex(start);
59.         destination = BitBoardTools.BitboardToIndex(destination);
60.         move = (ushort)(start | (destination << 6));
61.     }
62.
63.     public Move(byte start, byte destination, ushort flag) // overloading for flags
64.     {
65.         move = (ushort)(start | (destination << 6) | (flag << 12));
66.     }
67.
68.     public Move(ulong start, ulong destination, ushort flag)
69.     {
70.         start = BitBoardTools.BitboardToIndex(start);
71.         destination = BitBoardTools.BitboardToIndex(destination);
72.
73.         move = (ushort)(start | (destination << 6) | ((ulong)flag << 12));
74.     }
75.
76.     public byte startIndex => (byte)(move & startIndexMask);
77.
78.     public byte destinationIndex => (byte)((move & destinationIndexMask) >> 6);
79.
80.     public byte flag => (byte)((move & flagMask) >> 12);
81.
82.     public bool isPromotion => (flag > 3 && flag < 8);
83.
84.     public int pieceIndex => (flag & 3);
85.
86.     public bool isEmpty => (move == 0);
87.
88.     }
89. }

```

PolyGlott

PolyEntry

```

1. namespace Engine.PolyGlott
2. {
3.     internal struct PolyEntry
4.     {
5.         #region values
6.         internal ulong key;
7.         internal ushort move;
8.         internal ushort weight;
9.         internal uint learn;
10.        #endregion
11.
12.        #region masks
13.        private const ushort destFileMask = 0b111;
14.        private const ushort destRowMask = 0b111_000;

```

```

15.     private const ushort startFileMask = 0b111_000_000;
16.     private const ushort startRowMask = 0b111_000_000_000;
17.     private const ushort promotionMask = 0b111_000_000_000_000;
18.     #endregion
19.
20.     internal PolyEntry(ulong Key, ushort Move, ushort Weight, uint Learn)
21.     {
22.         key = Key;
23.         move = Move;
24.         weight = Weight;
25.         learn = Learn;
26.     }
27.
28.     #region Functions
29.     public byte destinationFile => (byte)(move & destFileMask);
30.
31.     public byte destinationRow => (byte)((move & destRowMask) >> 3);
32.
33.     public byte startFile => (byte)((move & startFileMask) >> 6);
34.
35.     public byte startRow => (byte)((move & startRowMask) >> 9);
36.
37.     public int pieceIndex => (byte)((move & startRowMask) >> 12) + 2;
38.
39.     public bool isPromotion => ((byte)((move & promotionMask) >> 12) > 0);
40.     #endregion
41. }
42. }

```

Polyglot

```

1. using System;
2. using System.IO;
3.
4. namespace Engine.PolyGlot
5. {
6.
7.     /// <summary>
8.     /// A static class which (should) handle any polyglot chess book
9.     /// </summary>
10.    /// Note: This code reads the entire book into memory so do not use very large books
11.    /// The polyglot format is open source and under the GPL License.
12.    internal static class Polyglot
13.    {
14.        static string bookPath; // the current path of the book which is being read
15.        private static PolyEntry[] entries; //the array of all of the opening book entries
16.        private static Random randomGen = new Random();
17.
18.        public static string BookPath
19.        {
20.            get { return bookPath; }
21.            set
22.            {
23.                // gets the dynamic path of the openings folder
24.                string root =
Directory.GetParent(Directory.GetCurrentDirectory()).FullName.Split(new string[] { "UI",
"Engine" }, StringSplitOptions.None)[0];
25.                // gets the user input name
26.                root = Path.Combine(root, "Openings", $"{value}.bin");
27.
28.                if (File.Exists(root))
29.                {
30.                    bookPath = root;

```

```

31.         readBook();
32.     }
33.     else
34.     {
35.         throw new Exception("Book name was not found");
36.     }
37. }
38. }
39. #region Endian Swaps
40.
41.
42.     //Endian Swap algorithms found
on:https://stackoverflow.com/questions/19560436/bitwise-endian-swap-for-various-types
43.
44.
45.     static ulong EndianSwapU64(ulong x)
46.     {
47.         // swap adjacent 32-bit blocks
48.         x = (x >> 32) | (x << 32);
49.         // swap adjacent 16-bit blocks
50.         x = ((x & 0xFFFF0000FFFF0000) >> 16) | ((x & 0x0000FFFF0000FFFF) << 16);
51.         // swap adjacent 8-bit blocks
52.         return ((x & 0xFF00FF00FF00FF00) >> 8) | ((x & 0x00FF00FF00FF00FF) << 8);
53.     }
54.
55.
56.     static uint EndianSwapU32(uint x)
57.     {
58.         // swap adjacent 16-bit blocks
59.         x = (x >> 16) | (x << 16);
60.         // swap adjacent 8-bit blocks
61.         return ((x & 0xFF00FF00) >> 8) | ((x & 0x00FF00FF) << 8);
62.     }
63.
64.     static ushort EndianSwapU16(ushort x)
65.     {
66.         x = (ushort)(
67.             (x >> 8) |
68.             (x << 8));
69.         return x;
70.     }
71. #endregion
72.
73.     /// <summary>
74.     /// Opens the current value of bookPath and tries to create the opening book
75.     /// </summary>
76.     static void readBook()
77.     {
78.         byte[] bytes = File.ReadAllBytes(bookPath);
79.
80.         // each entry is 16 bytes long
81.         int numEntries = bytes.Length / 16;
82.
83.         // create a new opening book array
84.         entries = new PolyEntry[numEntries];
85.
86.         int i = 0;
87.
88.         for (int j = 0; j < numEntries; j++)
89.         {
90.             // first 8 bytes are the key
91.             // 2 bytes for the move
92.             // 2 bytes for the weight
93.             // 4 bytes for the learn
94.             // the infomation is in the big endian format so it needs to be converted.
95.

```

```

96.         ulong key = BitConverter.ToUInt64(bytes, i);
97.         key = EndianSwapU64(key);
98.
99.         ushort move = BitConverter.ToUInt16(bytes, i + 8);
100.        move = EndianSwapU16(move);
101.
102.        ushort weight = BitConverter.ToUInt16(bytes, i + 10);
103.        weight = EndianSwapU16(weight);
104.
105.        uint learn = BitConverter.ToUInt32(bytes, i + 12);
106.
107.        // store the information
108.        entries[i / 16] = new PolyEntry(key, move, weight, learn);
109.
110.        // increment the index by 16 bytes
111.        i += 16;
112.    }
113.}
114.
115.    /// <summary>
116.    /// returns the index of the first element in the opening book which is
associated with the given key
117.    /// </summary>
118.    /// <param name="startindex">The index of the first found entry</param>
119.    /// <param name="key">The key of the position to lookup</param>
120.    /// <returns></returns>
121.    static int findStartIndex(int startindex, ulong key)
122.    {
123.        bool stop = false;
124.
125.        do
126.        {
127.            // check if the index is outside the bounds of the array
128.            if (startindex < 0)
129.            {
130.                stop = true;
131.                startindex = 0;
132.            }
133.            // check if the entry has the desired key
134.            else if (entries[startindex].key == key)
135.            {
136.                startindex--;
137.            }
138.            // the element does not so the element after this must be the start of
the instances
139.            else
140.            {
141.                stop = true;
142.                startindex++;
143.            }
144.        } while (!stop);
145.        return startindex;
146.    }
147.
148.    /// <summary>
149.    /// Finds the range of the entries with the desired key
150.    /// </summary>
151.    /// <param name="startIndex">The index which is where the entries start</param>
152.    /// <param name="key">The desired key of the position</param>
153.    /// <returns>The number of entries with the desired key</returns>
154.    static int findRange(int startIndex, ulong key)
155.    {
156.        int counter = 0;
157.        // loop through until the key does not match
158.        for (int i = startIndex; i < entries.Length; i++)
159.        {

```



```

160.         // check if the key matches
161.         if (entries[i].key == key)
162.         {
163.             counter++;
164.         }
165.
166.         else
167.         {
168.             // it does not so we can return
169.             return counter;
170.         }
171.     }
172.     return counter;
173. }
174.
175.     /// <summary>
176.     /// Loops through at the start index and outputs the entry information
177.     /// </summary>
178.     /// <param name="Startindex">The index which is where the entries start</param>
179.     /// <param name="key">The desired key of the position</param>
180.     static void listBook(int Startindex, ulong key)
181.     {
182.         // find true startpoint
183.
184.         Startindex = findStartIndex(Startindex, key);
185.
186.         for (int i = Startindex; i < entries.Length; i++)
187.         {
188.             if (entries[i].key == key)
189.             {
190.                 // get the move information
191.                 PolyEntry move = entries[i];
192.
193.                 // convert it to the move struct
194.                 int fromLoc = 8 * move.startRow + (7 - move.startFile);
195.                 int toLoc = 8 * move.destinationRow + (7 - move.destinationFile);
196.                 ushort promotion = (ushort)move.pieceIndex;
197.
198.                 Move temp = new Move((byte)fromLoc, (byte)toLoc, promotion);
199.
200.                 // output the information
201.                 Console.WriteLine($"bookinfo weight {move.weight} move
{Tools.MoveToString(temp)}");
202.
203.                 if (CommandHandler.Debug)
204.                 {
205.                     Console.Write($"Key: {move.key.ToString("X")}  ");
206.                     Console.Write($"Move: {Tools.MoveToString(temp)}  ");
207.                     Console.Write($"Index: {i}");
208.                     Console.Write($"weight {move.weight}\n");
209.                 }
210.             }
211.             // if the element if not the desired key we can just return
212.             else
213.             {
214.                 return;
215.             }
216.         }
217.     }
218.
219.     /// <summary>
220.     /// Performs a recursive binary search to find an element with the desired key
    in the entry array
221.     /// </summary>
222.     /// <param name="left">The leftmost index of the current iteration (when
    calling it is 0)</param>

```

```

223.          /// <param name="right">The rightmost index of the current iteration (when
    calling it is the last index of the array)</param>
224.          /// <param name="desiredKey"></param>
225.          /// <returns>The index of the element or -1 if one is not found</returns>
226.          static int binarySearch(int left, int right, ulong desiredKey)
227.          {
228.              //
229.              if (right >= left)
230.              {
231.                  // find the middle of the array
232.                  int mid = left + (right - left) / 2;
233.
234.                  // check if the middle is the desired element
235.                  if (entries[mid].key == desiredKey)
236.                      return mid;
237.                  // if the probed key is higher then we know that all the elements are
    irrelevant
238.                  if (entries[mid].key > desiredKey)
239.                  // make the rightmost index the mid -1 as we know the desired
    element is not at the mid indexv
240.                      return binarySearch(left, mid - 1, desiredKey);
241.
242.                  // we know the opposite so we look at everything after mid
243.                  return binarySearch(mid + 1, right, desiredKey);
244.              }
245.
246.              // a value was not found so return -1
247.              return -1;
248.          }
249.
250.          /// <summary>
251.          /// Lists all of the book moves for a position, if one was not found then a
    error message is shown
252.          /// </summary>
253.          /// <param name="desiredKey">The key of the position that is to be found in the
    book</param>
254.          public static void showBookMoves(ulong desiredKey)
255.          {
256.              int index = binarySearch(0, entries.Length, desiredKey);
257.              if (index == -1)
258.              {
259.                  Console.WriteLine("Could not find");
260.                  return;
261.              }
262.              listBook(index, desiredKey);
263.
264.              return;
265.          }
266.
267.          /// <summary>
268.          /// Probes the book and finds the best entry
269.          /// </summary>
270.          /// <param name="kingloc"></param>
271.          /// <param name="desiredKey"></param>
272.          /// <returns>The best opening book move</returns>
273.          public static Move queryBook(int kingloc, ulong desiredKey)
274.          {
275.              // perform a binary search to find an index
276.              int index = binarySearch(0, entries.Length, desiredKey);
277.
278.              if (index == -1) // a entry was not found so a empty move is returned
279.              {
280.                  return new Move(true);
281.              }
282.
283.              int startIndex = findStartIndex(index, desiredKey);

```

```

284.
285.         int bestWeight = -10000;
286.         Move bestMove = new Move(true);
287.         int counter = 0;
288.         try
289.         {
290.             // check if the book uses the same weight for all of the moves
291.             if (entries[startIndex].weight == entries[startIndex + 1].weight)
292.             {
293.                 // pick a random move which is in the range
294.                 int length = findRange(startIndex, desiredKey);
295.                 index = randomGen.Next(startIndex, startIndex + length);
296.             }
297.
298.             else
299.             {
300.                 // pick the first move as it is stored in decending weight order
301.                 index = startIndex;
302.             }
303.         }
304.         catch
305.         {
306.             // if something goes wrong return the first index
307.             index = startIndex;
308.         }
309.
310.         // get the move information (the start and destination index)
311.         PolyEntry entry = entries[index];
312.         int fromLoc = 8 * entry.startRow + (7 - entry.startFile);
313.         int toLoc = 8 * entry.destinationRow + (7 - entry.destinationFile);
314.
315.         // castling moves are in a untradiditonal format so we need to correct this
316.         (e1h1 replace with e1g1)
317.         if (fromLoc == kingloc)
318.         {
319.             // check if it is catling king side
320.             if (entry.destinationFile == 7)
321.             {
322.                 toLoc = 8 * entry.destinationRow + (8 - entry.destinationFile);
323.             }
324.             else if (entry.destinationFile == 0)
325.             {
326.                 toLoc = 8 * entry.destinationRow + (5 - entry.destinationFile);
327.             }
328.         }
329.
330.         // check if it is a promotion
331.         ushort promotion = (ushort)entry.pieceIndex;
332.
333.         return new Move((byte)fromLoc, (byte)toLoc, promotion);
334.
335.     }
336.
337.
338.
339.     static Polyglot()
340.     {
341.         bookPath =
342.         Directory.GetParent(Directory.GetCurrentDirectory()).FullName.Split(new string[] { "UI",
343.         "Engine" }, StringSplitOptions.None)[0];
344.         bookPath = Path.Combine(bookPath, "Openings", "codekiddy.bin");
345.         readBook();
346.     }

```

```

347.
348.      /*
349.      Books with Weight:
350.      codekiddy.bin
351.      DCbook_large.bin
352.      Elo2400.bin
353.      final-book.bin
354.      gm2600.bin
355.      SmallBook.bin
356.      Titans.bin
357.
358.      castling moves are encoded as e1h1 not e1g1 which is annoying.
359.      */

```

Keys

```

1.  0x9D39247E33776D41, 0x2AF7398005AAA5C7, 0x44DB015024623547, 0x9C15F73E62A76AE2,
2.  0x75834465489C0C89, 0x3290AC3A203001BF, 0x0FBBAD1F61042279, 0xE83A908FF2FB60CA,
3.  0x0D7E765D58755C10, 0x1A083822CEAFE02D, 0x9605D5F0E25EC3B0, 0xD021FF5CD13A2ED5,
4.  0x40BDF15D4A672E32, 0x011355146FD56395, 0x5DB4832046F3D9E5, 0x239F8B2D7FF719CC,
5.  0x05D1A1AE85B49AA1, 0x679F848F6E8FC971, 0x7449BBFF801FED0B, 0x7D11CDB1C3B7ADF0,
6.  0x82C7709E781EB7CC, 0xF3218F1C9510786C, 0x331478F3AF51BBE6, 0x4BB38DE5E7219443,
7.  0xAA649C6EBCFD50FC, 0x8DBD98A352AFD40B, 0x87D2074B81D79217, 0x19F3C751D3E92AE1,
8.  0xB4AB30F062B19ABF, 0x7B0500AC42047AC4, 0xC9452CA81A09D85D, 0x24AA6C514DA27500,
9.  0x4C9F34427501B447, 0x14A68FD73C910841, 0xA71B9B83461CBD93, 0x03488B95B0F1850F,
10. 0x637B2B34FF93C040, 0x09D1BC9A3DD90A94, 0x3575668334A1DD3B, 0x735E2B97A4C45A23,
11. 0x1872707F1BD400B, 0x1FCBACD259BF02E7, 0xD310A7C2CE9B6555, 0xBF983FE0FE5D8244,
12. 0x9F74D14F7454A824, 0x51EBDC4AB9BA3035, 0x5C82C505DB9AB0FA, 0xFCF7FE8A3430B241,
13. 0x3253A729B9BA3DDE, 0x8C74C368081B3075, 0xB9BC6C87167C33E7, 0x7EF48F2B83024E20,
14. 0x11D505D4C351BD7F, 0x6568FCA92C76A243, 0x4DE0B0F40F32A7B8, 0x96D693460CC37E5D,
15. 0x42E240CB63689F2F, 0x6D2BDCDAE2919661, 0x42880B0236E4D951, 0x5F0F4A5898171BB6,
16. 0x39F890F579F92F88, 0x93C5B5F47356388B, 0x63DC359D8D231B78, 0xEC16CA8AEA98AD76,
17. 0x5355F90C2A82DC7, 0x07FB9F855A997142, 0x5093417AA8A7ED5E, 0x7BCBC38DA25A7F3C,
18. 0x19FC8A768CF4B6D4, 0x637A7780DEFC0D9, 0x8249A47AE0E41F7, 0x79AD695501E7D1E8,
19. 0x14ACBAF4777D5776, 0xF145B6BECCDEA195, 0xDABF2AC8201752FC, 0x24C3C94DF9C8D3F6,
20. 0xBB6E2924F03912EA, 0x0CE26C0B95C980D9, 0xA49CD132BFBF7CC4, 0xE99D662AF4243939,
21. 0x27E6AD7891165C3F, 0x8535F040B9744FF1, 0x54B3F4FA5F40D873, 0x72B12C32127FED2B,
22. 0xEE954D3C7B411F47, 0x9A85AC909A24EAA1, 0x70AC4CD9F04F21F5, 0xF9B89D3E99A075C2,
23. 0x87B3E2B2B5C907B1, 0xA366E5B8C54F48B8, 0xAE4A9346CC3F7CF2, 0x1920C04D47267BBD,
24. 0x87BF02C6B49E2AE9, 0x092237AC237F3859, 0xFF07F64EF8ED14D0, 0x8DE8DCA9F03CC54E,
25. 0x9C1633264DB49C89, 0xB3F22C3D0B0B38ED, 0x390E5FB44D01144B, 0x5BFEA5B4712768E9,
26. 0x1E1032911FA78984, 0x9A74ACB964E78CB3, 0x4F80F7A035DAFB04, 0x6304D09A0B3738C4,
27. 0x2171E64683023A08, 0x5B9B63EB9CEFF80C, 0x506AACF489889342, 0x1881AF9A3A701D6,
28. 0x6503080440750644, 0xDFD395339CDBF4A7, 0xEF927DBCF00C20F2, 0x7B32F7D1E03680EC,
29. 0xB9FD7620E7316243, 0x05A7E8A57DB91B77, 0xB5889C6E15630A75, 0x4A750A09CE9573F7,
30. 0xCF464CEC899A2F8A, 0xF538639CE705B824, 0x3C79A0FF5580EF7F, 0xEDE6C87F8477609D,
31. 0x799E81F05BC93F31, 0x86536B8CF3428A8C, 0x97D7374C60087B73, 0xA246637CFF328532,
32. 0x043FCAE60CC0EBA0, 0x920E449535DD359E, 0x70EB093B15B290CC, 0x73A1921916591CBD,
33. 0x56436C9FE1A1AA8D, 0xEFAC4B70633B8F81, 0xBB215798D45DF7AF, 0x45F2B0042F24F1768,
34. 0x930F80F4E8EB7462, 0xFF6712FFCFD75EA1, 0xAE623FD67468AA70, 0xDD2C5BC84BC8D8FC,
35. 0x7EED120D54CF2DD9, 0x22FE545401165F1C, 0xC91800E98FB99929, 0x808BD68E6AC10365,
36. 0xDEC468145B7605F6, 0x1BEDE3A3AEF53302, 0x43539603D6C55602, 0xAA969B5C691CCB7A,
37. 0xA87832D392EFEE56, 0x65942C7B3C7E11AE, 0xDE2D633CAD004F6, 0x21F08570F420E565,
38. 0xB415938D7DA9AE3C, 0x91B859E59ECB6350, 0x10CFF333E0ED804A, 0x28AED140BE0BB7DD,
39. 0xC5CC1D89724FA456, 0x5648F680F11A2741, 0x2D255069F0B7DAB3, 0x9BC5A38EF729ABD4,
40. 0xEF2F054308F6A2BC, 0xAF2042F5CC5C2858, 0x480412BAB7F5BE2A, 0xAEF3AF4A563DFE43,
41. 0x19AFE59AE451497F, 0x52593803DFF1E840, 0xF4F076E65F2CE6F0, 0x11379625747D5AF3,
42. 0xBCE5D2248682C115, 0x9DA4243DE836994F, 0x066F70B33FE09017, 0x4DC4DE189B671A1C,
43. 0x51039AB7712457C3, 0xC07A3F80C31FB4B4, 0xB46EE9C5E64A6E7C, 0xB3819A42ABE61C87,
44. 0x21A007933A522A20, 0x2DF16F761598AA4F, 0x763C4A1371B368FD, 0xF793C46702E086A0,
45. 0xD7288E012AEB8D31, 0xDE336A2A4BC1C44B, 0x0BF692B38D079F23, 0x2C604A7A177326B3,
46. 0x4850E73E03EB6064, 0xCFC447F1E53C8E1B, 0xB05CA3F564268D99, 0x9AE182C8BC9474E8,

```

47. 0xA4FC4BD4FC5558CA, 0xE755178D58FC4E76, 0x69B97DB1A4C03DFE, 0xF9B5B7C4ACC67C96,
 48. 0xFC6A82D64B8655FB, 0x9C684CB6C4D24417, 0x8EC97D2917456ED0, 0x6703DF9D2924E97E,
 49. 0xC547F57E42A7444E, 0x78E37644E7CAD29E, 0xFE9A44E9362F05FA, 0x08BD35CC38336615,
 50. 0x9315E5EB3A129ACE, 0x94061B871E04DF75, 0xDF1D9F9D784BA010, 0x3BBA57B68871B59D,
 51. 0xD2B7ADEEDED1F73F, 0xF7A255D83BC373F8, 0xD7F4F2448C0CEB81, 0xD95BE88CD210FFA7,
 52. 0x336F52F8FF4728E7, 0xA74049DAC312AC71, 0xA2F61BB6E437FDB5, 0x4F2A5CB07F6A35B3,
 53. 0x87D380BDA5BF7859, 0x16B9F7E06C453A21, 0x7BA2484C8A0FD54E, 0xF3A678CAD9A2E38C,
 54. 0x39B0BF7DDE437BA2, 0xFCF55C1BF8A4424, 0x18FCF680573FA594, 0x4C0563B89F495AC3,
 55. 0x40E087931A00930D, 0x8CFFA9412EB642C1, 0x68CA39053261169F, 0x7A1EE967D27579E2,
 56. 0x9D1D60E5076F5B6F, 0x3810E399B6F65BA2, 0x32095B6D4AB5F9B1, 0x35CAB62109DD038A,
 57. 0xA90B24499FCFAFB1, 0x77A225A07CC2C6BD, 0x513E5E634C70E331, 0x4361C0CA3F692F12,
 58. 0xD941ACA44B20A45B, 0x528F7C8602C5807B, 0x52AB92BEB9613989, 0x9D1DFA2EFC557F73,
 59. 0x722FF175F572C348, 0x1D1260A51107FE97, 0x7A249A57EC0C9BA2, 0x04208FE9E8F7F2D6,
 60. 0x5A110C6058B920A0, 0x0CD9A497658A5698, 0x56FD23C8F9715A4C, 0x284C847B9D887AAE,
 61. 0x04FEABFBDB619CB, 0x742E1E651C60BA83, 0x9A9632E65904AD3C, 0x881B82A13B51B9E2,
 62. 0x506E6744CD974924, 0xB0183DB56FFC6A79, 0x0ED9B915C66ED37E, 0x5E11E86D5873D484,
 63. 0xF678647E3519AC6E, 0x1B85D48D0F20CC5, 0xDAB9FE6525D89021, 0x0D151D86ADB73615,
 64. 0xA865A54EDCC0F019, 0x93C42566AEF98FFB, 0x99E7AFEAEBE000731, 0x48CBFF086DDF285A,
 65. 0x7F9B6AF1EBF78BAF, 0x58627E1A149BBA21, 0x2CD16E2ABD791E33, 0xD363EFF5F0977996,
 66. 0x0CE2A38C344A6EED, 0x1A804AADB9CFA741, 0x907F30421D78C5DE, 0x501F65EDB3034D07,
 67. 0x37624AE5A48FA6E9, 0x957BAF61700CFF4E, 0x3A6C27934E31188A, 0xD49503536ABCA345,
 68. 0x088E049589C432E0, 0xF943AEE7FEBF21B8, 0x6C3B8E3E336139D3, 0x3646FFFA464EE52E,
 69. 0xD60F6DCEDC314222, 0x56963B0DCA418FC0, 0x16F50EDF91E513AF, 0xEF1955914B609F93,
 70. 0x565601C0364E3228, 0xECB53939887E8175, 0xBAC7A9A18531294B, 0xB344C470397BBA52,
 71. 0x65D34954DAF3CEBD, 0xB4B81B3FA97511E2, 0xB422061193D6F6A7, 0x071582401C38434D,
 72. 0x7A13F18BBEDC4FF5, 0xBC4097B116C524D2, 0x59B97885E2F2EA28, 0x99170A5DC3115544,
 73. 0x6F423357E7C6A9F9, 0x325928EE6E6F8794, 0xD0E4366228B03343, 0x565C31F7DE89EA27,
 74. 0x30F5611484119414, 0xD873DB391292ED4F, 0x7BD94E1D8E17DEBC, 0xC7D9F16864A76E94,
 75. 0x947AE053EE56E63C, 0xC8C93882F9475F5F, 0x3A9BF55BA91F81CA, 0xD9A11FBB3D9808E4,
 76. 0x0FD22063EDC29FCA, 0xB3F256D8ACA0B0B9, 0xB03031A8B4516E84, 0x35DD37D5871448AF,
 77. 0xE9F6082B05542E4E, 0xEBFAFA33D7254B59, 0x9255ABB50D532280, 0xB9AB4CE57F2D34F3,
 78. 0x693501D628297551, 0xC62C58F97DD949BF, 0xCD454F8F19C5126A, 0xBBE83F4ECC2BDECB,
 79. 0xDC842B7E2819E230, 0xBA89142E007503B8, 0xA3BC941D0A5061CB, 0xE9F6760E32CD8021,
 80. 0x09C7E552BC76492F, 0x852F54934DA55CC9, 0x8107FCCF064FCF56, 0x098954D51FFF6580,
 81. 0x23B70EDB1955C4BF, 0xC330DE426430F69D, 0x4715ED43E8A45C0A, 0xA8D7E4DAB780A08D,
 82. 0x0572B974F03CE0BB, 0xB57D2E985E1419C7, 0xE8D9ECBE2CF3D73F, 0x2FE4B17170E59750,
 83. 0x11317BA87905E790, 0x7FBF21EC8A1F45EC, 0x1725CABFCB045B00, 0x964E915CD5E2B207,
 84. 0x3E2B8BCBF016D66D, 0xBE7444E39328A0AC, 0xF85B2B4FBCDE44B7, 0x49353FEA39BA63B1,
 85. 0x1DD01AAFCF53486A, 0x1FCA8A92FD719F85, 0xFC7C95D827357AFA, 0x18A6A990C8B35EBD,
 86. 0xCCCCB7005C6B9C28D, 0x3BDBB92C43B17F26, 0xAA70B5B4F89695A2, 0xE94C39A54A98307F,
 87. 0xB7A0B174CFF6F36E, 0xD4DBA84729AF48AD, 0x2E18BC1AD9704A68, 0x2DE0966DAF2F8B1C,
 88. 0xB9C11D5B1E43A07E, 0x64972D68DEE33360, 0x94628D38D0C20584, 0xDBC0D2B6AB90A559,
 89. 0xD2733C4335C6A72F, 0x7E75D99D94A70F4D, 0x6CED1983376FA72B, 0x97FCAACBF030BC24,
 90. 0x7B77497B32503B12, 0x8547EDDFB81CCB94, 0x79999CDDFF70902CB, 0xCFFE1939438E9B24,
 91. 0x829626E3892D95D7, 0x92FAE24291F2B3F1, 0x63E22C147B9C3403, 0xC678B6D860284A1C,
 92. 0x5873888850659AE7, 0x0981DCD296A8736D, 0x9F65789A6509A440, 0x9FF38FD72E9052F,
 93. 0xE479EE5B9930578C, 0xE7F28ECD2D49EECD, 0x56C074A581EA17FE, 0x5544F7D774B14AEF,
 94. 0x7B3F0195FC6F290F, 0x12153635B2C0CF57, 0x7F5126DBBA5E0CA7, 0x7A76956C3EAFB413,
 95. 0x3D5774A11D31AB39, 0x8A1B083821F40CB4, 0x7B4A38E32537DF62, 0x950113646D1D6E03,
 96. 0x4DA8979A0041E8A9, 0x3BC36E078F7515D7, 0x5D0A12F27AD310D1, 0x7F9D1A2E1EBE1327,
 97. 0xDA3A361B1C5157B1, 0xDCDD7D20903D0C25, 0x36833336D068F707, 0xCE68341F79893389,
 98. 0xAB9090168DD05F34, 0x43954B3252DC25E5, 0xB438C2B67F98E5E9, 0x10DCD78E3851A492,
 99. 0xDBC27AB5447822BF, 0x9B3CDB65F82CA382, 0xB67B7896167B4C84, 0xBFCE1B0048EAC50,
 100. 0xA9119B60369FFEBD, 0x1FFF7AC80904BF45, 0xAC12FB171817EEE7, 0xAF08DA9177DDA93D,
 101. 0x1B0CAB936E65C744, 0xB559EB1D04E5E932, 0xC37B45B3F8D6F2BA, 0xC3A9DC228CAAC9E9,
 102. 0xF3B8B6675A6507FF, 0x9FC477DE4ED681DA, 0x67378D8ECCEF96CB, 0x6DD856D94D259236,
 103. 0xA319CE15B0B4DB31, 0x073973751F12DD5E, 0x8A8E849EB32781A5, 0xE1925C71285279F5,
 104. 0x74C04BF1790C0EFE, 0x4DDA48153C94938A, 0x9D266D6A1CC0542C, 0x7440FB816508C4FE,
 105. 0x13328503DF48229F, 0xD6BF7BAEE43CAC40, 0x4838D65F6EF6748F, 0x1E152328F3318DEA,
 106. 0x8F8419A348F296BF, 0x72C8834A5957B511, 0xD7A023A73260B45C, 0x94EBC8ABCFB56DAE,
 107. 0x9FC10D0F989993E0, 0xDE68A2355B93CAE6, 0xA44CFE79AE538BBE, 0x9D1D84FCCE371425,
 108. 0x51D2B1AB2DDFB636, 0x2FD7E4B9E72CD38C, 0x65CA5B96B7552210, 0xDD69A0D8AB3B546D,
 109. 0x604D51B25FBF70E2, 0x73AA8A564FB7AC9E, 0x1A8C1E992B941148, 0xAAC40A2703D9BEA0,
 110. 0x764DBEAE7FA4F3A6, 0x1E99B96E70A9BE8B, 0x2C5E9DEB57EF4743, 0x3A938FEE32D29981,
 111. 0x26E6DB8FFDF5ADFE, 0x469356C504EC9F9D, 0xC8763C5B08D1908C, 0x3F6C6AF859D80055,
 112. 0x7F7CC39420A3A545, 0x9BFB227EBDF4C5CE, 0x89039D79D6FC5C5C, 0x8FE88B57305E2AB6,

Candidate Number:

Harry Foster

Centre Number:

113.	0xA09E8C8C35AB96DE,	0xFA7E393983325753,	0xD6B6D0ECC617C699,	0xDFEA21EA9E7557E3,
114.	0xB67C1FA481680AF8,	0xCA1E3785A9E724E5,	0x1CFC8BED0D681639,	0xD18D8549D140CAEA,
115.	0x4ED0FE7E9DC91335,	0xE4DBF0634473F5D2,	0x1761F93A44D5AEFE,	0x53898E4C3910DA55,
116.	0x734DE8181F6EC39A,	0x2680B122BAA28D97,	0x298AF231C85BAFAB,	0x7983EED3740847D5,
117.	0x66C1A2A1A60CD889,	0x9E17E49642A3E4C1,	0xEDB454E7BAD0805,	0x50B704CAB602C329,
118.	0x4CC317FB9CDD023,	0x66B4835D9EAFEA22,	0x219B97E26FFC81BD,	0x261E4E4C0A333A9D,
119.	0x1FE2CCA76517DB90,	0xD7504DFA8816EDBB,	0xB9571FA04DC089C8,	0x1DDC0325259B27DE,
120.	0xCF3F4688801EB9AA,	0xF4F5D05C10CAB243,	0x38B6525C21A42B0E,	0x36F60E2BA4FA6800,
121.	0xEB3593803173E0CE,	0x9C4CD6257C5A3603,	0xAF0C317D32ADAA8A,	0x258E5A80C7204C4B,
122.	0x8B889D624D44885D,	0xF4D14597E660F855,	0xD4347F66EC8941C3,	0xE699ED85B0DFB40D,
123.	0x2472F6207C2D0484,	0xC2A1E7B5B459AEB5,	0xAB4F6451CC1D45EC,	0x63767572AE3D6174,
124.	0xA59E0BD101731A28,	0x116D0016CB948F09,	0x2CF9C8CA052F6E9F,	0x0B090A7560A968E3,
125.	0xABEEDDB2DDE06FF1,	0x58EFC10B06A2068D,	0xC6E57A78FBD986E0,	0x2EAB8CA63CE802D7,
126.	0x14A195640116F336,	0x7C0828DD624EC390,	0xD74BBE77E6116AC7,	0x804456AF10F5FB53,
127.	0xEBE9EA2ADF4321C7,	0x03219A39EE587A30,	0x49787FEF17AF9924,	0xA1E9300CD8520548,
128.	0x5B45E522E4B1B4EF,	0xB49C3B3995091A36,	0xD4490AD526F14431,	0x12A8F216AF9418C2,
129.	0x001F837CC7350524,	0x1877B51E57A764D5,	0xA2853B80F17F58EE,	0x993E1DE72D36D310,
130.	0xB3598080CE64A656,	0x252F59CF0D9F04BB,	0xD23C8E176D113600,	0x1BDA0492E7E4586E,
131.	0x21E0BD5026C619BF,	0x3B097ADAF088F94E,	0x8D14DEDB30BE846E,	0xF95CFFA23AF5F6F4,
132.	0x3871700761B3F743,	0xCA672B91E9E4FA16,	0x64C8E531BFF53B55,	0x241260ED4AD1E87D,
133.	0x106C09B972D2E822,	0x7FBA195410E5CA30,	0x7884D9BC6CB569D8,	0x0647DFEDCD894A29,
134.	0x63573FF03E224774,	0x4FC8E9560F91B123,	0x1DB956E450275779,	0xB8D91274B9E9D4FB,
135.	0xA2EBEE47E2FBFCE1,	0xD9F1F30CCD97FB09,	0xEFED53D75FD64E6B,	0x2E6D02C36017F67F,
136.	0xA9AA4D20DB084E9B,	0xB64BE8D8B25396C1,	0x70CB6AF7C2D5BCF0,	0x98F076A4F7A2322E,
137.	0xBF84470805E69B5F,	0x94C3251F06F90CF3,	0x3E003E616A6591E9,	0xB925A6CD0421AFF3,
138.	0x61BDD1307C66E300,	0xBF8D5108E27E0D48,	0x240AB57A8B888B20,	0xFC87614BAF287E07,
139.	0xEF02CDD06FFDB432,	0xA1082C0466DF6C0A,	0x8215E577001332C8,	0xD39BB9C3A48DB6CF,
140.	0x2738259634305C14,	0x61CF4F94C97DF93D,	0x1B6BACA2AE4E125B,	0x758F450C88572E0B,
141.	0x959F587D507A8359,	0xB063E962E045F54D,	0x60E8ED72C0DFF5D1,	0x7B64978555326F9F,
142.	0xFD080D236DA814BA,	0x8C90FD9B083F4558,	0x106F72FE81E2C590,	0x7976033A39F7D952,
143.	0xA4EC0132764CA04B,	0x733EA705FAE4FA77,	0xB4D8F77BC3E56167,	0x9E21F4F903B33FD9,
144.	0x9D765E419FB69F6D,	0xD30C088BA61EA5EF,	0x5D94337FBFAF7F5B,	0x1A4E4822EB4D7A59,
145.	0x6FFE73E81B637FB3,	0xDDF957BC36D8B9CA,	0x64D0E29EEA8838B3,	0x08DD9BDFD96B9F63,
146.	0x087E79E5A57D1D13,	0xE328E230E3E2B3FB,	0x1C2559E30F0946BE,	0x720BF5F26F4D2EAA,
147.	0xB0774D261CC609DB,	0x443F64EC5A371195,	0x4112CF68649A260E,	0xD813F2FAB7F5C5CA,
148.	0x660D3257380841EE,	0x59AC2C7873F910A3,	0xE846963877671A17,	0x93B633ABFA3469F8,
149.	0xC0C0F5A60EF4CDCF,	0xCAF21ECD4377B28C,	0x57277707199B8175,	0x506C11B9D90E8B1D,
150.	0xD83CC2687A19255F,	0x4A29C6465A314CD1,	0xED2DF21216235097,	0xB5635C95FF7296E2,
151.	0x22AF003AB672E811,	0x52E762596BF68235,	0x9AEB3A3AC6ECC6B0,	0x944F6DE09134DFB6,
152.	0x6C47BEC883A7DE39,	0x6AD047C430A12104,	0xA5B1CFDBA0AB4067,	0x7C45D833AFF07862,
153.	0x5092EF950A16DA0B,	0x9338E69C052B8E7B,	0x455A4B4CFE30E3F5,	0x6B02E63195AD0CF8,
154.	0x6B17B224BAD6BF27,	0xD1E0CCD25B89C169,	0xDE0C89A556B9AE70,	0x50065E535A213CF6,
155.	0x9C1169FA2777B874,	0x78EDEFD694AF1EED,	0x6DC93D9526A50E68,	0xEE97F453F06791ED,
156.	0x32AB0EDB696703D3,	0x3A6853C7E70757A7,	0x31865CED6120F37D,	0x67FEF95D92607890,
157.	0x1F2B1D1F15F6DC9C,	0xB69E38A8965C6B65,	0xAA9119FF184CCCF4,	0xF43C732873F24C13,
158.	0xFBA43D794A9A80D2,	0x3550C2321FD6109C,	0x371F77E76BB8417E,	0x6BFA9AAE5EC05779,
159.	0xCD04F3FF001A4778,	0xE3273522064480CA,	0x9F91508BFFCFC14A,	0x049A7F41061A9E60,
160.	0xFCB6BE43A9F2FE9B,	0x08DE8A1C7797DA9B,	0x8F9887E6078735A1,	0xB5B4071DBFC73A66,
161.	0x230E343DFBA08D33,	0x43ED7F5A0FAE657D,	0x3A88A0FBBCB05C63,	0x21874B8B4D2DBC4F,
162.	0x1BDEA12E35F6A8C9,	0x53C065C6C8E63528,	0xE34A1D250E7A8D6B,	0xD6B04D3B7651DD7E,
163.	0x5E90277E7CB39E2D,	0x2C046F22062DC67D,	0xB10BB459132D0A26,	0x3FA9DDFB67E2F199,
164.	0x0E09B88E1914F7AF,	0x10E8B35AF3EEAB37,	0x9EEDECA8E272B933,	0xD4C718BC4AE8AE5F,
165.	0x81536D601170FC20,	0x91B534F885818A06,	0xEC8177F83F900978,	0x190E714FADA5156E,
166.	0xB592BF39B0364963,	0x89C350C893AE7DC1,	0xAC042E70F8B383F2,	0xB49B52E587A1EE60,
167.	0xFB152FE3FF26DA89,	0x3E666E6F69AE2C15,	0x3B544EBE544C19F9,	0xE805A1E290CF2456,
168.	0x24B33C9D7ED25117,	0xE74733427B72F0C1,	0x0A804D18B7097475,	0x57E3306D881EDB4F,
169.	0x4AE7D6A36EB5DBC8,	0x2D8D5432157064C8,	0xD1E649DE1E7F268B,	0x8A328A1CEDFE552C,
170.	0x07A3AEC79624C7DA,	0x84547DDC3E203C94,	0x990A98FD5071D263,	0x1A4FF12616EEFC89,
171.	0xF6F7FD1431714200,	0x30C05B1BA332F41C,	0x8D2636B81555A786,	0x46C9FEB55D120902,
172.	0xCCEC0A73B49C9921,	0x4E9D2827355FC492,	0x19EBB029435DCB0F,	0x4659D2B743848A2C,
173.	0x963EF2C96B33BE31,	0x74F85198B05A2E7D,	0x5A0F544DD2B1FB18,	0x0372D073C2E134B1,
174.	0xC7F6AA2DE59AEA61,	0x352787BAA0D7C22F,	0x9853EAB63B5E0B35,	0xABBD0CDD7ED5C0860,
175.	0xCF05DAF5AC8D77B0,	0x49CAD48CEBF4A71E,	0x7A4C10EC2158C4A6,	0xD9E92AA246BF719E,
176.	0x13AE978D09FE5557,	0x730499AF921549FF,	0x4E4B705B92903BA4,	0xFF577222C14F0A3A,
177.	0x55B6344CF97AAFAE,	0xB862225B055B6960,	0xCAC09AFBDD2CDB4,	0xDAF8E9829FE96B5F,
178.	0xB5FDFC5D3132C498,	0x310CB380DB6F7503,	0xE87FBB46217A360E,	0x2102AE466EBB1148,

```

179.      0xF8549E1A3AA5E00D, 0x07A69AFDCC42261A, 0xC4C118BFE78FEAAE, 0xF9F4892ED96BD438,
180.      0x1AF3DBE25D8F45DA, 0xF5B4B0B0D2DEEEB4, 0x962ACEEFA82E1C84, 0x046E3ECAAF453CE9,
181.      0xF05D129681949A4C, 0x964781CE734B3C84, 0x9C2ED44081CE5FBD, 0x522E23F3925E319E,
182.      0x177E00F9FC32F791, 0x2BC60A63A6F3B3F2, 0x222BBFAE61725606, 0x486289DDCC3D6780,
183.      0x7DC7785B8EFD8C0, 0x8AF38731C02BA980, 0x1FAB64EA29A2DDF7, 0xE4D9429322CD065A,
184.      0x9DA058C67844F20C, 0x24C0E332B70019B0, 0x233003B5A6CFE6AD, 0xD586BD01C5C217F6,
185.      0x5E5637885F29BC2B, 0x7EBA726D8C94094B, 0x0A56A5F0BFE39272, 0xD79476A84EE20D06,
186.      0x9E4C1269BAA4BF37, 0x17EFEE45B0DEE640, 0x1D95B0A5FCF90BC6, 0x93CBE0B699C2585D,
187.      0x65FA4F227A2B6D79, 0xD5F9E858292504D5, 0xC2B5A03F71471A6F, 0x59300222B4561E00,
188.      0xCE2F8642CA0712DC, 0x7CA9723FBB2E8988, 0x2785338347F2BA08, 0xC61BB3A141E50E8C,
189.      0x150F361DAB9DEC26, 0x9F6A419D382595F4, 0x64A53DC924FE7AC9, 0x142DE49FFF7A7C3D,
190.      0x0C335248857FA9E7, 0x0A9C32D5EAE45305, 0xE6C42178C4BBB92E, 0x71F1CE2490D20B07,
191.      0xF1BCC3D275AFE51A, 0xE728E8C83C334074, 0x96FBF83A12884624, 0x81A1549FD6573DA5,
192.      0x5FA7867CAF35E149, 0x56986E2EF3ED091B, 0x917F1DD5F8886C61, 0xD20D8C88C8FFE65F,
193.      0x31D71DCE64B2C310, 0xF165B587DF898190, 0xA57E6339DD2CF3A0, 0x1EF6E6DBB1961EC9,
194.      0x70CC73D90BC26E24, 0xE21A6B35DF0C3AD7, 0x003A93D8B2806962, 0x1C99DED33CB890A1,
195.      0xCF3145DE0ADD4289, 0xD0E4427A5514FB72, 0x77C621CC9FB3A483, 0x67A34DAC4356550B,
196.      0xF8D626AAAF278509

```

Zobrsit

```

1.  using System;
2.  using System.IO;
3.  using System.Text;
4.
5.  namespace Engine.Representaion
6.  {
7.
8.      /*
9.          Hashing Values and ployglot hashing algorithm rules from:
          http://hgm.nubati.net/book_format.html
10.         */
11.
12.     [Serializable]
13.     public struct ZobristKey // a zobrsit key can be split up into 4 components
14.     {
15.         public ulong pieceKey;
16.         public ulong enPassantKey;
17.         public ulong castlingKey;
18.         public ulong moveKey;
19.         public ulong Key => (pieceKey ^ enPassantKey ^ castlingKey ^ moveKey); // a function as
            the key should be changed via its components
20.
21.         public ZobristKey(bool a = false) // bool a is a wasted value as parameterless
            constructors are not supported
22.         {
23.             pieceKey = 0;
24.             enPassantKey = 0;
25.             castlingKey = 0;
26.             moveKey = 0;
27.         }
28.
29.     }
30.
31.     internal static class Zobrist
32.     {
33.         /*
34.             zobristTables =
35.             {
36.                 0 - white pawn
37.                 1 - black pawn
38.
39.                 2 - white knight
40.                 3 - black knight
41.
42.                 4 - white bishop

```

```

43.         5 - black bishop
44.
45.         6 - white rook
46.         7 - black rook
47.
48.         8 - white queen
49.         9 - black queen
50.
51.        10 - white king
52.        11 - black king
53.
54.        12 - Castling
55.
56.        13 - enPassant
57.        14 - SideToMove
58.    }
59.    */
60.
61.
62.    public static ulong[][] zobristTables = new ulong[15][];
63.
64.
65.    public static ulong[] zobristCastlingTable = new ulong[4];
66.
67.    // each ulong represent the file that will contribute to the hash 0 (A file), 7 (H
file)
68.    public static ulong[] zobristEnpassantFile = new ulong[8];
69.
70.
71.    // because of the way i have chosen to represent the indices of my bitboard they need
to be swapped to be compatible with the polyglot values
72.    public static int[] indexTable = {
73.        7,  6,  5,  4,  3,  2,  1,  0,
74.        15, 14, 13, 12, 11, 10, 9, 8,
75.        23, 22, 21, 20, 19, 18, 17, 16,
76.        31, 30, 29, 28, 27, 26, 25, 24,
77.        39, 38, 37, 36, 35, 34, 33, 32,
78.        47, 46, 45, 44, 43, 42, 41, 40,
79.        55, 54, 53, 52, 51, 50, 49, 48,
80.        63, 62, 61, 60, 59, 58, 57, 56
81.    };
82.
83.    public static ulong blackToMove;
84.
85.
86.    public static void changePieceSquare(ushort index, int pieceType, ref ulong key)
87.    {
88.        // convert the index to one that is used in the hashtable
89.        int zobristIndex = indexTable[index];
90.        // xor the key with the looked up value
91.        key ^= zobristTables[pieceType][zobristIndex];
92.    }
93.
94.
95.    public static ZobristKey CalculateZobrist(Bitboard board)
96.    {
97.        ZobristKey key = new ZobristKey(true);
98.        ulong PieceBB = 0;
99.        int turnAdder = board.WhiteToPlay ? 0 : 1;
100.        int counter = 0;
101.        for (int i = 2; i < board.Board.Length; i++) // for each piece bitboard i
(2 is the index of the pawn bitboard)
102.        {
103.            for (int j = 0; j < 2; j++) // loop over each colour
104.            {

```



```

105.                PieceBB = board.Board[i] & board.Board[j]; // gets the bitboard of
the piece colours (white pawns, black pawns ect.)
106.                while (PieceBB != 0) // loops over each piece
107.                {
108.                    ulong piece = BitBoardTools.popLSB(ref PieceBB); // get the LSB
109.                    int index = BitBoardTools.BitboardToIndex(piece);
110.
111.                    changePieceSquare((ushort)index, counter, ref key.pieceKey); //
add the hash
112.                }
113.                counter++;
114.            }
115.        }
116.
117.        if (board.enPassantFile != -1)
118.        {
119.            int index = board.enPassantFile - 1;
120.            // if the squares directly next to the pushed pawn are enemy pawns then
apply the hash
121.            int file = 7 - index;
122.            int pieceindex;
123.
124.            if (turnAdder == 1)
125.            {
126.                pieceindex = 2 * 8 + file;
127.            }
128.
129.            else
130.            {
131.                pieceindex = 5 * 8 + file;
132.            }
133.            ulong pawnloc = BitBoardTools.IndexToBitboard((byte)pieceindex);
134.            ulong ajecentMask = BitBoardTools.getPawnAttacks(pawnloc, turnAdder ^
1);
135.            ulong enemyPawns = Bitboard.getPawns(board.Board, turnAdder);
136.            if ((ajecentMask & enemyPawns) > 0)
137.            {
138.                key.enPassantKey ^= zobristTables[13][index];
139.            }
140.        }
141.
142.        // calculate Castling Hash
143.        ulong tempCastling = board.castlingRights;
144.        while (tempCastling != 0)
145.        {
146.            ulong piece = BitBoardTools.popLSB(ref tempCastling);
147.            int index = BitBoardTools.BitboardToIndex(piece);
148.            key.castlingKey ^= zobristTables[12][index];
149.        }
150.
151.        if (board.WhiteToPlay)
152.        {
153.            key.moveKey ^= zobristTables[14][0];
154.        }
155.
156.        return key;
157.    }
158.
159.
160.    static Zobrist()
161.    {
162.        // initilise piece tables
163.        string loc =
Directory.GetParent(Directory.GetCurrentDirectory()).FullName.Split(new string[] { "UI",
"Engine" }, StringSplitOptions.None)[0];

```

```

164.         loc = Path.Combine(loc, "Engine", "Engine", "PolyGlot",
    "PolyZobristKeys.txt");
165.
166.         zobristTables[12] = new ulong[4];
167.         zobristTables[13] = new ulong[8];
168.         zobristTables[14] = new ulong[1];
169.
170.
171.         string fileText = File.ReadAllText(loc);
172.         string[] keys = fileText.Split(',');
173.         int piecetableCounter = -1;
174.
175.         // loop over each key
176.         for (int i = 0; i < keys.Length; i++)
177.         {
178.             string temp = keys[i].Replace("\n", "").Trim();
179.             ulong key = (ulong)Convert.ToUInt64(temp, 16);
180.             if (i > 767) // in enpassant or castling
181.             {
182.                 if (i < 772)
183.                 { // in castling
184.                     zobristTables[12][i - 768] = key;
185.                 }
186.                 else if (i < 780)
187.                 {
188.                     // in enpassant
189.                     zobristTables[13][i - 772] = key;
190.                 }
191.                 else
192.                 {
193.                     // black hash
194.                     zobristTables[14][0] = key;
195.                 }
196.             }
197.
198.             else
199.             { // it is a piece hash
200.
201.                 if (i % 64 == 0)
202.                 { // it is at the start of a new piece
203.                     // hash is in the format Pawn black, Pawn white, so convert
    that into white, black
204.
205.                     if (piecetableCounter % 2 == 0)
206.                     {
207.                         piecetableCounter += 3;
208.                     }
209.
210.                     else if (piecetableCounter == -1)
211.                     {
212.                         piecetableCounter += 2;
213.                     }
214.                     else
215.                     {
216.                         piecetableCounter--;
217.                     }
218.                     zobristTables[piecetableCounter] = new ulong[64];
219.
220.                 }
221.
222.                 zobristTables[piecetableCounter][i % 64] = key;
223.             }
224.         }
225.     }
226. }
227.

```

```

228.         }
229.     }

```

Search

MoveOrderer

```

1. using System.Collections.Generic;
2.
3. namespace Engine.Search
4. {
5.
6.
7.     internal static class Move_Orderer
8.     {
9.         // get each move and give it a rating on how good it might be
10.        // apply a sorting algorithm (merge sort maybe / quick sort / heap sort)
11.
12.        // general move ordering should be:
13.        // PV move
14.        // captures (mvv-lva)
15.        // killers
16.        // history score
17.
18.        static readonly int[] VictimScores = { 100, 200, 300, 400, 500, 600 };
19.
20.        static int[][] MvvLvaScores = new int[6][]; // {pawn[], knight[], bishop[], rook[],
queen[], king[]} [victim][attacker]
21.
22.        static Move_Orderer()
23.        {
24.            initMvvLva();
25.        }
26.
27.        internal static void initMvvLva()
28.        {
29.            for (int victim = 0; victim < MvvLvaScores.Length; victim++)
30.            {
31.                MvvLvaScores[victim] = new int[6];
32.                for (int attacker = 0; attacker < MvvLvaScores[victim].Length; attacker++)
33.                {
34.                    MvvLvaScores[victim][attacker] = VictimScores[victim] + 6 -
VictimScores[attacker] / 100;
35.                }
36.            }
37.        }
38.
39.        static void swap(double[] arr, ref List<Move> list, int i, int j)
40.        {
41.            double temp = arr[i];
42.            arr[i] = arr[j];
43.            arr[j] = temp;
44.
45.            Move tempmove = list[i];
46.            list[i] = list[j];
47.            list[j] = tempmove;
48.        }
49.
50.        static int partition(double[] arr, ref List<Move> moves, int low, int high)
51.        {
52.            // Choosing the pivot
53.            double pivot = arr[high];
54.

```

```

55.         // Index of smaller element and indicates
56.         // the right position of pivot found so far
57.         int i = (low - 1);
58.
59.         for (int j = low; j <= high - 1; j++)
60.         {
61.
62.             // If current element is smaller than the pivot
63.             if (arr[j] > pivot)
64.             {
65.
66.                 // Increment index of smaller element
67.                 i++;
68.                 swap(arr, ref moves, i, j);
69.             }
70.         }
71.         swap(arr, ref moves, i + 1, high);
72.         return (i + 1);
73.     }
74.
75.     public static void quickSort(double[] moveScores, ref List<Move> moves, int left, int
right) {
76.
77.         if (left < right)
78.         {
79.
80.             // pi is partitioning index, arr[p]
81.             // is now at right place
82.             int pi = partition(moveScores, ref moves, left, right);
83.
84.             // Separately sort elements before
85.             // and after partition index
86.             quickSort(moveScores, ref moves, left, pi - 1);
87.             quickSort(moveScores, ref moves, pi + 1, right);
88.         }
89.         //
90.     }
91.
92.     public static void orderMoves(ref List<Move> moves, Bitboard board)
93.     {
94.         // for each capture
95.         // calculate the mvv-lva score
96.         // sort the captures
97.         double[] moveScores = new double[moves.Count];
98.
99.         for(int i =0; i<moves.Count;i++) {
100.             moveScores[i] = EvaluateMove(moves[i], board);
101.         }
102.
103.         // sort moves
104.         quickSort(moveScores, ref moves, 0, moveScores.Length-1);
105.     }
106.
107.     private static double EvaluateMove(Move move, Bitboard b) {
108.         // unpack move information
109.         byte startindex = move.startIndex;
110.         ulong startBitboard = BitBoardTools.IndexToBitboard(startindex);
111.         byte destinationindex = move.destinationIndex;
112.         ulong destinationBitboard =
BitBoardTools.IndexToBitboard(destinationindex);
113.         int typeOfPieceMoving = Bitboard.getPieceType(b.Board, startBitboard) -2;
114.         int enemyColour = b.WhiteToPlay ? 1 : 0;
115.
116.         // if pv
117.
118.         // if capture

```

```

119.             if ((b.Board[enemyColour] & destinationBitboard) != 0) {
120.                 int enemyPieceType = Bitboard.getPieceType(b.Board,
destinationBitboard) -2;
121.                 return MvvLvaScores[enemyPieceType][typeOfPieceMoving] + 1000;
122.             }
123.
124.             // if killer
125.             // history
126.
127.
128.             return 0;
129.         }
130.
131.     }
132. }

```

Perft

```

1. using System;
2. using System.Collections.Generic;
3. using System.Diagnostics;
4. using System.Linq;
5.
6. namespace Engine.Search
7. {
8.
9.     public class PerftTest
10.    {
11.        public int depth;
12.        public ulong nodes;
13.        public string fen;
14.    }
15.    public class Perft
16.    {
17.        Bitboard board;
18.        int targetDepth;
19.        string[] debuggingstring;
20.        PerftTest[] tests;
21.        Generation generator = new Generation();
22.        bool stopSearch = false;
23.
24.        internal void stopsearch() {
25.            stopSearch = true;
26.        }
27.
28.        public void PerftTestBulkTest(Bitboard b, PerftTest[] testarray)
29.        {
30.            stopSearch = false;
31.            this.board = b;
32.            int n_passed = 0;
33.            foreach (PerftTest i in testarray)
34.            {
35.                if (stopSearch) return;
36.
37.                Console.WriteLine("FEN: " + i.fen);
38.                board = Tools.FENtoBitboard(i.fen);
39.                targetDepth = i.depth;
40.                Console.WriteLine($"Depth: {i.depth}");
41.                ulong nodes = RecursivePerftTest(i.depth);
42.                Console.WriteLine("Nodes: " + i.nodes);
43.                if (nodes == i.nodes)
44.                {
45.                    Console.ForegroundColor = ConsoleColor.Green;

```

```

46.         Console.Write(" Passed\n");
47.         n_passed++;
48.     }
49.     else
50.     {
51.         Console.ForegroundColor = ConsoleColor.Red;
52.         Console.Write(" Failed " + "EXPECTED: " + i.nodes);
53.     }
54.     Console.ForegroundColor = ConsoleColor.White;
55.     Console.WriteLine("\n");
56. }
57.
58. Console.WriteLine($"Passed: {n_passed}/{testarray.Length}");
59. }
60.
61. public void PerftTest(Bitboard b, int depth)
62. {
63.     stopSearch = false;
64.     this.board = b;
65.     targetDepth = depth;
66.     Console.WriteLine($"Depth: {depth}");
67.
68.     ulong nodes;
69.     Stopwatch timer = new Stopwatch();
70.     timer.Start();
71.     nodes = RecursivePerftTest(depth);
72.     timer.Stop();
73.     Console.WriteLine($"Nodes: {nodes}");
74.     Console.WriteLine($"Time Elapsed: {timer.Elapsed}");
75.     Console.WriteLine($"Million Nodes Per Second: {(double)(nodes / 1000) /
(double)(timer.ElapsedMilliseconds)}");
76.     Console.WriteLine("\n");
77. }
78.
79. public void PerftTestCaptures(Bitboard b, int depth)
80. {
81.     stopSearch = false;
82.     this.board = b;
83.     targetDepth = depth;
84.     Console.WriteLine($"Depth: {depth}");
85.
86.     ulong nodes;
87.     Stopwatch timer = new Stopwatch();
88.     timer.Start();
89.     nodes = RecursivePerftTestCapturesOnly(depth);
90.     timer.Stop();
91.     Console.WriteLine($"Nodes: {nodes}");
92.     Console.WriteLine($"Time Elapsed: {timer.Elapsed}");
93.     Console.WriteLine($"Million Nodes Per Second: {(double)(nodes / 1000) /
(double)(timer.ElapsedMilliseconds)}");
94.     Console.WriteLine("\n");
95. }
96.
97. public void PerftTestDebug(Bitboard b, int depth, string[] DEBUGING)
98. {
99.     stopSearch = false;
100.     this.board = b;
101.     this.debugingstring = DEBUGING;
102.     targetDepth = depth;
103.     Console.WriteLine($"Depth: {depth}");
104.     Console.WriteLine($"Nodes: {RecursiveDebugPerftTest(depth)}");
105.     Console.WriteLine("\n\n");
106. }
107.
108. public void PerftTestMoveDebug(Bitboard b, int depth, string[] DEBUGING)
109. {

```

```

110.         stopSearch = false;
111.         this.board = b;
112.         this.debuggingstring = DEBUGGING;
113.         targetDepth = depth;
114.         Console.WriteLine($"Depth: {depth}");
115.         Console.WriteLine($"Nodes: {RecursiveMoveDebugPerftTest(depth)}");
116.         Console.WriteLine("\n\n");
117.     }
118.
119.     private ulong RecursiveMoveDebugPerftTest(int depth)
120.     {
121.         List<Move> move_list = generator.GenerateMoves(board, false);
122.         int n_moves;
123.         n_moves = move_list.Count;
124.         ulong nodes = 0;
125.
126.         if (depth == 1)
127.         {
128.             return (ulong)n_moves;
129.         }
130.         else
131.         {
132.             for (int i = 0; i < n_moves; i++)
133.             {
134.                 ulong expected = 0;
135.
136.                 if (stopSearch) return nodes;
137.
138.                 if (depth == targetDepth)
139.                 {
140.                     string move = Tools.MoveToString(move_list[i]);
141.                     Console.Write(move + ": \n");
142.
143.                     Console.WriteLine("Before:");
144.                     Tools.printPieceBoard(board);
145.
146.                     foreach (string j in debuggingstring)
147.                     {
148.                         if (j.Contains(move))
149.                         {
150.                             expected = ulong.Parse(j.Split(' ')[1]);
151.                         }
152.                     }
153.                 }
154.                 board.MakeMove(move_list[i]);
155.                 ulong nodesSearched = RecursivePerftTest(depth - 1);
156.                 if (depth == targetDepth)
157.                 {
158.                     Console.WriteLine("After:");
159.                     Tools.printPieceBoard(board);
160.
161.                     Console.Write(nodesSearched.ToString());
162.                     if (nodesSearched == expected)
163.                     {
164.                         Console.ForegroundColor = ConsoleColor.Green;
165.                         Console.Write(" Passed\n");
166.                     }
167.                     else
168.                     {
169.                         Console.ForegroundColor = ConsoleColor.Red;
170.                         Console.Write($" Failed EXPECTED:{expected}\n");
171.                     }
172.                     Console.ForegroundColor = ConsoleColor.White;
173.                 }
174.                 nodes += nodesSearched;
175.

```

```
176.         board.UndoMove(move_list[i]);
177.     }
178.     return nodes;
179. }
180. }
181.
182. private ulong RecursiveDebugPerftTest(int depth)
183. {
184.     List<Move> move_list = generator.GenerateMoves(board, false);
185.
186.     int n_moves;
187.     n_moves = move_list.Count;
188.     ulong nodes = 0;
189.
190.     if (depth == 1)
191.     {
192.         if (depth == targetDepth)
193.         {
194.             Tools.printPieceBoard(board);
195.             string[] movesinstring = new string[n_moves];
196.
197.             for (int i = 0; i < n_moves; i++)
198.             {
199.                 movesinstring[i] = Tools.MoveToString(move_list[i]);
200.             }
201.
202.             foreach (string j in debuggingstring)
203.             {
204.                 string move = j.Split(':')[0];
205.                 Console.Write(move + ":");
206.                 if (movesinstring.Contains(move))
207.                 {
208.                     Console.ForegroundColor = ConsoleColor.Green;
209.                     Console.Write(" Passed\n");
210.                 }
211.                 else
212.                 {
213.                     Console.ForegroundColor = ConsoleColor.Red;
214.                     Console.Write(" Failed\n");
215.                 }
216.                 Console.ForegroundColor = ConsoleColor.White;
217.             }
218.         }
219.         return (ulong)n_moves;
220.     }
221.     else
222.     {
223.         for (int i = 0; i < n_moves; i++)
224.         {
225.             if (stopSearch) return nodes;
226.             ulong expected = 0;
227.             if (depth == targetDepth)
228.             {
229.                 string move = Tools.MoveToString(move_list[i]);
230.                 Console.Write(move + ": ");
231.                 foreach (string j in debuggingstring)
232.                 {
233.                     if (j.Contains(move))
234.                     {
235.                         expected = ulong.Parse(j.Split(' ')[1]);
236.                     }
237.                 }
238.             }
239.             board.MakeMove(move_list[i]);
240.             ulong nodesSearched = RecursivePerftTest(depth - 1);
241.             if (depth == targetDepth)
```



```
242.         {
243.             Console.Write(nodesSearched.ToString());
244.             if (nodesSearched == expected)
245.             {
246.                 Console.ForegroundColor = ConsoleColor.Green;
247.                 Console.Write(" Passed\n");
248.             }
249.
250.             else
251.             {
252.                 Console.ForegroundColor = ConsoleColor.Red;
253.                 Console.Write($" Failed EXPECTED:{expected}\n");
254.             }
255.             Console.ForegroundColor = ConsoleColor.White;
256.         }
257.         nodes += nodesSearched;
258.         board.UndoMove(move_list[i]);
259.     }
260.     return nodes;
261. }
262. }
263.
264. private ulong RecursivePerftTest(int depth)
265. {
266.     List<Move> move_list = generator.GenerateMoves(board, false);
267.
268.     int n_moves;
269.     n_moves = move_list.Count;
270.     ulong nodes = 0;
271.
272.     if (depth == 1)
273.     {
274.         return (ulong)n_moves;
275.     }
276.     else
277.     {
278.         for (int i = 0; i < n_moves; i++)
279.         {
280.             if (stopSearch) return nodes;
281.
282.             if (depth == targetDepth)
283.             {
284.                 Console.Write(Tools.MoveToString(move_list[i]) + ": ");
285.             }
286.
287.             board.MakeMove(move_list[i]);
288.             ulong nodesSearched = RecursivePerftTest(depth - 1);
289.             if (depth == targetDepth)
290.             {
291.                 Console.Write(nodesSearched.ToString() + "\n");
292.             }
293.             nodes += nodesSearched;
294.             board.UndoMove(move_list[i]);
295.         }
296.         return nodes;
297.     }
298. }
299.
300. private ulong RecursivePerftTestCapturesOnly(int depth)
301. {
302.     List<Move> move_list = generator.GenerateMoves(board, true);
303.
304.     int n_moves;
305.     n_moves = move_list.Count;
306.     ulong nodes = 0;
307. }
```

```

308.         if (depth == 1)
309.         {
310.             return (ulong)n_moves;
311.         }
312.         else
313.         {
314.             for (int i = 0; i < n_moves; i++)
315.             {
316.                 if (stopSearch) return nodes;
317.
318.                 if (depth == targetDepth)
319.                 {
320.                     Console.Write(Tools.MoveToString(move_list[i]) + ": ");
321.                 }
322.
323.                 board.MakeMove(move_list[i]);
324.                 ulong nodesSearched = RecursivePerftTestCapturesOnly(depth - 1);
325.                 if (depth == targetDepth)
326.                 {
327.                     Console.Write(nodesSearched.ToString() + "\n");
328.                 }
329.                 nodes += nodesSearched;
330.                 board.UndoMove(move_list[i]);
331.             }
332.             return nodes;
333.         }
334.     }
335.
336. }
337.

```

Searcher

```

1. using Engine.Evaluation;
2. using System;
3. using System.Collections.Generic;
4. using System.IO;
5.
6. namespace Engine.Search
7. {
8.     internal class Searcher
9.     {
10.         Generation generator = new Generation();
11.         Evaluator evaluator = new Evaluator();
12.         TranspositionTable transpositionTable = new TranspositionTable(200);
13.         Bitboard board;
14.         Random random = new Random();
15.         int MaxDepth;
16.         Move bestMove = new Move();
17.         Move bestMoveIteration = new Move();
18.
19.         double bestScoreSofar;
20.         int currentIterationDepth;
21.
22.         bool searchStopped = false;
23.         bool debug = false;
24.         ulong nodescount = 0;
25.         string debugpath = Path.Combine(Directory.GetCurrentDirectory(), "Debug.txt");
26.
27.
28.         /// <summary>
29.         /// Given the states of the two players it will calculate the amount of time the
        program should search for so that the player will not run out of time

```

```

30.         /// </summary>
31.         /// <param name="whiteTime">the amount of time in ms that white has left on the
    clock</param>
32.         /// <param name="blackTime">the amount of time in ms that black has left on the
    clock</param>
33.         /// <param name="whiteInc"> the amount of time in ms that white will add every move
    played</param>
34.         /// <param name="blackInc"> the amount of time in ms that black will add every move
    played</param>
35.         public static int CalculateSearchTime(int whiteTime, int blackTime, int whiteInc, int
    blackInc, bool WhiteToPlay)
36.         {
37.             int remainingTime = WhiteToPlay ? whiteTime : blackTime;
38.
39.             // If remaining time is less than or equal to zero, return a default value
40.             if (remainingTime <= 0)
41.             {
42.                 return 1000;
43.             }
44.
45.             // Calculate search time based on remaining time and increments
46.             int searchTime = remainingTime / 30;
47.
48.             // Add increment time if applicable
49.             if (WhiteToPlay)
50.             {
51.                 searchTime += whiteInc;
52.             }
53.             else
54.             {
55.                 searchTime += blackInc;
56.             }
57.
58.             // Ensure search time is at least a minimum value (adjust as needed)
59.             int minSearchTime = 100;
60.             if (searchTime < minSearchTime)
61.             {
62.                 searchTime = minSearchTime;
63.             }
64.
65.             return searchTime;
66.         }
67.
68.         internal void stopSearch()
69.         {
70.             searchStopped = true;
71.         }
72.
73.         private void resetSearchVars()
74.         {
75.             searchStopped = false;
76.             nodescount = 0;
77.             transpositionTable.resetDebuggingVars();
78.         }
79.
80.         public void Search(Bitboard b, int maxDepth = 256)
81.         {
82.             this.board = b;
83.             resetSearchVars();
84.             double score = 0;
85.             string pv = "";
86.
87.             double alpha = -500000;
88.             double beta = 500000;
89.
90.

```

```

91.         for (int depth = 1; depth <= maxDepth; depth++)
92.         {
93.             currentIterationDepth = depth;
94.             score = alphaBetaNega(depth, alpha, beta);
95.
96.             if ((score <= alpha) || (score >= beta))
97.             {
98.                 alpha = -50000;
99.                 beta = 50000;
100.                 continue;
101.             }
102.
103.             // set up the window for the next iteration
104.             alpha = score - 50;
105.             beta = score + 50;
106.
107.             pv = transpositionTable.getPV(board.zobristKey.Key, board);
108.
109.             Console.WriteLine($"info score cp {Math.Round(score)} depth {depth}
nodes {nodescount} pv {pv}");
110.
111.             if (searchStopped)
112.             {
113.                 break;
114.             }
115.             else
116.             {
117.                 bestMove = bestMoveIteration;
118.             }
119.
120.         }
121.
122.         transpositionTable.printStats();
123.         pv = transpositionTable.getPV(board.zobristKey.Key, board);
124.         Console.WriteLine($"info score cp {Math.Round(score)} depth {maxDepth}
nodes {nodescount} pv {pv}");
125.         Console.WriteLine("bestmove " + Tools.MoveToString(bestMove));
126.
127.     }
128.
129.     // search the position until the position has no more captures (a quiet
position)
130.     // removes the horizon effect
131.     internal double Quiescence(double alpha, double beta)
132.     {
133.         double staticEval = evaluator.EvaluateFromPov(board);
134.
135.         nodescount++;
136.
137.         if (searchStopped)
138.         {
139.             return staticEval;
140.         }
141.
142.         // fail hard beta cutoff
143.         if (staticEval >= beta)
144.         {
145.             return beta;
146.         }
147.
148.         // better move found
149.         if (staticEval > alpha)
150.         {
151.             alpha = staticEval;
152.         }
153.

```

```

154.         List<Move> moves = generator.GenerateMoves(board, true);
155.         // loop each capture
156.         foreach (Move i in moves)
157.         {
158.             board.MakeMove(i);
159.             double score = -Quiescence(-beta, -alpha);
160.             board.UndoMove(i);
161.
162.             // better move found
163.             if (score > alpha)
164.             {
165.                 alpha = score;
166.             }
167.
168.             // fail hard beta cutoff
169.             if (score >= beta)
170.             {
171.                 return beta;
172.             }
173.         }
174.
175.         return alpha;
176.
177.     }
178.
179.     internal double alphaBetaNega(int depth, double alpha, double beta)
180.     {
181.         bool isNotRootNode = (currentIterationDepth - depth) == 0 ? false : true;
182.
183.         // probe transposition table if not at root node
184.         if (isNotRootNode && transpositionTable.get(board.zobristKey.Key, alpha,
185.             beta, depth, out double LookUpResult))
186.         {
187.             return LookUpResult;
188.         }
189.
190.         if (searchStopped)
191.         {
192.             return 0;
193.         }
194.
195.         // if we reach the maximum search depth or the search is stopped
196.         if (depth == 0)
197.         {
198.             double QuiescenceEval = Quiescence(alpha, beta);
199.             return QuiescenceEval;
200.         }
201.
202.         // increment nodes count
203.         nodescount++;
204.
205.         // check for the 50 move rule
206.         if (board.plyCount >= 100)
207.         {
208.             return 0;
209.         }
210.
211.         List<Move> moves = generator.GenerateMoves(this.board);
212.
213.         if (moves.Count == 0)
214.         {
215.             if (generator.isCheck) // opponent has checkmated
216.             {
217.                 double plysSearched = currentIterationDepth - depth;
218.                 return -10000 + plysSearched; // return negative as it is always
219.                 bad from our pov

```

```

218.         }
219.
220.         return 0; // stalemate
221.     }
222.
223.     double originalAlpha = alpha;
224.     double recursiveScore;
225.     Move bestMove = new Move(true);
226.     TTFlag tFlag = TTFlag.Fail_Low_Alpha;
227.
228.     // sort moves
229.     Move_Orderer.orderMoves(ref moves, board);
230.     foreach (Move i in moves)
231.     {
232.         board.MakeMove(i);
233.         // calculate check extentions
234.         recursiveScore = -alphaBetaNega(depth - 1, -beta, -alpha);
235.         board.UndoMove(i);
236.
237.         // fail hard beta cutoff
238.         if (recursiveScore >= beta)
239.         {
240.             transpositionTable.add(board.zobristKey.Key, depth,
TTFlag.Fail_High_Beta, beta, i);
241.             return beta;
242.         }
243.
244.         // better move found
245.         if (recursiveScore > alpha)
246.         {
247.             alpha = recursiveScore;
248.             tFlag = TTFlag.Exact;
249.             bestMove = i;
250.         }
251.     }
252.
253.     // alpha has been improved
254.     // this is the new pv node
255.     if (alpha > originalAlpha)
256.     {
257.         bestMoveIteration = bestMove;
258.     }
259.
260.     if (!searchStopped) // if the search has been stopped the value is likely
to be inaccurate
261.     {
262.         transpositionTable.add(board.zobristKey.Key, depth, tFlag, alpha,
bestMove);
263.     }
264.
265.     return alpha;
266.
267. }
268.
269. internal int CalculateExtentions()
270. {
271.     return 0;
272. }
273. }
274. }
275.
276. /* Notes:
277.     When the search is stopped and all of the nodes are being searched from the TT then
the best move does not change from the last search.

```

278. */

Evaluation

Evaluator

```

1. namespace Engine.Evaluation
2. {
3.     internal struct BoardInfo
4.     {
5.
6.         internal int wPawnCount;
7.         internal int bPawnCount;
8.         internal int PawnCount => wPawnCount + bPawnCount;
9.
10.        internal int wKnightCount;
11.        internal int bKnightCount;
12.        internal int KnightCount => wKnightCount + bKnightCount;
13.
14.        internal int wBishopCount;
15.        internal int bBishopCount;
16.        internal int BishopCount => wBishopCount + bBishopCount;
17.
18.        internal int wRookCount;
19.        internal int bRookCount;
20.        internal int RookCount => wRookCount + bRookCount;
21.
22.
23.        internal int wQueenCount;
24.        internal int bQueenCount;
25.        internal int QueenCount => wQueenCount + bQueenCount;
26.
27.
28.        internal BoardInfo(Bitboard bitboard)
29.        {
30.            ulong wPieces = bitboard.Board[0];
31.            ulong bPieces = bitboard.Board[1];
32.
33.            wPawnCount = Tools.countOnes(bitboard.Board[(int)Tools.BB.Pawns] & wPieces);
34.            bPawnCount = Tools.countOnes(bitboard.Board[(int)Tools.BB.Pawns] & bPieces);
35.
36.            wKnightCount = Tools.countOnes(bitboard.Board[(int)Tools.BB.Knights] & wPieces);
37.            bKnightCount = Tools.countOnes(bitboard.Board[(int)Tools.BB.Knights] & bPieces);
38.
39.            bBishopCount = Tools.countOnes(bitboard.Board[(int)Tools.BB.Bishops] & bPieces);
40.            wBishopCount = Tools.countOnes(bitboard.Board[(int)Tools.BB.Bishops] & wPieces);
41.
42.            wRookCount = Tools.countOnes(bitboard.Board[(int)Tools.BB.Rooks] & wPieces);
43.            bRookCount = Tools.countOnes(bitboard.Board[(int)Tools.BB.Rooks] & bPieces);
44.
45.            wQueenCount = Tools.countOnes(bitboard.Board[(int)Tools.BB.Queen] & wPieces);
46.            bQueenCount = Tools.countOnes(bitboard.Board[(int)Tools.BB.Queen] & bPieces);
47.        }
48.    }
49.
50.    internal class Evaluator
51.    {
52.        Bitboard board;
53.
54.        #region Bonuses
55.        const double BishopPairBonus = 50;
56.        const double PassedPawnBonus = 50;
57.    }

```

```

58.      const double RookOnOpenFileBonus = 10;
59.      #endregion
60.
61.      #region Penalties
62.      const double DoublePawnsPenalty = -10;
63.      const double IsolatedPawnPenalty = -25;
64.      const double BackwardsPawnPenalty = -10;
65.      #endregion
66.
67.      static readonly double[][] materialScores = {
68.          // opening scores
69.          new double[] { 82, 337, 365, 477, 1025, 12000 },
70.          //endgame scores
71.          new double[] { 94, 291, 297, 512, 936, 12000 }
72.      };
73.
74.
75.      enum gamephase { opening, endgame, middlegame }
76.      const double openingPhaseValue = 6192;
77.      const double endgamePhaseValue = 518;
78.
79.      private double calculateGamePhase(BoardInfo info)
80.      {
81.          double pawnPhase = 0;
82.          double knightPhase = 1;
83.          double bishopPhase = 1;
84.          double rookPhase = 1;
85.          double queenPhase = 1;
86.
87.          double phase = 0;
88.
89.          phase += info.PawnCount * materialScores[0][0] * pawnPhase;
90.          phase += info.KnightCount * materialScores[0][1] * knightPhase;
91.          phase += info.BishopCount * materialScores[0][2] * bishopPhase;
92.          phase += info.RookCount * materialScores[0][3] * rookPhase;
93.          phase += info.QueenCount * materialScores[0][4] * queenPhase;
94.
95.          return phase;
96.      }
97.
98.      public double Evaluate(Bitboard board)
99.      {
100.          this.board = board;
101.          gamephase phase = gamephase.middlegame;
102.          BoardInfo info = new BoardInfo(board);
103.
104.          double value = 0;
105.
106.          double gamePhaseValue = calculateGamePhase(info);
107.
108.          if (gamePhaseValue > openingPhaseValue) phase = gamephase.opening;
109.          else if (gamePhaseValue < endgamePhaseValue) phase = gamephase.endgame;
110.
111.          // loop over each piece and give it a material score (exact or
interpolated) and a positional score
112.
113.          for (int i = 2; i < board.Board.Length; i++)
114.          {
115.
116.              // loop over each piece
117.              int pieceType = i - 2; // the piece bitboards start at index 2, the
indexes in material values start at 0
118.              ulong pieceBoard = board.Board[i];
119.
120.              while (pieceBoard != 0)
121.              {

```



```

122.          // get the location of the piece
123.          ulong loc = BitBoardTools.popLSB(ref pieceBoard);
124.          byte index = BitBoardTools.BitboardToIndex(loc);
125.          bool isWhite = board.IsColour(loc, 0);
126.          double ColourSign = isWhite ? 1 : -1; // if it is white then add,
          if it is black subtract
127.
128.          // check the phase
129.
130.          // use exact values
131.          if (phase != gamephase.middlegame)
132.          {
133.              // add the piece values
134.              value += materialScores[(int)phase][pieceType] * ColourSign;
135.
136.              // add positional values
137.              value += PieceTables.LookUp(pieceType, index, (int)phase,
          isWhite) * ColourSign;
138.
139.          }
140.
141.          // interpolate values
142.          else
143.          {
144.              double opEval = materialScores[0][pieceType];
145.              double egEval = materialScores[1][pieceType];
146.              value += ((opEval * (gamePhaseValue) + (egEval *
          (openingPhaseValue - gamePhaseValue))) / openingPhaseValue) * ColourSign;
147.
148.
149.              double opScore = 0;
150.              double egScore = 0;
151.              value += ((opScore * (gamePhaseValue) + (egScore *
          (openingPhaseValue - gamePhaseValue))) / openingPhaseValue) * ColourSign;
152.          }
153.
154.          // add piece specific bonuses/penalties
155.          if (pieceType == 0)
156.          { // pawns
157.              // passed pawn
158.              // double pawn
159.              // backwards pawn
160.              // isolated pawn
161.          }
162.
163.          else if (pieceType == 1)
164.          { // knight
165.              // add your own bounuses / penalties
166.          }
167.
168.          else if (pieceType == 2)
169.          { // bishop
170.              // add your own bounuses / penalties
171.          }
172.
173.          else if (pieceType == 3)
174.          { // rook
175.              // open file / semi open file
176.          }
177.
178.          else if (pieceType == 4)
179.          { // queen
180.
181.          }
182.
183.          else if (pieceType == 5)

```

```

184.                { // king
185.                    // pawn shield
186.                }
187.            }
188.        }
189.        return value;
190.
191.    }
192.
193.    public double EvaluateFromPov(Bitboard board)
194.    {
195.        double value = Evaluate(board);
196.
197.        return board.WhiteToPlay ? value : -value;
198.
199.    }
200.
201.    }
202. }

```

PieceTables

```

1. using System;
2.
3. namespace Engine.Evaluation
4. {
5.     internal static class PieceTables
6.     {
7.         #region Piece Square Tables
8.
9.         /* values from Rofchade:
10.         http://www.talkchess.com/forum3/viewtopic.php?f=2&t=68311&start=19 */
11.
12.         /*
13.         Mg - Middle game
14.         Eg - End game
15.
16.         In the format of
17.
18.         H1, G1, F1, E1, D1, C1, B1, A1
19.         H2, G2, F2, E2, D2, C2, B2, A2
20.         H3, G3, F3, E3, D3, C3, B3, A3
21.         H4, G4, F4, E4, D4, C4, B4, A4
22.         H5, G5, F5, E5, D5, C5, B5, A5
23.         H6, G6, F6, E6, D6, C6, B6, A6
24.         H7, G7, F7, E7, D7, C7, B7, A7
25.         H8, G8, F8, E8, D8, C8, B8, A8
26.         */
27.
28.         public static int[] Mg_Pawn = {
29.             0, 0, 0, 0, 0, 0, 0, 0,
30.             98, 134, 61, 95, 68, 126, 34, -11,
31.             -6, 7, 26, 31, 65, 56, 25, -20,
32.             -14, 13, 6, 21, 23, 12, 17, -23,
33.             -27, -2, -5, 12, 17, 6, 10, -25,
34.             -26, -4, -4, -10, 3, 3, 33, -12,
35.             -35, -1, -20, -23, -15, 24, 38, -22,
36.             0, 0, 0, 0, 0, 0, 0, 0,
37.         };
38.
39.         public static int[] Eg_Pawn = {
40.             0, 0, 0, 0, 0, 0, 0, 0,

```

```

41.      94, 100, 85, 67, 56, 53, 82, 84,
42.      32, 24, 13, 5, -2, 4, 17, 17,
43.      13, 9, -3, -7, -7, -8, 3, -1,
44.      4, 7, -6, 1, 0, -5, -1, -8,
45.      13, 8, 8, 10, 13, 0, 2, -7,
46.      0, 0, 0, 0, 0, 0, 0, 0,
47.      };
48.
49.      public static int[] Mg_Knight = {
50.      -167, -89, -34, -49, 61, -97, -15, -107,
51.      -73, -41, 72, 36, 23, 62, 7, -17,
52.      -47, 60, 37, 65, 84, 129, 73, 44,
53.      -9, 17, 19, 53, 37, 69, 18, 22,
54.      -13, 4, 16, 13, 28, 19, 21, -8,
55.      -23, -9, 12, 10, 19, 12, 25, -16,
56.      -29, -53, -12, -3, -1, 18, -14, -19,
57.      -105, -21, -58, -33, -17, -28, -19, -23,
58.      };
59.
60.      public static int[] Eg_Knight = {
61.      -58, -38, -13, -28, -31, -27, -63, -99,
62.      -25, -8, -25, -2, -9, -25, -24, -52,
63.      -24, -20, 10, 9, -1, -9, -19, -41,
64.      -17, 3, 22, 22, 22, 11, 8, -18,
65.      -18, -6, 16, 25, 16, 17, 4, -18,
66.      -23, -3, -1, 15, 10, -3, -20, -22,
67.      -42, -20, -10, -5, -2, -20, -23, -44,
68.      -29, -51, -23, -15, -22, -18, -50, -64,
69.      };
70.
71.      public static int[] Mg_Bishop = {
72.      -29, 4, -82, -37, -25, -42, 7, -8,
73.      -26, 16, -18, -13, 30, 59, 18, -47,
74.      -16, 37, 43, 40, 35, 50, 37, -2,
75.      -4, 5, 19, 50, 37, 37, 7, -2,
76.      -6, 13, 13, 26, 34, 12, 10, 4,
77.      0, 15, 15, 15, 14, 27, 18, 10,
78.      4, 15, 16, 0, 7, 21, 33, 1,
79.      -33, -3, -14, -21, -13, -12, -39, -21,
80.      };
81.
82.      public static int[] Eg_Bishop = {
83.      -14, -21, -11, -8, -7, -9, -17, -24,
84.      -8, -4, 7, -12, -3, -13, -4, -14,
85.      2, -8, 0, -1, -2, 6, 0, 4,
86.      -3, 9, 12, 9, 14, 10, 3, 2,
87.      -6, 3, 13, 19, 7, 10, -3, -9,
88.      -12, -3, 8, 10, 13, 3, -7, -15,
89.      -14, -18, -7, -1, 4, -9, -15, -27,
90.      -23, -9, -23, -5, -9, -16, -5, -17,
91.      };
92.
93.      public static int[] Mg_Rook = {
94.      32, 42, 32, 51, 63, 9, 31, 43,
95.      27, 32, 58, 62, 80, 67, 26, 44,
96.      -5, 19, 26, 36, 17, 45, 61, 16,
97.      -24, -11, 7, 26, 24, 35, -8, -20,
98.      -36, -26, -12, -1, 9, -7, 6, -23,
99.      -45, -25, -16, -17, 3, 0, -5, -33,
100.     -44, -16, -20, -9, -1, 11, -6, -71,
101.     -19, -13, 1, 17, 16, 7, -37, -26,
102.     };
103.
104.     public static int[] Eg_Rook = {
105.     13, 10, 18, 15, 12, 12, 8, 5,
106.     11, 13, 13, 11, -3, 3, 8, 3,

```

```

107.      7, 7, 7, 5, 4, -3, -5, -3,
108.      4, 3, 13, 1, 2, 1, -1, 2,
109.      3, 5, 8, 4, -5, -6, -8, -11,
110.     -4, 0, -5, -1, -7, -12, -8, -16,
111.     -6, -6, 0, 2, -9, -9, -11, -3,
112.     -9, 2, 3, -1, -5, -13, 4, -20,
113.      };
114.
115.      public static int[] Mg_Queen = {
116.     -28, 0, 29, 12, 59, 44, 43, 45,
117.     -24, -39, -5, 1, -16, 57, 28, 54,
118.     -13, -17, 7, 8, 29, 56, 47, 57,
119.     -27, -27, -16, -16, -1, 17, -2, 1,
120.     -9, -26, -9, -10, -2, -4, 3, -3,
121.     -14, 2, -11, -2, -5, 2, 14, 5,
122.     -35, -8, 11, 2, 8, 15, -3, 1,
123.     -1, -18, -9, 10, -15, -25, -31, -50,
124.      };
125.
126.      public static int[] Eg_Queen = {
127.     -9, 22, 22, 27, 27, 19, 10, 20,
128.     -17, 20, 32, 41, 58, 25, 30, 0,
129.     -20, 6, 9, 49, 47, 35, 19, 9,
130.     3, 22, 24, 45, 57, 40, 57, 36,
131.     -18, 28, 19, 47, 31, 34, 39, 23,
132.     -16, -27, 15, 6, 9, 17, 10, 5,
133.     -22, -23, -30, -16, -16, -23, -36, -32,
134.     -33, -28, -22, -43, -5, -32, -20, -41,
135.      };
136.
137.      public static int[] Mg_King = {
138.     -65, 23, 16, -15, -56, -34, 2, 13,
139.     29, -1, -20, -7, -8, -4, -38, -29,
140.     -9, 24, 2, -16, -20, 6, 22, -22,
141.     -17, -20, -12, -27, -30, -25, -14, -36,
142.     -49, -1, -27, -39, -46, -44, -33, -51,
143.     -14, -14, -22, -46, -44, -30, -15, -27,
144.     1, 7, -8, -64, -43, -16, 9, 8,
145.     -15, 36, 12, -54, 8, -28, 24, 14,
146.      };
147.
148.      public static int[] Eg_King = {
149.     -74, -35, -18, -18, -11, 15, 4, -17,
150.     -12, 17, 14, 17, 17, 38, 23, 11,
151.     10, 17, 23, 15, 20, 45, 44, 13,
152.     -8, 22, 24, 27, 26, 33, 26, 3,
153.     -18, -4, 21, 24, 27, 23, 9, -11,
154.     -19, -3, 11, 21, 23, 16, 7, -9,
155.     -27, -11, 4, 13, 14, 4, -5, -17,
156.     -53, -34, -21, -11, -28, -14, -24, -43
157.      };
158.
159.      public static int[] FlipTable = {
160.     63, 62, 61, 60, 59, 58, 57, 56,
161.     55, 54, 53, 52, 51, 50, 49, 48,
162.     47, 46, 45, 44, 43, 42, 41, 40,
163.     39, 38, 37, 36, 35, 34, 33, 32,
164.     31, 30, 29, 28, 27, 26, 25, 24,
165.     23, 22, 21, 20, 19, 18, 17, 16,
166.     15, 14, 13, 12, 11, 10, 9, 8,
167.     7, 6, 5, 4, 3, 2, 1, 0
168.      };
169.
170.      #endregion
171.
172.      static int[][] Mg_PieceSquareTable = {

```

```

173.             Mg_Pawn,
174.             Mg_Knight,
175.             Mg_Bishop,
176.             Mg_Rook,
177.             Mg_Queen,
178.             Mg_King
179.         };
180.
181.         static int[][] Eg_PieceSquareTable = {
182.             Eg_Pawn,
183.             Eg_Knight,
184.             Eg_Bishop,
185.             Eg_Rook,
186.             Eg_Queen,
187.             Eg_King
188.         };
189.
190.
191.         internal static double LookUp(int pieceType, int pieceIndex, int phase, bool
isWhite)
192.         {
193.             if (Convert.ToInt32(isWhite) == 1)
194.             {
195.                 pieceIndex = FlipTable[pieceIndex];
196.             }
197.
198.             if (phase == 0)
199.             { // middle game phase
200.                 return Mg_PieceSquareTable[pieceType][pieceIndex];
201.             }
202.             return Eg_PieceSquareTable[pieceType][pieceIndex];
203.
204.         }
205.
206.     }
207. }

```

Transposition table

```

1. using System;
2. using System.Linq;
3.
4. namespace Engine.Evaluation
5. {
6.
7.     public enum TTFlag
8.     {
9.         Exact, // PV Node
10.        Fail_Low_Alpha, // a better move was not found
11.        Fail_High_Beta, // beta cut off (the opponent could play a better move)
12.    }
13.
14.    internal class TTEntry // use a class because a struct is a value type and is not nullable
15.    {
16.        public ulong zobrist; // the hash of the position
17.        public int depth; // the depth that it was search at (the accuracy of the value)
18.        public TTFlag flag; // what type of node it is
19.        public double eval; // the evaluation of the position
20.        public Move bestMove; // the next best move that should be played for the position
21.
22.        public TTEntry(ulong zobrist, int depth, TTFlag flag, double eval, Move bestMove)
23.        {
24.            this.zobrist = zobrist;

```

```

25.         this.depth = depth;
26.         this.flag = flag;
27.         this.eval = eval;
28.         this.bestMove = bestMove;
29.     }
30. }
31.
32. internal class TranspositionTable
33. {
34.     TTEnter[] TTable; // the array of transposition entries
35.     uint Size; // the number of entries int the transposition table
36.
37.     int sizeMB; // the approximation of how many MB the table should occupy when full
38.
39.     // debugging vars
40.     ulong successfulAccess = 0;
41.     ulong unsuccessfulAccess = 0;
42.     ulong timesAccessed = 0;
43.     ulong writes = 0;
44.     ulong rewrites = 0;
45.     ulong rewriteCollisions = 0;
46.
47.     public TranspositionTable(int sizeMb)
48.     {
49.         sizeMB = sizeMb;
50.         int sizeBytes = sizeMB * 1000 * 1000;
51.         int EntrySizeBytes = 32; // rough estimate
52.         Size = (uint)(sizeBytes / EntrySizeBytes);
53.         TTable = new TTEnter[Size];
54.     }
55.
56.     public void add(ulong zobrist, int depth, TTFlag flag, double eval, Move bestMove)
57.     {
58.         // calculate the hash index
59.         int hashIndex = (int)(zobrist % Size);
60.         // retrieve the current item at the hash index
61.         TTEnter current = TTable[hashIndex];
62.
63.         // check if it is is free
64.         if (current == null || depth > current.depth)
65.         {
66.             // check for collision
67.             if (current != null)
68.             {
69.                 rewrites++;
70.                 if (current.zobrist != zobrist)
71.                 {
72.                     rewriteCollisions++;
73.                 }
74.             }
75.
76.             writes++;
77.             TTable[hashIndex] = new TTEnter(zobrist, depth, flag, eval, bestMove);
78.         }
79.     }
80.
81.     public bool get(ulong zobrist, double alpha, double beta, int depth, out double value)
82.     // returns the lookup value of the hash, if not found then returns false
83.     {
84.         timesAccessed++;
85.         value = 0;
86.         int hashIndex = (int)(zobrist % Size);
87.         TTEnter entry = TTable[hashIndex];
88.
89.         // check for a collision
90.         if (entry != null && entry.zobrist == zobrist)

```

```

90.         {
91.             // check to see if the position is evaluated to a higher depth than given
92.             if (entry.depth >= depth)
93.             {
94.                 successfulAccess++;
95.                 if (entry.flag == TTFlag.Exact)
96.                 {
97.                     value = entry.eval;
98.                 }
99.
100.                else if (entry.flag == TTFlag.Fail_Low_Alpha && entry.eval <=
alpha)
101.                {
102.                    value = alpha;
103.                }
104.                // caused a beta cut off (there was a better move)
105.                else if (entry.flag == TTFlag.Fail_High_Beta && alpha >= beta)
106.                {
107.                    value = entry.eval;
108.                }
109.
110.                return true;
111.            }
112.        }
113.
114.        unsuccessfulAccess++;
115.        return false;
116.    }
117.
118.
119.    public bool get(ulong zobrist, out TTEntry value) // a overload which is used
for the PV
120.    {
121.        timesAccessed++;
122.        int hashIndex = (int)(zobrist % Size);
123.        TTEntry entry = TTable[hashIndex];
124.
125.        // check for a collision
126.        if (entry != null && entry.zobrist == zobrist && entry.flag ==
TTFlag.Exact)
127.        {
128.            value = entry;
129.            return true;
130.        }
131.
132.        unsuccessfulAccess++;
133.        value = null;
134.        return false;
135.    }
136.
137.    public void ClearTable()
138.    {
139.
140.        Array.Clear(TTable, 0, (int)Size);
141.    }
142.
143.    public void resetDebuggingVars()
144.    {
145.        successfulAccess = 0;
146.        unsuccessfulAccess = 0;
147.        timesAccessed = 0;
148.        writes = 0;
149.        rewrites = 0;
150.        rewriteCollisions = 0;
151.    }
152.

```

```

153.         // gets stuck in a loop
154.     public string getPV(ulong rootZobrsit, Bitboard board)
155.     {
156.
157.         string PrincipalVariation = "";
158.         // create a copy of the position
159.         Bitboard copy = board.DeepClone();
160.
161.         ulong currentZobrist = rootZobrsit;
162.         // go down the PV until we find the leaf node
163.         while (get(currentZobrist, out TTEntry entry) && !entry.bestMove.isEmpty)
164.         {
165.             // check if the move is in the PV, if it is then it will cause a loop
166.             if (PrincipalVariation.Contains(Tools.MoveToString(entry.bestMove)))
167.             {
168.                 break;
169.             }
170.             // convert the move to a string to add to the PV
171.             PrincipalVariation += Tools.MoveToString(entry.bestMove) + " ";
172.             copy.MakeMove(entry.bestMove);
173.             // get the new zobrist hash to lookup and find the new best move down
the PV
174.             currentZobrist = copy.zobristKey.Key;
175.         }
176.
177.         return PrincipalVariation;
178.     }
179.
180.
181.     public void printStats()
182.     {
183.         Console.WriteLine($"info Size of table: {sizeMB}");
184.         Console.WriteLine($"info Times Accessed: {timesAccessed}");
185.         Console.WriteLine($"info Successful Accesses: {successfulAccess}");
186.         Console.WriteLine($"info Unsuccessful Accesses: {unsuccessfulAccess}");
187.         Console.WriteLine($"info Writes: {writes}");
188.         Console.WriteLine($"info Rewrites: {rewrites}");
189.         Console.WriteLine($"info Rewrite Collisions: {rewriteCollisions}");
190.         Console.WriteLine($"info TTEntries: {TTable.Count(s => s != null)}");
191.         Console.WriteLine($"info % of TT used: {Decimal.Divide(TTable.Count(s => s
!= null), TTable.Length) * 100}\n\n");
192.     }
193. }
194. }

```

UI

MainWindow

XAML

```

1. <Window x:Class="UI.MainWindow"
2.     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.     xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
5.     xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
6.     xmlns:local="clr-namespace:UI" xmlns:MVVM.Views="clr-namespace:UI.MVVM.Views"
7.     mc:Ignorable="d"
8.     Title="MainWindow" Height="450" Width="800">
9.     <Viewbox Stretch="Fill" StretchDirection="Both">
10.         <MVVM.Views:HomeView Height="450" Width="800"/>

```



```

11.         <!-- <MVVM.Views:PlayingScreen Height="450" Width="800"/> -->
12.     </Viewbox>
13. </Window>

```

C#

```

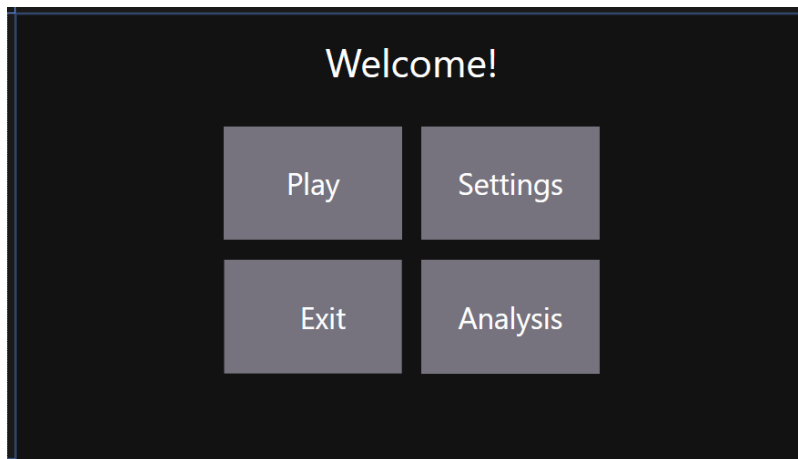
1. using System;
2. using System.Windows;
3. using UI.MVVM.Models.Players;
4.
5. namespace UI
6. {
7.     /// <summary>
8.     /// Interaction logic for MainWindow.xaml
9.     /// </summary>
10.    public partial class MainWindow : Window
11.    {
12.        public MainWindow()
13.        {
14.            InitializeComponent();
15.        }
16.
17.        protected override void OnClosed(EventArgs e)
18.        {
19.            base.OnClosed(e);
20.            Bot.connectionEnded = true;
21.            OnlinePlayer.connectionEnded = true;
22.            Application.Current.Shutdown();
23.        }
24.    }
25. }

```

Views

HomeView

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.HomeView"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:UserLoc ="clr-namespace:UI.MVVM.Models.Players"
8.             mc:Ignorable="d"
9.             d:DesignHeight="450" d:DesignWidth="800">

```

```

10.     <Grid Background="{Binding Source={x:Static UserLoc:User.Settings},
    Path=BackgroundColour.ColourBrush}">
11.         <TextBlock Text="Welcome!" FontSize="40" VerticalAlignment="Top"
    HorizontalAlignment="Center" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
    Path=TextColour.ColourBrush}" Margin="20"/>
12.
13.         <Grid Height="270" Width="400" Margin="0,25,0,0">
14.             <Grid.ColumnDefinitions>
15.                 <ColumnDefinition/>
16.                 <ColumnDefinition/>
17.             </Grid.ColumnDefinitions>
18.
19.             <Grid.RowDefinitions>
20.                 <RowDefinition/>
21.                 <RowDefinition/>
22.                 <!-- <RowDefinition/> -->
23.             </Grid.RowDefinitions>
24.
25.             <Label Content="Play" Grid.Row="0" Margin="10" Foreground="{Binding
    Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Background="{Binding
    Source={x:Static UserLoc:User.Settings}, Path=ButtonColour.ColourBrush}" MouseUp="Play_Click"
    FontSize="30" HorizontalContentAlignment="Center" VerticalContentAlignment="Center" />
26.             <!-- <Label Content="Play Online" Grid.Row="1" Margin="10" Foreground="{Binding
    Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Background="{Binding
    Source={x:Static UserLoc:User.Settings},
    Path=ButtonColour.ColourBrush}" MouseUp="PlayOnline_Click" FontSize="30"
    HorizontalContentAlignment="Center" VerticalContentAlignment="Center"/> -->
27.
28.             <Label Content="Settings" Grid.Column="1" Grid.Row="0" Margin="10"
    Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
    Background="{Binding Source={x:Static UserLoc:User.Settings}, Path=ButtonColour.ColourBrush}"
    MouseUp="Settings_Click" FontSize="30" HorizontalContentAlignment="Center"
    VerticalContentAlignment="Center"/>
29.             <Label Content="Analysis" Grid.Column="1" Grid.Row="1" Margin="10"
    Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
    Background="{Binding Source={x:Static UserLoc:User.Settings}, Path=ButtonColour.ColourBrush}"
    MouseUp="Analysis_Click" FontSize="30" HorizontalContentAlignment="Center"
    VerticalContentAlignment="Center"/>
30.
31.             <Label Content="Exit" Grid.ColumnSpan="1" Grid.Row="1" Margin="10" Width="200"
    Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
    Background="{Binding Source={x:Static UserLoc:User.Settings}, Path=ButtonColour.ColourBrush}"
    MouseUp="Exit_Click" FontSize="30" HorizontalContentAlignment="Center"
    VerticalContentAlignment="Center"/>
32.
33.
34.         </Grid>
35.     </Grid>
36. </UserControl>

```

C#

```

1. using System.Threading;
2. using System.Windows;
3. using System.Windows.Controls;
4. using UI.MVVM.Models;
5.
6. namespace UI.MVVM.Views
7. {
8.     /// <summary>
9.     /// Interaction logic for HomeView.xaml
10.    /// </summary>
11.    public partial class HomeView : UserControl
12.    {
13.        public HomeView()
14.        {

```

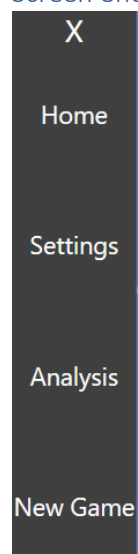
```

15.         InitializeComponent();
16.     }
17.
18.     private void Play_Click(object sender, RoutedEventArgs e)
19.     {
20.         NewGameView newGameView = new();
21.         newGameView.ShowDialog();
22.     }
23.
24.     private void PlayOnline_Click(object sender, RoutedEventArgs e)
25.     {
26.         // show the server browser
27.     }
28.
29.     private void Analysis_Click(object sender, RoutedEventArgs e)
30.     {
31.         Window mainWindow = App.Current.MainWindow;
32.         AnalysisScreen usercontrol = new();
33.         usercontrol.Height = 450;
34.         usercontrol.Width = 800;
35.         ((Viewbox)mainWindow.Content).Child = usercontrol;
36.         Thread.Sleep(500);
37.         ((AnalysisGame)usercontrol.LocalBoardView.viewModel.game).bot.SendMove();
38.     }
39.
40.     private void Settings_Click(object sender, RoutedEventArgs e)
41.     {
42.         Window mainWindow = App.Current.MainWindow;
43.         Settings usercontrol = new();
44.         usercontrol.Height = 450;
45.         usercontrol.Width = 800;
46.         ((Viewbox)mainWindow.Content).Child = usercontrol;
47.     }
48.
49.     private void Exit_Click(object sender, System.Windows.Input.MouseButtonEventArgs e)
50.     {
51.         MainWindow.GetWindow(this).Close();
52.     }
53. }
54. }

```

MainMenu

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.MainMenu"
2.     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.     xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.     xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.     xmlns:local="clr-namespace:UI.MVVM.Views"
7.     xmlns:UserLoc="clr-namespace:UI.MVVM.Models.Players"
8.     mc:Ignorable="d"
9.     d:DesignHeight="450" d:DesignWidth="100">
10.     <!-- Background="{Binding Source={x:Static UserLoc:User.Settings},
11.         Path=PanelColour.ColourBrush}"-->
12.     <Grid Background="{Binding Source={x:Static UserLoc:User.Settings},
13.         Path=PanelColour.ColourBrush}">
14.         <Grid.RowDefinitions>
15.             <RowDefinition MaxHeight="30"/>
16.             <RowDefinition/>
17.             <RowDefinition/>
18.             <RowDefinition/>
19.             <!-- <RowDefinition/> -->
20.         </Grid.RowDefinitions>
21.         <TextBlock x:Name="ExitButton" Text="X" FontSize="25" Foreground="{Binding
22.             Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
23.             VerticalAlignment="Center" Margin="0,0,0,0" TextAlignment="Center" Cursor="Hand"/>
24.         <TextBlock Grid.Row="1" Text="Home" VerticalAlignment="Center"
25.             TextAlignment="Center" TextWrapping="Wrap" FontSize="20" Foreground="{Binding Source={x:Static
26.             UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Cursor="Hand" MouseUp="Home_Click"/>
27.         <TextBlock Grid.Row="2" Text="Settings" VerticalAlignment="Center"
28.             TextAlignment="Center" TextWrapping="Wrap" FontSize="20" Foreground="{Binding Source={x:Static
29.             UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Cursor="Hand" MouseUp="Settings_Click"/>
30.         <TextBlock Grid.Row="3" Text="Analysis" VerticalAlignment="Center"
31.             TextAlignment="Center" TextWrapping="Wrap" FontSize="20" Foreground="{Binding Source={x:Static
32.             UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Cursor="Hand" MouseUp="Analysis_Click"/>
33.         <TextBlock Grid.Row="4" Text="New Game" VerticalAlignment="Center"
34.             TextAlignment="Center" TextWrapping="Wrap" FontSize="20" Foreground="{Binding Source={x:Static
35.             UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Cursor="Hand" MouseUp="NewGame_Click"/>
36.         <!-- <TextBlock Grid.Row="4" Text="Online?" VerticalAlignment="Center"
37.             TextAlignment="Center" FontSize="35" Foreground="{Binding Source={x:Static
38.             UserLoc:User.Settings}, Path=TextColour.ColourBrush}" -->
39.     </Grid>
40. </UserControl>

```

C#

```

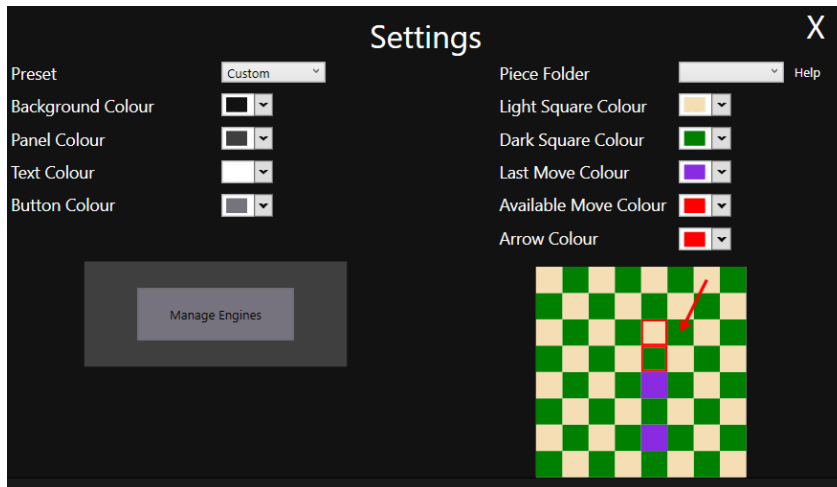
1. using System.Threading;
2. using System.Windows;
3. using System.Windows.Controls;
4. using UI.MVVM.Models;
5. using UI.MVVM.Models.Players;
6.
7. namespace UI.MVVM.Views
8. {
9.     /// <summary>
10.     /// Interaction logic for MainMenu.xaml
11.     /// </summary>
12.     public partial class MainMenu : UserControl
13.     {
14.         public MainMenu()
15.         {
16.             InitializeComponent();
17.         }

```

```
18.
19.     private void CloseThreads() {
20.         Bot.connectionEnded = true;
21.         OnlinePlayer.connectionEnded = true;
22.     }
23.
24.     private void Home_Click(object sender, RoutedEventArgs e)
25.     {
26.         Window mainWindow = App.Current.MainWindow;
27.         HomeView usercontrol = new();
28.         usercontrol.Height = 450;
29.         usercontrol.Width = 800;
30.         ((Viewbox)mainWindow.Content).Child = usercontrol;
31.         CloseThreads();
32.     }
33.
34.     private void Settings_Click(object sender, RoutedEventArgs e)
35.     {
36.         Window mainWindow = App.Current.MainWindow;
37.         Settings usercontrol = new();
38.         usercontrol.Height = 450;
39.         usercontrol.Width = 800;
40.         ((Viewbox)mainWindow.Content).Child = usercontrol;
41.         CloseThreads();
42.     }
43.
44.     private void Analysis_Click(object sender, RoutedEventArgs e)
45.     {
46.         CloseThreads();
47.         Window mainWindow = App.Current.MainWindow;
48.         AnalysisScreen usercontrol = new();
49.         usercontrol.Height = 450;
50.         usercontrol.Width = 800;
51.         ((Viewbox)mainWindow.Content).Child = usercontrol;
52.         Thread.Sleep(500);
53.         ((AnalysisGame)usercontrol.LocalBoardView.viewModel.game).bot.SendMove();
54.     }
55.
56.     private void NewGame_Click(object sender, RoutedEventArgs e)
57.     {
58.         NewGameView usercontrol = new();
59.         usercontrol.ShowDialog();
60.         CloseThreads();
61.     }
62. }
63. }
```

Settings

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.Settings"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:UserLoc="clr-namespace:UI.MVVM.Models.Players"
8.             xmlns:xctk="http://schemas.xceed.com/wpf/xaml/toolkit"
9.
10.            mc:Ignorable="d"
11.            d:DesignHeight="450" d:DesignWidth="800">
12.    <Grid x:Name="MainGrid" Background="{Binding Source={x:Static UserLoc:User.Settings},
13.        Path=BackgroundColour.ColourBrush, UpdateSourceTrigger=PropertyChanged, Mode=TwoWay}">
14.        <TextBlock Text="Settings" FontSize="30" Foreground="{Binding Source={x:Static
15.            UserLoc:User.Settings}, Path=TextColour.ColourBrush}" TextAlignment="Center"
16.            Margin="0,10,0,0"/>
17.        <TextBlock Text="X" FontSize="30" Foreground="{Binding Source={x:Static
18.            UserLoc:User.Settings}, Path=TextColour.ColourBrush}" TextAlignment="Right"
19.            Margin="762,4,20,413" Cursor="Hand" MouseUp="Exit_Click" VerticalAlignment="Center"/>
20.
21.        <Grid Margin="0, 50, 0, 0">
22.            <Grid.ColumnDefinitions>
23.                <ColumnDefinition/>
24.                <ColumnDefinition/>
25.            </Grid.ColumnDefinitions>
26.
27.            <local:BoardPreview x:Name="Boardpreview" Grid.Column="1" Height="190" Width="190"
28.                VerticalAlignment="Bottom" HorizontalAlignment="Center" Margin="5"/>
29.
30.            <Grid x:Name="PanelExample" Background="{Binding Source={x:Static
31.                UserLoc:User.Settings}, Path=PanelColour.ColourBrush, UpdateSourceTrigger=PropertyChanged,
32.                Mode=OneWay}" Grid.Column="0" Width="250" Height="100" Margin="00,90,0,0">
33.                <Button x:Name="ManageEnginesButton" Content="Manage Engines" Width="150"
34.                    Height="50" HorizontalAlignment="Center" VerticalAlignment="Center" Background="{Binding
35.                        Source={x:Static UserLoc:User.Settings}, Path=ButtonColour.ColourBrush,
36.                        UpdateSourceTrigger=PropertyChanged, Mode=OneWay}" Click="ManageEnginesButton_Click"/>
37.            </Grid>
38.
39.            <Grid Grid.Column="0">
40.                <Grid.ColumnDefinitions>
41.                    <ColumnDefinition/>
42.                    <ColumnDefinition/>
43.                </Grid.ColumnDefinitions>

```

```

33.         <StackPanel>
34.             <TextBlock Grid.Column = "0" HorizontalAlignment="Left" Text="Preset"
Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
FontSize="16" Margin="5"/>
35.             <TextBlock Grid.Column = "0" HorizontalAlignment="Left" Text="Background
Colour" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
Path=TextColour.ColourBrush}" FontSize="16" Margin="5"/>
36.             <TextBlock Grid.Column = "0" HorizontalAlignment="Left" Text="Panel
Colour" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
Path=TextColour.ColourBrush}" FontSize="16" Margin="5"/>
37.             <TextBlock Grid.Column = "0" HorizontalAlignment="Left" Text="Text Colour"
Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
FontSize="16" Margin="5"/>
38.             <TextBlock Grid.Column = "0" HorizontalAlignment="Left" Text="Button
Colour" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
Path=TextColour.ColourBrush}" FontSize="16" Margin="5"/>
39.         </StackPanel>
40.
41.         <StackPanel Grid.Column="1">
42.             <ComboBox Width="100" Margin="5" Height="20" HorizontalAlignment="Left"
x:Name="Preset" SelectionChanged="Preset_SelectionChanged">
43.                 <ComboBoxItem Content="Dark Mode"/>
44.                 <ComboBoxItem Content="Light Mode"/>
45.                 <ComboBoxItem Content="Custom" IsSelected="True"/>
46.             </ComboBox>
47.             <xctk:ColorPicker x:Name="_colorPicker" Grid.Column = "1" Width="50"
Margin="5" SelectedColor="{Binding Source={x:Static UserLoc:User.Settings},
Path=BackgroundColor.ColorPickerData, UpdateSourceTrigger=PropertyChanged, Mode=OneWay}"
HorizontalAlignment="Left" DisplayColorAndName="True"
SelectedColorChanged="Background_Select"/>
48.             <xctk:ColorPicker Grid.Column = "1" Width="50" Margin="5"
SelectedColor="{Binding Source={x:Static UserLoc:User.Settings},
Path=PanelColour.ColorPickerData, UpdateSourceTrigger=PropertyChanged, Mode=OneWay}"
HorizontalAlignment="Left" DisplayColorAndName="True" SelectedColorChanged="Panel_Select"/>
49.             <xctk:ColorPicker Grid.Column = "1" Width="50" Margin="5"
SelectedColor="{Binding Source={x:Static UserLoc:User.Settings},
Path=TextColour.ColorPickerData, UpdateSourceTrigger=PropertyChanged, Mode=OneWay}"
HorizontalAlignment="Left" DisplayColorAndName="True" SelectedColorChanged="Text_Select" />
50.             <xctk:ColorPicker Grid.Column = "1" Width="50" Margin="5"
SelectedColor="{Binding Source={x:Static UserLoc:User.Settings},
Path=ButtonColour.ColorPickerData, UpdateSourceTrigger=PropertyChanged, Mode=OneWay}"
HorizontalAlignment="Left" DisplayColorAndName="True" SelectedColorChanged="Button_Select" />
51.         </StackPanel>
52.     </Grid>
53.
54.     </Grid>
55.
56.     <Grid Grid.Column="1">
57.         <Grid.ColumnDefinitions>
58.             <ColumnDefinition Width="59*" />
59.             <ColumnDefinition Width="41*" />
60.         </Grid.ColumnDefinitions>
61.
62.         <StackPanel Grid.Column="0">
63.
64.             <TextBlock Text="Piece Folder" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="16" Margin="70 5, 0,5"/>
65.             <TextBlock Text="Light Square Colour" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="16" Margin="70, 5, 0,5"/>
66.             <TextBlock Text="Dark Square Colour" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="16" Margin="70, 5, 0,5"/>
67.             <TextBlock Text="Last Move Colour" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="16" Margin="70, 5, 0,5"/>
68.             <TextBlock Text="Available Move Colour" Foreground="{Binding
Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="16"
Margin="70, 5, 0,5"/>

```

```

69.         <TextBlock Text="Arrow Colour" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="16" Margin="70,5, 0,5"/>
70.
71.         </StackPanel>
72.
73.         <StackPanel Grid.Column="1">
74.             <StackPanel Orientation="Horizontal">
75.                 <ComboBox Width="100" Margin="5" Height="20" HorizontalAlignment="Left"
x:Name="PieceFolders" SelectionChanged="PieceFolders_SelectionChanged"/>
76.                 <TextBlock ToolTip="To add a custom piece set, copy the folder into
Assests -> Pieces and make sure the peice names are the same format as in the default folder"
Text="Help" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
Path=TextColour.ColourBrush}" VerticalAlignment="Center" Margin="5"/>
77.             </StackPanel>
78.             <xctk:ColorPicker Width="50" Margin="5" SelectedColor="{Binding
Source={x:Static UserLoc:User.Settings}, Path=LightSquareColour.ColorPickerData,
UpdateSourceTrigger=PropertyChanged, Mode=OneWay}" HorizontalAlignment="Left"
DisplayColorAndName="True" SelectedColorChanged="LightSquare_Select"/>
79.             <xctk:ColorPicker Width="50" Margin="5" SelectedColor="{Binding
Source={x:Static UserLoc:User.Settings}, Path=DarkSquareColour.ColorPickerData,
UpdateSourceTrigger=PropertyChanged, Mode=OneWay}" HorizontalAlignment="Left"
DisplayColorAndName="True" SelectedColorChanged="DarkSquare_Select" />
80.             <xctk:ColorPicker Width="50" Margin="5" HorizontalAlignment="Left"
SelectedColor="{Binding Source={x:Static UserLoc:User.Settings},
Path=LastMoveColour.ColorPickerData, UpdateSourceTrigger=PropertyChanged, Mode=OneWay}"
DisplayColorAndName="True" SelectedColorChanged="Last_Select" />
81.             <xctk:ColorPicker Width="50" Margin="5" HorizontalAlignment="Left"
SelectedColor="{Binding Source={x:Static UserLoc:User.Settings},
Path=AvailibleMoveColour.ColorPickerData, UpdateSourceTrigger=PropertyChanged, Mode=OneWay}"
DisplayColorAndName="True" SelectedColorChanged="Available_Select" />
82.             <xctk:ColorPicker Width="50" Margin="5" HorizontalAlignment="Left"
SelectedColor="{Binding Source={x:Static UserLoc:User.Settings},
Path=ArrowColour.ColorPickerData, UpdateSourceTrigger=PropertyChanged, Mode=OneWay}"
DisplayColorAndName="True" SelectedColorChanged="Arrow_Select" />
83.
84.         </StackPanel>
85.     </Grid>
86.
87. </Grid>
88. </Grid>
89. </UserControl>

```

C#

```

1. using System.Collections.Generic;
2. using System.IO;
3. using System.Windows;
4. using System.Windows.Controls;
5. using System.Windows.Input;
6. using System.Windows.Media;
7. using UI.MVVM.Models;
8. using UI.MVVM.Models.Players;
9. using Xceed.Wpf.Toolkit;
10.
11. namespace UI.MVVM.Views
12. {
13.     /// <summary>
14.     /// Interaction logic for Settings.xaml
15.     /// </summary>
16.
17.
18.     public partial class Settings : UserControl
19.     {
20.         List<TextBlock> textElements = new List<TextBlock>();
21.
22.         public Settings()

```



```
23.     {
24.         InitializeComponent();
25.         ExtractAllTextBlocks(MainGrid.Children);
26.         GetPieceFolders();
27.     }
28.
29.
30.     private void GetPieceFolders()
31.     {
32.         string RootFolder =
Directory.GetParent(System.IO.Directory.GetCurrentDirectory()).Parent.Parent.FullName;
33.         string PiecesPath = System.IO.Path.Combine(new string[] { RootFolder, "Assets",
"Pieces" });
34.
35.         string[] Folders = Directory.GetDirectories(PiecesPath);
36.         int selected = 0;
37.         List<string> acceptedFolders = new List<string>();
38.         for (int i = 0; i < Folders.Length; i++)
39.         {
40.             if (FolderContainsPieces(Folders[i]))
41.             {
42.                 Folders[i] = Folders[i].Split('\\')[Folders[i].Split('\\').Length - 1];
43.                 // make sure the folder contains all of the right pieces
44.
45.
46.
47.                 if (Folders[i] == User.Settings.PieceFolder)
48.                 {
49.                     selected = i;
50.                 }
51.                 acceptedFolders.Add(Folders[i]);
52.             }
53.
54.         }
55.         PieceFolders.ItemsSource = acceptedFolders;
56.         PieceFolders.SelectedIndex = selected;
57.     }
58.
59.     private bool FolderContainsPieces(string folername)
60.     {
61.         return true;
62.     }
63.
64.     private void ExtractAllTextBlocks(UIElementCollection Collection)
65.     {
66.         foreach (UIElement child in Collection)
67.         {
68.             if (child is TextBlock)
69.             {
70.                 textElements.Add((TextBlock)child);
71.             }
72.
73.             else if (child is Grid)
74.             {
75.                 ExtractAllTextBlocks(((Grid)child).Children);
76.             }
77.
78.             else if (child is StackPanel)
79.             {
80.                 ExtractAllTextBlocks(((StackPanel)child).Children);
81.             }
82.         }
83.     }
84.
85.     private void UpdateScreenColours()
86.     {
```

```

87.
88.         MainGrid.Background = User.Settings.BackgroundColor.ColourBrush;
89.         PanelExample.Background = User.Settings.PanelColour.ColourBrush;
90.         foreach (TextBlock i in textElements)
91.         {
92.             i.Foreground = User.Settings.TextColour.ColourBrush;
93.         }
94.         ManageEnginesButton.Background = User.Settings.ButtonColour.ColourBrush;
95.     }
96.
97.     private void BackGround_Select(object sender, RoutedEventArgs<Color?> e)
98.     {
99.         ColorPicker colorPicker = (ColorPicker)sender;
100.         Colour pickedColour = new Colour((Color)colorPicker.SelectedColor);
101.         User.Settings.BackgroundColor = pickedColour;
102.         MainGrid.Background = User.Settings.BackgroundColor.ColourBrush;
103.     }
104.
105.     private void Panel_Select(object sender, RoutedEventArgs<Color?>
106. e)
107.     {
108.         ColorPicker colorPicker = (ColorPicker)sender;
109.         Colour pickedColour = new Colour((Color)colorPicker.SelectedColor);
110.         User.Settings.PanelColour = pickedColour;
111.         PanelExample.Background = User.Settings.PanelColour.ColourBrush;
112.     }
113.     private void Text_Select(object sender, RoutedEventArgs<Color?>
114. e)
115.     {
116.         ColorPicker colorPicker = (ColorPicker)sender;
117.         Colour pickedColour = new Colour((Color)colorPicker.SelectedColor);
118.         User.Settings.TextColour = pickedColour;
119.         foreach (TextBlock i in textElements)
120.         {
121.             i.Foreground = User.Settings.TextColour.ColourBrush;
122.         }
123.     }
124.     private void PieceFolders_SelectionChanged(object sender,
125. SelectionChangedEventArgs e)
126.     {
127.         ComboBox combo = (ComboBox)sender;
128.         string selectedValue = combo.SelectedItem.ToString();
129.         User.Settings.PieceFolder = selectedValue;
130.         Boardpreview.viewModel.UpdatePreview();
131.     }
132.     private void Preset_SelectionChanged(object sender, SelectionChangedEventArgs
133. e)
134.     {
135.         ComboBox combo = (ComboBox)sender;
136.         string selectedValue =
137. ((ComboBoxItem)combo.SelectedItem).Content.ToString();
138.         if (selectedValue == "Dark Mode") User.Settings.resetDarkMode();
139.         else if (selectedValue == "Light Mode") User.Settings.resetLightMode();
140.         UpdateScreenColours();
141.     }
142.     private void LightSquare_Select(object sender,
143. RoutedEventArgs<Color?> e)
144.     {
145.         ColorPicker colorPicker = (ColorPicker)sender;
146.         Colour pickedColour = new Colour((Color)colorPicker.SelectedColor);
147.         User.Settings.LightSquareColour = pickedColour;
148.         Boardpreview.viewModel.UpdatePreview();

```

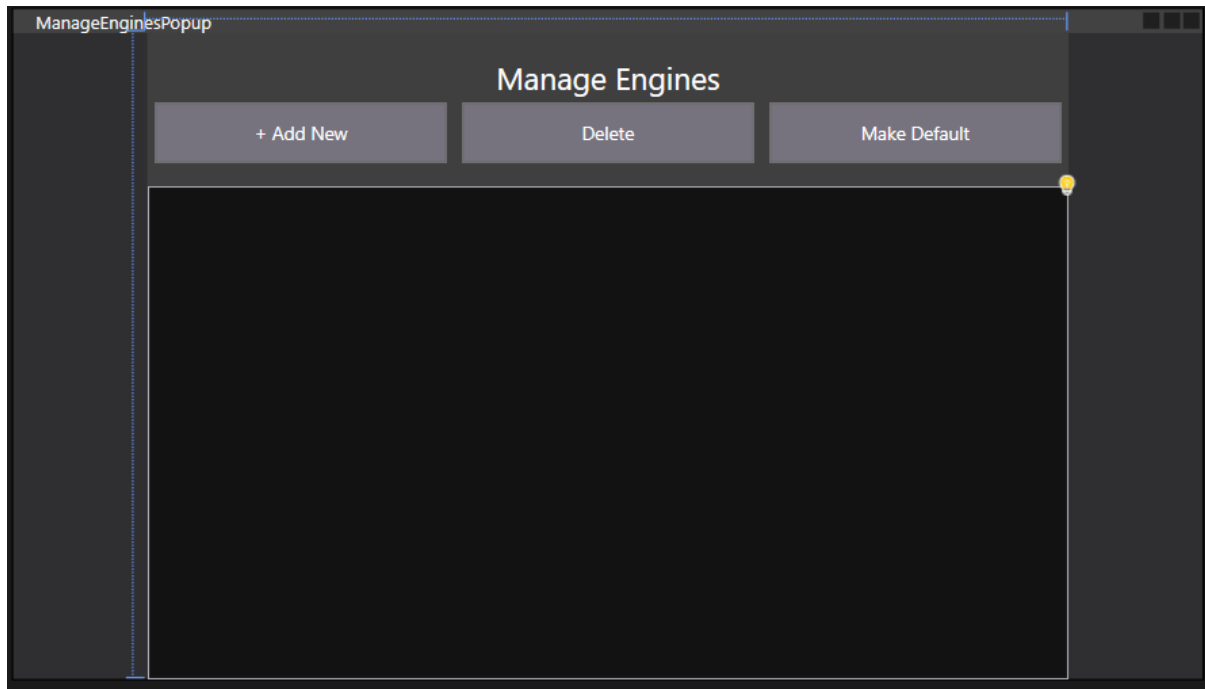
```

147.         }
148.
149.         private void DarkSquare_Select(object sender,
RoutedPropertyChangedEventArgs<Color?> e)
150.         {
151.             ColorPicker colorPicker = (ColorPicker)sender;
152.             Colour pickedColour = new Colour((Color)colorPicker.SelectedColor);
153.             User.Settings.DarkSquareColour = pickedColour;
154.             Boardpreview.viewModel.UpdatePreview();
155.         }
156.
157.         private void Available_Select(object sender,
RoutedPropertyChangedEventArgs<Color?> e)
158.         {
159.             ColorPicker colorPicker = (ColorPicker)sender;
160.             Colour pickedColour = new Colour((Color)colorPicker.SelectedColor);
161.             User.Settings.AvailableMoveColour = pickedColour;
162.             Boardpreview.viewModel.UpdatePreview();
163.         }
164.
165.         private void Arrow_Select(object sender, RoutedPropertyChangedEventArgs<Color?>
e)
166.         {
167.             ColorPicker colorPicker = (ColorPicker)sender;
168.             Colour pickedColour = new Colour((Color)colorPicker.SelectedColor);
169.             User.Settings.ArrowColour = pickedColour;
170.             Boardpreview.viewModel.UpdatePreview();
171.         }
172.
173.         private void Last_Select(object sender, RoutedPropertyChangedEventArgs<Color?>
e)
174.         {
175.             ColorPicker colorPicker = (ColorPicker)sender;
176.             Colour pickedColour = new Colour((Color)colorPicker.SelectedColor);
177.             User.Settings.LastMoveColour = pickedColour;
178.             Boardpreview.viewModel.UpdatePreview();
179.         }
180.
181.         private void Button_Select(object sender,
RoutedPropertyChangedEventArgs<Color?> e)
182.         {
183.             ColorPicker colorPicker = (ColorPicker)sender;
184.             Colour pickedColour = new Colour((Color)colorPicker.SelectedColor);
185.             User.Settings.ButtonColour = pickedColour;
186.             ManageEnginesButton.Background = User.Settings.ButtonColour.ColourBrush;
187.         }
188.
189.         private void ManageEnginesButton_Click(object sender, RoutedEventArgs e)
190.         {
191.             ManageEnginesPopup window = new ManageEnginesPopup();
192.             window.ShowDialog();
193.         }
194.
195.         private void Exit_Click(object sender, MouseButtonEventArgs e)
196.         {
197.             Window mainWindow = App.Current.MainWindow;
198.             HomeView usercontrol = new();
199.             usercontrol.Height = 450;
200.             usercontrol.Width = 800;
201.             ((Viewbox)mainWindow.Content).Child = usercontrol;
202.         }
203.     }
204. }

```

Manage Engines popup

Screen-Shot



XAML

```

1. <Window x:Class="UI.MVVM.Views.ManageEnginesPopup"
2.     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.     xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
5.     xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
6.     xmlns:local="clr-namespace:UI.MVVM.Views"
7.     xmlns:UserLoc = "clr-namespace:UI.MVVM.Models.Players"
8.
9.     mc:Ignorable="d"
10.    Title="ManageEnginesPopup" Height="450" Width="800" Background="#FF2F2E30">
11.    <Viewbox Stretch="Uniform">
12.        <Grid Background="{Binding Source={x:Static UserLoc:User.Settings},
13.            Path=PanelColour.ColourBrush}">
14.            <TextBlock Text="Manage Engines" Foreground="{Binding Source={x:Static
15.                UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="20" VerticalAlignment="Top"
16.                HorizontalAlignment="Center" Margin="15"/>
17.            <Grid Width="600" Height="380" Margin="0, 40,0,0">
18.                <Grid Height="50" VerticalAlignment="Top">
19.                    <Grid.ColumnDefinitions>
20.                        <ColumnDefinition/>
21.                        <ColumnDefinition/>
22.                        <ColumnDefinition/>
23.                    </Grid.ColumnDefinitions>
24.                    <Button Grid.Column="0" Margin="5" Content="+ Add New" Background="{Binding
25.                        Source={x:Static UserLoc:User.Settings}, Path=ButtonColour.ColourBrush}" Foreground="{Binding
26.                        Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Click="Add_Click"/>
27.                    <Button Grid.Column="1" Margin="5" Content="Delete" Background="{Binding
28.                        Source={x:Static UserLoc:User.Settings}, Path=ButtonColour.ColourBrush}" Foreground="{Binding
29.                        Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Click="Delete_Click"/>
30.                    <Button Grid.Column="2" Margin="5" Content="Make Default"
31.                        Background="{Binding Source={x:Static UserLoc:User.Settings}, Path=ButtonColour.ColourBrush}"
32.                        Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
33.                        Click="MakeDefault_Click"/>
34.                </Grid>
35.                <ListBox x:Name="Engines" Height="320" Background="{Binding Source={x:Static
36.                    UserLoc:User.Settings}, Path=BackgroundColour.ColourBrush}" VerticalAlignment="Bottom"

```

```

    Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
    BorderThickness="1"/>
26.
27.         </Grid>
28.     </Grid>
29. </Viewbox>
30. </Window>

```

C#

```

1. using Microsoft.Win32;
2. using System;
3. using System.Diagnostics;
4. using System.Threading;
5. using System.Threading.Tasks;
6. using System.Windows;
7. using System.Windows.Controls;
8. using System.Windows.Media;
9. using UI.MVVM.Models.Engine;
10. using UI.MVVM.Models.Players;
11.
12. namespace UI.MVVM.Views
13. {
14.     /// <summary>
15.     /// Interaction logic for ManageEnginesPopup.xaml
16.     /// </summary>
17.     public partial class ManageEnginesPopup : Window
18.     {
19.
20.         bool validEngine = false;
21.         bool connectionEnded = false;
22.         Process process;
23.         EngineInfo tempEngine = new EngineInfo();
24.
25.         public ManageEnginesPopup()
26.         {
27.             InitializeComponent();
28.             for (int i = 0; i < User.Engines.Count; i++)
29.             {
30.                 string engine = User.Engines[i].name;
31.
32.                 Engines.Items.Add(new ListBoxItem() { Content = engine });
33.
34.                 if (i == User.Settings.DefaultEngine)
35.                 {
36.                     ListBoxItem item = (ListBoxItem)Engines.Items[i];
37.                     item.Background = Brushes.DarkGreen;
38.                 }
39.             }
40.         }
41.
42.         private void Add_Click(object sender, RoutedEventArgs e)
43.         {
44.             OpenFileDialog fileSelect = new();
45.             fileSelect.Filter = "Executable Files(*.EXE)|*.EXE|All files (*.*)|*.*";
46.             if (fileSelect.ShowDialog() == true) // user has selected a file
47.             {
48.                 //Get the path of specified file
49.
50.                 string filePath = fileSelect.FileName;
51.
52.                 // check if it is already in the list
53.                 if (checkEngineContains(filePath))
54.                 {
55.                     MessageBox.Show("The item you selected is already in the list");
56.                     return;

```

```
57.         }
58.
59.         // validate the engine and get the information
60.         checkEngineValidity(filePath);
61.
62.         //wait for 100ms to let the validation finish
63.         Thread.Sleep(100);
64.
65.         // add the engine
66.         if (validEngine)
67.         {
68.             User._Engines.Add(tempEngine);
69.             Engines.Items.Add(new ListBoxItem() { Content = tempEngine.name });
70.         }
71.
72.         // show a error to the user
73.         else
74.         {
75.             MessageBox.Show("Not a valid engine");
76.             return;
77.         }
78.     }
79. }
80.
81. private void Delete_Click(object sender, RoutedEventArgs e)
82. {
83.     int index = Engines.SelectedIndex;
84.     if (index == 0)
85.     {
86.         MessageBox.Show("Can not delete the default engine");
87.         return;
88.     }
89.
90.     else if (index == -1)
91.     {
92.         MessageBox.Show("No item selected");
93.         return;
94.     }
95.
96.     else if (index == User.Settings.DefaultEngine)
97.     {
98.         ChangeDefault(0);
99.     }
100.
101.     User.Engines.RemoveAt(index);
102.     Engines.Items.RemoveAt(index);
103. }
104.
105. private void ChangeDefault(int index)
106. {
107.     ListBoxItem item = (ListBoxItem)Engines.Items[User.Settings.DefaultEngine];
108.     item.Background = null;
109.     item = (ListBoxItem)Engines.Items[index];
110.     item.Background = Brushes.DarkGreen;
111.     User.Settings.DefaultEngine = index;
112. }
113.
114. private void MakeDefault_Click(object sender, RoutedEventArgs e)
115. {
116.     int index = Engines.SelectedIndex;
117.
118.     if (index == -1)
119.     {
120.         MessageBox.Show("No item selected");
121.         return;
122.     }
```

```

123.
124.         ChangeDefault(index);
125.     }
126.
127.     private void checkEngineValidity(string path)
128.     {
129.         validEngine = false;
130.         connectionEnded = false;
131.
132.         tempEngine = new EngineInfo();
133.         tempEngine.path = path;
134.
135.         process = new Process()
136.         {
137.             StartInfo = new ProcessStartInfo()
138.             {
139.                 FileName = path,
140.                 UseShellExecute = false,
141.                 RedirectStandardInput = true,
142.                 RedirectStandardOutput = true,
143.                 //CreateNoWindow = true,
144.                 RedirectStandardError = true,
145.             }
146.         };
147.         /* Set your output and error (asynchronous) handlers
148.         process.OutputDataReceived += new DataReceivedEventHandler(OutputHandler);
149.         process.ErrorDataReceived += new DataReceivedEventHandler(OutputHandler);
150.         /* Start process and handlers
151.         process.Start();
152.         process.BeginOutputReadLine();
153.         process.BeginErrorReadLine();
154.         process.StandardInput.WriteLine("uci");
155.         do
156.         {
157.             Task.Delay(new TimeSpan(0, 0, 1)).ContinueWith(o => { connectionEnded =
true; });
158.         } while (!connectionEnded);
159.         process.Kill();
160.     }
161.
162.     private void OutputHandler(object sender, DataReceivedEventArgs e)
163.     {
164.         if (e.Data == null)
165.         {
166.             return;
167.         }
168.
169.         else if (e.Data.Contains("id"))
170.         {
171.             // split the id command
172.             if (e.Data.Contains("name"))
173.             {
174.                 string name = e.Data.Split(' ')[2];
175.                 tempEngine.name = name;
176.             }
177.
178.             else
179.             {
180.                 // it is the author
181.                 string author = e.Data.Split(' ')[2];
182.                 tempEngine.author = author;
183.             }
184.         }
185.
186.         else if (e.Data.Contains("option"))
187.         {

```

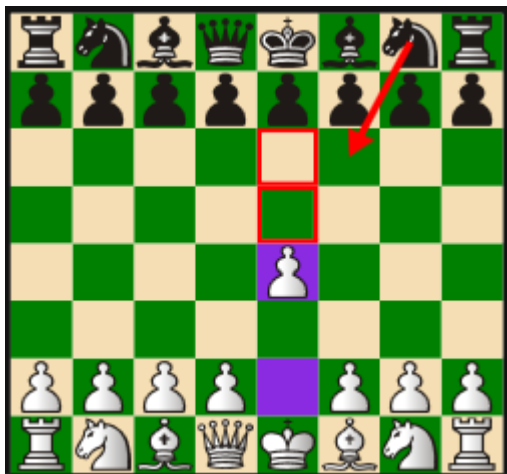
```

188.          // split and parse the option command
189.      }
190.
191.      else if (e.Data.Contains("uciok"))
192.      {
193.          process.StandardInput.WriteLine("quit");
194.          validEngine = true;
195.          connectionEnded = true;
196.      }
197.  }
198.
199.  private bool checkEngineContains(string path)
200.  {
201.      // perform a linear search
202.      foreach (EngineInfo i in User._Engines)
203.      {
204.          if (i.path == path) return true;
205.      }
206.
207.      return false;
208.  }
209.  }
210.  }
211.
212.
213.      // return the name of the engine also allow the users to input the parameters they want
      (edit engine popup)

```

BoardPreviewView

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.BoardPreview"
2.     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.     xmlns:mc="http://schemas.openxmlformats.org/markup-
   compatibility/2006"
5.     xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.     xmlns:local="clr-namespace:UI.MVVM.Views"
7.     mc:Ignorable="d"
8.     d:DesignHeight="200" d:DesignWidth="200">
9.     <Grid>
10.         <Canvas x:Name="BoardCanvas" Background="white" Height="200" Width="200"
   VerticalAlignment="Center" HorizontalAlignment="Left" ></Canvas>
11.

```



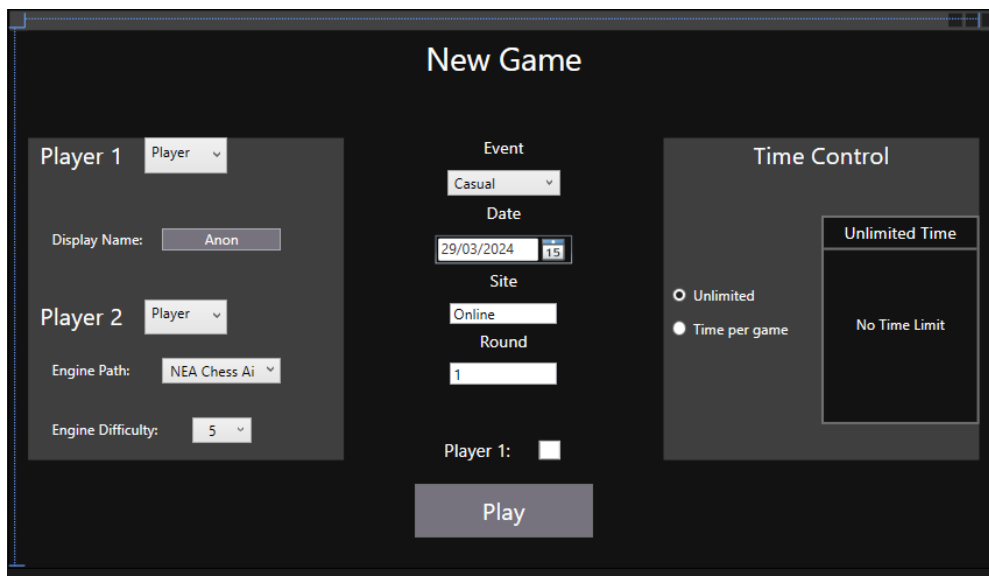
```
12.     </Grid>
13. </UserControl>
```

C#

```
1. using System.Windows.Controls;
2. using UI.MVVM.ViewModels;
3.
4. namespace UI.MVVM.Views
5. {
6.     /// <summary>
7.     /// Interaction logic for BoardPreview.xaml
8.     /// </summary>
9.     public partial class BoardPreview : UserControl
10.    {
11.        internal BoardPreiewModel viewModel;
12.
13.        public BoardPreview()
14.        {
15.            InitializeComponent();
16.            viewModel = new BoardPreiewModel(BoardCanvas);
17.        }
18.    }
19. }
```

NewGameView

Screen-Shot



XAML

```
1. <Window x:Class="UI.MVVM.Views.NewGameView"
2.         xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.         xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.         xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.         xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.         xmlns:local="clr-namespace:UI.MVVM.Views"
7.         xmlns:UserLoc ="clr-namespace:UI.MVVM.Models.Players"
8.         xmlns:sys="clr-namespace:System;assembly=mscorlib"
9.         mc:Ignorable="d">
```

```

10.         d:DesignHeight="450" d:DesignWidth="800" Background="{Binding Source={x:Static
    UserLoc:User.Settings}, Path=BackgroundColour.ColourBrush}">
11.     <Viewbox Stretch="Uniform">
12.         <Grid Background="{Binding Source={x:Static UserLoc:User.Settings},
    Path=BackgroundColour.ColourBrush}" Height="450" Width="800">
13.             <TextBlock Text="New Game" Foreground="{Binding Source={x:Static
    UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="27"
    HorizontalAlignment="Center" Margin="5"/>
14.             <Grid>
15.                 <Grid.ColumnDefinitions>
16.                     <ColumnDefinition/>
17.                     <ColumnDefinition/>
18.                     <ColumnDefinition/>
19.                 </Grid.ColumnDefinitions>
20.
21.                 <!--#region Player Select Panel-->
22.                 <Grid Grid.Column="0" Background="{Binding Source={x:Static
    UserLoc:User.Settings}, Path=PanelColour.ColourBrush}" Height="270" Width="265">
23.                     <Grid.RowDefinitions>
24.                         <RowDefinition/>
25.                         <RowDefinition/>
26.                     </Grid.RowDefinitions>
27.
28.                     <!-- Select Screen Name for human-->
29.                     <!-- Select Engine path for bot-->
30.                     <!-- This needs to be declared before the players since they use it-->
31.                     <Grid x:Name="Player1Grid" Height="100" VerticalAlignment="Bottom">
32.                         <local:NewGamePlayerSelectView/>
33.                     </Grid>
34.
35.                     <Grid Grid.Row="2" x:Name="Player2Grid" Height="100"
    VerticalAlignment="Bottom">
36.                         <local:NewGameBotSelectView/>
37.                     </Grid>
38.
39.                     <!-- Player 1 -->
40.                     <TextBlock Text="Player 1" Grid.Row="0" Foreground="{Binding
    Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
    HorizontalAlignment="Left" FontSize="20" Margin="10,0,0,0"/>
41.                     <ComboBox x:Name="Player1Combo" Height="30" Width="70"
    VerticalAlignment="Top" SelectionChanged="ComboBox_SelectionChanged" Tag="0">
42.                         <ComboBoxItem Content="Player" IsSelected="True"/>
43.                         <ComboBoxItem Content="Bot"/>
44.                     </ComboBox>
45.
46.
47.
48.                     <!-- Player 2-->
49.                     <TextBlock Text="Player 2" Grid.Row="1" Foreground="{Binding
    Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
    HorizontalAlignment="Left" FontSize="20" Margin="10,0,0,0"/>
50.                     <ComboBox x:Name="Player2Combo" Grid.Row="1" Height="30" Width="70"
    VerticalAlignment="Top" SelectionChanged="ComboBox_SelectionChanged" Tag="0">
51.                         <ComboBoxItem Content="Player" IsSelected="True"/>
52.                         <ComboBoxItem Content="Bot"/>
53.                     </ComboBox>
54.
55.                 </Grid>
56.                 <!--#endregion-->
57.
58.                 <!--#region Game Information Panel-->
59.                 <StackPanel Grid.Column="1" VerticalAlignment="Center">
60.                     <TextBlock Text="Event" Foreground="{Binding Source={x:Static
    UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="14"
    HorizontalAlignment="Center" Margin="0, 0, 0, 10"/>

```

```

61.         <ComboBox x:Name="EventCombo" Margin="0, 0, 0, 4" Width="95"
        Background="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextBoxColour.ColourBrush}">
62.             <ComboBoxItem Content="Casual" IsSelected="True"/>
63.             <ComboBoxItem Content="Competitive"/>
64.
65.         </ComboBox>
66.         <TextBlock Text="Date" Foreground="{Binding Source={x:Static
        UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="14"
        HorizontalAlignment="Center" Margin="0, 0, 0, 10"/>
67.         <DatePicker x:Name="GameDate" Margin="0, 0, 0, 4" Width="115"
        SelectedDate="{x:Static sys:DateTime.Now}"
        SelectedDateChanged="DatePicker_SelectedDateChanged"/>
68.         <TextBlock Text="Site" Foreground="{Binding Source={x:Static
        UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="14"
        HorizontalAlignment="Center" Margin="0, 0, 0, 10" LostFocus="TextBlock_LostFocus"/>
69.         <TextBox x:Name="SiteTextBox" Margin="0, 0, 0, 4" Width="90"
        Text="Online"/>
70.         <TextBlock Text="Round" Foreground="{Binding Source={x:Static
        UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="14"
        HorizontalAlignment="Center" Margin="0, 0, 0, 10"/>
71.         <TextBox x:Name="RoundTextBox" Margin="0, 0, 0, 45" Width="90" Text="1"
        TextChanged="TextBox_TextChanged" LostFocus="RoundTextBox_LostFocus"/>
72.         <StackPanel Orientation="Horizontal" Width="100">
73.             <TextBlock Text="Player 1: " Foreground="{Binding Source={x:Static
        UserLoc:User.Settings}, Path=TextColour.ColourBrush}" FontSize="15"/>
74.             <Border x:Name="Player1Border" Background="White"
        BorderBrush="LightGray" BorderThickness="1" Height="16" Width="18" Margin="20,0,0,0"
        Cursor="Hand" MouseUp="Border_MouseUp"/>
75.         </StackPanel>
76.     </StackPanel>
77.     <!--#endregion-->
78.
79.     <Grid Grid.Column="2" Background="{Binding Source={x:Static
        UserLoc:User.Settings}, Path=PanelColour.ColourBrush}" Height="270" Width="265">
80.         <TextBlock Text="Time Control" Foreground="{Binding Source={x:Static
        UserLoc:User.Settings}, Path=TextColour.ColourBrush}" HorizontalAlignment="Center"
        FontSize="20"/>
81.         <!-- Add radar buttons -->
82.         <Grid Margin="0,35,0,0">
83.             <Grid.ColumnDefinitions>
84.                 <ColumnDefinition/>
85.                 <ColumnDefinition/>
86.             </Grid.ColumnDefinitions>
87.
88.
89.             <!-- Add Dynamic Screen that will provide more option depending on the
        choice-->
90.             <Grid Grid.Column="1" x:Name="TimeControlSelectGrid" Height="175"
        VerticalAlignment="Center">
91.                 <local:UnlimitedTime/>
92.             </Grid>
93.
94.             <!-- Radio Buttons-->
95.             <StackPanel x:Name="RadioButtonsStackPanel" VerticalAlignment="Center">
96.                 <RadioButton Margin="6,0,0,13"
        Content="Unlimited" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
        Path=TextColour.ColourBrush}" Tag="0" Checked="RadioButton_Checked"
        IsChecked="True"></RadioButton>
97.                 <RadioButton Margin="6,0,0,13" Content="Time per
        game" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
        Path=TextColour.ColourBrush}" Tag="1" Checked="RadioButton_Checked" ></RadioButton>
98.             </StackPanel>
99.
100.
101.         </Grid>
102.

```

```

103.                </Grid>
104.
105.
106.
107.                </Grid>
108.                <Button Height="45" Width="150" VerticalAlignment="Bottom"
Margin="0,0,0,25" Background="{Binding Source={x:Static UserLoc:User.Settings},
Path=ButtonColour.ColourBrush}" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
Path=TextColour.ColourBrush}" Content="Play" FontSize="20" Click="Button_Click"/>
109.            </Grid>
110.        </Viewbox>
111.    </Window>

```

C#

```

1. using System;
2. using System.Linq;
3. using System.Threading;
4. using System.Windows;
5. using System.Windows.Controls;
6. using System.Windows.Input;
7. using System.Windows.Media;
8. using UI.MVVM.Models;
9. using UI.MVVM.Models.Players;
10.
11. namespace UI.MVVM.Views
12. {
13.     /// <summary>
14.     /// Interaction logic for NewGameView.xaml
15.     /// </summary>
16.     public partial class NewGameView : Window
17.     {
18.         public NewGameView()
19.         {
20.             InitializeComponent();
21.             RoundTextBox.IsUndoEnabled = true;
22.         }
23.
24.
25.
26.         private void ComboBox_SelectionChanged(object sender, SelectionChangedEventArgs e)
27.         {
28.             ComboBox playerSelect = (ComboBox)sender;
29.             Grid PlayerInformationContent;
30.             if (Grid.GetRow(playerSelect) == 0)
31.             {
32.                 PlayerInformationContent = Player1Grid;
33.             }
34.
35.             else
36.             {
37.                 PlayerInformationContent = Player2Grid;
38.             }
39.
40.             PlayerInformationContent.Children.Remove(PlayerInformationContent.Children[0]);
41.
42.             if (playerSelect.SelectedIndex == 0)
43.             {
44.                 PlayerInformationContent.Children.Add(new NewGamePlayerSelectView());
45.             }
46.             else
47.             {
48.                 PlayerInformationContent.Children.Add(new NewGameBotSelectView());
49.             }
50.         }
51.

```

```

52.     private void Button_Click(object sender, RoutedEventArgs e)
53.     {
54.         int player1Type = Player1Combo.SelectedIndex;
55.         int player2Type = Player2Combo.SelectedIndex;
56.         int[] playerTypes = { player1Type, player2Type };
57.
58.         string player1Info = Player1Grid.Children[0].ToString();
59.
60.         string player2Info = Player2Grid.Children[0].ToString();
61.
62.         string[] playerArguments = { player1Info, player2Info };
63.
64.         // validate the arguments
65.
66.         // get the time control
67.         (TimeSpan, double) TimeControl = getTimeControl();
68.
69.         if (TimeControl.Item1 == TimeSpan.Zero)
70.         { // the user did not enter any time
71.             MessageBox.Show("The playing time must be non-Zero");
72.             return;
73.         }
74.
75.         // switch the playing screen
76.         Window mainWindow = App.Current.MainWindow;
77.         PlayingScreen usercontrol = new();
78.         usercontrol.Height = mainWindow.Height;
79.         usercontrol.Width = mainWindow.Width;
80.         ((Viewbox)mainWindow.Content).Child = usercontrol;
81.
82.         string EventType = ((ComboBoxItem)EventCombo.SelectedItem).Content.ToString();
83.         string RoundNo = RoundTextBox.Text;
84.         DateTime date = ((DateTime)GameDate.SelectedDate).Date;
85.         string Site = SiteTextBox.Text;
86.
87.         // create the game
88.         CurrentGame game = new CurrentGame(usercontrol.LocalBoardView.viewModel, EventType,
89.         int.Parse(RoundNo), Site);
90.
91.         // set the game information panel
92.         usercontrol.LocalGameInformationView.ChangeGameinfo(EventType, Site, RoundNo,
93.         date);
94.
95.         // get if player 1 was black or white (true - white , false black)
96.         bool player1White = getPlayer1Colour();
97.
98.         // if player 1 is black then swap the player types and arguments
99.         if (!player1White)
100.        {
101.            playerTypes = new int[] { player2Type, player1Type };
102.            playerArguments = new string[] { player2Info, player1Info };
103.        }
104.
105.        // create the players
106.        for (int i = 0; i < 2; i++)
107.        {
108.            if (playerTypes[i] == 0)
109.            { // it is a player
110.                game.players[i] = new Models.Players.Player(game,
111.                playerArguments[i]);
112.                game.players[i].isWhite = i == 0;
113.                game.players[i].timeOnClock = TimeControl.Item1;
114.                game.players[i].clockIncrementMS = TimeControl.Item2;
115.            }
116.        }
117.        else

```

```

115.         { // it is a bot
116.             int engineIndex =
117.             int.Parse(playerArguments[i].Split("Index:")[1].Split(' ')[0]);
118.             string path = User.Engines[engineIndex].path;
119.             string depth = playerArguments[i].Split("Level:")[1];
120.             game.players[i] = new Models.Players.Bot(game, path, depth);
121.             game.players[i].isWhite = i == 0;
122.             game.players[i].timeOnClock = TimeControl.Item1;
123.             game.players[i].clockIncrementMS = TimeControl.Item2;
124.
125.             if (i == 0)
126.             { // it is the starting player
127.                 Thread.Sleep(200); // wait for the communication thread to
128.                 start
129.                 game.players[i].SendMove(); // make the bot play the first move
130.             }
131.         }
132.
133.         // set the game information
134.         usercontrol.LocalBoardView.viewModel.game = game;
135.         usercontrol.PlayerInfoView.Game = game;
136.         usercontrol.LocalBoardView.viewModel.game.playerInfo =
137.         usercontrol.PlayerInfoView;
138.
139.         // set the player information
140.         usercontrol.PlayerInfoView.ChangePlayerInfo(0, playerArguments[0], 800,
141.         "UAE");
142.         usercontrol.PlayerInfoView.ChangePlayerInfo(1, playerArguments[1], 1000,
143.         "UAE");
144.
145.         if (((ComboBoxItem)EventCombo.SelectedItem).Content.ToString() ==
146.         "Competitive") {
147.             usercontrol.PlayerInfoView.CompetitiveMode();
148.         }
149.
150.         this.Close();
151.
152.     }
153.
154.     private (TimeSpan, double) getTimeControl()
155.     {
156.         UserControl selection = (UserControl)TimeControlSelectGrid.Children[0];
157.         string time = selection.ToString();
158.
159.         if (time.Contains("Unlimited"))
160.         {
161.             return (TimeSpan.MaxValue, 0);
162.         }
163.
164.         string[] args = time.Split(' '); // in the format "Time {inMs} Inc {inMs}"
165.         double inc = 0;
166.         TimeSpan timeSpan = TimeSpan.Zero;
167.         try
168.         {
169.             timeSpan = TimeSpan.FromMilliseconds(double.Parse(args[1]));
170.             inc = double.Parse(args[3]);
171.         }
172.         catch
173.         {
174.

```

```

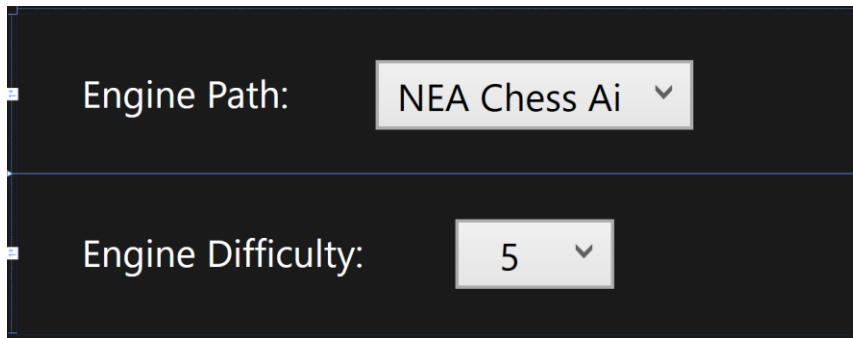
175.         return (timeSpan, inc);
176.     }
177.
178.     private void RadioButton_Checked(object sender, RoutedEventArgs e)
179.     {
180.         int button = int.Parse(((RadioButton)sender).Tag.ToString());
181.         foreach (RadioButton radio in
182.             RadioButtonsStackPanel.Children.OfType<RadioButton>())
183.         { // loop through each button and deselect each but the currently selected
184.             one
185.             if (radio != sender)
186.             {
187.                 radio.IsChecked = false;
188.             }
189.         }
190.         // change the time control grid
191.         UserControl selection = (UserControl)TimeControlSelectGrid.Children[0];
192.         TimeControlSelectGrid.Children.Remove(selection);
193.         switch (button)
194.         {
195.             case 0:
196.                 TimeControlSelectGrid.Children.Add(new UnlimitedTime() {
197.                     HorizontalAlignment = HorizontalAlignment.Center });
198.                 break;
199.             case 1:
200.                 TimeControlSelectGrid.Children.Add(new TimePerGame() {
201.                     HorizontalAlignment = HorizontalAlignment.Center });
202.                 break;
203.         }
204.     }
205.
206.     private void DatePicker_SelectedDateChanged(object sender,
207.         SelectionChangedEventArgs e)
208.     {
209.         DateTime newDate = (DateTime)e.AddedItems[0];
210.         if (newDate.Date > DateTime.Today)
211.         {
212.             DateTime original = (DateTime)e.RemovedItems[0];
213.             MessageBox.Show("You can not pick a date in the future");
214.             ((DatePicker)sender).SelectedDate = original;
215.         }
216.     }
217.
218.     private void TextBox_TextChanged(object sender, TextChangedEventArgs e)
219.     {
220.         TextBox source = (TextBox)sender;
221.         string newText = (source).Text;
222.         bool succesful = int.TryParse(newText, out int result);
223.         if (!succesful && newText != "" || (result < 1 && succesful))
224.         {
225.             // we need to call a dispatcher as the undo operation is still being
226.             prepared. https://stackoverflow.com/questions/11115777/unable-to-use-undo-in-textchanged
227.             Dispatcher.BeginInvoke(new Action(() => source.Undo()));
228.         }
229.     }
230.
231.     private void RoundTextBox_LostFocus(object sender, RoutedEventArgs e)
232.     {
233.         TextBox source = (TextBox)sender;
234.         string newText = (source).Text;
235.         bool succesful = int.TryParse(newText, out int result);
236.         if (!succesful || result < 1)
237.         {

```

```
235.         source.Text = "1";
236.     }
237. }
238.
239. private void TextBlock_LostFocus(object sender, RoutedEventArgs e)
240. {
241.     TextBox source = (TextBox)sender;
242.     string newText = (source).Text;
243.     if (newText == "")
244.     {
245.         source.Text = "Online";
246.     }
247. }
248.
249. private void Border_MouseUp(object sender, MouseButtonEventArgs e)
250. {
251.     Border player1Colour = (Border)sender;
252.     if (player1Colour.Background == Brushes.White)
253.         player1Colour.Background = Brushes.Black;
254.
255.     else if (player1Colour.Background == Brushes.Black)
256.     {
257.         player1Colour.Background = null;
258.         // add ?
259.         player1Colour.Child = new TextBlock() { Text = "?", Foreground =
User.Settings.TextColour.ColourBrush, FontSize = 10, TextAlignment = TextAlignment.Center };
260.     }
261.
262.     else if (player1Colour.Background == null)
263.     {
264.         // remove ?
265.         player1Colour.Child = null;
266.         player1Colour.Background = Brushes.White;
267.     }
268. }
269.
270. private bool getPlayer1Colour()
271. {
272.
273.     if (Player1Border.Background == Brushes.White) return true;
274.
275.     else if (Player1Border.Background == Brushes.Black) return false;
276.
277.     // random chance that player 1 is white
278.     Random random = new Random();
279.
280.     if (random.Next(1, 100) > 50) return true;
281.
282.     return false;
283. }
284. }
285. }
```


NewGameBotSelect

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.NewGameBotSelectView"
2.     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.     xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.     xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.     xmlns:local="clr-namespace:UI.MVVM.Views"
7.     mc:Ignorable="d"
8.     xmlns:UserLoc = "clr-namespace:UI.MVVM.Models.Players"
9.
10.    d:DesignHeight="100" d:DesignWidth="265">
11.    <Grid>
12.        <Grid.RowDefinitions>
13.            <RowDefinition/>
14.            <RowDefinition/>
15.        </Grid.RowDefinitions>
16.        <TextBlock Text="Engine Path:" VerticalAlignment="Center" Margin="20,0,0,0" Width="80"
17.            Height="18" HorizontalAlignment="Left" Foreground="{Binding Source={x:Static
18.                UserLoc:User.Settings}, Path=TextColour.ColourBrush}" />
19.        <ComboBox x:Name="EnginePathCombo" Text="Anon" HorizontalContentAlignment="Center"
20.            VerticalAlignment="Center" Margin="60,0, 0, 0" Width="100" HorizontalAlignment="Center"
21.            Foreground="Black" ItemsSource="{Binding Source={x:Static UserLoc:User.EngineName}}"
22.            SelectedIndex="0"/>
23.
24.        <TextBlock Grid.Row="1" Text="Engine Difficulty:" VerticalAlignment="Center"
25.            Margin="20,0,0,0" Width="90" HorizontalAlignment="Left" Foreground="{Binding Source={x:Static
26.                UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Height="18"
27.            RenderTransformOrigin="0.5,0.5">
28.            </TextBlock>
29.        <ComboBox Grid.Row="1" x:Name="EngineDifficulty" Text="Anon"
30.            HorizontalContentAlignment="Center" VerticalAlignment="Center" Margin="60,0, 0, 0" Width="50"
31.            HorizontalAlignment="Center" Foreground="Black" SelectedIndex="4">
32.            <ComboBoxItem Content="1"/>
33.            <ComboBoxItem Content="2"/>
34.            <ComboBoxItem Content="3"/>
35.            <ComboBoxItem Content="4"/>
36.            <ComboBoxItem Content="5"/>
37.            <ComboBoxItem Content="6"/>
38.        </ComboBox>
39.
40.    </Grid>
41. </UserControl>

```

C#

```

1. using System.Windows.Controls;
2. using UI.MVVM.Models.Players;
3.
4. namespace UI.MVVM.Views
5. {

```

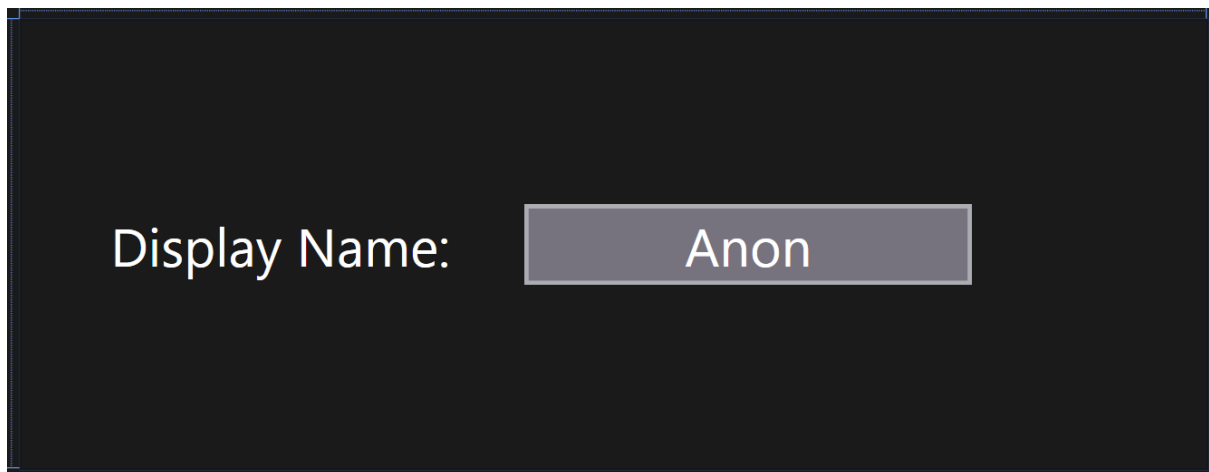
```

6.    /// <summary>
7.    /// Interaction logic for NewGameBotSelectView.xaml
8.    /// </summary>
9.    public partial class NewGameBotSelectView : UserControl
10.   {
11.       public NewGameBotSelectView()
12.       {
13.           InitializeComponent();
14.           EnginePathCombo.SelectedIndex = User.Settings.DefaultEngine;
15.       }
16.
17.       public override string ToString()
18.       {
19.           string difficultyLevel = (EngineDifficulty.SelectedIndex + 1).ToString();
20.           return $"Name:{EnginePathCombo.Text} Index:{EnginePathCombo.SelectedIndex}
21.           Level:{difficultyLevel}";
22.       }
23.   }

```

NewPlayerSelectView

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.NewGamePlayerSelectView"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:UserLoc = "clr-namespace:UI.MVVM.Models.Players"
8.
9.             mc:Ignorable="d"
10.            d:DesignHeight="100" d:DesignWidth="265">
11.     <Grid>
12.         <TextBlock Text="Display Name:" VerticalAlignment="Center" Margin="20" Width="80"
13.             HorizontalAlignment="Left" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
14.             Path=TextColour.ColourBrush}"/>
15.         <TextBox x:Name="NameTextBox" Text="Anon" HorizontalContentAlignment="Center"
16.             VerticalAlignment="Center" Margin="60,0, 0, 0" Width="100" HorizontalAlignment="Center"
17.             Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
18.             Background="{Binding Source={x:Static UserLoc:User.Settings},
19.             Path=TextBoxColour.ColourBrush}"/>
20.     </Grid>

```

15. </UserControl>

C#

```

1. using System.Windows.Controls;
2.
3. namespace UI.MVVM.Views
4. {
5.     /// <summary>
6.     /// Interaction logic for NewGamePlayerSelectView.xaml
7.     /// </summary>
8.     public partial class NewGamePlayerSelectView : UserControl
9.     {
10.         public NewGamePlayerSelectView()
11.         {
12.             InitializeComponent();
13.         }
14.
15.         public override string ToString()
16.         {
17.             return NameTextBox.Text;
18.         }
19.     }
20. }

```

TimePerGame

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.TimePerGame"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:UserLoc="clr-namespace:UI.MVVM.Models.Players"
8.             mc:Ignorable="d"
9.             d:DesignHeight="174" d:DesignWidth="130">
10.     <Border BorderBrush="#707070" BorderThickness="2">
11.         <Grid Background="{Binding Source={x:Static UserLoc:User.Settings},
Path=BackgroundColour.ColourBrush}">

```

```

12.         <TextBlock Text="Time Per Game" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" HorizontalAlignment="Center"
FontSize="14" Margin="3"/>
13.
14.         <!-- Time Select Content-->
15.         <Border BorderBrush="#707070" BorderThickness="0,2,0,0" Margin="0,25,0,0">
16.             <Grid>
17.                 <Grid.RowDefinitions>
18.                     <RowDefinition/>
19.                     <RowDefinition/>
20.                 </Grid.RowDefinitions>
21.                 <!-- Time input -->
22.                 <Border BorderBrush="#707070" BorderThickness="0,0,0,1">
23.                     <Grid>
24.                         <Grid.RowDefinitions>
25.                             <RowDefinition MaxHeight="50"/>
26.                             <RowDefinition/>
27.                             <RowDefinition/>
28.                             <RowDefinition/>
29.                         </Grid.RowDefinitions>
30.
31.                         <Grid.ColumnDefinitions>
32.                             <ColumnDefinition/>
33.                             <ColumnDefinition/>
34.                             <ColumnDefinition/>
35.                         </Grid.ColumnDefinitions>
36.
37.                         <TextBlock Text="Time" Grid.Row="0" Grid.ColumnSpan="4"
Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
HorizontalAlignment="Center" Margin="0,3,0,14" Grid.RowSpan="2"/>
38.                         <!-- HH -->
39.                         <TextBox x:Name="TimeHH" Text="00" Grid.Row="2" Height="18"
Width="20" HorizontalAlignment="Center" SelectionChanged="TextBox_SelectionChanged"
GotFocus="TextBox_Focus" LostFocus="TextBox_LostFocus"/>
40.                         <RepeatButton Height="10" Width="10"
Grid.Row="1" HorizontalAlignment="Center" Content="^" Tag="0 U" Click="RepeatButton_Click"/>
41.                         <RepeatButton Height="10" Width="10"
Grid.Row="3" HorizontalAlignment="Center" Content="V" Tag="0 D" Click="RepeatButton_Click"/>
42.
43.                         <TextBlock Text=":" Grid.Row="2" Grid.ColumnSpan="2"
Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
HorizontalAlignment="Center" Grid.RowSpan="2"/>
44.
45.
46.                         <!-- MM -->
47.                         <TextBox x:Name="TimeMM" Text="00" Grid.Row="2" Grid.Column="1"
Height="18" Width="20" HorizontalAlignment="Center" SelectionChanged="TextBox_SelectionChanged"
GotFocus="TextBox_Focus" LostFocus="TextBox_LostFocus"/>
48.                         <RepeatButton Height="10" Width="10" Grid.Row="1"
Grid.Column="1" HorizontalAlignment="Center" Content="^" Tag="1 U"
Click="RepeatButton_Click"/>
49.                         <RepeatButton Height="10" Width="10" Grid.Row="3"
Grid.Column="1" HorizontalAlignment="Center" Content="V" Tag="1 D"
Click="RepeatButton_Click"/>
50.
51.                         <TextBlock Text=":" Grid.Row="2" Grid.Column="1"
Grid.ColumnSpan="2" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
Path=TextColour.ColourBrush}" HorizontalAlignment="Center" Grid.RowSpan="2"/>
52.
53.                         <!-- SS -->
54.                         <TextBox x:Name="TimeSS" Text="00" Grid.Row="2" Grid.Column="2"
Height="18" Width="20" HorizontalAlignment="Center" SelectionChanged="TextBox_SelectionChanged"
GotFocus="TextBox_Focus" LostFocus="TextBox_LostFocus"/>
55.                         <RepeatButton Height="10" Width="10" Grid.Row="1" Grid.Column="2"
HorizontalAlignment="Center" Content="^" Tag="2 U" Click="RepeatButton_Click" />

```

```

56.         <RepeatButton Height="10" Width="10" Grid.Row="3" Grid.Column ="2"
HorizontalAlignment="Center" Content="V" Tag="2 D" Click="RepeatButton_Click"/>
57.     </Grid>
58. </Border>
59. <!-- Inc -->
60.     <Grid Grid.Row="1">
61.         <Grid.RowDefinitions>
62.             <RowDefinition MaxHeight="50"/>
63.             <RowDefinition/>
64.             <RowDefinition/>
65.             <RowDefinition/>
66.         </Grid.RowDefinitions>
67.
68.         <Grid.ColumnDefinitions>
69.             <ColumnDefinition/>
70.             <ColumnDefinition/>
71.             <ColumnDefinition/>
72.             <ColumnDefinition/>
73.         </Grid.ColumnDefinitions>
74.
75.         <TextBlock Text="Increment" Grid.Row="0" Grid.ColumnSpan="4"
Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
HorizontalAlignment="Center" Margin="0,3,0,14" Grid.RowSpan="2"/>
76.         <!-- HH -->
77.         <TextBox x:Name="InchH" Text="00" Grid.Row="2" Height="18" Width="20"
HorizontalAlignment="Center" SelectionChanged="TextBox_SelectionChanged"
GotFocus="TextBox_Focus" LostFocus="TextBox_LostFocus"/>
78.         <RepeatButton Height="10" Width="10"
Grid.Row="1" HorizontalAlignment="Center" Content="3 U" Click="RepeatButton_Click"/>
79.         <RepeatButton Height="10" Width="10"
Grid.Row="3" HorizontalAlignment="Center" Content="V" Tag="3 D" Click="RepeatButton_Click"/>
80.
81.         <TextBlock Text=":" Grid.Row="2" Grid.ColumnSpan="2"
Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
HorizontalAlignment="Center" Grid.RowSpan="2"/>
82.
83.
84.         <!-- MM -->
85.         <TextBox x:Name="IncMM" Text="00" Grid.Row="2" Grid.Column="1"
Height="18" Width="20" HorizontalAlignment="Center" SelectionChanged="TextBox_SelectionChanged"
GotFocus="TextBox_Focus" LostFocus="TextBox_LostFocus"/>
86.         <RepeatButton Height="10" Width="10" Grid.Row="1"
Grid.Column="1" HorizontalAlignment="Center" Content="^" Tag="4 U"
Click="RepeatButton_Click"/>
87.         <RepeatButton Height="10" Width="10" Grid.Row="3"
Grid.Column="1" HorizontalAlignment="Center" Content="V" Tag="4 D"
Click="RepeatButton_Click"/>
88.
89.         <TextBlock Text=":" Grid.Row="2" Grid.ColumnSpan="2"
Grid.Column="1" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
Path=TextColour.ColourBrush}" HorizontalAlignment="Center" Grid.RowSpan="2"/>
90.
91.
92.         <!-- SS -->
93.         <TextBox x:Name="IncSS" Text="00" Grid.Row="2" Grid.Column ="2"
Height="18" Width="20" HorizontalAlignment="Center" SelectionChanged="TextBox_SelectionChanged"
GotFocus="TextBox_Focus" LostFocus="TextBox_LostFocus"/>
94.         <RepeatButton Height="10" Width="10" Grid.Row="1" Grid.Column ="2"
HorizontalAlignment="Center" Content="^" Tag="5 U" Click="RepeatButton_Click"/>
95.         <RepeatButton Height="10" Width="10" Grid.Row="3" Grid.Column ="2"
HorizontalAlignment="Center" Content="V" Tag="5 D" Click="RepeatButton_Click"/>
96.
97.         <TextBlock Text=":" Grid.Row="2" Grid.ColumnSpan="2"
Grid.Column="2" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
Path=TextColour.ColourBrush}" HorizontalAlignment="Center" Grid.RowSpan="2"/>
98.

```

```

99.
100.                <!-- SS -->
101.                <TextBox x:Name="IncMS" Text="00" Grid.Row="2"    Grid.Column =
    "3" Height="18" Width="20" HorizontalAlignment="Center"
    SelectionChanged="TextBox_SelectionChanged" GotFocus="TextBox_Focus"
    LostFocus="TextBox_LostFocus"/>
102.                <RepeatButton Height="10" Width="10" Grid.Row="1"
    Grid.Column = "3" HorizontalAlignment="Center" Content="^" Tag="6 U"
    Click="RepeatButton_Click"/>
103.                <RepeatButton Height="10" Width="10" Grid.Row="3"
    Grid.Column = "3" HorizontalAlignment="Center" Content="V" Tag="6 D"
    Click="RepeatButton_Click"/>
104.                </Grid>
105.
106.                </Grid>
107.                </Border>
108.            </Grid>
109.        </Border>
110.    </UserControl>

```

C#

```

1. using System;
2. using System.Windows;
3. using System.Windows.Controls;
4. using System.Windows.Controls.Primitives;
5.
6. namespace UI.MVVM.Views
7. {
8.     /// <summary>
9.     /// Interaction logic for TimePerGame.xaml
10.    /// </summary>
11.    public partial class TimePerGame : UserControl
12.    {
13.
14.        TextBox[] textBoxes;
15.        string OldText = string.Empty;
16.
17.        public TimePerGame()
18.        {
19.            InitializeComponent();
20.            textBoxes = new TextBox[] { TimeHH, TimeMM, TimeSS, IncHH, IncMM, IncSS, IncMS };
21.        }
22.
23.        public override string ToString()
24.        {
25.            string time = "Time";
26.            int timeinMs = 0;
27.            timeinMs += int.Parse(TimeHH.Text) * 3600000;
28.            timeinMs += int.Parse(TimeMM.Text) * 60000;
29.            timeinMs += int.Parse(TimeSS.Text) * 1000;
30.
31.            time += $" {timeinMs} Inc";
32.
33.            int incinMs = 0;
34.            incinMs += int.Parse(IncHH.Text) * 3600000;
35.            incinMs += int.Parse(IncMM.Text) * 60000;
36.            incinMs += int.Parse(IncSS.Text) * 1000;
37.            incinMs += int.Parse(IncMS.Text);
38.
39.            time += $" {incinMs}";
40.            return time;
41.        }
42.
43.        private void RepeatButton_Click(object sender, RoutedEventArgs e)
44.        {

```

```
45.         string[] tag = ((string)((RepeatButton)sender).Tag).Split(' ');
46.         int index = int.Parse(tag[0]);
47.
48.         TextBox box = textBoxes[index];
49.         int current = int.Parse(box.Text);
50.         if (tag[1] == "U") // inc
51.         {
52.             if (current == 99)
53.             {
54.                 MessageBox.Show("99 is the max");
55.                 return;
56.             }
57.
58.             current++;
59.             box.Text = current.ToString("00");
60.
61.         }
62.         else
63.         { // dec
64.             if (current == 0)
65.             {
66.                 MessageBox.Show("Cannot go below 0");
67.                 return;
68.             }
69.             current--;
70.             box.Text = current.ToString("00");
71.
72.         }
73.     }
74.
75.     private void TextBox_Focus(object sender, EventArgs e)
76.     {
77.         TextBox box = (TextBox)sender;
78.         OldText = box.Text;
79.     }
80.
81.     private void TextBox_LostFocus(object sender, EventArgs e)
82.     {
83.         TextBox box = (TextBox)sender;
84.         if (box.Text == "")
85.         {
86.             box.Text = "00";
87.         }
88.     }
89.
90.     private void TextBox_SelectionChanged(object sender, RoutedEventArgs e)
91.     {
92.         TextBox box = (TextBox)sender;
93.         int _temp;
94.         if (int.TryParse(box.Text, out _temp) && !(_temp > 99 || _temp < 0) || box.Text ==
95.         "" )
96.         {
97.             return;
98.         }
99.         else
100.        {
101.            box.Text = OldText;
102.        }
103.
104.        // add inc/dec button
105.
106.
107.    }
```

```
108.    }
```

UnlimitedTime

Screen-Shot



XAML

```
1. <UserControl x:Class="UI.MVVM.Views.UnlitedTime"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:UserLoc="clr-namespace:UI.MVVM.Models.Players"
8.             mc:Ignorable="d"
9.             d:DesignHeight="174" d:DesignWidth="130">
10.    <Border BorderBrush="#707070" BorderThickness="2">
11.        <Grid Background="{Binding Source={x:Static UserLoc:User.Settings},
12.            Path=BackgroundColour.ColourBrush}">
13.            <TextBlock Text="Unlimited Time" Foreground="{Binding Source={x:Static
14.                UserLoc:User.Settings}, Path=TextColour.ColourBrush}" HorizontalAlignment="Center"
15.                FontSize="14" Margin="3"/>
16.            <!-- Time Select Content-->
17.            <Border BorderBrush="#707070" BorderThickness="0,2,0,0" Margin="0,25,0,0">
18.                <Grid Margin="0,0,0,0">
19.                    <TextBlock Text="No Time Limit" TextWrapping="Wrap" Margin="0,0,0,20"
20.                        HorizontalAlignment="Center" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
21.                            Path=TextColour.ColourBrush}" VerticalAlignment="Center"/>
22.                </Grid>
23.            </Border>
24.        </Grid>
25.    </Border>
26. </UserControl>
```

C#

```
1. using System.Windows.Controls;
2.
3. namespace UI.MVVM.Views
4. {
5.     /// <summary>
6.     /// Interaction logic for UnlitedTime.xaml
7.     /// </summary>
```



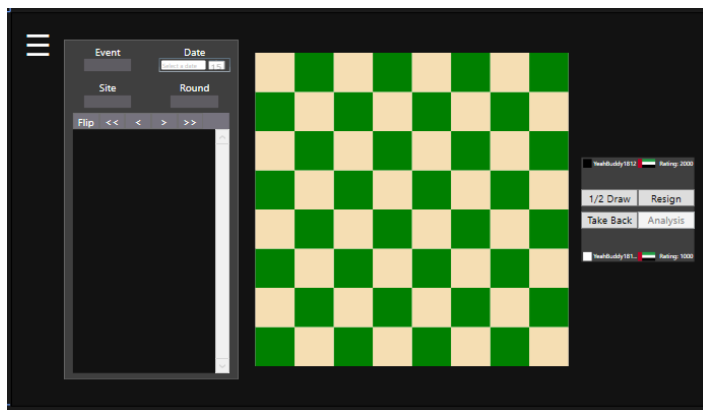
```

8.     public partial class UnlimitedTime : UserControl
9.     {
10.        public UnlimitedTime()
11.        {
12.            InitializeComponent();
13.        }
14.
15.        public override string ToString()
16.        {
17.            return "Time Unlimited Inc None";
18.        }
19.    }
20. }

```

PlayingScreen

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.PlayingScreen"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:localPlayingScreen="clr-namespace:UI.MVVM.Views.Playing_Screen"
8.
9.             xmlns:UserLoc ="clr-namespace:UI.MVVM.Models.Players"
10.            mc:Ignorable="d"
11.            d:DesignHeight="450" d:DesignWidth="800">
12.
13.
14.    <Grid Background="{Binding Source={x:Static UserLoc:User.Settings},
15.        Path=BackgroundColour.ColourBrush}">
16.
17.        <TextBlock
18.            Text="≡"
19.            FontSize="35"
20.            Height="auto"
21.            Width="auto"
22.            Margin="14,11,754,392"
23.            Foreground="{Binding Source={x:Static UserLoc:User.Settings},
24.                Path=TextColour.ColourBrush}"
25.            Cursor="Hand"
26.            MouseUp="ShowMenu"
27.        />

```

```

28.         <local:MainMenu x:Name="MainMenu" Margin="0 10 0 0" Height="420" Width="100"
           HorizontalAlignment="Left" Visibility="Hidden" Panel.ZIndex="1"/>
29.
30.
31.         <local:GameInformationView x:Name="LocalGameInformationView" Margin="60 0 0 0"
           Height="390" Width="200" HorizontalAlignment="Left"/>
32.
33.
34.         <local:BoardView x:Name="LocalBoardView" Height="360" Width="360" Margin="120 0 0 0"/>
35.
36.         <local:PlayerInfoView x:Name="PlayerInfoView" Height="150" Width="130"
           HorizontalAlignment="Right" Margin="0 0 15 0"/>
37.
38.         <local:PlayingScreen.GameOver x:Name="GameOverOverlay" Height="450" Width="800"
           Visibility="Hidden"/>
39.
40.     </Grid>
41. </UserControl>

```

C#

```

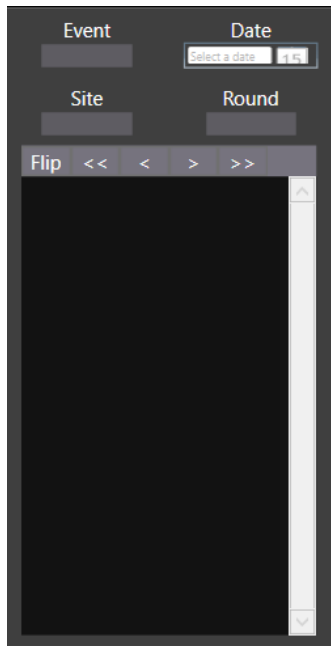
1. using System.Security.Permissions;
2. using System.Windows;
3. using System.Windows.Controls;
4. using System.Windows.Input;
5. using UI.MVVM.Models;
6.
7. namespace UI.MVVM.Views
8. {
9.     /// <summary>
10.    /// Interaction logic for PlayingScreen.xaml
11.    /// </summary>
12.    public partial class PlayingScreen : UserControl
13.    {
14.        public PlayingScreen()
15.        {
16.            InitializeComponent();
17.            LocalGameInformationView._FlipButton.Click += FlipBoard_Click;
18.            LocalGameInformationView._FirstMove.Click += BoardStateNavigation_Click;
19.            LocalGameInformationView._LastMove.Click += BoardStateNavigation_Click;
20.            LocalGameInformationView._NextMove.Click += BoardStateNavigation_Click;
21.            LocalGameInformationView._PreviousMove.Click += BoardStateNavigation_Click;
22.            PlayerInfoView.Game = LocalBoardView.viewModel.game;
23.            LocalBoardView.viewModel.game.playerInfo = PlayerInfoView;
24.            MainMenu.ExitButton.MouseUp += ShowMenu;
25.            GameOverOverlay.OkButton.MouseUp += hideGameOver;
26.        }
27.
28.        private void ShowMenu(object sender, MouseButtonEventArgs e)
29.        {
30.            if (MainMenu.Visibility == Visibility.Hidden)
31.                MainMenu.Visibility = Visibility.Visible;
32.
33.            else
34.                MainMenu.Visibility = Visibility.Hidden;
35.
36.            e.Handled = true;
37.        }
38.
39.        private void FlipBoard_Click(object sender, RoutedEventArgs e)
40.        {
41.            LocalBoardView.viewModel.WhiteView = !LocalBoardView.viewModel.WhiteView;
42.        }
43.
44.        private void BoardStateNavigation_Click(object sender, RoutedEventArgs e)
45.        {

```

```
46.         string senderTag = ((Button)sender).Tag.ToString();
47.     try
48.     {
49.         switch (senderTag)
50.         {
51.             case "0":
52.                 LocalBoardView.viewModel.game.currentMoveNo = 0;
53.                 break;
54.             case "1":
55.                 LocalBoardView.viewModel.game.currentMoveNo -= 1;
56.                 break;
57.             case "2":
58.                 LocalBoardView.viewModel.game.currentMoveNo += 1;
59.
60.                 break;
61.             case "3":
62.                 LocalBoardView.viewModel.game.currentMoveNo =
CurrentGame.movesPlayed.Count - 1;
63.
64.                 break;
65.         }
66.     }
67.     catch
68.     {
69.         // do nothing
70.     }
71. }
72.
73. public void displayGameOver(string result)
74. {
75.
76.     GameOverOverlay.changeMesage(result);
77.     GameOverOverlay.Visibility = Visibility.Visible;
78.     PlayerInfoView.endgame();
79. }
80.
81. void hideGameOver(object sender, RoutedEventArgs e) {
82.     GameOverOverlay.Visibility = Visibility.Hidden;
83. }
84.
85. }
86. }
```

GameInformationView

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.GameInformationView"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:UserLoc="clr-namespace:UI.MVVM.Models.Players"
8.             mc:Ignorable="d"
9.             d:DesignHeight="390" d:DesignWidth="200">
10.     <Grid Background="White">
11.         <StackPanel Background="{Binding Source={x:Static UserLoc:User.Settings},
12.             Path=PanelColour.ColourBrush}" Width="200" Height="390" HorizontalAlignment="Left"
13.             VerticalAlignment="Top">
14.             <!--The move and game information panel-->
15.             <Grid Margin="0 5 0 0">
16.                 <Grid.ColumnDefinitions>
17.                     <ColumnDefinition/>
18.                     <ColumnDefinition/>
19.                 </Grid.ColumnDefinitions>
20.                 <StackPanel Grid.Column="0">
21.                     <TextBlock Text="Event" Foreground="{Binding Source={x:Static
22.                         UserLoc:User.Settings}, Path=TextColour.ColourBrush}" HorizontalAlignment="Center"/>
23.                     <TextBox x:Name="EventBox" IsEnabled="False" Background="{Binding
24.                         Source={x:Static UserLoc:User.Settings}, Path=TextBoxColour.ColourBrush}" Width="57"
25.                         Height="16" BorderBrush="{x:Null}" Foreground="{Binding Source={x:Static
26.                         UserLoc:User.Settings}, Path=TextColour.ColourBrush}"/>
27.                     <TextBlock Text="Site" Foreground="{Binding Source={x:Static
28.                         UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Margin="0 10 0 0"
29.                         HorizontalAlignment="Center"/>
30.                     <TextBox x:Name="SiteBox" IsEnabled="False" Background="{Binding
31.                         Source={x:Static UserLoc:User.Settings}, Path=TextBoxColour.ColourBrush}" Width="57"
32.                         Height="16" BorderBrush="{x:Null}" Foreground="{Binding Source={x:Static
33.                         UserLoc:User.Settings}, Path=TextColour.ColourBrush}"/>
34.                     </StackPanel>
35.                 <StackPanel Grid.Column="1">

```

```

28.         <TextBlock Text="Date" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" HorizontalAlignment="Center"/>
29.         <DatePicker x:Name="GameDate" IsEnabled="False" Width="82" Height="16"
FontSize="7"/>
30.
31.         <TextBlock Text="Round" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Margin="0 10 0 0"
HorizontalAlignment="Center"/>
32.         <TextBox x:Name="RoundBox" IsEnabled="False" Background="{Binding
Source={x:Static UserLoc:User.Settings}, Path=TextBoxColour.ColourBrush}" Width="57"
Height="16" BorderBrush="{x:Null}" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}"/>
33.     </StackPanel>
34. </Grid>
35.     <Grid Background="{Binding Source={x:Static UserLoc:User.Settings},
Path=ButtonColour.ColourBrush}" Margin="10 5 10 0">
36.         <Grid.ColumnDefinitions>
37.             <ColumnDefinition/>
38.             <ColumnDefinition/>
39.             <ColumnDefinition/>
40.             <ColumnDefinition/>
41.             <ColumnDefinition/>
42.             <ColumnDefinition/>
43.         </Grid.ColumnDefinitions>
44.         <Button Grid.Column="0" x:Name="_FlipButton" Content="Flip" Height="19"
Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
Background="{x:Null}"/>
45.         <Button Grid.Column="1" Tag="0" x:Name="_FirstMove" Content="<<" Height="19"
Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
Background="{x:Null}"/>
46.         <Button Grid.Column="2" Tag="1" x:Name="_PreviousMove" Content="<"
Height="19" Foreground="{Binding Source={x:Static UserLoc:User.Settings},
Path=TextColour.ColourBrush}" Background="{x:Null}"/>
47.         <Button Grid.Column="3" Tag="2" x:Name="_NextMove" Content=">" Height="19"
Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
Background="{x:Null}"/>
48.         <Button Grid.Column="4" Tag="3" x:Name="_LastMove" Content=">>" Height="19"
Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
Background="{x:Null}"/>
49.     </Grid>
50.     <ScrollViewer Height="280" Margin="10 0" Background="{Binding Source={x:Static
UserLoc:User.Settings}, Path=BackgroundColour.ColourBrush}" BorderBrush="{Binding
Source={x:Static UserLoc:User.Settings}, Path=PanelColour.ColourBrush}"
BorderThickness="10,10,10,10">
51.         <StackPanel x:Name="Moves">
52.             </StackPanel>
53.     </ScrollViewer>
54.
55. </StackPanel>
56. </Grid>
57. </UserControl>

```

C#

```

1. using System;
2. using System.Windows.Controls;
3.
4. namespace UI.MVVM.Views
5. {
6.     /// <summary>
7.     /// Interaction logic for GameInformationView.xaml
8.     /// </summary>
9.     public partial class GameInformationView : UserControl
10.    {
11.        public static StackPanel movesStackPanel = new StackPanel();
12.        public GameInformationView()

```

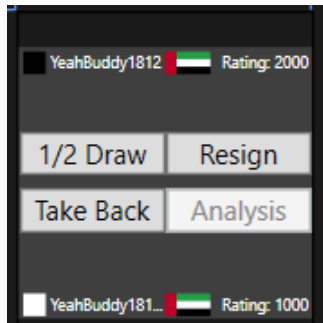
```

13.     {
14.         InitializeComponent();
15.         movesStackPanel = Moves;
16.     }
17.
18.
19.     public void ChangeGameinfo(string Event, string Site, string Round, DateTime Date)
20.     {
21.         EventBox.Text = Event;
22.         SiteBox.Text = Site;
23.         RoundBox.Text = Round;
24.         GameDate.SelectedDate = Date;
25.     }
26. }
27. }

```

PlayerInformationView

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.PlayerInfoView"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:UserLoc="clr-namespace:UI.MVVM.Models.Players"
8.             mc:Ignorable="d"
9.             d:DesignHeight="150" d:DesignWidth="130">
10.     <Grid>
11.         <TextBlock x:Name="player2Clock" Text="00:00:00" HorizontalAlignment="Center"
12.             VerticalAlignment="Top" Height="11" Width="50" FontSize="8" TextAlignment="Center"
13.             Margin="0,0,0,2" Visibility="Hidden" Foreground="{Binding Source={x:Static
14.                 UserLoc:User.Settings}, Path=TextColour.ColourBrush}"/>
15.         <Grid Background="{Binding Source={x:Static UserLoc:User.Settings},
16.             Path=PanelColour.ColourBrush}" Height="120" Width="130">
17.             <Grid.ColumnDefinitions>
18.                 <ColumnDefinition/>
19.                 <ColumnDefinition/>
20.             </Grid.ColumnDefinitions>
21.             <StackPanel Grid.Column="0">
22.                 <StackPanel Orientation="Horizontal">
23.                     <Border x:Name="player2Colour" Height="10" Width="10" Background="Black"
24.                         Margin="2 2 0 0"/>
25.                     <TextBlock x:Name="player2Name" Text="YeahBuddy1812" Foreground="{Binding
26.                         Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Margin="2 2 0 0"
27.                         TextWrapping="Wrap" Width="51" FontSize="7"/>
28.                 </StackPanel>
29.             </Grid>
30.         </Grid>
31.     </UserControl>

```

```

24.         <Button x:Name="DrawButton" Content="1/2 Draw" Margin="0 25 0 0"
Click="Draw_Click"/>
25.         <Button x:Name="TakeBackButton" Content="Take Back" Margin="0 5 0 25"
Click="TakeBack_Click"/>
26.         <StackPanel Orientation="Horizontal">
27.             <Border x:Name="player1Colour" Height="10" Width="10" Background="White"
Margin="2 2 0 0"/>
28.             <TextBlock x:Name="player1Name" Text="YeahBuddy181..." Foreground="{Binding
Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Margin="2 2 0 0"
TextWrapping="Wrap" Width="51" FontSize="7"/>
29.         </StackPanel>
30.     </StackPanel>
31.
32.     <StackPanel Grid.Column="1">
33.         <StackPanel Orientation="Horizontal">
34.             <Image x:Name="player2Flag" Width="20" Height="10" Margin="0 2 1 0"
Source="D:\OneDrive\OneDrive - Bury Grammar Schools\A level\Computer science\NEA Chess Ai
Project\UI\UI\Assets\Flags\flag_uae.gif"/>
35.             <TextBlock x:Name="player2Rating" Text="Rating: 2000" Foreground="{Binding
Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Margin="4 2 0 0"
TextWrapping="Wrap" FontSize="7"/>
36.         </StackPanel>
37.         <Button x:Name="ResignButton" Margin="0 25 0 0" Content="Resign"
Click="Resign_Click"/>
38.         <Button x:Name="AnalysisButton" Margin="0 5 0 25" Content="Analysis"
IsEnabled="False" Click="Analysis_Click"/>
39.         <StackPanel Orientation="Horizontal">
40.             <Image x:Name="player1Flag" Width="20" Height="10" Margin="0 2 1 0"
Source="D:\OneDrive\OneDrive - Bury Grammar Schools\A level\Computer science\NEA Chess Ai
Project\UI\UI\Assets\Flags\flag_uae.gif"/>
41.             <TextBlock x:Name="player1Rating" Text="Rating: 1000" Foreground="{Binding
Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}" Margin="4 2 0 0"
TextWrapping="Wrap" Width="40" FontSize="7"/>
42.         </StackPanel>
43.     </StackPanel>
44. </Grid>
45.
46.     <TextBlock x:Name="player1Clock" Text="00:00:00" HorizontalAlignment="Center"
VerticalAlignment="Bottom" Height="11" Width="50" FontSize="8" TextAlignment="Center"
Margin="0,0,0,2" Visibility="Hidden" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" />
47. </Grid>
48. </UserControl>

```

C#

```

1. using System;
2. using System.Collections.Generic;
3. using System.Diagnostics;
4. using System.Linq;
5. using System.Threading;
6. using System.Windows;
7. using System.Windows.Controls;
8. using System.Windows.Threading;
9. using UI.MVVM.Models;
10. using UI.MVVM.Models.Players;
11.
12. namespace UI.MVVM.Views
13. {
14.     /// <summary>
15.     /// Interaction logic for PlayerInfoView.xaml
16.     /// </summary>
17.     public partial class PlayerInfoView : UserControl
18.     {
19.         bool timeUnlimited = true;
20.

```

```

21.         bool competitive = false;
22.
23.         private CurrentGame game;
24.
25.         DispatcherTimer clockTimer;
26.         Stopwatch timer = new Stopwatch();
27.
28.         TextBlock[] clocks;
29.
30.         internal CurrentGame Game
31.         {
32.             get { return game; }
33.             set
34.             {
35.                 game = value;
36.                 if (game.players[0].timeOnClock != TimeSpan.MaxValue)
37.                 {
38.                     timeUnlimited = false;
39.                     showClocks();
40.                 }
41.             }
42.         }
43.
44.         public PlayerInfoView()
45.         {
46.             InitializeComponent();
47.             clocks = new TextBlock[] { player1Clock, player2Clock };
48.         }
49.
50.         public void switchClocks()
51.         { // adds the clock increment and starts the clock on the first move
52.             if (!timeUnlimited)
53.             {
54.                 Player currentPlayer = game.players[game.board.WhiteToPlay ? 1 : 0];
55.                 TextBlock ClockTextBlock = clocks[game.board.WhiteToPlay ? 1 : 0];
56.                 currentPlayer.timeOnClock += TimeSpan.FromMilliseconds(currentPlayer.clockIncrement);
57.                 ClockTextBlock.Text =
58.                 $"{currentPlayer.timeOnClock.Hours.ToString("D2")}:{currentPlayer.timeOnClock.Minutes.ToString("D2")}";
59.                 if (game.MovesPlayedCount == 2)
60.                 {
61.                     ToggleTimer();
62.                 }
63.             }
64.
65.         private void ToggleTimer()
66.         {
67.             if (clockTimer.IsEnabled)
68.             {
69.                 clockTimer.Stop();
70.                 timer.Stop();
71.                 timer.Reset();
72.             }
73.             else
74.             {
75.                 timer.Start();
76.                 clockTimer.Start();
77.             }
78.         }
79.
80.         public void showClocks()
81.         {
82.             clocks[0].Visibility = Visibility.Visible;
83.             TimeSpan player1clock = game.players[0].timeOnClock;
84.             clocks[0].Text = $"{player1clock.Hours.ToString("00")}:{player1clock.Minutes.ToString("00")}";
85.

```



```

86.         clocks[1].Visibility = Visibility.Visible;
87.         TimeSpan player2clock = game.players[1].timeOnClock;
88.         clocks[1].Text = $"{player2clock.Hours.ToString("D2")}:{player2clock.Minutes.ToString("D2")}:{player2clock.Seconds.ToString("D2")}:{player2clock.Milliseconds.ToString("D3")}";
89.
90.         clockTimer = new DispatcherTimer();
91.         clockTimer.Tick += updateClock;
92.         clockTimer.Interval = TimeSpan.FromMilliseconds(10);
93.     }
94.
95.     public void updateClock(object sender, EventArgs e)
96.     {
97.         // change time on clock
98.         bool isWhite = (game.MovesPlayedCount % 2) == 1;
99.         Player currentPlayer = game.players[isWhite ? 0 : 1];
100.         TextBlock ClockTextBlock = clocks[isWhite ? 0 : 1];
101.         currentPlayer.timeOnClock -= timer.Elapsed;
102.
103.         ClockTextBlock.Text =
104.             $"{CurrentPlayer.timeOnClock.Hours.ToString("D2")}:{CurrentPlayer.timeOnClock.Minutes.ToString("D2")}:{CurrentPlayer.timeOnClock.Seconds.ToString("D2")}:{CurrentPlayer.timeOnClock.Milliseconds.ToString("D3")}";
105.         timer.Restart();
106.
107.         if (CurrentPlayer.timeOnClock.TotalMilliseconds <= 0)
108.         {
109.             clockTimer.Stop();
110.             timer.Stop();
111.             ((CurrentGame)game).endGame(isWhite ? 1 : 0);
112.             currentPlayer.timeOnClock = new TimeSpan(0);
113.             ClockTextBlock.Text =
114.                 $"{CurrentPlayer.timeOnClock.Hours.ToString("D2")}:{CurrentPlayer.timeOnClock.Minutes.ToString("D2")}:{CurrentPlayer.timeOnClock.Seconds.ToString("D2")}:{CurrentPlayer.timeOnClock.Milliseconds.ToString("D3")}";
115.         }
116.     }
117.
118.     public void ChangePlayerInfo(int playerNo, string UserName, int Elo, string flag)
119.     {
120.         if (playerNo == 0)
121.         {
122.             //player1Colour;
123.             //player1Flag;
124.             player1Name.Text = UserName;
125.             player1Name.ToolTip = UserName;
126.             player1Rating.Text = "Rating: " + Elo.ToString();
127.         }
128.         else
129.         {
130.             player2Rating.Text = "Rating: " + Elo.ToString();
131.             player2Name.Text = UserName;
132.         }
133.     }
134.
135.     public void CompetitiveMode()
136.     {
137.         TakeBackButton.IsEnabled = false;
138.         competitive = true;
139.     }
140.
141.     private void Resign_Click(object sender, RoutedEventArgs e)
142.     {
143.         bool isWhite = (game.MovesPlayedCount % 2) == 1;
144.         Player currentPlayer = game.players[isWhite ? 0 : 1];
145.         Player OtherPlayer = game.players[isWhite ? 1 : 0];
146.
147.

```

```

148.         if (CurrentPlayer is Player) // it is the players turn
149.         {
150.             ((CurrentGame)game).endGame(isWhite ? 1 : 0);
151.         }
152.         else if (OtherPlayer is Player)
153.         {
154.             game.endGame(isWhite ? 1 : 0);
155.         }
156.     }
157.
158.     private void Draw_Click(object sender, RoutedEventArgs e)
159.     {
160.         // get current player
161.         bool isWhite = (game.MovesPlayedCount % 2) == 1;
162.         Player CurrentPlayer = game.players[isWhite ? 0 : 1];
163.         Player OtherPlayer = game.players[isWhite ? 1 : 0];
164.
165.         if (CurrentPlayer is Player) // it is the players turn
166.         {
167.             if (OtherPlayer is Player)
168.             {
169.                 ((CurrentGame)game).endGame(2);
170.             }
171.
172.             else
173.             { // the other player is the engine
174.                 if (competitive)
175.                 { // reject if it is competitive
176.                     return;
177.                 }
178.                 ((Bot)OtherPlayer).SendRequest("draw");
179.             }
180.         }
181.
182.         else
183.         { // it is not the players turn
184.             if (competitive)
185.             { // reject if it is competitive
186.                 return;
187.             }
188.             ((Bot)OtherPlayer).SendRequest("draw");
189.         }
190.     }
191.
192.     private void TakeBack_Click(object sender, RoutedEventArgs e)
193.     {
194.         Player CurrentPlayer = game.players[game.board.WhiteToPlay ? 0 : 1];
195.         Player OtherPlayer = game.players[game.board.WhiteToPlay ? 1 : 0];
196.
197.         if (CurrentPlayer is Bot)
198.         { // the player can not request a takeback when it is not their turn
199.             return;
200.         }
201.
202.         if (CurrentPlayer is Player && OtherPlayer is Bot)
203.         {
204.             CurrentPlayer.SendRequest("TakeBack1");
205.             return;
206.         }
207.
208.         if (CurrentPlayer is Player && OtherPlayer is Player)
209.         {
210.             CurrentPlayer.SendRequest("TakeBack2");
211.             return;
212.         }
213.

```

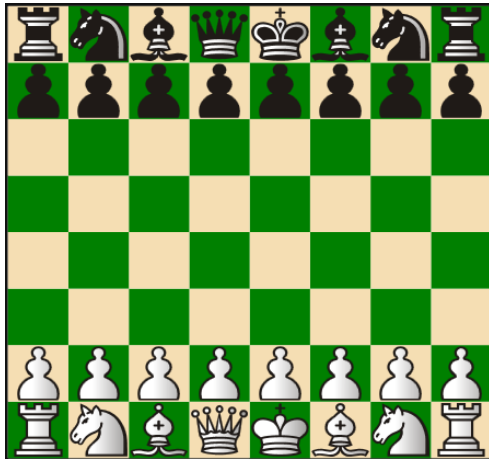
```

214.
215.     }
216.
217.     private void Analysis_Click(object sender, RoutedEventArgs e)
218.     {
219.         Window mainWindow = App.Current.MainWindow;
220.         List<UIMove> moves = UI.MVVM.Models.Game.movesPlayed;
221.         Grid[] gameInformationView = GameInformationView.movesStackPanel.Children.OfTy
222.         GameInformationView.movesStackPanel.Children.Clear();
223.         AnalysisScreen usercontrol = new();
224.         usercontrol.Height = 450;
225.         usercontrol.Width = 800;
226.         ((Viewbox)mainWindow.Content).Child = usercontrol;
227.         UI.MVVM.Models.Game.movesPlayed = moves;
228.
229.
230.         for (int i = 0; i < gameInformationView.Length; i++)
231.         {
232.             // add a new event handler to be handled by the analysis board
233.             foreach (Button j in gameInformationView[i].Children.OfType<Button>())
234.             {
235.                 j.Click += (sender, e) =>
236.                 {
237.                     ((AnalysisGame)usercontrol.LocalBoardView.viewModel.game).changeBo
238.                 };
239.             }
240.             // aad each element to the stack panel
241.             GameInformationView.movesStackPanel.Children.Add(gameInformationView[i]);
242.         }
243.
244.         // wait for the bot to set up
245.         Thread.Sleep(500);
246.         // start the analysis
247.         ((AnalysisGame)usercontrol.LocalBoardView.viewModel.game).bot.SendMove();
248.     }
249.
250.     public void ShowTakeBackRequest() // for online
251.     {
252.
253.     }
254.
255.     public void ShowDrawRequest() // for online
256.     {
257.
258.     }
259.
260.     public void endgame()
261.     {
262.
263.         if (clockTimer != null && clockTimer.IsEnabled)
264.         {
265.             ToggleTimer();
266.         }
267.
268.         ResignButton.IsEnabled = false;
269.         TakeBackButton.IsEnabled = false;
270.         DrawButton.IsEnabled = false;
271.         AnalysisButton.IsEnabled = true;
272.     }
273. }
274. }

```

BoardView

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.BoardView"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:viewModel="clr-namespace:UI.MVVM.ViewModels"
8.
9.             mc:Ignorable="d"
10.            d:DesignHeight="360" d:DesignWidth="360">
11.
12.        <Canvas x:Name="BoardCanvas" Background="white" Height="360" Width="360"
13.            VerticalAlignment="Center" HorizontalAlignment="Left" ></Canvas>
14. </UserControl>

```

C#

```

1. using System.Windows.Controls;
2. using UI.MVVM.ViewModels;
3.
4. namespace UI.MVVM.Views
5. {
6.     /// <summary>
7.     /// Interaction logic for BoardView.xaml
8.     /// </summary>
9.     public partial class BoardView : UserControl
10.    {
11.        internal BoardViewModel viewModel;
12.
13.        public BoardView()
14.        {
15.            InitializeComponent();
16.            viewModel = new BoardViewModel(BoardCanvas);
17.        }
18.
19.
20.    }
21. }

```

Promotion

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.Promotion"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:svg="https://github.com/dotnetprojects/SVGImage"
8.             xmlns:UserLoc="clr-namespace:UI.MVVM.Models.Players"
9.             mc:Ignorable="d"
10.            d:DesignHeight="180" d:DesignWidth="45">
11.    <StackPanel x:Name="promotion" Background="{Binding Source={x:Static
    UserLoc:User.Settings}, Path=PanelColour.ColourBrush}">
12.        <svg:SVGImage Tag="q" Height="45" MouseDown="PieceClick"/> <!-- Queen -->
13.        <svg:SVGImage Tag="r" Height="45" MouseDown="PieceClick"/> <!-- Rook -->
14.        <svg:SVGImage Tag="b" Height="45" MouseDown="PieceClick"/> <!-- Bishop -->
15.        <svg:SVGImage Tag="n" Height="45" MouseDown="PieceClick"/> <!-- Knight -->
16.
17.
18.    </StackPanel>
19. </UserControl>

```

C#

```

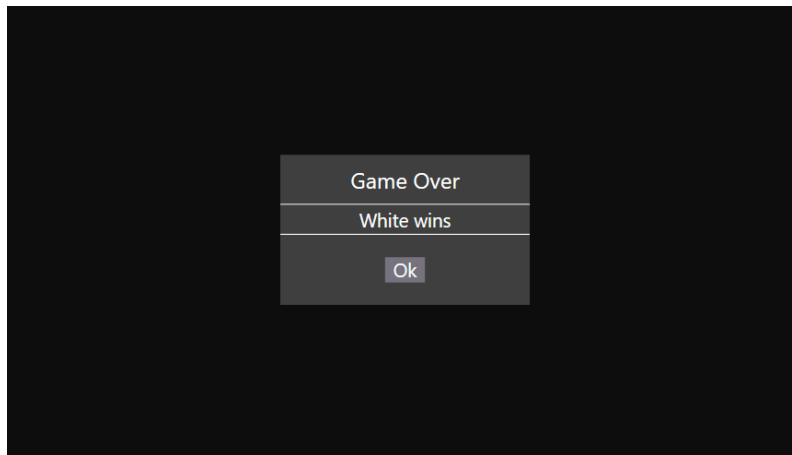
1. using Engine;
2. using System;
3. using System.Linq;
4. using System.Windows;
5. using System.Windows.Controls;
6. using System.Windows.Input;
7. using UI.MVVM.Models.Players;
8.
9. namespace UI.MVVM.Views
10. {
11.     /// <summary>
12.     /// Interaction logic for Promotion.xaml
13.     /// </summary>
14.
15.     public partial class Promotion : UserControl
16.     {
17.         internal char promotionClick = 'q';
18.         internal StackPanel promotionStack;
19.         internal Move move;
20.         internal string colour;
21.
22.         public Promotion(Move move, string colour)
23.         {

```

```
24.         InitializeComponent();
25.         promotionStack = promotion;
26.         this.move = move;
27.         this.colour = colour;
28.         setImageSource();
29.     }
30.
31.     private void PieceClick(object sender, MouseEventArgs e)
32.     {
33.         char type = ((string)((SVGImage.SVG.SVGImage)sender).Tag)[0];
34.         promotionClick = type;
35.         genMove();
36.     }
37.
38.     private void setImageSource()
39.     {
40.
41.         string filePath = User.assetsFolder + $"\\Pieces\\{User.Settings.PieceFolder}";
42.
43.         foreach (SVGImage.SVG.SVGImage i in
promotion.Children.OfType<SVGImage.SVG.SVGImage>())
44.         {
45.             i.HorizontalContentAlignment = HorizontalAlignment.Center;
46.             i.VerticalContentAlignment = VerticalAlignment.Center;
47.             i.UriSource = new Uri(filePath + $"\\{colour +
((string)i.Tag).ToUpper()}.svg");
48.         }
49.     }
50.
51.     private void genMove()
52.     {
53.         byte from = move.startIndex;
54.         byte to = move.destinationIndex;
55.         byte flag = 0;
56.         switch (promotionClick)
57.         {
58.             case 'q':
59.                 flag = 0b0111;
60.                 break;
61.
62.             case 'r':
63.                 flag = 0b0110;
64.                 break;
65.
66.             case 'b':
67.                 flag = 0b0101;
68.                 break;
69.
70.             case 'n':
71.                 flag = 0b0100;
72.                 break;
73.         }
74.
75.         move = new Move(from, to, flag);
76.     }
77.
78. }
79. }
```

GameOver

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.Playing_Screen.GameOver"
2.     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.     xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.     xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.     xmlns:local="clr-namespace:UI.MVVM.Views.Playing_Screen"
7.     xmlns:UserLoc="clr-namespace:UI.MVVM.Models.Players"
8.     mc:Ignorable="d"
9.     d:DesignHeight="450" d:DesignWidth="800">
10.     <Grid>
11.
12.         <Rectangle Height="450" Width="800" Fill="Black" Opacity="0.5"/>
13.
14.         <Grid VerticalAlignment="Center" HorizontalAlignment="Center" Width="250" Height="150"
15.             Background="{Binding Source={x:Static UserLoc:User.Settings}, Path=PanelColour.ColourBrush}">
16.             <Grid.RowDefinitions>
17.                 <RowDefinition MaxHeight="50"/>
18.                 <RowDefinition MaxHeight="30"/>
19.             </Grid.RowDefinitions>
20.             <Border BorderThickness="0 0 0 1" BorderBrush="{Binding Source={x:Static
21.                 UserLoc:User.Settings}, Path=TextColour.ColourBrush}">
22.                 <TextBlock Text="Game Over" VerticalAlignment="Center"
23.                     HorizontalAlignment="Center" FontSize="22" Foreground="{Binding Source={x:Static
24.                         UserLoc:User.Settings}, Path=TextColour.ColourBrush}"/>
25.             </Border>
26.
27.             <Border Grid.Row="1" BorderThickness="0 0 0 1" BorderBrush="{Binding
28.                 Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}">
29.                 <TextBlock x:Name="MessageTextBlock" Text="White
30.                     wins" VerticalAlignment="Center" HorizontalAlignment="Center" FontSize="19"
31.                     Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"/>
32.             </Border>
33.
34.             <TextBlock x:Name="OkButton" Grid.Row="2" Width="40" TextAlignment="Center"
35.                 Text="Ok" Cursor="Hand" VerticalAlignment="Center" HorizontalAlignment="Center" FontSize="19"
36.                 Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
37.                 Background="{Binding Source={x:Static UserLoc:User.Settings}, Path=ButtonColour.ColourBrush}"/>
38.             </TextBlock>
39.         </Grid>
40.     </Grid>
41. </UserControl>

```

C#

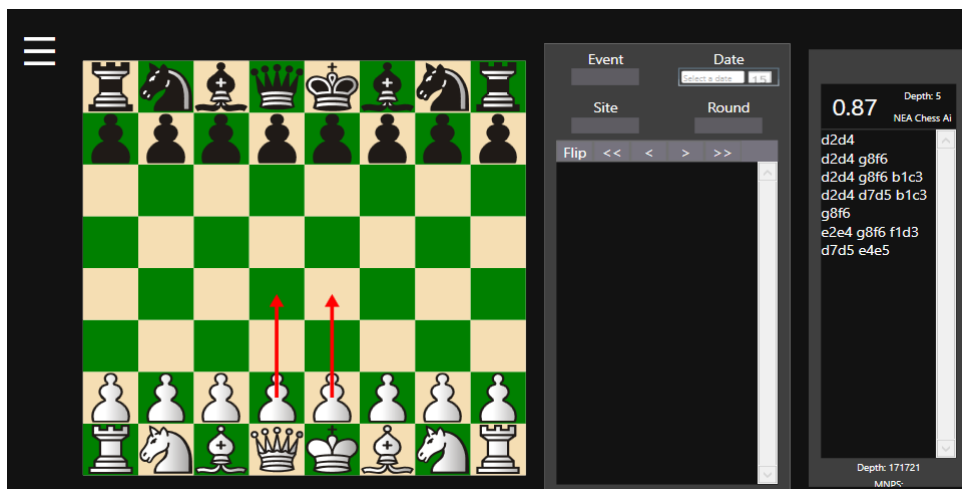
```

1. using System.Windows.Controls;
2.
3. namespace UI.MVVM.Views.Playing_Screen
4. {
5.     /// <summary>
6.     /// Interaction logic for GameOver.xaml
7.     /// </summary>
8.     public partial class GameOver : UserControl
9.     {
10.         public GameOver()
11.         {
12.             InitializeComponent();
13.         }
14.
15.         public void changeMesage(string message)
16.         {
17.             MessageTextBlock.Text = message;
18.         }
19.     }
20. }

```

AnalysisScreen

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.AnalysisScreen"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:UserLoc="clr-namespace:UI.MVVM.Models.Players"
8.             mc:Ignorable="d"
9.             d:DesignHeight="450" d:DesignWidth="800">
10.
11.     <Grid Background="{Binding Source={x:Static UserLoc:User.Settings},
12.         Path=BackgroundColour.ColourBrush}">
13.

```



```

14.         <TextBlock Text="≡" FontSize="35" Height="auto" Width="auto" Margin="14,11,754,392"
           Foreground="{Binding Source={x:Static UserLoc:User.Settings}, Path=TextColour.ColourBrush}"
           Cursor="Hand" MouseUp="ShowMenu"/>
15.
16.         <local:MainMenu x:Name="MainMenu" Margin="0 10 0 0" Height="420" Width="100"
           HorizontalAlignment="Left" Visibility="Hidden" Panel.ZIndex="1"/>
17.
18.         <local:EngineAnalysisView x:Name="EngineInfoView" Height="380" Width="130"
           HorizontalAlignment="Right" Margin="0 0 15 0"/>
19.
20.         <local:GameInformationView x:Name="LocalGameInformationView" Margin="280 0 0 0"
           Height="390" Width="200" HorizontalAlignment="Center"/>
21.
22.         <local:AnalysisBoardView x:Name="LocalBoardView" Height="360" Width="360" Margin="65 0
           0 0" HorizontalAlignment="Left"/>
23.
24.     </Grid>
25. </UserControl>

```

C#

```

1. using System.Windows;
2. using System.Windows.Controls;
3. using System.Windows.Input;
4. using UI.MVVM.Models;
5.
6. namespace UI.MVVM.Views
7. {
8.     /// <summary>
9.     /// Interaction logic for AnalysisScreen.xaml
10.    /// </summary>
11.    public partial class AnalysisScreen : UserControl
12.    {
13.        public static EngineAnalysisView analysisView;
14.        public AnalysisScreen()
15.        {
16.            InitializeComponent();
17.            analysisView = EngineInfoView;
18.            LocalGameInformationView._FlipButton.Click += FlipBoard_Click;
19.            LocalGameInformationView._FirstMove.Click += BoardStateNavigation_Click;
20.            LocalGameInformationView._LastMove.Click += BoardStateNavigation_Click;
21.            LocalGameInformationView._NextMove.Click += BoardStateNavigation_Click;
22.            LocalGameInformationView._PreviousMove.Click += BoardStateNavigation_Click;
23.            MainMenu.ExitButton.MouseUp += ShowMenu;
24.        }
25.
26.        private void ShowMenu(object sender, MouseButtonEventArgs e)
27.        {
28.            if (MainMenu.Visibility == Visibility.Hidden)
29.                MainMenu.Visibility = Visibility.Visible;
30.
31.            else
32.                MainMenu.Visibility = Visibility.Hidden;
33.
34.            e.Handled = true;
35.        }
36.
37.        private void FlipBoard_Click(object sender, RoutedEventArgs e)
38.        {
39.            LocalBoardView.viewModel.WhiteView = !LocalBoardView.viewModel.WhiteView;
40.        }
41.
42.        private void BoardStateNavigation_Click(object sender, RoutedEventArgs e)
43.        {
44.            string senderTag = ((Button)sender).Tag.ToString();
45.            try

```

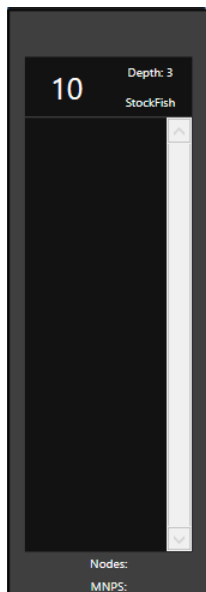
```

46.         {
47.             switch (senderTag)
48.             {
49.                 case "0":
50.                     ((AnalysisGame)LocalBoardView.viewModel.game).currentMoveNo = 0;
51.                     break;
52.                 case "1":
53.                     ((AnalysisGame)LocalBoardView.viewModel.game).currentMoveNo -= 1;
54.                     break;
55.                 case "2":
56.                     ((AnalysisGame)LocalBoardView.viewModel.game).currentMoveNo += 1;
57.                     break;
58.                 case "3":
59.                     ((AnalysisGame)LocalBoardView.viewModel.game).currentMoveNo =
AnalysisGame.movesPlayed.Count - 1;
60.
61.                     break;
62.             }
63.         }
64.         catch
65.         {
66.             // do nothing
67.         }
68.     }
69. }
70. }

```

EngineAnalysisView

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.EngineAnalysisView"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:UserLoc="clr-namespace:UI.MVVM.Models.Players"
8.             mc:Ignorable="d" d:DesignWidth="130" Height="385">
9.     <Grid Background="{Binding Source={x:Static UserLoc:User.Settings},
Path=PanelColour.ColourBrush}" Height="385" Width="130">
10.

```

```

11.         <Grid>
12.             <Grid.RowDefinitions>
13.                 <RowDefinition Height="30"/>
14.                 <RowDefinition/>
15.                 <RowDefinition Height="30"/>
16.             </Grid.RowDefinitions>
17.
18.             <!-- Player 1-->
19.             <Grid Grid.Row="0"/>
20.
21.             <!-- Engine Analysis -->
22.
23.             <Grid Grid.Row="1" Width="110" Background="{Binding Source={x:Static
UserLoc:User.Settings}, Path=BackgroundColour.ColourBrush}">
24.                 <Grid.RowDefinitions>
25.                     <RowDefinition Height="40"/>
26.                     <RowDefinition/>
27.                 </Grid.RowDefinitions>
28.
29.                 <!-- Engine Info Grid -->
30.                 <Border BorderThickness="0,0,0,1" BorderBrush="{Binding
Source={x:Static UserLoc:User.Settings}, Path=PanelColour.ColourBrush}">
31.                     <Grid Grid.Row="0" >
32.                         <Grid.ColumnDefinitions>
33.                             <ColumnDefinition/>
34.                             <ColumnDefinition/>
35.                         </Grid.ColumnDefinitions>
36.
37.                         <Grid.RowDefinitions>
38.                             <RowDefinition/>
39.                             <RowDefinition/>
40.                         </Grid.RowDefinitions>
41.
42.                         <TextBlock x:Name="EvaluationTextBlock" Text="10"
Grid.RowSpan="2" Grid.Column="0" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" TextAlignment="Center"
VerticalAlignment="Center" FontSize="20"/>
43.
44.                         <TextBlock x:Name="DepthTextBlock" Text="Depth: 3"
Grid.Column="1" Grid.Row="0" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" TextAlignment="Center"
VerticalAlignment="Center" FontSize="8"/>
45.                         <TextBlock x:Name="EngineNameTextBlock" Text="StockFish"
Grid.Column="1" Grid.Row="1" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" TextAlignment="Center"
VerticalAlignment="Center" FontSize="8"/>
46.
47.                     </Grid>
48.                 </Border>
49.                 <ScrollViewer Grid.Row="1">
50.                     <StackPanel x:Name="PVLinesStackPanel"/>
51.                 </ScrollViewer>
52.
53.             </Grid>
54.
55.             <!-- NodesSearched -->
56.             <Grid Grid.Row="2">
57.                 <Grid.RowDefinitions>
58.                     <RowDefinition/>
59.                     <RowDefinition/>
60.                 </Grid.RowDefinitions>
61.
62.                 <TextBlock x:Name="NodesSearchedTextBlock"
Text="Nodes:" Grid.Row="0" Foreground="{Binding Source={x:Static
UserLoc:User.Settings}, Path=TextColour.ColourBrush}" TextAlignment="Center"
VerticalAlignment="Center" FontSize="8"/>

```

```

63.         <TextBlock x:Name="NPSTextBlock"
        Text="MNPS:" Grid.Row="1" Foreground="{Binding Source={x:Static
        UserLoc:User.Settings}, Path=TextColour.ColourBrush}" TextAlignment="Center"
        VerticalAlignment="Center" FontSize="8"/>
64.
65.         </Grid>
66.
67.     </Grid>
68.
69. </Grid>
70. </UserControl>

```

C#

```

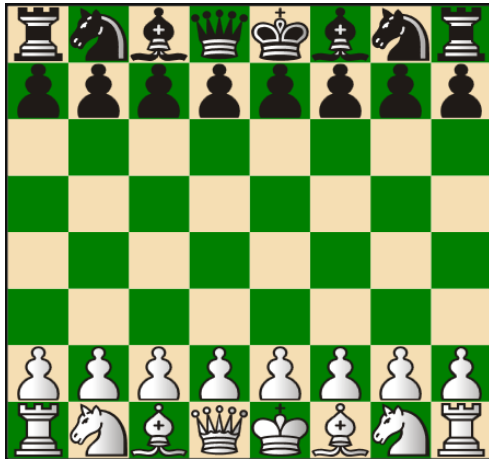
1. using System.Windows;
2. using System.Windows.Controls;
3. using UI.MVVM.Models.Players;
4.
5. namespace UI.MVVM.Views
6. {
7.     /// <summary>
8.     /// Interaction logic for EngineAnalysisView.xaml
9.     /// </summary>
10.    public partial class EngineAnalysisView : UserControl
11.    {
12.        public EngineAnalysisView()
13.        {
14.            InitializeComponent();
15.            EngineNameTextBlock.Text =
        User.Engines[User.Settings.DefaultEngine].name;
16.        }
17.
18.        public void Clear()
19.        {
20.            PVLinesStackPanel.Children.Clear();
21.        }
22.
23.
24.        public void AddInfo(string arg)
25.        {
26.            // scrape the data
27.
28.            // nodes number_of_nodes_searched
29.            if (arg.Contains("nodes"))
30.            {
31.                NodesSearchedTextBlock.Text = DepthTextBlock.Text = "Depth: " +
        arg.Split("nodes ")[1].Split(" ")[0];
32.            }
33.            // depth
34.            if (arg.Contains("depth"))
35.            {
36.                DepthTextBlock.Text = "Depth: " + arg.Split("depth ")[1].Split("
        ")[0];
37.            }
38.            // nps
39.            if (arg.Contains("nps"))
40.            {
41.                NPSTextBlock.Text = "MNPS: " + arg.Split("nps ")[1].Split("
        ")[0];
42.            }
43.            // score <cp | mate | lowerbound | upperbound> <value>
44.            if (arg.Contains("score"))
45.            {
46.                string typeofScore = arg.Split("score ")[1].Split(" ")[0];
47.                if (typeofScore == "mate")
48.                {

```

```
49.         EvaluationTextBlock.Text = "#" + arg.Split("mate
50.     ")[1].Split(' ')[0];
51.     }
52.     else if (typeofScore == "cp")
53.     {
54.         double evalInCp = double.Parse(arg.Split("cp ")[1].Split('
55.     ')[0]);
56.         double eval = evalInCp / 100;
57.         EvaluationTextBlock.Text = eval.ToString();
58.     }
59.     else
60.     {
61.         EvaluationTextBlock.Text = arg.Split(typeofScore + "
62.     ")[1].Split(' ')[0];
63.     }
64. }
65. // currmove
66. // currmove number
67. //hashful
68.
69. if (arg.Contains("pv"))
70. {
71.     string pv = arg.Split("pv ")[1];
72.     TextBlock textBlock = new TextBlock()
73.     {
74.         Text = pv,
75.         Foreground = User.Settings.TextColour.ColourBrush,
76.         TextWrapping = TextWrapping.Wrap,
77.     };
78.     PVLinesStackPanel.Children.Add(textBlock);
79.
80.     // add arrow for the best move
81.     string best = pv.Split(' ')[0];
82.     //
83. }
84. }
85. }
```

AnalysisBoardView

Screen-Shot



XAML

```

1. <UserControl x:Class="UI.MVVM.Views.AnalysisBoardView"
2.             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
5.             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
6.             xmlns:local="clr-namespace:UI.MVVM.Views"
7.             xmlns:viewModel="clr-namespace:UI.MVVM.ViewModels"
8.
9.             mc:Ignorable="d"
10.            d:DesignHeight="360" d:DesignWidth="360">
11.
12.        <Canvas x:Name="BoardCanvas" Background="white" Height="360" Width="360"
13.            VerticalAlignment="Center" HorizontalAlignment="Left" ></Canvas>
14.    </UserControl>
15.

```

C#

```

1. using System.Windows.Controls;
2. using UI.MVVM.ViewModels;
3.
4. namespace UI.MVVM.Views
5. {
6.     /// <summary>
7.     /// Interaction logic for AnalysisBoardView.xaml
8.     /// </summary>
9.     public partial class AnalysisBoardView : UserControl
10.    {
11.        internal AnalysisBoardViewModel viewModel;
12.
13.        public AnalysisBoardView()
14.        {
15.            InitializeComponent();
16.            viewModel = new AnalysisBoardViewModel(BoardCanvas);
17.        }
18.    }
19. }
20.
21.
22.

```

ViewModels

BoardViewModelBase

C#

```

1. using Engine;
2. using System;
3. using System.Collections.Generic;
4. using System.Reflection;
5. using System.Windows;
6. using System.Windows.Controls;
7. using System.Windows.Input;
8. using System.Windows.Media;
9. using System.Windows.Shapes;
10. using UI.MVVM.Models;
11. using UI.MVVM.Models.Players;
12. using UI.MVVM.Views;
13.
14. namespace UI.MVVM.ViewModels
15. {
16.     public abstract class BoardViewModelBase
17.     {
18.         #region White View Bool
19.         protected static bool whiteView = true;
20.
21.         public bool WhiteView
22.         {
23.             get { return whiteView; }
24.
25.             set
26.             {
27.                 whiteView = value;
28.                 DrawBoard();
29.                 DrawPieces(this.game.board);
30.                 if (CurrentGame.movesPlayed.Count != 0)
31.                 {
32.                     DrawComments(CurrentGame.movesPlayed[game.currentMoveNo].Comments);
33.                 }
34.                 selectedIndexLeft = -1;
35.                 selectedIndexRight = -1;
36.             }
37.         }
38.         #endregion
39.
40.         internal Game game;
41.
42.         #region Private Variables
43.         public Canvas boardCanvas;
44.         protected bool LeftMouse;
45.         protected bool RightMouse;
46.         protected SVGImage.SVGImage MovingPiece;
47.         protected int selectedIndexLeft = -1; // use two different variables so that both of
the buttons will work without breaking the other
48.         protected int selectedIndexRight = -1;
49.         protected List<int> LegalMoves = new List<int>();
50.         protected Polygon arrow;
51.         protected Promotion promotionWindow;
52.         #endregion
53.
54.         #region Static Variables
55.         static readonly string[] pieceNames = { "P", "N", "B", "R", "Q", "K" };
56.
57.         protected string appFolderPath =
System.IO.Path.GetDirectoryName(Assembly.GetEntryAssembly().Location);
58.
59.         protected static Square[] squares = new Square[64];

```

```

60.
61.     protected static double SquareLength = 0;
62.
63.     protected static string assetsPath = "";
64.     #endregion
65.
66.
67.     public BoardViewModelBase()
68.     {
69.
70.     }
71.
72.     public virtual void OnRightMouseDown(object sender, MouseEventArgs e)
73.     {
74.         RightMouse = true;
75.         Border SquareClicked = (Border)sender;
76.         int index = (int)SquareClicked.Tag;
77.         int boardIndex = getBitboardIndex(index, whiteView);
78.         selectedIndexRight = index;
79.         // create arrow
80.         double borderX = Canvas.GetRight(squares[selectedIndexRight].border);
81.         double startX = boardCanvas.Width - (borderX + SquareLength / 2);
82.         double startY = boardCanvas.Height -
83.         (Canvas.GetBottom(squares[selectedIndexRight].border) + SquareLength / 2);
84.         Point mousePos = Mouse.GetPosition(boardCanvas);
85.         PointCollection arrowPoints = calculateArrowPoints(startX, startY, mousePos.X,
86.         mousePos.Y);
87.         arrow = new Polygon();
88.         arrow.Points = arrowPoints;
89.         arrow.Fill = Brushes.Red;
90.
91.         boardCanvas.Children.Add(arrow);
92.     }
93.
94.     public abstract void OnLeftMouseDown(object sender, MouseEventArgs e);
95.
96.     public virtual void OnRightMouseUp(object sender, MouseEventArgs e)
97.     {
98.         int index = calculateBitBoardIndexAtMousePosition();
99.         int boardViewIndex = getBitboardIndex(index, whiteView);
100.        int selectedIndexRight = getBitboardIndex(selectedIndexRight, whiteView);
101.        RightMouse = false;
102.        boardCanvas.Children.Remove(arrow);
103.        if (boardViewIndex == selectedIndexRight)
104.        {
105.            // add to move notes - if there has been no moves/startpos then do
106.            nothing
107.            string squareFrom =
108.            Engine.Tools.IndexToSquare((byte)selectedIndexRight);
109.            string squareTo = Engine.Tools.IndexToSquare((byte)index);
110.            string Comment = $"[{squareFrom},{squareTo},SolidColour]";
111.            if (Game.movesPlayed.Count != 0)
112.            {
113.                if
114.                (Game.movesPlayed[game.currentMoveNo].Comments.Contains(Comment))
115.                {
116.                    Game.movesPlayed[game.currentMoveNo].Comments.Remove(Comment);
117.                    DrawBoard();
118.                    DrawPieces(game.board);
119.                    DrawComments(Game.movesPlayed[game.currentMoveNo].Comments);
120.                    if (selectedIndexLeft != -1)
121.                    {
122.                        DrawAvailibleMoves(LegalMoves);
123.                    }
124.                    selectedIndexRight = -1;

```



```

121.         return;
122.     }
123.
124.         Game.movesPlayed[game.currentMoveNo].Comments.Add(Comment);
125.     }
126.         squares[boardViewIndex].border.Background =
    User.Settings.ArrowColour.ColourBrush;
127.     }
128.
129.         else if (selectedIndexRight != -1)
130.         {
131.             // create the arrow comment
132.             // Comment in the format of "[SqaureFrom,SquareTo,Arrow/SolidColour]"
133.             string squareFrom =
    Engine.Tools.IndexToSquare((byte)selectedIndex);
134.             string squareTo = Engine.Tools.IndexToSquare((byte)index);
135.             string Comment = $"[{squareFrom},{squareTo},Arrow]";
136.             // we are making sure that the current game is greater than zero
    otherwise the comment is not related to any move
137.             if (Game.movesPlayed.Count != 0)
138.             {
139.                 // check if the user is removing the comment
140.                 if
    (Game.movesPlayed[game.currentMoveNo].Comments.Contains(Comment))
141.                 {
142.                     Game.movesPlayed[game.currentMoveNo].Comments.Remove(Comment);
143.                     DrawBoard();
144.                     DrawPieces(game.board);
145.                     DrawComments(Game.movesPlayed[game.currentMoveNo].Comments);
146.                     selectedIndexLeft = -1;
147.                     selectedIndexRight = -1;
148.                     return;
149.                 }
150.
151.                 // else add the arrow comment and the new arrow
152.                 Game.movesPlayed[game.currentMoveNo].Comments.Add(Comment);
153.             }
154.
155.             double startX = boardCanvas.Width -
    (Canvas.GetRight(squares[selectedIndexRight].border) + SquareLength / 2);
156.             double startY = boardCanvas.Height -
    (Canvas.GetBottom(squares[selectedIndexRight].border) + SquareLength / 2);
157.             double endX = boardCanvas.Width -
    (Canvas.GetRight(squares[boardViewIndex].border) + SquareLength / 2);
158.             double endY = boardCanvas.Height -
    (Canvas.GetBottom(squares[boardViewIndex].border) + SquareLength / 2);
159.             PointCollection arrowPoints = calculateArrowPoints(startX, startY,
    endX, endY);
160.             Polygon arrow1 = new Polygon();
161.             arrow1.Points = arrowPoints;
162.             arrow1.Fill = User.Settings.ArrowColour.ColourBrush;
163.
164.             boardCanvas.Children.Add(arrow1);
165.         }
166.
167.         selectedIndexRight = -1;
168.     }
169.
170.     public abstract void OnLeftMouseUp(object sender, MouseEventArgs e);
171.
172.
173.     public void ShowPromotion(Move move, int index)
174.     {
175.         string colour = game.board.WhiteToPlay ? "w" : "b";
176.         promotionWindow = new Promotion(move, colour);
177.

```

```

178.         promotionWindow.promotionStack.MouseUp += PromotionStack_MouseUp;
179.
180.         promotionWindow.Height = SquareLength * 4;
181.         promotionWindow.Width = SquareLength;
182.
183.         if (index < 8) // promotion at the bottom of the screen
184.         {
185.             index += 8 * 3; // set it three squares higher so it is still on the
board
186.         }
187.
188.         Canvas.SetTop(promotionWindow, (7 - (index / 8)) * SquareLength); // set
the x and y of the screen to be originating from the square
189.         Canvas.SetRight(promotionWindow, (index % 8) * SquareLength);
190.         boardCanvas.Children.Add(promotionWindow);
191.     }
192.
193.     public void RemovePromotion()
194.     {
195.         try
196.         {
197.             boardCanvas.Children.Remove(promotionWindow);
198.             promotionWindow = null;
199.         }
200.         catch
201.         {
202.             return;
203.         }
204.     }
205.
206.     protected void PromotionStack_MouseUp(object sender, MouseButtonEventArgs e)
207.     {
208.
209.         game.MakeMove(promotionWindow.move);
210.         DrawBoard();
211.         DrawPieces(game.board);
212.         selectedIndexLeft = -1;
213.     }
214.
215.     public int calculateBitBoardIndexAtMousePosition()
216.     {
217.         Point point = Mouse.GetPosition(boardCanvas);
218.         int y = (int)(point.Y / SquareLength);
219.         int x = (int)(point.X / SquareLength);
220.         int index = ((x + y * 8));
221.         return getBoardViewIndex(index, !whiteView);
222.     }
223.
224.     public void MouseMove(object sender, MouseEventArgs e)
225.     {
226.         if (LeftMouse)
227.         {
228.             Point position = Mouse.GetPosition(boardCanvas);
229.             Canvas.SetLeft(MovingPiece, position.X - MovingPiece.Width / 2);
230.             Canvas.SetTop(MovingPiece, position.Y - MovingPiece.Height / 2);
231.         }
232.
233.         if (RightMouse)
234.         {
235.             double startX = boardCanvas.Width -
(Canvas.GetRight(squares[selectedIndexRight].border) + SquareLength / 2);
236.             double startY = boardCanvas.Height -
(Canvas.GetBottom(squares[selectedIndexRight].border) + SquareLength / 2);
237.             Point mousePos = Mouse.GetPosition(boardCanvas);
238.             PointCollection arrowPoints = calculateArrowPoints(startX, startY,
mousePos.X, mousePos.Y);

```

```
239.         arrow.Points = arrowPoints;
240.     }
241. }
242.
243. public List<int> filterMoves(List<Move> moves, int targetSource)
244. {
245.     List<int> destinations = new List<int>();
246.     foreach (Move move in moves)
247.     {
248.         if (move.startIndex == targetSource)
249.         {
250.             destinations.Add(move.destinationIndex);
251.         }
252.     }
253.
254.     return destinations;
255. }
256.
257. public int getBitboardIndex(int clickIndex, bool whiteView)
258. {
259.     if (whiteView) return clickIndex;
260.
261.     return (Math.Abs(clickIndex - 63));
262. }
263.
264. public int getBoardViewIndex(int clickIndex, bool whiteView)
265. {
266.     if (whiteView) return clickIndex;
267.
268.     return (Math.Abs(clickIndex - 63));
269. }
270.
271. public void DrawBoard()
272. {
273.     boardCanvas.Children.Clear();
274.
275.     for (int i = 0; i < 8; i++)
276.     {
277.         for (int j = 0; j < 8; j++)
278.         {
279.             int index = ((j + i * 8));
280.             squares[index] = new Square();
281.             squares[index].border = new Border
282.             {
283.                 Tag = index,
284.                 Height = SquareLength,
285.                 Width = SquareLength,
286.                 BorderBrush = Brushes.Red
287.             };
288.             squares[index].border.MouseLeftButtonDown += OnLeftMouseDown;
289.             squares[index].border.MouseRightButtonDown += OnRightMouseDown;
290.
291.             squares[index].border.BorderThickness = new Thickness();
292.
293.             if ((index + (-1 * (i))) % 2 == 0)
294.             {
295.                 squares[index].border.Background =
296.                 User.Settings.LightSquareColour.ColourBrush;
297.             }
298.             else
299.             {
300.                 squares[index].border.Background =
301.                 User.Settings.DarkSquareColour.ColourBrush;
302.             }
303.             boardCanvas.Children.Add(squares[index].border);
304.         }
305.     }
306. }
```

```

302.                Canvas.SetRight(squares[index].border, j *
    squares[index].border.Height);
303.                Canvas.SetBottom(squares[index].border, i *
    squares[index].border.Height);
304.            }
305.        }
306.
307.    }
308.
309.    public void clearBoard()
310.    {
311.        boardCanvas.Children.RemoveRange(64, boardCanvas.Children.Count - 64);
312.        foreach (Square i in squares)
313.        {
314.            i.border.Child = null;
315.        }
316.    }
317.
318.    public void DrawPieces(Engine.Bitboard b)
319.    {
320.
321.        this.clearBoard();
322.        ulong allPieces = b.Board[0] | b.Board[1];
323.        while (allPieces != 0)
324.        {
325.            ulong pieceLoc = BitBoardTools.popLSB(ref allPieces);
326.            int Piecetype = Bitboard.getPieceType(b.Board, pieceLoc);
327.            char isWhite = (pieceLoc & b.Board[0]) != 0 ? 'w' : 'b';
328.
329.            int index = whiteView ? (BitBoardTools.BitboardToIndex(pieceLoc)) : (63
- BitBoardTools.BitboardToIndex(pieceLoc));
330.
331.            squares[index].border.Child = createPieceImage(Piecetype, isWhite);
332.        }
333.
334.        DrawLastMove();
335.    }
336.
337.    public virtual void DrawAvailibleMoves(List<int> moves)
338.    {
339.        foreach (Square square in squares)
340.        {
341.            square.border.BorderThickness = new Thickness(0);
342.        }
343.
344.        foreach (int move in moves)
345.        {
346.            int index = getBoardViewIndex(move, whiteView);
347.            squares[index].border.BorderThickness = new Thickness(2, 2, 2, 2);
348.            squares[index].border.BorderBrush =
    User.Settings.AvailibleMoveColour.ColourBrush;
349.        }
350.    }
351.
352.    public virtual void DrawLastMove()
353.    {
354.        if (game.currentMoveNo != 0)
355.        {
356.            Move LastMove = CurrentGame.movesPlayed[game.currentMoveNo].Move;
357.            int StartIndex = getBoardViewIndex(LastMove.startIndex, whiteView);
358.            int DestinationIndex = getBoardViewIndex(LastMove.destinationIndex,
    whiteView);
359.            squares[StartIndex].border.Background =
    User.Settings.LastMoveColour.ColourBrush;
360.            squares[DestinationIndex].border.Background =
    User.Settings.LastMoveColour.ColourBrush;

```

```

361.         }
362.     }
363.
364.     public void DrawComments(List<string> comments)
365.     {
366.         foreach (string comment in comments)
367.         {
368.             // remove [ ]
369.             string Comment;
370.             Comment = comment.Substring(1, comment.Length - 2);
371.
372.             // split into its components
373.             string[] commentComponents = Comment.Split(',');
374.             string startSqaure = commentComponents[0];
375.             string endSqaure = commentComponents[1];
376.             string Type = commentComponents[2];
377.
378.             if (Type == "Arrow")
379.             {
380.                 // get the
381.                 int startSquareIndex =
382.                     getBoardViewIndex(Engine.Tools.SquareToIndex(startSqaure), whiteView);
383.                 int endSquareIndex =
384.                     getBoardViewIndex(Engine.Tools.SquareToIndex(endSqaure), whiteView);
385.                 double startX = boardCanvas.Width -
386.                     (Canvas.GetRight(squares[startSquareIndex].border) + SquareLength / 2);
387.                 double startY = boardCanvas.Height -
388.                     (Canvas.GetBottom(squares[startSquareIndex].border) + SquareLength / 2);
389.                 double endX = boardCanvas.Width -
390.                     (Canvas.GetRight(squares[endSquareIndex].border) + SquareLength / 2);
391.                 double endY = boardCanvas.Height -
392.                     (Canvas.GetBottom(squares[endSquareIndex].border) + SquareLength / 2);
393.                 PointCollection arrowPoints = calculateArrowPoints(startX, startY,
394.                     endX, endY);
395.
396.                 Polygon arrow1 = new Polygon();
397.                 arrow1.Points = arrowPoints;
398.                 arrow1.Fill = arrow1.Fill = User.Settings.ArrowColour.ColourBrush;
399.
400.                 boardCanvas.Children.Add(arrow1);
401.             }
402.
403.             else if (Type == "SolidColour")
404.             {
405.                 squares[getBoardViewIndex(Engine.Tools.SquareToIndex(startSqaure),
406.                     whiteView)].border.Background = User.Settings.ArrowColour.ColourBrush;
407.             }
408.         }
409.     }
410.
411.     public PointCollection calculateArrowPoints(double startX, double startY,
412.         double endX, double endY)
413.     {
414.         Point end = new Point(endX, endY);
415.         Point start = new Point(startX, startY);
416.
417.         // Code from https://stackoverflow.com/questions/16714011/how-to-draw-an-
418.         // arrow-in-wpf-programatically
419.
420.         Vector direction = end - start;
421.
422.         Vector normalizedDirection = direction;
423.         normalizedDirection.Normalize();
424.
425.         Vector normalizedlineWidenVector = new Vector(-normalizedDirection.Y,
426.             normalizedDirection.X); // Rotate by 90 degrees

```

```

416.         Vector lineWidenVector = normalizedlineWidenVector * 1.5;
417.
418.         double lineLength = direction.Length;
419.
420.         double defaultArrowLength = 3 * 3.73205081;
421.
422.         // Prepare usedArrowLength
423.         // if the length is bigger than 1/3 (_maxArrowLengthPercent) of the line
         length adjust the arrow length to 1/3 of line length
424.
425.         double usedArrowLength;
426.         if (lineLength * 0.3 < defaultArrowLength)
427.             usedArrowLength = lineLength * 0.3;
428.         else
429.             usedArrowLength = defaultArrowLength;
430.
431.         // Adjust arrow thickness for very thick lines
432.         double arrowWidthFactor = 6;
433.
434.         Vector arrowWidthVector = normalizedlineWidenVector * arrowWidthFactor;
435.         Point endArrowCenterPosition = end - (normalizedDirection *
         usedArrowLength);
436.
437.         PointCollection pointCollection = new PointCollection(7)
438.         {
439.             end,
440.             endArrowCenterPosition + arrowWidthVector,
441.             endArrowCenterPosition + lineWidenVector,
442.             start + lineWidenVector,
443.             start - lineWidenVector,
444.             endArrowCenterPosition - lineWidenVector,
445.             endArrowCenterPosition - arrowWidthVector
446.         };
447.
448.         return pointCollection;
449.     }
450.
451.     public static SVGImage.SVG.SVGImage createPieceImage(int PieceType, char
         isWhite)
452.     {
453.         string filePath = assetsPath +
         $"\\Pieces\\{User.Settings.PieceFolder}\\{isWhite + pieceNames[PieceType - 2]}.svg";
454.         SVGImage.SVG.SVGImage piece = new SVGImage.SVG.SVGImage();
455.         piece.Width = SquareLength - 2;
456.         piece.Height = SquareLength - 2;
457.         piece.HorizontalContentAlignment = HorizontalAlignment.Center;
458.         piece.VerticalContentAlignment = VerticalAlignment.Center;
459.         piece.VerticalAlignment = VerticalAlignment.Center;
460.         piece.HorizontalAlignment = HorizontalAlignment.Center;
461.         piece.SizeType = SVGImage.SVG.SVGImage.eSizeType.ContentToSizeNoStretch;
462.
463.         piece.UriSource = new Uri(filePath);
464.
465.         return piece;
466.     }
467. }
468. }
469. }

```

BoardViewModel

C#

```

1. using Engine;
2. using System;
3. using System.IO;

```

```

4. using System.Windows;
5. using System.Windows.Controls;
6. using System.Windows.Input;
7. using UI.MVVM.Models;
8.
9. namespace UI.MVVM.ViewModels
10. {
11.     public class BoardViewModel : BoardViewModelBase
12.     {
13.         public BoardViewModel(Canvas root)
14.         {
15.             boardCanvas = root;
16.             game = new CurrentGame(this);
17.             boardCanvas.MouseMove += MouseMove;
18.             boardCanvas.MouseRightButtonUp += OnRightMouseUp;
19.             boardCanvas.MouseLeftButtonUp += OnLeftMouseUp;
20.             squares = new Square[64];
21.             assetsPath =
System.IO.Path.Combine(Directory.GetParent(appFolderPath).Parent.Parent.FullName, "Assets");
22.             SquareLength = boardCanvas.Height / 8;
23.             DrawBoard();
24.             DrawPieces(this.game.board);
25.         }
26.
27.
28.         public override void OnLeftMouseUp(object sender, MouseEventArgs e)
29.         {
30.             LeftMouse = false;
31.             int index = calculateBitBoardIndexAtMousePosition();
32.             int boardIndex = getBoardViewIndex(index, whiteView);
33.             boardCanvas.Children.Remove(MovingPiece);
34.
35.             if (boardIndex == selectedIndexLeft) // user clicks on the same square
36.             {
37.                 squares[boardIndex].border.Child = MovingPiece; // replace the piece
38.                 if (promotionWindow != null)
39.                 {
40.                     RemovePromotion();
41.                 }
42.             }
43.
44.             else if (LegalMoves.Contains(index) && selectedIndexLeft != -1 &&
game.currentMoveNo == (game.MovesPlayedCount - 1)) // it is a valid move and we are on the
lastest board and a piece was originally selected
45.             {
46.                 // make move
47.                 Move moveMade = game.createMove(getBitboardIndex(selectedIndexLeft, whiteView),
index);
48.
49.                 if (moveMade.isPromotion)
50.                 {
51.                     if (promotionWindow != null) // the user selects a different promotion move
52.                     {
53.                         RemovePromotion();
54.                     }
55.
56.                     ShowPromotion(moveMade, getBitboardIndex(index, whiteView));
57.                     return;
58.                 }
59.
60.                 game.MakeMove(moveMade);
61.                 DrawBoard();
62.                 DrawPieces(game.board);
63.                 selectedIndexLeft = -1;
64.             }
65.

```

```

66.         else if (selectedIndexLeft != -1) // the user did not select a valid move
67.         {
68.
69.             try
70.             {
71.                 squares[selectedIndexLeft].border.Child = MovingPiece; // replace the piece
72.                 RemovePromotion();
73.             }
74.
75.             catch (Exception)
76.             {
77.
78.             }
79.
80.         }
81.     }
82.
83.     public override void OnLeftMouseDown(object sender, MouseEventArgs e)
84.     {
85.         Border SquareClicked = (Border)sender;
86.         int index = (int)SquareClicked.Tag;
87.         int boardIndex = getBitboardIndex(index, whiteView);
88.         ulong boardIndexBitboard = BitBoardTools.IndexToBitboard((byte)boardIndex);
89.         if ((boardIndexBitboard & game.board.Board[game.board.WhiteToPlay ? 0 : 1]) != 0 &&
game.players[game.board.WhiteToPlay ? 0 : 1].canMove)
90.         {
91.             if ((SVGImage.SVG.SVGImage)SquareClicked.Child != null)
92.             { // this when the promotion screen is active and the player clicks on the
original square
93.                 MovingPiece = (SVGImage.SVG.SVGImage)SquareClicked.Child; // set the moving
piece to whatever is in the square
94.                 SquareClicked.Child = null; // empty the square
95.
96.                 Point position = Mouse.GetPosition(boardCanvas);
97.
98.                 Canvas.SetLeft(MovingPiece, position.X - MovingPiece.Width / 2); // set the
position of the moving piece to the centre of the mouse
99.                 Canvas.SetTop(MovingPiece, position.Y - MovingPiece.Height / 2);
100.                 boardCanvas.Children.Add(MovingPiece);
101.
102.             }
103.
104.             LeftMouse = true;
105.             selectedIndexLeft = index;
106.
107.             LegalMoves = filterMoves(game.genMoves(), boardIndex);
108.             DrawAvailibleMoves(LegalMoves);
109.         }
110.
111.     }
112. }
113. }

```

AnalysisBoardViewModel

C#

```

1. using Engine;
2. using System;
3. using System.IO;
4. using System.Windows;
5. using System.Windows.Controls;
6. using System.Windows.Input;
7. using UI.MVVM.Models;
8.
9. namespace UI.MVVM.ViewModels

```



```

10. {
11.     public class AnalysisBoardViewModel : BoardViewModelBase
12.     {
13.         public AnalysisBoardViewModel(Canvas root)
14.         {
15.             boardCanvas = root;
16.             game = new AnalysisGame(this);
17.             boardCanvas.MouseMove += MouseMove;
18.             boardCanvas.MouseRightButtonUp += OnRightMouseUp;
19.             boardCanvas.MouseLeftButtonUp += OnLeftMouseUp;
20.             squares = new Square[64];
21.             assetsPath =
System.IO.Path.Combine(Directory.GetParent(appFolderPath).Parent.Parent.FullName, "Assets");
22.             SquareLength = boardCanvas.Height / 8;
23.             DrawBoard();
24.             DrawPieces(this.game.board);
25.         }
26.
27.
28.         public override void OnLeftMouseDown(object sender, MouseEventArgs e)
29.         {
30.             Border SquareClicked = (Border)sender;
31.             int index = (int)SquareClicked.Tag;
32.             int boardIndex = getBitboardIndex(index, whiteView);
33.             ulong boardIndexBitboard = BitBoardTools.IndexToBitboard((byte)boardIndex);
34.             if ((boardIndexBitboard & game.board.Board[game.board.WhiteToPlay ? 0 : 1]) != 0)
35.             {
36.                 if ((SVGImage.SVG.SVGImage)SquareClicked.Child != null)
37.                 { // this when the promotion screen is active and the player clicks on the
original square
38.                     MovingPiece = (SVGImage.SVG.SVGImage)SquareClicked.Child; // set the moving
piece to whatever is in the square
39.                     SquareClicked.Child = null; // empty the square
40.
41.                     Point position = Mouse.GetPosition(boardCanvas);
42.
43.                     Canvas.SetLeft(MovingPiece, position.X - MovingPiece.Width / 2); // set the
position of the moving piece to the centre of the mouse
44.                     Canvas.SetTop(MovingPiece, position.Y - MovingPiece.Height / 2);
45.                     boardCanvas.Children.Add(MovingPiece);
46.
47.                 }
48.
49.                 LeftMouse = true;
50.                 selectedIndexLeft = index;
51.
52.                 LegalMoves = filterMoves(game.genMoves(), boardIndex);
53.                 DrawAvailableMoves(LegalMoves);
54.             }
55.
56.         }
57.
58.
59.
60.         public override void OnLeftMouseUp(object sender, MouseEventArgs e)
61.         {
62.             LeftMouse = false;
63.             int index = calculateBitBoardIndexAtMousePosition();
64.             boardCanvas.Children.Remove(MovingPiece);
65.
66.             if (index == selectedIndexLeft) // user clicks on the same square
67.             {
68.                 squares[index].border.Child = MovingPiece; // replace the piece
69.                 if (promotionWindow != null)
70.                 {
71.                     RemovePromotion();

```

```

72.         }
73.     }
74.
75.     else if (LegalMoves.Contains(index) && selectedIndexLeft != -1) // it is a valid
move and we are on the latest board and a piece was originally selected
76.     {
77.         // make move
78.         Move moveMade = game.createMove(getBitboardIndex(selectedIndexLeft, whiteView),
index);
79.
80.         if (moveMade.isPromotion)
81.         {
82.             if (promotionWindow != null) // the user selects a different promotion move
83.             {
84.                 RemovePromotion();
85.             }
86.
87.             ShowPromotion(moveMade, getBitboardIndex(index, whiteView));
88.             return;
89.         }
90.
91.         game.MakeMove(moveMade);
92.         DrawBoard();
93.         DrawPieces(game.board);
94.         selectedIndexLeft = -1;
95.     }
96.
97.     else if (selectedIndexLeft != -1) // the user did not select a valid move
98.     {
99.
100.        try
101.        {
102.            squares[selectedIndexLeft].border.Child = MovingPiece; // replace
the piece
103.            RemovePromotion();
104.        }
105.
106.        catch (Exception)
107.        {
108.
109.        }
110.
111.    }
112. }
113. }
114.
115. }
```

BoardPreviewModel

C#

```

1. using Engine;
2. using System.Collections.Generic;
3. using System.IO;
4. using System.Windows.Controls;
5. using System.Windows.Input;
6. using UI.MVVM.Models;
7. using UI.MVVM.Models.Players;
8.
9. namespace UI.MVVM.ViewModels
10. {
11.     public class BoardPreiewModel : BoardViewModelBase
12.     {
13.
```

```

14.         static Bitboard b = Tools.FENtoBitboard("rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR
    b KQkq - 0 1");
15.
16.
17.         public BoardPreviewModel(Canvas root)
18.         {
19.
20.             boardCanvas = root;
21.             squares = new Square[64];
22.             assetsPath =
    System.IO.Path.Combine(Directory.GetParent(appFolderPath).Parent.Parent.FullName, "Assets");
23.             SquareLength = boardCanvas.Height / 8;
24.             UpdatePreview();
25.         }
26.
27.         public void UpdatePreview()
28.         {
29.             DrawBoard();
30.             DrawPieces(b);
31.             DrawAvailibleMoves(new List<int> { 43, 35 });
32.             DrawLastMove();
33.             DrawComments(new List<string> { "[g8,f6,Arrow]" });
34.         }
35.
36.         public override void DrawLastMove()
37.         {
38.             squares[11].border.Background = User.Settings.LastMoveColour.ColourBrush;
39.             squares[27].border.Background = User.Settings.LastMoveColour.ColourBrush;
40.         }
41.
42.         public override void OnLeftMouseDown(object sender, MouseEventArgs e)
43.         {
44.
45.         }
46.
47.         public override void OnLeftMouseUp(object sender, MouseEventArgs e)
48.         {
49.
50.         }
51.     }
52. }

```

Models

User

C#

```

1. using System.Collections.Generic;
2. using System.IO;
3. using System.Reflection;
4. using System.Windows.Media;
5. using UI.MVVM.Models.Engine;
6.
7. namespace UI.MVVM.Models.Players
8. {
9.     internal static class User
10.    {
11.        private static BrushConverter converter = new BrushConverter();
12.
13.        public static List<EngineInfo> _Engines = new List<EngineInfo>();
14.
15.        public static readonly string rootFolder;
16.        public static readonly string assetsFolder;
17.
18.
19.        public static List<EngineInfo> Engines

```

```

20.     {
21.         get { return _Engines; }
22.         set { }
23.     }
24.
25.     public static List<string> EngineName
26.     {
27.         get
28.         {
29.             List<string> list = new List<string>();
30.             foreach (EngineInfo engine in _Engines)
31.             {
32.                 list.Add(engine.name);
33.             }
34.             return list;
35.         }
36.         set { }
37.     }
38.
39.     public static string UserName { get; set; }
40.     public static string flag { get; set; }
41.
42.     public static UISettings Settings { get; set; }
43.
44.     static User()
45.     {
46.         UserName = "Anon";
47.         flag = "Default";
48.         Settings = new UISettings();
49.         rootFolder =
Directory.GetParent(System.IO.Directory.GetCurrentDirectory()).Parent.Parent.Parent.Parent.FullName
50.         string appFolderPath =
System.IO.Path.GetDirectoryName(Assembly.GetEntryAssembly().Location);
51.         assetsFolder =
System.IO.Path.Combine(Directory.GetParent(appFolderPath).Parent.Parent.FullName, "Assets");
52.         string EngineString = System.IO.Path.Combine(new string[] { rootFolder, "Engine",
"Engine", "bin", "Release", "Engine.exe" });
53.
54.         EngineInfo temp = new EngineInfo("NEA Chess Ai", EngineString, "Harry Foster", new
List<EngineOption>());
55.
56.         _Engines.Add(temp);
57.         Settings.DefaultEngine = 0;
58.
59.     }
60. }
61. }

```

*UISettings**C#*

```

1. using System.Windows.Media;
2.
3. namespace UI.MVVM.Models
4. {
5.     public struct Colour
6.     {
7.         private static BrushConverter converter = new BrushConverter();
8.         private string _HexColourString;
9.         private Color _color;
10.
11.         public string HexColourString
12.         {
13.             get { return _HexColourString; }
14.             set

```

```
15.         {
16.             _HexColourString = value;
17.             _ColourBrush = (Brush)converter.ConvertFromString(value);
18.             _color = ConvertBrushToColor(_ColourBrush);
19.         }
20.     }
21.
22.     public Color ColorPickerData
23.     {
24.         get { return _color; }
25.         set
26.         {
27.             _color = value;
28.             _ColourBrush = ConvertColorToBrush(value);
29.             _HexColourString = converter.ConvertToString(value);
30.         }
31.     }
32.
33.     private Brush _ColourBrush;
34.
35.     public Brush ColourBrush
36.     {
37.         get { return _ColourBrush; }
38.
39.         set
40.         {
41.             _ColourBrush = value;
42.             _HexColourString = converter.ConvertToString(value);
43.             _color = ConvertBrushToColor(value);
44.         }
45.     }
46.
47. }
48.
49. public Colour(string hex)
50. {
51.     _HexColourString = hex;
52.     _ColourBrush = (Brush)converter.ConvertFromString(hex);
53.     _color = ConvertBrushToColor(_ColourBrush);
54.
55. }
56.
57. public Colour(Brush colourBrush)
58. {
59.     _HexColourString = converter.ConvertToString(colourBrush);
60.     _ColourBrush = colourBrush;
61.     _color = ConvertBrushToColor(colourBrush);
62. }
63.
64. public Colour(Color colour)
65. {
66.     _color = colour;
67.     _ColourBrush = ConvertColorToBrush(_color);
68.     _HexColourString = converter.ConvertToString(_ColourBrush);
69. }
70.
71. public static Color ConvertBrushToColor(Brush brush)
72. {
73.     SolidColorBrush newBrush = (SolidColorBrush)brush;
74.     return newBrush.Color;
75. }
76. public static Brush ConvertColorToBrush(Color color)
77. {
78.     SolidColorBrush newBrush = new SolidColorBrush(color);
79.     return (Brush)newBrush;
80. }
```

```

81.
82.     }
83.
84.     public class UISettings
85.     {
86.         public Colour _BackgroundColour;
87.         public Colour _ButtonColour;
88.         public Colour _TextBoxColour;
89.         public Colour _LightSquareColour;
90.         public Colour _DarkSquareColour;
91.         public Colour _PanelColour;
92.         public Colour _TextColour;
93.         public Colour _LastMoveColour;
94.         public Colour _ArrowColour;
95.         public Colour _AvailibleMoveColour;
96.
97.         public Colour BackgroundColour { get { return _BackgroundColour; } set {
98.             _BackgroundColour = value; } }
99.         public Colour ButtonColour { get { return _ButtonColour; } set { _ButtonColour = value;
100.             } }
101.         public Colour TextBoxColour { get { return _TextBoxColour; } set { _TextBoxColour =
102.             value; } }
103.         public Colour LightSquareColour { get { return _LightSquareColour; } set {
104.             _LightSquareColour = value; } }
105.         public Colour DarkSquareColour { get { return _DarkSquareColour; } set {
106.             _DarkSquareColour = value; } }
107.         public Colour PanelColour { get { return _PanelColour; } set { _PanelColour =
108.             value; } }
109.         public Colour TextColour { get { return _TextColour; } set { _TextColour =
110.             value; } }
111.         public Colour LastMoveColour { get { return _LastMoveColour; } set {
112.             _LastMoveColour = value; } }
113.         public Colour ArrowColour { get { return _ArrowColour; } set { _ArrowColour =
114.             value; } }
115.         public Colour AvailibleMoveColour { get { return _AvailibleMoveColour; } set {
116.             _AvailibleMoveColour = value; } }
117.
118.         internal string PieceFolder;
119.
120.         internal int DefaultEngine;
121.
122.         public UISettings()
123.         {
124.             resetDarkMode();
125.             resetBoard();
126.             PieceFolder = "_Default";
127.         }
128.
129.         // _Default
130.         internal void resetDarkMode()
131.         {
132.             _BackgroundColour = new Colour("#121212");
133.             _TextColour = new Colour(Brushes.White);
134.             _TextBoxColour = new Colour("#76737E");
135.             _ButtonColour = new Colour("#76737E");
136.             _PanelColour = new Colour("#3F3F3F");
137.         }
138.
139.         internal void resetLightMode()
140.         {
141.             _BackgroundColour = new Colour("#F5F5F5");
142.             _TextColour = new Colour(Brushes.Black);
143.             _TextBoxColour = new Colour("#FAF8FF");
144.             _ButtonColour = new Colour("#CCCCC");
145.             _PanelColour = new Colour("#979797");

```

```

137.         }
138.
139.         internal void resetBoard()
140.         {
141.             _LastMoveColour = new Colour(Brushes.BlueViolet);
142.             _AvailableMoveColour = new Colour(Brushes.Red);
143.             _ArrowColour = new Colour(Brushes.Red);
144.             _LightSquareColour = new Colour(Brushes.Wheat);
145.             _DarkSquareColour = new Colour(Brushes.Green);
146.         }
147.
148.     }
149. }

```

EngineInfo

C#

```

1. using System.Collections.Generic;
2.
3. namespace UI.MVVM.Models.Engine
4. {
5.     internal class EngineInfo
6.     {
7.         public string name = "";
8.         public string path = "";
9.         public string author = "";
10.        public List<EngineOption> options = new List<EngineOption>();
11.
12.        public EngineInfo(string name, string path, string author, List<EngineOption> options)
13.        {
14.            this.name = name;
15.            this.path = path;
16.            this.author = author;
17.            this.options = options;
18.        }
19.
20.        public EngineInfo()
21.        {
22.
23.        }
24.
25.        public List<string> genOptionCommands()
26.        {
27.            return null;
28.        }
29.    }
30. }

```

EngineOption

C#

```

1. namespace UI.MVVM.Models.Engine
2. {
3.     internal class EngineOption
4.     {
5.         string optionName;
6.         string optionType;
7.         string maxValue;
8.         string minValue;
9.         string currentValue;
10.        string userChangeValue;
11.
12.    }

```

```
13. }
```

Square

C#

```
1. using System.Windows.Controls;
2.
3. namespace UI.MVVM.Models
4. {
5.     public class Square
6.     {
7.         public Border border = new Border();
8.
9.         public Square() { }
10.    }
11. }
```

Game

C#

```
1. using Engine;
2. using System;
3. using System.Collections.Generic;
4. using UI.MVVM.Models.Players;
5. using UI.MVVM.ViewModels;
6.
7. namespace UI.MVVM.Models
8. {
9.     internal abstract class Game
10.    {
11.        internal Bitboard board;
12.        internal Generation generator = new Generation();
13.        internal bool startFromFEN = false;
14.        internal string FEN = "";
15.        internal string Event;
16.        internal int Round;
17.        internal string Site;
18.        internal DateTime dateTime = DateTime.Now;
19.        protected int CurrentMoveNo = 0;
20.        public int currentMoveNo
21.        {
22.            get { return CurrentMoveNo; }
23.
24.            set
25.            {
26.                if (value < 0 || value >= movesPlayed.Count)
27.                {
28.                    throw new Exception("Index Exception, the move number given was not within
range");
29.                }
30.                CurrentMoveNo = value;
31.                changeBoardViewToMove(CurrentMoveNo);
32.
33.            }
34.        }
35.        public int MovesPlayedCount
36.        {
37.            get { return movesPlayed.Count; }
38.            set { }
39.        }
40.
41.        internal static List<UIMove> movesPlayed = new List<UIMove>();
42.        internal Player[] players = new Player[2];
43.        internal BoardViewModel boardModel;
44.    }
```



```
45.
46.     public string genUCIPositionCommand(int stopindex = -1)
47.     {
48.
49.         string Command = "position ";
50.         if (startFromFEN)
51.         {
52.             Command += "fen " + FEN;
53.         }
54.
55.         else Command += "startpos ";
56.
57.         if (movesPlayed.Count == 0 || stopindex == 0) { return Command; }
58.
59.         Command += "moves ";
60.
61.         Command += movesPlayed[currentMoveNo].uciCommand;
62.
63.         return Command;
64.     }
65.
66.     public virtual void MakeMove(Move move)
67.     {
68.
69.     }
70.
71.
72.     // end the game
73.     public virtual void endGame()
74.     {
75.
76.     }
77.
78.     public void changeBoardViewToMove(int index)
79.     {
80.         CurrentMoveNo = index;
81.         board = new Bitboard();
82.
83.         for (int i = 1; i <= index; i++)
84.         {
85.             board.MakeMove(movesPlayed[i].Move);
86.         }
87.
88.         boardModel.DrawBoard();
89.         boardModel.DrawPieces(board);
90.         boardModel.DrawComments(movesPlayed[index].Comments);
91.     }
92.
93.     public List<Move> genMoves()
94.     {
95.         return generator.GenerateMoves(this.board);
96.     }
97.
98.     virtual public Move createMove(int start, int destination)
99.     {
100.         board = new Bitboard();
101.         if (currentMoveNo != 0)
102.         {
103.             string uci = movesPlayed[currentMoveNo].uciCommand;
104.             string[] moveslist = uci.Split(' ');
105.             foreach (string move in moveslist)
106.             {
107.                 if (move == "")
108.                 {
109.                     break;
110.                 }
111.             }
112.         }
113.     }
```

```

111.                // try and convert the move from a string
112.                if (CommandHandler.stringToMove(move, board, out Move target))
113.                {
114.                    board.MakeMove(target);
115.                }
116.            }
117.        }
118.
119.
120.        foreach (Move i in generator.GenerateMoves(board))
121.        {
122.            if (i.startIndex == start && i.destinationIndex == destination)
123.            {
124.                return i;
125.            }
126.        }
127.
128.        throw new Exception("Move Not Found");
129.    }
130.
131.    public void SaveAsPGN()
132.    {
133.
134.    }
135.
136.    }
137.
138.    }

```

CurrentGame

C#

```

1. using Engine;
2. using System;
3. using System.Collections.Generic;
4. using System.Windows;
5. using System.Windows.Controls;
6. using UI.MVVM.Models.Players;
7. using UI.MVVM.ViewModels;
8. using UI.MVVM.Views;
9.
10. namespace UI.MVVM.Models
11. {
12.     internal class CurrentGame : Game
13.     {
14.
15.         internal PlayerInfoView playerInfo;
16.
17.         int winner = -1; // 0 for white 1 for black, 2 draw, -1 no winner
18.
19.         internal CurrentGame(BoardViewModel model, string _fen, string _event = "Casual", int
20. _round = 1, string _site = "Online")
21.         {
22.             GameInformationView.movesStackPanel.Children.Clear();
23.
24.             try
25.             {
26.                 board = Tools.FENtoBitboard(_fen);
27.             }
28.
29.             catch (Exception e)
30.             {
31.                 MessageBox.Show(e.Message);
32.             }
33.         }
34.     }
35. }

```

```

33.         movesPlayed = new List<UIMove>();
34.         movesPlayed.Add(new UIMove(new Move(), board));
35.
36.         this.startFromFEN = true;
37.         FEN = _fen;
38.         Event = _event;
39.         Round = _round;
40.         Site = _site;
41.         boardModel = model;
42.         players[0] = new Player(this);
43.         players[1] = new Player(this);
44.
45.
46.     }
47.
48.     internal CurrentGame(BoardViewModel model, string _event = "Casual", int _round = 1,
string _site = "Online")
49.     {
50.         GameInformationView.movesStackPanel.Children.Clear();
51.         board = new Bitboard();
52.         boardModel = model;
53.         players[0] = new Player(this);
54.         players[1] = new Player(this);
55.         movesPlayed = new List<UIMove>();
56.         movesPlayed.Add(new UIMove(new Move(), board));
57.         Event = _event;
58.         Round = _round;
59.         Site = _site;
60.     }
61.
62.
63.     public override void MakeMove(Move move)
64.     {
65.         if (winner == -1) // check if the game is over
66.         {
67.             string lastMovePlayed = "";
68.             try
69.             {
70.                 lastMovePlayed = movesPlayed[movesPlayed.Count - 1].uciCommand;
71.             }
72.             catch
73.             {
74.                 lastMovePlayed = "";
75.             }
76.
77.             movesPlayed.Add(new UIMove(move, board, lastMovePlayed));
78.             board.MakeMove(move);
79.             CurrentMoveNo = movesPlayed.Count - 1;
80.             DisplayMove(movesPlayed[movesPlayed.Count - 1].MoveStringAN, board.moveCount);
81.             int turn = board.WhiteToPlay ? 0 : 1;
82.
83.             // check for checkmate
84.             if (board.isMate())
85.             {
86.                 MessageBox.Show("CheckMate");
87.                 endGame(turn ^ 1);
88.                 return;
89.             }
90.
91.
92.             else if (board.isStalemate())
93.             {
94.                 endGame(2);
95.                 return;
96.             }
97.

```

```

98.         players[turn].SendMove();
99.
100.        playerInfo.switchClocks();
101.    }
102. }
103.
104.     private void DisplayMove(string moveString, int moveNo) // add the move to the
Move list that will display the move history
105.     {
106.         StackPanel movesStack = GameInformationView.movesStackPanel;
107.
108.         if (movesStack.Children.Count != 0 &&
((Grid)(movesStack.Children[movesStack.Children.Count - 1])).Children.Count != 3) // in a full
grid the moveNo, white move and black move is displayed
109.         { // it is blacks move on the panel and it will add it to the existing grid
110.
111.             Grid moveGrid = ((Grid)(movesStack.Children[movesStack.Children.Count -
112.             1]));
113.             Button tempMoveButton = new Button();
114.             tempMoveButton.Tag = movesPlayed.Count - 1;
115.             tempMoveButton.Click += (sender, e) => {
changeBoardViewToMove((int)((Button)sender).Tag); };
116.             tempMoveButton.Content = moveString;
117.             Grid.SetColumn(tempMoveButton, 2);
118.             moveGrid.Children.Add(tempMoveButton);
119.             return;
120.         }
121.
122.         // create a new move grid
123.         Grid movegrid = new Grid()
124.         {
125.             ColumnDefinitions =
126.             {
127.                 new ColumnDefinition(),
128.                 new ColumnDefinition(),
129.                 new ColumnDefinition(),
130.             };
131.
132.             movegrid.ColumnDefinitions[0].Width = new GridLength(0.333333333333,
GridUnitType.Star);
133.
134.             TextBlock moveNoTextBlock = new TextBlock();
135.             moveNoTextBlock.HorizontalAlignment = HorizontalAlignment.Center;
136.             moveNoTextBlock.Foreground = User.Settings.TextColour.ColourBrush;
137.             moveNoTextBlock.Text = moveNo.ToString();
138.             Grid.SetColumn(moveNoTextBlock, 0);
139.
140.             Button moveButton = new Button();
141.             moveButton.Tag = movesPlayed.Count - 1;
142.             moveButton.Click += (sender, e) => {
changeBoardViewToMove((int)((Button)sender).Tag); };
143.             moveButton.Content = moveString;
144.             Grid.SetColumn(moveButton, 1);
145.
146.             movegrid.Children.Add(moveNoTextBlock);
147.             movegrid.Children.Add(moveButton);
148.             movesStack.Children.Add(movegrid);
149.         }
150.
151.     public void takeBack()
152.     {
153.         // decrement the variables which store the move count information
154.         CurrentMoveNo--;
155.
156.         // undo the move on the board

```

```

157.         Move lastMove = movesPlayed[movesPlayed.Count - 1].Move;
158.         board.UndoMove(lastMove);
159.         movesPlayed.RemoveAt(movesPlayed.Count - 1);
160.         playerInfo.switchClocks();
161.
162.         // get the move on the move grid
163.         StackPanel movesStack = GameInformationView.movesStackPanel;
164.         Grid LastMoveGrid = (Grid)(movesStack.Children[movesStack.Children.Count -
155.         1]);
166.
167.         // check if the last move played was whites move
168.         if (LastMoveGrid.Children.Count != 3)
169.         {
170.             // we can just remove the last grid from the stack
171.             int lastIndex = movesStack.Children.Count - 1;
172.             movesStack.Children.RemoveAt(lastIndex);
173.         }
174.         else
175.         { // it was blacks turn, we need to remove the last element in the grid
176.             int lastIndex = LastMoveGrid.Children.Count - 1;
177.             LastMoveGrid.Children.RemoveAt(lastIndex);
178.         }
179.
180.         currentMoveNo = CurrentMoveNo;
181.
182.     }
183.
184.     // end the game
185.     // winner: 2 Draw, 0 White, 1 Black
186.     public void endGame(int winningcolour)
187.     {
188.         winner = winningcolour;
189.         PlayingScreen playingscreen =
190.         (PlayingScreen)((Grid)((BoardView)(boardModel.boardCanvas.Parent)).Parent).Parent;
191.
192.         // draw
193.         if (winner == 2)
194.         {
195.             // show draw
196.             playingscreen.displayGameOver("Draw");
197.         }
198.         else if (winner == 0)
199.         {
200.             // white wins
201.             playingscreen.displayGameOver("White Wins");
202.         }
203.         else
204.         {
205.             // black wins
206.             playingscreen.displayGameOver("Black wins");
207.         }
208.
209.         // diable the users from making any more moves
210.         players[0].canMove = false;
211.         players[1].canMove = false;
212.
213.         // end the connection with the bot/ the online player
214.         Bot.connectionEnded = true;
215.         OnlinePlayer.connectionEnded = true;
216.
217.
218.     }
219.
220. }

```

```
221.    }
```

AnalysisGame

C#

```
1. using Engine;
2. using System;
3. using System.Collections.Generic;
4. using System.Windows;
5. using System.Windows.Controls;
6. using UI.MVVM.Models.Players;
7. using UI.MVVM.ViewModels;
8. using UI.MVVM.Views;
9.
10. namespace UI.MVVM.Models
11. {
12.     internal class AnalysisGame : Game
13.     {
14.
15.         internal new AnalysisBoardViewModel boardModel;
16.         internal AnalysisBot bot;
17.
18.         public new int currentMoveNo
19.         {
20.             get { return CurrentMoveNo; }
21.
22.             set
23.             {
24.                 if (value < 0 || value >= movesPlayed.Count)
25.                 {
26.                     throw new Exception("Index Exception, the move number given was not within
range");
27.                 }
28.                 CurrentMoveNo = value;
29.                 changeBoardViewToMove(CurrentMoveNo);
30.
31.             }
32.         }
33.
34.         internal AnalysisGame(AnalysisBoardViewModel model, string _fen, string _event =
"Casual", int _round = 1, string _site = "Online")
35.         {
36.             GameInformationView.movesStackPanel.Children.Clear();
37.
38.             try
39.             {
40.                 board = Tools.FENtoBitboard(_fen);
41.             }
42.
43.             catch (Exception e)
44.             {
45.                 MessageBox.Show(e.Message);
46.             }
47.
48.             movesPlayed = new List<UIMove>();
49.             movesPlayed.Add(new UIMove(new Move(), board));
50.             this.startFromFEN = true;
51.             FEN = _fen;
52.             Event = _event;
53.             Round = _round;
54.             Site = _site;
55.             boardModel = model;
56.             bot = new AnalysisBot(this, User.Engines[User.Settings.DefaultEngine].path);
57.
58.         }
```

```

59.         }
60.
61.         internal AnalysisGame(AnalysisBoardViewModel model, string _event = "Casual", int
        _round = 1, string _site = "Online")
62.         {
63.             GameInformationView.movesStackPanel.Children.Clear();
64.             board = new Bitboard();
65.             boardModel = model;
66.             bot = new AnalysisBot(this, User.Engines[User.Settings.DefaultEngine].path);
67.             movesPlayed = new List<UIMove>();
68.             movesPlayed.Add(new UIMove(new Move(), board));
69.             Event = _event;
70.             Round = _round;
71.             Site = _site;
72.         }
73.
74.         public override void MakeMove(Move move)
75.         {
76.             string uciposition = "";
77.             try
78.             {
79.                 uciposition = movesPlayed[currentMoveNo].uciCommand;
80.             }
81.             catch
82.             {
83.                 uciposition = "";
84.             }
85.             movesPlayed.Add(new UIMove(move, board, uciposition));
86.             board.MakeMove(move);
87.             CurrentMoveNo = movesPlayed.Count - 1;
88.             DisplayMove(movesPlayed[movesPlayed.Count - 1].MoveStringAN, board.moveCount);
89.
90.
91.             // set the bot to the new position
92.             bot.SendMove();
93.
94.             // clear the old analysis
95.             AnalysisScreen.analysisView.Clear();
96.
97.         }
98.
99.         private void DisplayMove(string moveString, int moveNo) // add the move to the Move
        list that will display the move history
100.        {
101.            StackPanel movesStack = GameInformationView.movesStackPanel;
102.
103.            if (movesStack.Children.Count != 0 &&
            ((Grid)(movesStack.Children[movesStack.Children.Count - 1])).Children.Count != 3)
104.            { // it is blacks move on the panel and it will add it to the existing grid
105.
106.                Grid moveGrid = ((Grid)(movesStack.Children[movesStack.Children.Count -
            1]));
107.                Button tempMoveButton = new Button();
108.                tempMoveButton.Tag = movesPlayed.Count - 1;
109.                tempMoveButton.Click += (sender, e) => {
            changeBoardViewToMove((int)((Button)sender).Tag); };
110.                tempMoveButton.Content = moveString;
111.                Grid.SetColumn(tempMoveButton, 2);
112.                moveGrid.Children.Add(tempMoveButton);
113.                return;
114.            }
115.
116.            // create a new move grid
117.            Grid movegrid = new Grid()
118.            {
119.                ColumnDefinitions =

```

```

120.            {
121.                new ColumnDefinition(),
122.                new ColumnDefinition(),
123.                new ColumnDefinition(),
124.            }
125.        };
126.
127.        movegrid.ColumnDefinitions[0].Width = new GridLength(0.333333333333,
GridUnitType.Star);
128.
129.        TextBlock moveNoTextBlock = new TextBlock();
130.        moveNoTextBlock.HorizontalAlignment = HorizontalAlignment.Center;
131.        moveNoTextBlock.Foreground = User.Settings.TextColour.ColourBrush;
132.        moveNoTextBlock.Text = moveNo.ToString();
133.        Grid.SetColumn(moveNoTextBlock, 0);
134.
135.        Button moveButton = new Button();
136.        moveButton.Tag = movesPlayed.Count - 1;
137.        moveButton.Click += (sender, e) => {
changeBoardViewToMove((int)((Button)sender).Tag); };
138.        moveButton.Content = moveString;
139.        Grid.SetColumn(moveButton, 1);
140.
141.        movegrid.Children.Add(moveNoTextBlock);
142.        movegrid.Children.Add(moveButton);
143.        movesStack.Children.Add(movegrid);
144.    }
145.
146.    public new void changeBoardViewToMove(int index)
147.    {
148.        CurrentMoveNo = index;
149.        board = new Bitboard();
150.
151.        if (currentMoveNo != 0)
152.        {
153.            string uci = movesPlayed[currentMoveNo].uciCommand;
154.            string[] moveslist = uci.Split(' ');
155.            foreach (string move in moveslist)
156.            {
157.                if (move == "")
158.                {
159.                    break;
160.                }
161.                // try and convert the move from a string
162.                if (CommandHandler.stringToMove(move, board, out Move target))
163.                {
164.                    board.MakeMove(target);
165.                }
166.            }
167.        }
168.
169.        // draw the board in the new position
170.        boardModel.DrawBoard();
171.        boardModel.DrawPieces(board);
172.        boardModel.DrawComments(movesPlayed[index].Comments);
173.
174.        // clear the analysis view for the new position
175.        AnalysisScreen.analysisView.Clear();
176.        bot.SendMove();
177.    }
178.
179.    public void addCommentToCurrentPosition(string start, string end, string type)
180.    {
181.        string Comment = $"[{start},{end},{type}]";
182.        // we are making sure that the current game is greater than zero otherwise
the comment is not related to any move

```



```

183.         if (movesPlayed[currentMoveNo].Comments.Contains(Comment))
184.         {
185.             // the comment is already in the move
186.             return;
187.         }
188.         movesPlayed[currentMoveNo].Comments.Add(Comment);
189.         boardModel.DrawBoard();
190.         boardModel.DrawPieces(board);
191.         boardModel.DrawComments(movesPlayed[currentMoveNo].Comments);
192.     }
193.
194.     }
195. }

```

UIMove

C#

```

1. using Engine;
2. using System.Collections.Generic;
3.
4. namespace UI.MVVM.Models
5. {
6.     internal class UIMove
7.     {
8.         public Move Move { get; set; }
9.         public string MoveStringAN = ""; //stores the algebraic notation of the move eg e4, e5,
Nf3
10.        public string MoveStringUCI = ""; //stores the UCI notation for the move eg e2e4, e7e5,
g1f3
11.        public List<string> Comments = new List<string>(); // comments stored in the format
12.        public readonly string uciCommand = "";
13.
14.
15.        public UIMove(Move move, Bitboard positionBeforeMove, string uciCommand)
16.        {
17.            Move = move;
18.            MoveStringAN = MoveToAN(move, positionBeforeMove);
19.            MoveStringUCI = MoveToUCI(move, positionBeforeMove);
20.            this.uciCommand = uciCommand + MoveStringUCI + " ";
21.        }
22.
23.        public UIMove(Move move, Bitboard positionBeforeMove)
24.        {
25.            Move = move;
26.            MoveStringAN = MoveToAN(move, positionBeforeMove);
27.            MoveStringUCI = MoveToUCI(move, positionBeforeMove);
28.            this.uciCommand = "";
29.        }
30.        public static string MoveToAN(Move move, Bitboard positionBeforeMove)
31.        {
32.            string ANMove = "";
33.            int enemyIndex = positionBeforeMove.WhiteToPlay ? 1 : 0;
34.
35.            if (move.flag == Move.castling)
36.            {
37.                switch (move.destinationIndex)
38.                {
39.                    case 1:
40.                    case 57:
41.                        return "O-O";
42.                    case 5:
43.                    case 61:
44.                        return "O-O-O";
45.                }
46.            }

```

```

47.
48.         // find which piece is moving
49.         switch (Bitboard.getPieceType(positionBeforeMove.Board,
BitBoardTools.IndexToBitboard(move.startIndex)))
50.         {
51.             case 3: //knight
52.                 ANMove += "n";
53.                 break;
54.             case 4: //bishop
55.                 ANMove += "b";
56.                 break;
57.
58.             case 5: //rook
59.                 ANMove += "r";
60.                 break;
61.
62.             case 6: //queen
63.                 ANMove += "q";
64.                 break;
65.
66.             case 7: //king
67.                 ANMove += "k";
68.                 break;
69.         }
70.
71.
72.         if ((positionBeforeMove.Board[enemyIndex] &
BitBoardTools.IndexToBitboard(move.destinationIndex)) != 0)
73.         { // it is a capture
74.             if (ANMove == "")
75.                 { // pawn is capturing so the start file needs to be appended
76.
77.                     ANMove += (char)(BitBoardTools.CalculateFile(move.startIndex) + 96); // 96
is the char code before a, Calculate file returns 1-8. This will result in 1=a 2=b ect..
78.                 }
79.
80.             return ANMove + "x" + Tools.IndexToSquare(move.destinationIndex);
81.         }
82.
83.         // check if piece of the same type can move to the same square
84.
85.         ANMove += Tools.IndexToSquare(move.destinationIndex);
86.
87.         if (move.isPromotion)
88.         {
89.             ANMove += "=";
90.             switch (move.pieceIndex)
91.             {
92.                 case 0: //knight
93.                     ANMove += "n";
94.                     break;
95.                 case 1: //bishop
96.                     ANMove += "b";
97.                     break;
98.
99.                 case 2: //rook
100.                     ANMove += "r";
101.                     break;
102.
103.                 case 3: //queen
104.                     ANMove += "q";
105.                     break;
106.
107.             }
108.         }
109.

```

```

110.         return ANMove;
111.     }
112.
113.     public static string MoveToUCI(Move move, Bitboard positionBeforeMove)
114.     {
115.         string result = string.Empty;
116.         result += Tools.IndexToSquare(move.startIndex);
117.         result += Tools.IndexToSquare(move.destinationIndex);
118.
119.         if (move.isPromotion)
120.         {
121.             switch (move.pieceIndex)
122.             {
123.                 case 0: //knight
124.                     result += "n";
125.                     break;
126.                 case 1: //bishop
127.                     result += "b";
128.                     break;
129.
130.                 case 2: //rook
131.                     result += "r";
132.                     break;
133.
134.                 case 3: //queen
135.                     result += "q";
136.                     break;
137.             }
138.         }
139.     }
140.
141.     return result;
142. }
143. }
144. }

```

Player

C#

```

1. internal class Player
2.     {
3.         public bool isWhite;
4.         public bool canMove;
5.         public string UserName;
6.         public int Elo;
7.         public string flag;
8.         public TimeSpan timeOnClock = TimeSpan.MaxValue;
9.         public double clockIncrementMS = 0;
10.        internal Game game;
11.
12.        public Player(Game game)
13.        {
14.            canMove = true;
15.            this.game = game;
16.        }
17.
18.        public Player(Game game, string UserName)
19.        {
20.            canMove = true;
21.            this.game = game;
22.        }
23.
24.        public virtual void SendMove()
25.        {
26.            // do nothing

```

```

27.     }
28.
29.     public virtual void SendRequest(string type)
30.     {
31.         // handle as always accept
32.         if (type == "TakeBack1")
33.         {
34.             if (game.MovesPlayedCount < 2) return;
35.             ((CurrentGame)game).takeBack();
36.             Thread.Sleep(100);
37.             ((CurrentGame)game).takeBack();
38.         }
39.
40.         else if (type == "TakeBack2")
41.         {
42.             if (game.MovesPlayedCount < 2) return;
43.             ((CurrentGame)game).takeBack();
44.         }
45.     }
46. }

```

Bot

C#

```

1. internal class Bot : Player
2. {
3.     protected string path;
4.     protected static Thread? CommunicationThread;
5.     public static bool connectionEnded;
6.     protected Process process;
7.     protected bool ignoreResponse = false;
8.     protected Canvas? boardCanvas;
9.     protected bool readyok = false;
10.    private string depth;
11.
12.    public Bot(Game game, string botLoc, string depthToSearch = "5") : base(game)
13.    {
14.        this.game = game;
15.        this.depth = depthToSearch;
16.        canMove = false;
17.
18.        if (game.boardModel != null)
19.        {
20.            this.boardCanvas = game.boardModel.boardCanvas;
21.        }
22.
23.        path = botLoc;
24.        connectionEnded = false;
25.        CommunicationThread = new ThreadStart(this.CreateConnection);
26.        CommunicationThread.Start();
27.    }
28.
29.    protected void CreateConnection()
30.    {
31.        process = new Process()
32.        {
33.            StartInfo = new ProcessStartInfo()
34.            {
35.                FileName = path,
36.                UseShellExecute = false,
37.                RedirectStandardInput = true,
38.                RedirectStandardOutput = true,
39.                CreateNoWindow = true,
40.                RedirectStandardError = true,
41.            }

```

```
42.         };
43.         /* Set your output and error (asynchronous) handlers
44.         process.OutputDataReceived += new DataReceivedEventHandler(this.OutputHandler);
45.         process.ErrorDataReceived += new DataReceivedEventHandler(this.OutputHandler);
46.         /* Start process and handlers
47.         process.Start();
48.         process.BeginOutputReadLine();
49.         process.BeginErrorReadLine();
50.         process.StandardInput.WriteLine("uci");
51.         process.StandardInput.WriteLine("ucinewgame");
52.         process.StandardInput.WriteLine("isready");
53.         do
54.         {
55.         } while (!connectionEnded);
56.         process.Close();
57.     }
58.
59.     virtual protected void OutputHandler(object sendingProcess, DataReceivedEventArgs
outline)
60.     {
61.
62.         if (outline.Data == null)
63.         {
64.             return;
65.         }
66.
67.         if (outline.Data.Contains("bestmove") && !ignoreResponse)
68.         {
69.             // best move in the format: bestmove {move} Eval: {value}
70.             string move = outline.Data.Split(' ')[1];
71.             string start = move.Substring(0, 2);
72.             string end = move.Substring(2, 2);
73.             int promotion = -1;
74.
75.             if (move.Length > 4)
76.             {
77.                 switch (move[4])
78.                 {
79.                     case 'n':
80.                         promotion = 4;
81.                         break;
82.
83.                     case 'b':
84.                         promotion = 5;
85.                         break;
86.
87.                     case 'r':
88.                         promotion = 6;
89.                         break;
90.
91.                     case 'q':
92.                         promotion = 7;
93.                         break;
94.                 }
95.             }
96.             byte startIndex = Tools.SquareToIndex(start);
97.             byte endIndex = Tools.SquareToIndex(end);
98.             Move moveMade = new Move();
99.             try
100.            {
101.                if (promotion != -1) // Create a promotion
102.                {
103.                    moveMade = new Move(startIndex, endIndex, (ushort)promotion);
104.                }
105.                else
106.                {
```

```

107.         moveMade = game.createMove(startIndex, endIndex);
108.     }
109.     }
110.     catch
111.     {
112.         // an illegal move was made
113.         // end the game on an illegal move by the engine
114.         boardCanvas.Dispatcher.Invoke(() =>
115.         { // make the move
116.             ((CurrentGame)game).endGame(isWhite ? 0 : 1);
117.             MessageBox.Show($"Engine made invallid move: {move}");
118.         });
119.         return;
120.     }
121.
122.     boardCanvas.Dispatcher.Invoke(() =>
123.     { // make the move
124.         game.MakeMove(moveMade);
125.         game.boardModel.DrawBoard();
126.         game.boardModel.DrawPieces(game.board);
127.     });
128. }
129.
130. else if (outLine.Data == "readyok")
131. {
132.     readyok = true;
133. }
134.
135. else if (outLine.Data.Contains("Exception"))
136. {
137.     MessageBox.Show("An error has occured, the connection has been ended");
138.     Bot.connectionEnded = true;
139. }
140. }
141.
142. public override void SendMove()
143. { // send the search command
144.     if (!connectionEnded)
145.     {
146.         while (!readyok)
147.         {
148.             // wait for readyok
149.         }
150.         string PosCommand = game.genUCIPositionCommand();
151.         process.StandardInput.WriteLine(PosCommand); // set the position
152.         process.StandardInput.WriteLine("go depth " + depth);
153.     }
154. }
155.
156. public override void SendRequest(string type)
157. {
158.     // get the value of the position
159.     // if it is wining then reject the draw / accept resignation
160. }
161. }

```

AnalysisBot

C#

```

1. internal class AnalysisBot : Bot
2.     {
3.         bool ownbookoff = false;
4.         public AnalysisBot(AnalysisGame game, string botLoc) : base(game, botLoc)
5.         {
6.             this.game = game;

```

```

7.         canMove = false;
8.         this.boardCanvas = game.boardModel.boardCanvas;
9.     }
10.
11.     void turnOffOwnBook()
12.     {
13.         process.StandardInput.WriteLine("setoption name OwnBook value false");
14.     }
15.
16.     protected override void OutputHandler(object sendingProcess, DataReceivedEventArgs
outline)
17.     {
18.
19.         if (outline.Data == null)
20.         {
21.             return;
22.         }
23.
24.         if (outline.Data.StartsWith("info"))
25.         {
26.             // pass the data to the UI
27.             AnalysisScreen.analysisView.Dispatcher.Invoke(() =>
28.             {
29.                 AnalysisScreen.analysisView.AddInfo(outline.Data);
30.             });
31.
32.             if (outline.Data.Contains("pv"))
33.             {
34.                 string data = outline.Data.Replace("multiply", "");
35.                 string move = data.Split("pv ")[1].Split(' ')[0];
36.                 if (move == "")
37.                 {
38.                     return;
39.                 }
40.                 string start = move.Substring(0, 2);
41.                 string end = move.Substring(2, 2);
42.                 boardCanvas.Dispatcher.Invoke(() =>
43.                 {
44.                     ((AnalysisGame)game).addCommentToCurrentPosition(start, end, "Arrow");
45.                 });
46.             }
47.
48.         }
49.
50.         else if (outline.Data.Contains("Exception"))
51.         {
52.             MessageBox.Show("An error has occurred, the connection has been ended");
53.             connectionEnded = true;
54.         }
55.     }
56.
57.     public override void SendMove()
58.     {
59.         // check if opening book is turned off
60.         if (!ownbookoff)
61.         {
62.             turnOffOwnBook();
63.             ownbookoff = !ownbookoff;
64.         }
65.
66.         // send the search command
67.         // stop the current search
68.         process.StandardInput.WriteLine("stop");
69.         Thread.Sleep(10);
70.
71.         // create the position command and send it to the engine

```

```

72.         string PosCommand = game.genUCIPositionCommand(game.currentMoveNo);
73.         process.StandardInput.WriteLine(PosCommand);
74.
75.         // start an infinite search
76.         process.StandardInput.WriteLine("go");
77.     }
78. }

```

OnlinePlayer

C#

```

1. internal class OnlinePlayer : Player
2. {
3.     string path;
4.     static Thread CommunicationThread;
5.     public static bool connectionEnded;
6.     Process process;
7.     Canvas boardCanvas;
8.
9.     public OnlinePlayer(Game game) : base(game)
10.    {
11.        this.canMove = false;
12.    }
13. }
14.
15. [DllImport("user32.dll")]
16. private static extern Boolean ShowWindow(IntPtr hWnd, Int32 nCmdShow);
17. public void CreateConnection()
18. {
19.
20. }
21.
22. void OutputHandler(object sendingProcess, DataReceivedEventArgs outLine)
23. {
24. }
25.
26. public override void SendMove()
27. {
28.
29. }
30.
31. public override void SendRequest(string type)
32. {
33.     // send request to online user
34.     // recive answer
35.     // handle response
36. }
37. }

```

Client

Program

C#

```

1. using System;
2. using System.Collections.Generic;
3. using System.Linq;
4. using System.Text;
5. using System.Threading.Tasks;
6. using System.Net;
7. using System.Net.Sockets;
8. using System.Net.NetworkInformation;
9. using System.Threading;
10. using System.Diagnostics;
11. using System.Xml;

```



```
12. using System.Xml.Serialization;
13.
14. namespace Client
15. {
16.     internal class Program
17.     {
18.
19.         static Thread BroadcastThread;
20.         static bool readBroadcast = true;
21.
22.         static Socket TCPSocket;
23.         static Thread TCPThread;
24.         static bool readTCP = true;
25.
26.         static void Main(string[] args)
27.         {
28.
29.             // Client Code
30.             // search for servers
31.             // get responses
32.             // output them
33.             // let the ui handle them
34.             // get input from the ui to see which one to connect to / re-search the servers
35.             // try connect
36.             // if successful then output something
37.             // communicate
38.
39.             string user = "";
40.
41.             BroadcastThread = new Thread(ReadBraudcast);
42.             BroadcastThread.Start();
43.
44.
45.             do
46.             {
47.                 user = Console.ReadLine();
48.                 user = user.Trim();
49.                 if (user == "write")
50.                 {
51.                     WriteBraudcast();
52.                 }
53.
54.                 else if (user.Contains("send")) {
55.                     string dataToSend = user.Split(new[] { "send " },
StringSplitOptions.None)[1];
56.                     TCPSend(dataToSend);
57.                 }
58.
59.                 else if (user.Contains("connect"))
60.                 {
61.                     string ip = user.Split(' ')[1];
62.                     int port = int.Parse(user.Split(' ')[2]);
63.
64.                     SetupTCPSocket(ip, port);
65.
66.                     readBroadcast = false;
67.
68.                     TCPThread = new Thread(TCPRead);
69.                     TCPThread.Start();
70.
71.                 }
72.
73.             } while (user != "exit");
74.
75.             TCPSocket.Close();
76.
```

```

77.     }
78.
79.
80.     static void ReadBraudcast() {
81.         Console.WriteLine("Waiting For Server Response");
82.         byte[] buffer = new byte[1024];
83.         var socket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram,
ProtocolType.Udp);
84.         socket.SetSocketOption(SocketOptionLevel.Socket, SocketOptionName.Broadcast, 1);
85.         IPEndPoint recieveEnd = new IPEndPoint(IPAddress.Any, 16790);
86.         socket.Bind(recieveEnd);
87.
88.         EndPoint Remote = new IPEndPoint(IPAddress.Any, 0);
89.
90.         while (readBroadcast)
91.         {
92.             int recv = socket.ReceiveFrom(buffer, ref Remote);
93.             Console.WriteLine("Message received from {0}:", Remote.ToString());
94.             string data = Encoding.ASCII.GetString(buffer, 0, recv);
95.             Console.WriteLine(data);
96.         }
97.         socket.Close();
98.     }
99.
100.    static void SetupTCPsocket(string ipToConnect, int port) {
101.        IPAddress ip = IPAddress.Parse(ipToConnect);
102.        TCPsocket = new Socket(AddressFamily.InterNetwork, SocketType.Stream,
ProtocolType.Tcp);
103.        IPEndPoint ep = new IPEndPoint(ip, port);
104.        TCPsocket.Connect(ep);
105.        Console.WriteLine("Connected Successfully");
106.    }
107.
108.    static void TCPSend(string data) {
109.        TCPsocket.Send(Encoding.UTF8.GetBytes(data));
110.    }
111.
112.    static void TCPRead() {
113.        try
114.        {
115.            byte[] buffer = new byte[1024];
116.
117.            while (readTCP)
118.            {
119.                int recv = TCPsocket.Receive(buffer);
120.                string data = Encoding.ASCII.GetString(buffer, 0, recv);
121.                Console.WriteLine(data);
122.            }
123.        }
124.        catch (Exception e) {
125.            Console.WriteLine(e);
126.            readTCP = false;
127.            TCPsocket.Close();
128.        }
129.    }
130.
131.
132.    static void WriteBraudcast()
133.    {
134.        int PORT = 9050;
135.        UdpClient udpClient = new UdpClient();
136.        var data = Encoding.UTF8.GetBytes("request servers");
137.        udpClient.Send(data, data.Length, "255.255.255.255", PORT);
138.    }
139.

```

```

140.         static IPAddress ComputeBroadcastIP() // (ip address | !subnet_mask) =
        broadcast Address
141.         {
142.             IPEndPoint host = Dns.GetHostEntry(Dns.GetHostName());
143.             IPAddress ipAddress = host.AddressList[0];
144.
145.             foreach (IPAddress address in host.AddressList) {
146.                 if (address.AddressFamily == AddressFamily.InterNetwork) {
147.                     ipAddress = address;
148.                     break;
149.                 }
150.             }
151.
152.             IPAddress mask = GetSubnetMask(ipAddress);
153.
154.             byte[] IPaddress = ipAddress.GetAddressBytes();
155.             byte[] SubnetMask = mask.GetAddressBytes();
156.             byte[] broadcastBytes = new byte[IPaddress.Length];
157.
158.             for (int i = 0; i < broadcastBytes.Length; i++)
159.             {
160.                 broadcastBytes[i] = (byte)(IPaddress[i] | ~SubnetMask[i]);
161.             }
162.
163.             return new IPAddress(broadcastBytes);
164.         }
165.
166.         public static IPAddress GetSubnetMask(IPAddress address)
167.         {
168.             foreach (NetworkInterface adapter in
169.                 NetworkInterface.GetAllNetworkInterfaces())
170.             {
171.                 foreach (UnicastIPAddressInformation unicastIPAddressInformation in
172.                     adapter.GetIPProperties().UnicastAddresses)
173.                 {
174.                     if (unicastIPAddressInformation.Address.AddressFamily ==
175.                         AddressFamily.InterNetwork)
176.                     {
177.                         if (address.Equals(unicastIPAddressInformation.Address))
178.                         {
179.                             return unicastIPAddressInformation.IPv4Mask;
180.                         }
181.                     }
182.                 }
183.             }
184.
185.             throw new ArgumentException(string.Format("Can't find subnetmask for IP
186. address '{0}'", address));
187.         }
188.
189.
190.         // System.Net.NetworkInformation.NetworkInterface.GetIsNetworkAvailable(); - add to
        UI code and display a error message if network is not available (do not let the client create a
        new host (save memory and processing))

```

Server

Program

C#

```

1. using System;
2. using System.Collections.Generic;

```

```
3. using System.Linq;
4. using System.Text;
5. using System.Threading.Tasks;
6. using System.Net;
7. using System.Net.Sockets;
8. using System.Net.NetworkInformation;
9. using System.Threading;
10.
11. namespace Server
12. {
13.     internal class Program
14.     {
15.         static IPAddress localIP;
16.         static Thread thread;
17.         static int TCP_Port = 9054;
18.         static bool ListenBroadCast = true;
19.
20.         static void Main(string[] args)
21.         {
22.             // listen to the broadcast ip
23.             // after two connect responses set up listener
24.             // send a connect request to the client via broadcast / client ip
25.             // wait for connect confirmation then write code to re-transmit the messages sent
26.
27.             //getLocalIp();
28.             thread = new Thread(listenBroadcast); // set up the broadcast loop that will
broadcast its information
29.             thread.Start();
30.             TCPConnection();
31.         }
32.
33.         static void getLocalIp() {
34.             IPHostEntry host = Dns.GetHostEntry(Dns.GetHostName());
35.             IPAddress ipAddress = host.AddressList[0];
36.
37.             foreach (IPAddress address in host.AddressList)
38.             {
39.                 if (address.AddressFamily == AddressFamily.InterNetwork)
40.                 {
41.                     localIP = address;
42.                     break;
43.                 }
44.             }
45.         }
46.
47.
48.         static void sendBroadcast(string data)
49.         {
50.             {
51.                 int PORT = 16790;
52.                 UdpClient udpClient = new UdpClient();
53.                 var Bytedata = Encoding.UTF8.GetBytes(data);
54.                 udpClient.Send(Bytedata, Bytedata.Length, "255.255.255.255", PORT);
55.                 udpClient.Close();
56.             }
57.
58.             static void TCPConnection() {
59.                 Socket TCPClient = new Socket(AddressFamily.InterNetwork, SocketType.Stream,
ProtocolType.Tcp);
60.                 IPEndPoint ep = new IPEndPoint(IPAddress.Any, TCP_Port);
61.                 TCPClient.Bind(ep);
62.                 TCPClient.Listen(1000);
63.                 TCPClient.BeginAccept(new AsyncCallback(AcceptCallback), TCPClient);
64.             }
65.
66.
```

```
67.     private static void AcceptCallback(IAsyncResult ar)
68.     {
69.         Socket listener = (Socket)ar.AsyncState;
70.         Socket handler = listener.EndAccept(ar);
71.
72.         // Handle the connection
73.         sendBroadcast("Server No Longer Available");
74.
75.         ListenBroadCast = false;
76.
77.         handler.Send(Encoding.ASCII.GetBytes("Hello, World!"));
78.     }
79.
80.     static void listenBroadcast() {
81.
82.         byte[] buffer = new byte[1024];
83.
84.         var socket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram,
            ProtocolType.Udp);
85.         IPEndPoint recieveEnd = new IPEndPoint(IPAddress.Any, 9050);
86.         socket.Bind(recieveEnd);
87.
88.         Console.WriteLine("Waiting for client");
89.
90.         IPEndPoint sender = new IPEndPoint(IPAddress.Any, 0);
91.         EndPoint Remote = (EndPoint)(sender);
92.         while (ListenBroadCast)
93.         {
94.             int recv = socket.ReceiveFrom(buffer, ref Remote);
95.             Console.WriteLine("Message received from {0}:", Remote.ToString());
96.             string data = Encoding.ASCII.GetString(buffer, 0, recv);
97.             Console.WriteLine(data);
98.
99.             if (data == "request servers")
100.            {
101.                // send response server name and other info and ip
102.                sendBroadcast("Server: " + localIP.ToString() + " PORT: 9054");
103.            }
104.        }
105.
106.        socket.Close();
107.    }
108. }
109. }
```